

Table of Contents

Contents

DAY 1:ARRAYS-I.....	11
Problem 1: Set Matrix Zeros	12
Problem 2: Pascal's Triangle	14
Problem 3: Next Permutation	15
Problem 4:Maximum SubArray:	17
Problem 5: Sort Colors	17
Problem 6: 121. Best Time to Buy and Sell Stock	18
DAY 2:ARRAYS-II.....	20
Problem 7: Rotate Matrix	21
Problem 8: 56. Merge Intervals:	22
Problem 9: 88. Merge Sorted Array:	23
Problem 10 :287. Find the Duplicate Number	24
Problem 11: Missing and repeating numbers:	25
Problem 12: Count Inversions	26
DAY 3: Arrays Part-III	29
Problem 13: 74. Search a 2D Matrix	30
Problem 14: 50. Pow(x, n):	31
Problem 15: 169. Majority Element	32
Problem 16: 229. Majority Element II	33
Problem 17: 62. Unique Paths	35
Problem 18: 493. Reverse Pairs	36
Day 4: Arrays Part-IV	38
Problem 19: 1. Two Sum	39
Problem 20: 18. 4Sum	40
Problem 21: 128. Longest Consecutive Sequence	42
Problem 22: Longest Subarray Zero Sum	43
Problem 23: Subarrays with XOR 'K'	45
Problem 24: 3. Longest Substring Without Repeating Characters	49

Problem 25: 1004. Max Consecutive Ones III	51
Day 5: Linked List	53
Problem 26: 1004. Max Consecutive Ones III	54
Problem 27: 876. Middle of the Linked List	54
Problem 28: 21. Merge Two Sorted Lists	55
Problem 29: 19. Remove Nth Node From End of List	58
Problem 30: 2. Add Two Numbers	59
Problem 31: 237. Delete Node in a Linked List	62
DAY 6: Linked List Part-II	64
Problem 32: 160. Intersection of Two Linked Lists	65
Problem 33: 141. Linked List Cycle	67
Problem 34: 25. Reverse Nodes in k-Group	68
Problem 35: 234. Palindrome Linked List	72
Problem 36: 142. Linked List Cycle II	74
Problem 37: Flatten A Linked List	76
Day 7: Linked List and Arrays	79
Problem 38: 61. Rotate List	80
Problem 39: 138. Copy List with Random Pointer	82
Problem 40: 15. 3Sum	84
Problem 41: 42. Trapping Rain Water	87
Problem 42: 26. Remove Duplicates from Sorted Array	88
Problem 43: 485. Max Consecutive Ones	89
Day 8: Greedy Algorithm	91
Problem 44: N meetings in one room	92
Problem 45: Minimum Number of Platforms	93
Problem 46: Job Sequencing Problem	95
Problem 47: Fractional Knapsack	97
Problem 48: Maximum activities	99
DAY 9: RECURSION	101
Problem: 49: Subset Sum	102
Problem 50: 90. Subsets II	103

Problem 51: 39. Combination Sum.....	104
Problem 52: 40. Combination Sum II.....	106
Problem 53: 131. Palindrome Partitioning.....	109
Day 10: RECURSION AND BACKTRACKING.....	115
Problem 54: 46. Permutations.....	116
Problem 55: 51. N-Queens.....	117
Problem 55-II: 52. N-Queens II.....	120
Problem 56: 37. Sudoku Solver.....	121
Problem 57: M-Coloring Problem.....	124
Problem 58: Rat In a Maze All Paths.....	126
Problem 59: 140. Word Break II.....	128
Problem 59-I:200. Number of Islands.....	130
DAY 11: BINARY SEARCH.....	133
Problem 60: Find Nth Root Of M.....	134
Problem 61: Matrix Median.....	135
Problem 62: 540. Single Element in a Sorted Array.....	138
Problem 63: 4. Median of Two Sorted Arrays.....	139
Problem 64: 33. Search in Rotated Sorted Array.....	142
Problem 65: Kth Element of Two Sorted Arrays.....	144
Problem 66: Allocate Books.....	146
Problem 67: Chess Tournament.....	148
Problem 67-II: 1011. Capacity To Ship Packages Within D Days.....	152
Day 12: HEAPS.....	155
Problem 68: MinHeap.....	156
Problem 69: 215. Kth Largest Element in an Array.....	157
Problem 70: K Max Sum Combinations.....	158
Problem 71: 295. Find Median from Data Stream.....	161
Problem 72: Merge K Sorted Arrays.....	162
Problem 73: 347. Top K Frequent Elements.....	164
Day 13: STACKS AND QUEUES.....	166
Problem 74:Maximum XOR With an Element From Array.....	167

Problem 75:Implement a Queue	168
Problem 76: 225. Implement Stack using Queues	172
Problem 77: 232. Implement Queue using Stacks	175
Problem 78: 20. Valid Parentheses	177
Problem:78-I. 1249. Minimum Remove to Make Valid Parentheses	178
Problem 79: 496. Next Greater Element I	180
Problem 80: Sort a Stack.....	182
Problem 81-Count Distinct Element in Every K Size Window.....	186
Day 14: STACKS AND QUEUE-II	189
Problem 82- Next Smaller Element	190
Problem 83-I:146. LRU Cache	191
Problem 83-II : 460. LFU Cache.....	198
Problem 84. 84. Largest Rectangle in Histogram.....	205
Problem 85. 239. Sliding Window Maximum.....	208
Problem 86: 155. Min Stack	210
Problem 87 : 994. Rotting Oranges	212
Problem 88: 901. Online Stock Span.....	215
Problem 89: Maximum of minimum for every window size	216
Problem 89-I: The Celebrity Problem.....	219
DAY 15: String.....	222
Problem 90: 151. Reverse Words in a String	223
Problem 91: 5. Longest Palindromic Substring.....	225
Problem 92: 13. Roman to Integer	226
Problem 93: 8. String to Integer (atoi)	228
Problem 94: 14. Longest Common Prefix	231
Problem 95: 686. Repeated String Match	234
Day 16: String Part-II	236
Problem 96,97:28. Find the Index of the First Occurrence in a String	237
Problem 98: Minimum Characters For Palindrome.....	238
Problem 99: 242. Valid Anagram.....	239
Problem 100: 38. Count and Say.....	241
Problem:101. 165. Compare Version Numbers.....	243

Day 17: Binary Tree-I	247
Problem:101: 94. Binary Tree Inorder Traversal.....	248
Problem:102: 144. Binary Tree Preorder Traversal	249
Problem:103: 145. Binary Tree Postorder Traversal	251
Problem:104: 145. Morris Inorder Traversal	253
Problem:105: Left View Of a Binary Tree	255
Problem:106: Bottom View Of Binary Tree	257
Problem:107: Top View Of Binary Tree.....	259
Problem:108: Tree Traversals-Preorder inorder postorder in a single traversal.....	262
Problem:109: 987. Vertical Order Traversal of a Binary Tree	264
Problem:109: All Root to Leaf Paths In Binary Tree.....	267
Problem:110: 662. Maximum Width of Binary Tree.....	270
Day 18: Binary Tree part-II	273
Problem:111: 102. Binary Tree Level Order Traversal.....	274
Problem:112: 104. Maximum Depth of Binary Tree.....	275
Problem:113: 543. Diameter of Binary Tree.....	277
Problem:114: 110. Balanced Binary Tree	279
Problem:115: 236. Lowest Common Ancestor of a Binary Tree.....	281
Problem:116: 100. Same Tree	283
Problem:117: 103. Binary Tree Zigzag Level Order Traversal	286
DAY 19: BINARY TREE-III	288
Problem:118: 124. Binary Tree Maximum Path Sum.....	289
Problem:119: 105. Construct Binary Tree from Preorder and Inorder Traversal	291
Problem:120: 106. Construct Binary Tree from Inorder and Postorder Traversal.....	292
Problem:121: 101. Symmetric Tree	295
Problem:122: 114. Flatten Binary Tree to Linked List	297
Problem:123: 100. Same Tree	299
Problem:124: Invert a Binary Tree	300
Problem:125: Children Sum Property.....	303

DAY 20: BINARY SEARCH TREE-I:	307
Problem:126:116. Populating Next Right Pointers in Each Node You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:	308
Problem:127: Search In BST	310
Problem:128: 108. Convert Sorted Array to Binary Search Tree	311
Problem:129: 1008. Construct Binary Search Tree from Preorder Traversal	313
Problem:130: 98. Validate Binary Search Tree	314
Problem:131: 235. Lowest Common Ancestor of a Binary Search Tree	317
Problem:132: Predecessor And Successor In BST	319
DAY 21: BINARY SEARCH TREE-I:	322
Problem:133: Floor in BST	323
Problem:134: Ceil from BST	324
Problem:135: 230. Kth Smallest Element in a BST	327
Problem:136: 653. Two Sum IV - Input is a BST	329
Problem:137: 173. Binary Search Tree Iterator	331
Problem:138: 1373. Maximum Sum BST in Binary Tree	333
Problem:139: 297. Serialize and Deserialize Binary Tree	335
Day 22: BINARY TREES(MISCELLANEOUS)	338
Problem:140: 114. Flatten Binary Tree to Linked List	339
Problem:141: 295. Find Median from Data Stream	341
Problem:142: 703. Kth Largest Element in a Stream	343
Problem:143: Count Distinct Element in Every K Size Window	345
Problem:143-I: 215. Kth Largest Element in an Array	347
Problem:144: 733. Flood Fill	349
DAY 23: GRAPHS	351
Problem:144: 133. Clone Graph	352
Problem:145: DFS OF GRAPH	354
Problem:146: BFS OF GRAPH	356
Problem:147: 207. Course Schedule(UNDIRECTED GRAPH)	358
Problem:148: 207. Course Schedule-DFS(UNDIRECTED GRAPH)	360
Problem:149: 200. Number of Islands	362

Problem:150(A): 785. Is Graph Bipartite(BFS)?.....	363
Problem:150-(B): 785. Is Graph Bipartite(DFS)?.....	365
DAY 24:GRAPHS-II	367
Problem:151: 1520. Maximum Number of Non-Overlapping Substrings	368
Problem:152: Implementing Dijkstra Algorithm.....	369
Problem:153: Distance from the Source (Bellman-Ford Algorithm).....	373
Problem:154: Floyd Warshall	375
Problem:155: 1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance.....	377
Problem:156: Minimum Spanning Tree	380
Problem:157: 1584. Min Cost to Connect All Points	382
Problem:158: Kruskal's Minimum Spanning Tree Algorithm	386
Problem:159: Strongly Connected Components (Tarjan's Algorithm)	389
Day 25: DYNAMIC PROGRAMMING	393
Problem:160: 152. Maximum Product Subarray	394
Problem:161: 300. Longest Increasing Subsequence	395
Problem:162: 1143. Longest Common Subsequence	397
Problem:163: 474. Ones and Zeroes	401
Problem:164: 72. Edit Distance.....	403
Problem:165: Maximum sum increasing subsequence	407
Problem:166: Matrix Chain Multiplication.....	408
DAY 26: DYNAMIC PROGRAMMING-II.....	414
Problem:167: 64. Minimum Path Sum	415
Problem:168: 322. Coin Change.....	417
Problem:169: 416. Partition Equal Subset Sum.....	418
Problem:170: 1547. Minimum Cost to Cut a Stick.....	421
Problem:171: 887. Super Egg Drop	426
Problem:171: 139. Word Break	429
Problem:172: 131. Palindrome Partitioning.....	431
Problem:173: 1235. Maximum Profit in Job Scheduling.....	433
DAY 27:TRIE:	436
Problem:174: 208. Implement Trie (Prefix Tree).....	437

Problem:175: Implement Trie II.....	439
Problem:176: Complete String.....	445
Problem:177: Count Distinct Substrings.....	448
Problem:178: Power Set(VERY IMPORTANT).....	449
Problem:179: 421. Maximum XOR of Two Numbers in an Array.....	450
Problem:180: 1707. Maximum XOR With an Element From Array.....	451
DESIGN.....	455
Problem:181. 622. Design Circular Queue.....	456
Problem:182. 355. Design Twitter.....	458
Problem:183. 284. Peeking Iterator.....	463
Problem:184. 1396. Design Underground System.....	465
OTHER PROBLEMS.....	469
Problem:185.WORD LADDER-I.....	470
Problem:186.WORD LADDER-II.....	472
Problem:187. 44. Wildcard Matching.....	479
Problem:188. 10. Wildcard Matching.....	482
Problem:189. 17. Letter Combinations of a Phone Number.....	484
Problem:190. 632. Smallest Range Covering Elements from K Lists.....	488
Problem:191. 787. Cheapest Flights Within K Stops.....	491
Problem:192. 417. Pacific Atlantic Water Flow.....	494
Problem:193. 1463. Cherry Pickup II.....	498
Problem:194. 934. Shortest Bridge(DFS & BFS).....	504
Problem:195-I. 403. Frog Jump.....	506
Problem:195-II. 70. Climbing Stairs.....	511
Problem:196. 34. Binary Search-Find First and Last Position of Element in Sorted Array(Upper & LowerBound).....	512
Problem:197. 825. Friends Of Appropriate Ages.....	514
Problem:198. 2262. Total Appeal of A String.....	517
Problem:199. 547.Number of Provinces-Union by Rank(Path Compression).....	521
MATH PROBLEMS.....	527
Problem-1:7. Reverse Integer.....	528
Problem-2:1492. The kth Factor of n.....	529

Problem-3:1621. Number of Sets of K Non-Overlapping Line Segments	529
Problem-4:1447. Simplified Fractions.....	531
Problem-5: 29. Divide Two Integers.....	532
Problem-6: 1317. Convert Integer to the Sum of Two No-Zero Integers.....	533
Problem-7: 69. Sqrt(x)	534
Problem-8: 50. Pow(x, n).....	536
Problem-9: 367. Valid Perfect Square.....	537
Problem-10: 204. Count Primes.....	540
Problem-11: 342. Power of Four.....	540
Problem-11b: 231. Power of Two.....	542
Problem-11c: 326. Power of Three	543
Problem-12: 1415. The k-th Lexicographical String of All Happy Strings of Length n.....	546
BIT MANIPULATION	548
Problem-1. 705. Design HashSet.....	560
Problem-2. 1097B - Petr and a Combination Lock(CODE FORCES)	562
Problem-3: 2151. Maximum Good People Based on Statements(BITMASK)	564
Problem-4: 1799. Maximize Score After N Operations.....	568
Problem-5: 2002. Maximum Product of the Length of Two Palindromic Subsequence	571
Problem-6: 2212. Maximum Points in an Archery Competition.....	573
RECURSION & Backtracking PROBLEMS	578
Problem-1.39. Combination Sum(BACKTRACKING)	595
Problem-2.40. Combination Sum II(BACKTRACKING).....	597
Problem-3: 1723. Find Minimum Time to Finish All Jobs (Backtracking).....	599
Problem-4: 698. Partition to K Equal Sum Subsets(BACKTRACKING, BUCKET, BITMASK).....	601
Problem-5: 93. Restore IP Addresses(Reduce by Index vs Unprocessed String)	606
Problem-6: 1593. Split a String Into the Max Number of Unique Substrings(BACKTRACKING).....	610
Problem-7: 2056. Number of Valid Move Combinations On Chessboard	614

Problem-8: 967. Numbers With Same Consecutive Differences(Backtrack primitive Object)	620
Problem-9: 691. Stickers to Spell Word(Recursion+MAP as Memoization key) ..	623
Problem-10: 1476. Probability of a Two Boxes Having The Same Number of Distinct Balls	627
Problem-11-I. 403. Frog Jump	630
Problem-12: 124. Binary Tree Maximum Path Sum(Pass Global Shared Object in RECURSION)	634
Problem-13: 1774. Closest Dessert Cost(RECURSION + pass Global Shared Object in RECURSION)	636
Problem-14: 2151. Maximum Good People Based on Statements(BITMASK)	638
Problem-15: 1307. Verbal Arithmetic Puzzle (Backtracking)	648

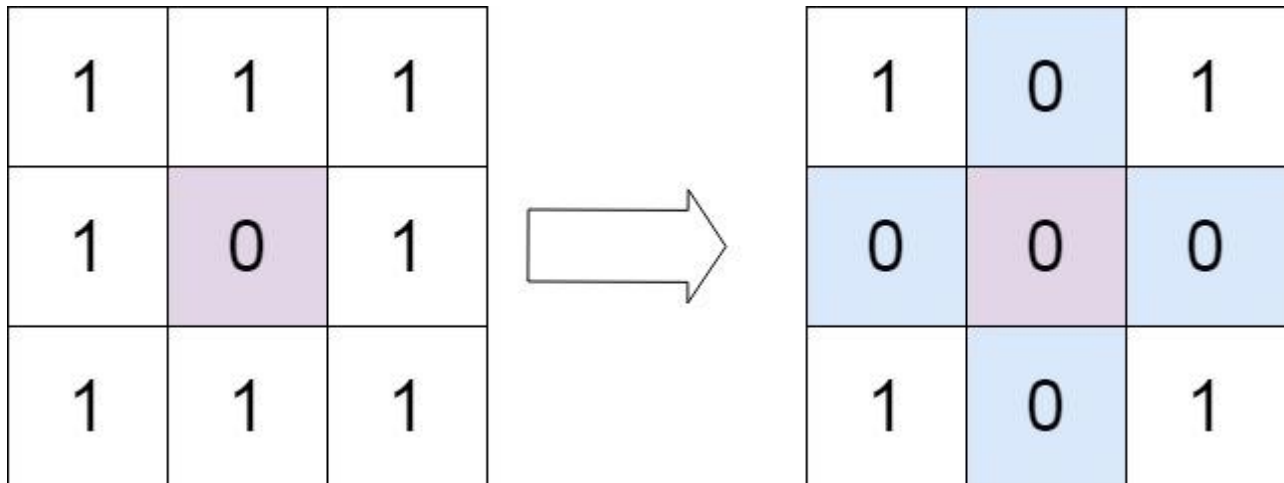
DAY 1:ARRAYS-I

Problem 1: Set Matrix Zeros

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it [in place](#).

Example 1:



Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

Solution:

```
class Solution {
// Runtime: 1 ms, faster than 96.37% of Java online submissions for Set
// Matrix Zeroes.
```

// Memory Usage: 54.4 MB, less than 28.80% of Java online submissions for Set Matrix Zeroes.

```
public void setZeroes(int[][] matrix) {
    // maintain column which says 0 or 1.
    int col0 = 1;
    int r = matrix.length;
    int c = matrix[0].length;

    // x/y 0 1 2
    // 1
    // 2

    // if we take first row as a row matrix to store whether zero
exists
    // and first col matrix as zero then matrix[0][0] has both i,j
has
    // common to row vs matrix so we need to track firstRow,firstCol
i.e index (0,0)
    // separately

    for (int i = 0; i < r; i++) {
        if (matrix[i][0] == 0)
            col0 = 0;
        for (int j = 1; j < c; j++) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }

    // Also if we do row first we might end up changing based
previously changed row
    // so first change all records columns for which e.g
matrix[i][j]==0
    // matrix[1][j..n] is zero

    // traverse reverse order
    // since otherwise if Row is 0 or col is zero entire matrix
becomes 0.
    for (int i = r - 1; i >= 0; i--) {
        for (int j = c - 1; j >= 1; j--) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
}
```

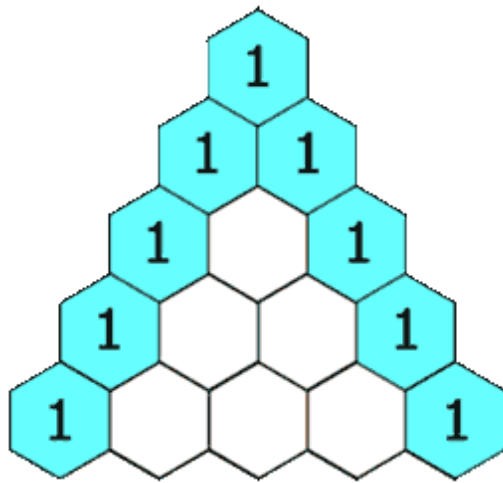
```
        if (col0 == 0)
            matrix[i][0] = 0;
    }

}
```

Problem 2: Pascal's Triangle

Given an integer numRows, return the first numRows of **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:



Solution:

```
import java.util.ArrayList;
import java.util.List;

class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < numRows; i++) {
            ArrayList<Integer> cur = new ArrayList<>();
            for (int j = 0; j <= i; j++) {
                if (j == 0 || j == i) {
                    cur.add(1);
                    continue;
                }
            }
            List<Integer> prev = result.get(i - 1);
```

```
        int temp = prev.get(j - 1) + prev.get(j);
        cur.add(temp);
    }

    result.add(cur);
}
return result;
}
}
```

Problem 3: Next Permutation

A **permutation** of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2, 1, 3], [2, 3, 1], [3,1,2], [3,2,1].

The **next permutation** of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the **next permutation** of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of arr = [1,2,3] is [1,3,2].
- Similarly, the next permutation of arr = [2,3,1] is [3,1,2].
- While the next permutation of arr = [3,2,1] is [1,2,3] because [3,2,1] does not have a lexicographical larger rearrangement.

Given an array of integers nums, *find the next permutation of nums*.

The replacement must be in place and use only constant extra memory.

Example 1:

Input: nums = [1,2,3]

Output: [1,3,2]

Example 2:

Input: nums = [3,2,1]

Output: [1,2,3]

Example 3:

Input: nums = [1,1,5]

Output: [1,5,1]

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 100$

Solution:

```
class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int start = getSmallestValueFromLast(nums);
        int end = -1;
        if (start != -1) {
            end = getNextLargestValFromStart(nums, start);

            if (end != -1) {
                swap(nums, start, end);
            }
        }

        reverse(nums, start + 1, n - 1);
    }

    private void reverse(int nums[], int start, int end) {
        while (start < end) {
            swap(nums, start++, end--);
        }
    }

    private void swap(int nums[], int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    private int getNextLargestValFromStart(int nums[], int start) {
        int end = nums.length - 1;

        while (start < end && nums[start] >= nums[end]) {
```



```
        end--;
    }

    return (nums[start] != nums[end]) ? end : -1;
}

private int getSmallestValueFromLast(int nums[]) {
    int n = nums.length;
    for (int i = n - 2; i >= 0; i--) {
        if (nums[i] < nums[i + 1]) {
            return i;
        }
    }

    return -1;
}
}
```

Problem 4:Maximum SubArray:

Given an integer array *nums*, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: *nums* = [-2,1,-3,4,-1,2,1,-5,4]

Output: 6

Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Solution:

```
class Solution {
    public int maxSubArray(int[] nums) {
        int max = Integer.MIN_VALUE;
        int curMax = 0;
        for (int num : nums) {
            curMax += num;
            curMax = Math.max(num, curMax);
            max = Math.max(max, curMax);
        }
        return max;
    }
}
```

Problem 5: Sort Colors

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Solution:

```
class Solution {
    public void sortColors(int[] nums) {
        int n = nums.length;
        int arr[] = new int [3];
        for(int val : nums){
            arr[val]++;
        }
        int index = 0;
        for(int i =0;i<3;i++){
            int count= arr[i];
            while(count-->0){
                nums[index++]=i;
            }
        }
    }
}
```

Problem 6: 121. Best Time to Buy and Sell Stock

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Solution:

```
class Solution {
    public int maxProfit(int[] prices) {
        int n = prices.length;
        int buy = 0;
        int sell = 0;
        int profitSoFar = 0;
        int maxProfit = 0;
        for (int i = 1; i < n; i++) {
            buy = -prices[i - 1];
            sell = prices[i];
            profitSoFar += (buy + sell);
            if (profitSoFar < 0) {
                profitSoFar = 0;
            }
            maxProfit = Math.max(maxProfit, profitSoFar);
        }
        return maxProfit;
    }
}
```

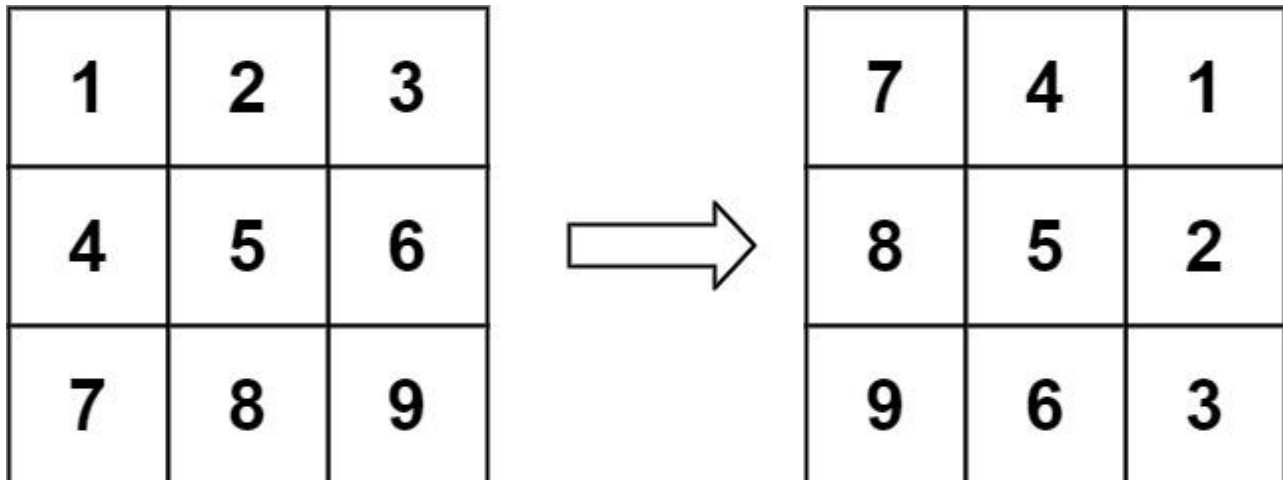
DAY 2:ARRAYS-II

Problem 7: Rotate Matrix

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

**Solution:**

```
import java.util.HashMap;
```

```
class Solution {
    public void rotate(int[][] matrix) {
        int r = matrix.length;
        int c = matrix[0].length;
        HashMap<String, Integer> map = new HashMap<>();
        int x = r - 1;
        int y = 0;
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                map.put(i + "_" + j, matrix[i][j]);
                if (map.get(x + "_" + y) == null) {
                    matrix[i][j] = matrix[x][y];
                } else {
                    matrix[i][j] = map.get(x + "_" + y);
                }
                x--;
            }
            x = r - 1;
            y++;
        }
    }
}
```

```
}
```

Problem 8: 56. Merge Intervals:

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`

Output: `[[1,5]]`

Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Solution:

```
import java.util.*;
import java.io.*;
/*****
```

Following is the Interval class structure

```
class Interval {
    int start, int finish;

    Interval(int start, int finish) {
        this.start = start;
        this.finish = finish;
    }
}
```

```
*****/
```

```
import java.util.List;
import java.util.ArrayList;

public class Solution {
    public static List<Interval> mergeIntervals(Interval[][] intervals) {
```

```

Arrays.sort(intervals, (a,b)-> a.start-b.start);

//[1, 4], [3, 5], [6, 8], [10, 12], [8, 9].
//Step 1:sorted
//[1,4] ,[3,5], [6,8],[8,9],[10,12]//sorted
//Step 2: merge till end
//[1,5] ->merge
//[1,5] [6,8] ->add

//[1,5],[6,8],[8,9] -> [6,9]->merge
//[1,5],[6,9],[8,9] ->merge
//[1,5],[6,9] ,[10,12]
//[1,5],[6,9],[10,12]->add

List<Interval> result = new ArrayList<>();

Interval prev = null,cur = null;
for(Interval interval: intervals){
    cur = interval;
    if(prev == null){
        prev =cur;
        continue;
    }
    if(prev.finish< cur.start) {
        //[1,5] [6,9]
        result.add(prev);
        prev = cur;
        //add prev and set cur to prev
    }else{
        //[1,2],[5,3]=>[]
        //[6,10],[8,9]=>[6,10]
        //[6,7],[8,9]=> [6,9]
        prev.finish = Math.max(prev.finish, cur.finish);
        //merge prev
    }
}
result.add(prev);

return result;
}
}

```

Problem 9: 88. Merge Sorted Array:

You are given two integer arrays nums1 and nums2, sorted in **non-decreasing order**, and two integers m and n, representing the number of elements in nums1 and nums2 respectively.

Merge nums1 and nums2 into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* nums1. To accommodate this, nums1 has a length of $m + n$, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n .

Example 1:

Input: nums1 = [1,2,3,0,0,0], $m = 3$, nums2 = [2,5,6], $n = 3$

Output: [1,2,2,3,5,6]

Explanation: The arrays we are merging are [1,2,3] and [2,5,6].

The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.

Solution:

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (i >= 0 || j >= 0) {
            if (j < 0 || (i >= 0 && nums1[i] >= nums2[j])) {
                nums1[k--] = nums1[i--];
            } else {
                nums1[k--] = nums2[j--];
            }
        }
    }
}
```

Problem 10 :287. Find the Duplicate Number

Given an array of integers nums containing $n + 1$ integers where each integer is in the range [1, n] inclusive.

There is only **one repeated number** in nums, return *this repeated number*.

You must solve the problem **without** modifying the array nums and uses only constant extra space.

Example 1:**Input:** nums = [1,3,4,2,2]**Output:** 2**Solution:**

```
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        fast = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

Problem 11: Missing and repeating numbers:**Solution:**

```
import java.util.ArrayList;
import java.util.*;

public class Solution {

    public static int[] missingAndRepeating(ArrayList<Integer> arr, int n)
    {

        // 4 5 2 9 8 1 1 7 10 3
        // 1 1 2 3 4 5 7 8 9 10
        // 1 2 4 7 11 16 23 31 40 50(sum)
        // dup ->1
    }
}
```

```
// 1 2 3 => 3*2/2 =6/2 =3 ->n*n-1/2

// 4 9 11 20 28 29 30 37 47 50
// n*(n-1)/2 =>10*9 = 90/2= 45(total)
// sum-(total-dup) = 50 - (45-1)=50-44 = 6

int res[] = new int[2];
Set<Integer> st = new HashSet<>();
int dup = 0;
int sum = 0;
int total = (n * (n + 1)) / 2;
// 4
// 30/2 = 15-(10-4)= 15-6
for (int i = 0; i < n; i++) {
    int cur = arr.get(i);
    sum += cur;
    if (st.contains(cur)) {
        dup = cur;
    }
    st.add(cur);
}

int missing = Math.abs(total - (sum - dup));
res[0] = missing;
res[1] = dup;

return res;
}
```

Problem 12: Count Inversions

Sample Input 1 :

3
3 2 1

Sample Output 1 :

3

Explanation of Sample Output 1:

We have a total of 3 pairs which satisfy the condition of inversion. (3, 2), (2, 1) and (3, 1).

Sample Input 2 :

5
2 5 1 3 4
Sample Output 2 :

4
Explanation of Sample Output 1:

We have a total of 4 pairs which satisfy the condition of inversion. (2, 1), (5, 1), (5, 3) and (5, 4).

Solution:

```
import java.util.*;
import java.io.*;

public class Solution {

    // [2,1,5,4,3]
    // 0 1 2 3 4
    // |
    // / \
    // / \
    // [2,1,5] [4,3]
    // / \ / \
    // [2,1] [5] [4] [3] => [3,4], [5]
    // | \
    // [2] [1] => sort [1,2]
    // [1,2,3,4,5]
    public static long getInversions(long arr[], int n) {
        long[] temp = new long[n];
        return mergeSort(arr, temp, 0, n - 1);
    }

    private static long mergeSort(long arr[], long temp[], int low, int
high) {
        long inversion_count = 0;
        long result = 0;
        int n = arr.length;
        // if(low<0 || high>=n) return 0;

        if (low < high) {
            int mid = low + (high - low) / 2;
            inversion_count += mergeSort(arr, temp, low, mid);
```

```
        inversion_count += mergeSort(arr, temp, mid + 1,
high);

        inversion_count += merge(arr, temp, low, mid + 1, high);
    }
    return inversion_count;
}

private static long merge(long arr[], long temp[], int low, int mid,
int high) {
    int i = low;
    int j = mid;

    int count = low;
    long inversion_count = 0;
    while (i <= mid - 1 && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[count++] = arr[i++];
        } else {
            temp[count++] = arr[j++];
            inversion_count = inversion_count + (mid - i);
        }
    }

    while (i <= mid - 1) {
        temp[count++] = arr[i++];
    }
    while (j <= high) {
        temp[count++] = arr[j++];
    }

    for (int x = low; x <= high; x++) {
        arr[x] = temp[x];
    }
    return inversion_count;
}
}
```

DAY 3: Arrays Part-III

Problem 13: 74. Search a 2D Matrix

Medium

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`

Output: `true`

Solution:

```
public class Solution {  
    static boolean findTargetInMatrix(ArrayList<ArrayList<Integer>> mat,  
    int m, int n, int target) {
```

```
//      3(r) 4(c) 8(target)
//      col1 col2 col3 col4
//      1   2   3   4   - row1
//      (0) (1) (2) (3)
//      5   6   7   8   - row2
//      (4) (5) (6) (7)
//      9  10  11  12   - row3
//      (8) (9) (10) (11)

// 5- 5/3 =1, 5%4 =1

return helper(mat, target);
}

private static boolean helper(ArrayList<ArrayList<Integer>> mat, int
target) {
    int r = mat.size();
    int c = mat.get(0).size();
    int low = 0;
    int high = (r * c - 1);

    // 11 -> 11/
    while (low <= high) {
        int mid = low + (high - low) / 2;

        int i = mid / c;
        int j = mid % c;
        if (mat.get(i).get(j) == target) {
            return true;
        }
        if (mat.get(i).get(j) < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return false;
}
}
```

Problem 14: 50. Pow(x, n):

Implement [`pow\(x, n\)`](#), which calculates x raised to the power n (i.e., x^n).

Example 1:**Input:** x = 2.00000, n = 10**Output:** 1024.00000**Solution:**

```
public class Solution {
    public static double myPow(double x, int n) {
        double ans = 1.00000000;
        // Declaring boolean variable, that will be used
        // to check whether 'n' is neg or positive
        boolean flag = false;
        if (n < 0) {
            flag = true;
            n = -n;
        }
        // Binary Exponentiation
        while (n > 0) {
            // If 'n' is divisible by '2'
            if (n % 2 == 0) {
                n /= 2;
            } else {
                // 'n' is not divisible by 2
                ans = ans * x;
                // Dividing 'n' by 2
                n /= 2;
            }
            // Multiplying 'x' by itself
            x = x * x;
        }
        // If 'flag' is true, that means, n was negative
        if (flag) {
            ans = 1.0 / ans;
        }
        return ans;
    }
}
```

Problem 15: 169. Majority Element

Easy

Given an array nums of size n, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Solution:

```
import java.util.*;
import java.io.*;

public class Solution {
    public static int findMajority(int[] arr, int n) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int val : arr) {
            map.put(val, map.getOrDefault(val, 0) + 1);
            if (map.get(val) > n / 2)
                return val;
        }
        return -1;
    }
}
```

Problem 16: 229. Majority Element II

Medium

Given an integer array of size n , find all elements that appear more than $\lfloor n/3 \rfloor$ times.

Example 1:

Input: nums = [3,2,3]

Output: [3]

Example 2:

Input: nums = [1]

Output: [1]

Example 3:

Input: nums = [1,2]

Output: [1,2]

Solution:

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;

public class Solution {
    public static ArrayList<Integer> majorityElementII(ArrayList<Integer>
arr) {

        Set<Integer> result = new HashSet<>();

        HashMap<Integer, Integer> map = new HashMap<>();
        int n = arr.size();
        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
            if (map.get(num) > n / 3) {
                result.add(num);
            }
        }

        return new ArrayList<Integer>(result);
    }
}
```

Problem 17: 62. Unique Paths

here is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Solution:

```
class Solution {
    public int uniquePaths(int m, int n) {

        int cache[][] = new int[m][n];

        return helper(m, n, 0, 0, cache);
    }

    private int helper(int m, int n, int i, int j, int[][] cache) {
        if (i < 0 || j < 0 || i >= m || j >= n)
            return 0;
        if (cache[i][j] != 0)
            return cache[i][j];
        if (i == m - 1 || j == n - 1) {
            return 1;
        }

        int right = 0, down = 0;
        right += helper(m, n, i + 1, j, cache);
        down += helper(m, n, i, j + 1, cache);
        cache[i][j] = right + down;
        return right + down;
    }
}
```

Problem 18: 493. Reverse Pairs

Hard

Given an integer array `nums`, return *the number of reverse pairs in the array*.

A reverse pair is a pair (i, j) where:

- $0 \leq i < j < \text{nums.length}$ and
- $\text{nums}[i] > 2 * \text{nums}[j]$.

Example 1:

Input: `nums = [1,3,2,3,1]`

Output: 2

Explanation: The reverse pairs are:

(1, 4) --> `nums[1] = 3, nums[4] = 1, 3 > 2 * 1`

(3, 4) --> `nums[3] = 3, nums[4] = 1, 3 > 2 * 1`

Solution:

```
import java.util.* ;
import java.io.*;
import java.util.ArrayList;

public class Solution
{
    public static int reversePairs(ArrayList<Integer> arr)
    {
        int pairs=0;
        for (int i = 0; i < arr.size(); i++) {
            for (int j = i + 1; j < arr.size(); j++) {
                if (arr.get(i) > 2 * arr.get(j)) pairs++;
            }
        }
        return pairs;
    }
}
```

Day 4: Arrays Part-IV

Problem 19: 1. Two Sum**1. Two Sum**

Easy

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have ***exactly one solution***, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:**Input:** `nums = [2,7,11,15]`, `target = 9`**Output:** `[0,1]`**Explanation:** Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.**Example 2:****Input:** `nums = [3,2,4]`, `target = 6`**Output:** `[1,2]`**Example 3:****Input:** `nums = [3,3]`, `target = 6`**Output:** `[0,1]`**Solution:**

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        // x+y = target;
        // target-num
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            int cur = nums[i];
            if (map.get(target - cur) != null) {
                int x = map.get(target - cur);
                // num+target-num = target
                return new int[] { x, i };
            }
            map.put(cur, i);
        }
        return new int[] { -1, -1 };
    }
}
```

Problem 20: 18. 4Sum

Given an array `nums` of `n` integers, return *an array of all the unique quadruplets* `[nums[a], nums[b], nums[c], nums[d]]` such that:

- $0 \leq a, b, c, d < n$
- `a`, `b`, `c`, and `d` are **distinct**.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in **any order**.

Example 1:

Input: `nums = [1,0,-1,0,-2,2]`, `target = 0`

Output: `[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]`

Example 2:

Input: `nums = [2,2,2,2,2]`, `target = 8`

Output: `[[2,2,2,2]]`

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$

Solution:

```
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        int n = nums.length;
        List<List<Integer>> result = new ArrayList<>();
        if (nums == null || nums.length == 0)
            return result;
        Arrays.sort(nums);
        for (int i = 0; i < n - 3; i++) {
            if (i != 0 && nums[i] == nums[i - 1])
                continue;
```



```
for (int j = i + 1; j < n - 2; j++) {
    if (j != i + 1 && nums[j] == nums[j - 1])
        continue;
    int low = j + 1;
    int high = n - 1;
    while (low < high) {
        List<Integer> list = new ArrayList<>();
        int a = nums[i];
        int b = nums[j];
        int c = nums[low];
        int d = nums[high];
        double sum = 0;
        // to handle integer overflow.
        //
        [1000000000,1000000000,1000000000,1000000000]
        // -294967296
        if (sum + a < Double.MAX_VALUE) {
            sum += a;
        }
        if (sum + b < Double.MAX_VALUE) {
            sum += b;
        }
        if (sum + c < Double.MAX_VALUE) {
            sum += c;
        }
        if (sum + d < Double.MAX_VALUE) {
            sum += d;
        }
        if (sum == target) {
            list.add(a);
            list.add(b);
            list.add(c);
            list.add(d);
            result.add(list);
            low++;
            high--;

            while (low < high
                && nums[high + 1] == nums[high]) {
                high--;
            }
            while (low < high
                && nums[low - 1] == nums[low]) {
                low++;
            }
        } else if (sum < target) {
            low++;
        } else {
            high--;
        }
    }
}
```

```
        high--;  
    }  
    }  
    }  
    }  
    return result;  
}
```

Problem 21: 128. Longest Consecutive Sequence

Given an unsorted array of integers *nums*, return *the length of the longest consecutive elements sequence*.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: *nums* = [100,4,200,1,3,2]

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: *nums* = [0,3,7,2,5,8,4,6,0,1]

Output: 9

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Solution:

```
import java.util.*;  
import java.io.*;  
  
public class Solution {  
    public static int lengthOfLongestConsecutiveSequence(int[] arr, int n)  
    {  
        // Write your code here.  
  
        // [9,5,4,9,10,10,6]  
        // step 1: check no-1 lesser exist  
        // : if exist move to next index.
```

```
//
// : do while exist check no+1 exist
// : update current count:
// : update max

// if we check both ways +1,-1 it will be n*n because
// for each value need to iterate prev of it and next of it.

int cur = 1;
int max = 1;
HashSet<Integer> st = new HashSet<>();

for (int val : arr) {
    st.add(val);
}

int i = 0;
while (i < n) {
    cur = arr[i];
    int prev = arr[i] - 1;
    if (st.contains(prev)) {
        i++;
        continue;
    }

    int next = cur + 1;
    int curMax = 1;
    while (st.contains(next)) {
        curMax++;
        next++;
    }
    max = Math.max(max, curMax);
    i++;
}

return max;
}
```

Problem 22: Longest Subarray Zero Sum

Sample Input 1:

```
2
5
1 3 -1 4 -4
4
```

1 -1 2 -2

Sample Output 1:

2

4

Explanation for Sample Output 1:

In the first test case, the given array is (1, 3, -1, 4, -4). The sub-arrays we can create are (1), (3), (-1), (4), (-4), (1, 3), (3, -1), (-1, 4), (4, -4), (1, 3, -1), (3, -1, 4), (-1, 4, -4), (1, 3, -1, 4), (3, -1, 4, -4), (1, 3, -1, 4, -4). Out of them only (4, -4) is the sub array whose sum is zero. Length of this sub array is 2 and hence we return 2 as the final answer.

In the second test case, the given array is (1, -1, 2, -2). The sub-arrays we can create are (1), (-1), (2), (-2), (1, -1), (-1, 2), (2, -2), (1, -1, 2), (-1, 2, -2), (1, -1, 2, -2). Out of them sub arrays with zero sum are (1, -1), (2, -2), (1, -1, 2, -2). Out of them only (1, -1, 2, -2) has the longest length of 4. Hence we return 4 as the final answer.

Sample Input 2:

2

3

4 -5 1

4

1 2 1 -2

Sample Output 2 :

3

0

Solution:

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;

public class Solution {

    public static int LongestSubsetWithZeroSum(ArrayList<Integer> arr) {

        // 1 3 -1 4 -4
        // 1 4 -3 1 -3
        // 0 1 2 3 4
        // stores {val, index}
        // 1 (not exist add to map) {1,0}
        // 4 (not exist add to map) {4,1}
        // -3 (not exist add to Map) {-3,2}
        // -2 (not exist add to Map) {-2,3}
        // -3 (exist so max len = 4-map.get(-3)=> 4-2=2)
```

```
HashMap<Integer, Integer> map = new HashMap<>();

int sum = 0;
int max = 0;
int n = arr.size();

for (int i = 0; i < n; i++) {
    int cur = arr.get(i);
    sum += cur;
    if (sum == 0) {
        // total length = current index
        max = i + 1;
        continue;
    }
    if (map.get(sum) == null) {
        map.put(sum, i);
    } else {
        int prevIndex = map.get(sum);
        max = Math.max(i - prevIndex, max);
    }
}

return max;
}
```

Problem 23: Subarrays with XOR 'K'

Sample Input 1:

4 2

1 2 3 2

Sample Output 1 :

3

Explanation Of Sample Input 1:

Input: 'N' = 4 'B' = 2

'A' = [1, 2, 3, 2]

Output: 3

Explanation: Subarrays have bitwise xor equal to '2' are: [1, 2, 3, 2], [2], [2].

Sample Input 2:

4 3

1 2 3 3

Sample Output 2:

4

Sample Input 3:

5 6

1 3 3 3 5

Sample Output 3:

2

Solution:

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;

public class Solution {
    public static int subarraysXor(ArrayList<Integer> arr, int x) {

        // 1 2 3 4
        // 1 3 6 10 prefix sum

        // 8421
        // 0001 -(1)
        // 0010 -(2)
        // ----
        // 0011 -[3] temp
        // 0011 -(3)
        // 0000 -[0]
        // 0100 -(4)
        // 0100 -[4]

        // ----->
        // <--p1--><--p2->
        // <-----psum----->

        // p1^p2 = total_sum
        // p1 =total_sum^p2

        // 10 1 0 3 4 7 //target =11
        //
        HashMap<Integer, Integer> map = new HashMap<>();
        int prefixsum = 0;
        int count = 0;
        // e.g 2 + 4
```

```
// target is 4

// 10 1 0 3 4 7//target 11
// 8421

// -----
// input1 -10
// -----

// 8421
// 1010
// 0000
// 1010 //output:10 {10,1}

// 1011 //xor 11
// 0001 //output 1 not exist in map

// -----
// input1 -1
// -----

// 8421
// 1010 // previous prefix sum
// 0001 // current input=1
// 1011 // output: 11 // count ->1

// 1011 // xor 11
// 0000 //0

// output 10 {10,1}
// -----
// input1 -0
// -----
// 8421
// 1011 //previous prefix sum
// 0000 //current input=0
// 1011 //output : 11 //count ->2

// 1011 //xor 11
// 1011 //11
// output 11{11,1}

// -----
// input1 -3
// -----

// 8421
```

```
// 1011 //previous prefix sum
// 0011 //current input=3
// 1000 //output: 8 //count ->2

// 1011 //xor 11
// 0011 //output 3
// { {10,1},{11,1},{8,1}}
// -----
// input1 -4
// -----
// 8421
// 1000 //previous prefix sum
// 0100 //current input =4
// 1100 //output : 12 // count ->2

// 1011 //xor 11
// 0111 //output 7

// { {10,1},{11,1},{8,1},{12,1}}
// -----
// input1 -7
// -----
// 8421
// 1100 //previous prefix sum
// 0111 //current input =7
// 1011 // output : 11 //count ->3

// 1011 //xor with 11
// 1011 //output 11
// -----

int n = arr.size();

int prefixSum = 0;

for (int i = 0; i < n; i++) {
    int cur = arr.get(i);
    prefixSum = prefixSum ^ cur;
    // if current reaches x
    if (prefixSum == x) {
        count++;
    }
    // cur xor remaining
    if (map.get(prefixSum ^ x) != null) {
        count += map.get(prefixSum ^ x);
    }
    map.put(prefixSum, map.getOrDefault(prefixSum, 0) + 1);
}
```



```
        return count;
    }
}
```

Problem 24: 3. Longest Substring Without Repeating Characters

Given a string *s*, find the length of the **longest substring** without repeating characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- *s* consists of English letters, digits, symbols and spaces.

Solution:

```
class Solution {  
    // Runtime: 6 ms, faster than 70.51% of Java online submissions for Longest  
    Substring Without Repeating Characters.  
    // Memory Usage: 39.4 MB, less than 61.14% of Java online submissions for  
    Longest Substring Without Repeating Characters.
```

```
    public int lengthOfLongestSubstring(String s) {  
        int max = 0;  
        char arr[] = s.toCharArray();  
        HashSet<Character> st = new HashSet<>();  
        int n = s.length();  
        int j = 0;  
        int i = 0;  
  
        // 2 pointer  
        // "anviaj"  
        // |||| i (4 max)  
        // | duplicate  
        // | j(remove a then try)  
        // |||| i(now 5 max)  
  
        // step 1: have 2 pointer , 1pt increment when no duplicates(i),  
and  
        // 2 pointer inc when duplicate(j).  
        // step 2: if found duplicate found remove j pointer i.e remove  
first character  
        // step 3: then add again (i) pt character if no duplicates  
        // maxsofar(kadane)  
        // 012345  
  
        int left = 0, right = 0;  
  
        while (left <= right && right < n) {  
            char cur = arr[right];  
            if (!st.contains(cur)) {  
                st.add(cur); // if not contains add to st  
                right++;  
                continue;  
            }  
            // once found try to shrink the window.  
            if (st.contains(cur)) {
```

```

        // wont add to set so update max and remove that char
        max = Math.max(max, st.size());
        st.remove(arr[left]);

        left++;
    }
}
// if input is a only 1 character then it wont go to dup found so
max
//wont get updated.
max = Math.max(max, st.size());

return max;
}
}

```

Problem 25: 1004. Max Consecutive Ones III

Medium

Given a binary array `nums` and an integer `k`, return *the maximum number of consecutive 1's in the array if you can flip at most k 0's*.

Example 1:

Input: `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2`

Output: 6

Explanation: `[1,1,1,0,0,1,1,1,1,1]`

Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Example 2:

Input: `nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1]`, `k = 3`

Output: 10

Explanation: `[0,0,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]`

Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Solution:

```
import java.util.LinkedList;
```

```

class Solution {
    public int longestOnes(int[] A, int K) {
        LinkedList<Integer> queue = new LinkedList<>();
        // [1,1,1,0,0,0,1,1,1,1,0]
        // [1,1,1,0,0]
        // [ 1,1,0,0]
        // [ 1,0,0]
        // [ 0,0,0]
        // [ 0,0,1,1,1,1]
    }
}

```

```
// [ 0,1,1,1,1,0]
// 1 1 1 0 0
int n = A.length;
int zero = 0;
int i = 0, j = 0, max = 0;
for (j = 0; j < n; j++) {
    if (A[j] == 0) {
        zero++;
    }
    while (zero > K) {
        if (A[i] == 0)
            zero--;
        i++;
    }
    max = Math.max(max, (j - i + 1));
}

return max;
}
```

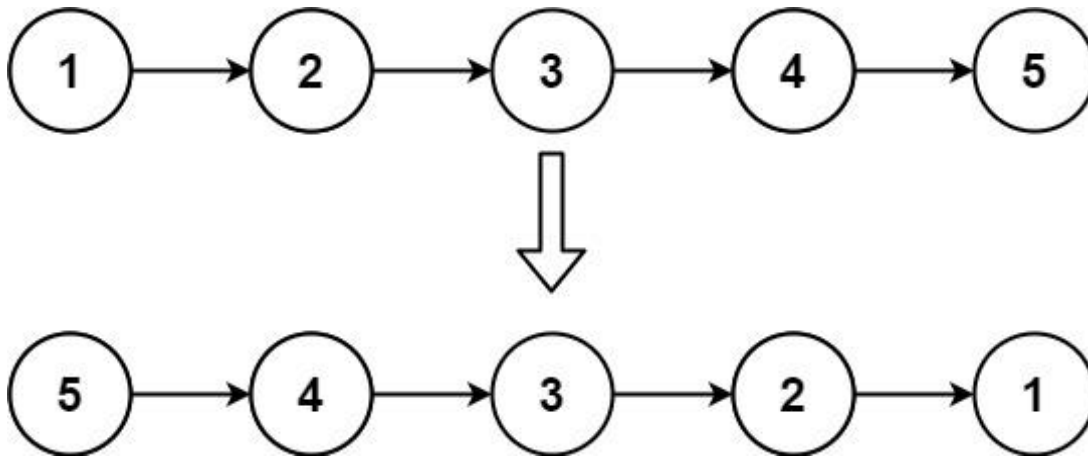
Day 5: Linked List

Problem 26: 1004. Max Consecutive Ones III

206. Reverse Linked List

Easy

Given the head of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:**Solution:**

```

public class Solution
{
    public static LinkedListNode<Integer>
reverseLinkedList(LinkedListNode<Integer> head)
    {
        LinkedListNode<Integer> pre = null;
        LinkedListNode<Integer> cur = head;

        while(cur!=null){
            LinkedListNode<Integer> next=cur.next;
            cur.next = pre;
            pre = cur;

            cur = next;
        }

        return pre;
    }
}

```

Problem 27: 876. Middle of the Linked List

876. Middle of the Linked List

Easy

Given the head of a singly linked list, return *the middle node of the Linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:



Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Solution:

```

import java.util.*;
import java.io.*;

public class Solution {
    public static Node findMiddle(Node head) {

        Node fast = head;
        Node slow = head;

        // EVEN
        // 1->2->3->4
        // s f
        // s f
        // s f

        // odd
        // 1 ->2->3
        // s f
        // s
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
  
```

Problem 28: 21. Merge Two Sorted Lists

21. Merge Two Sorted Lists

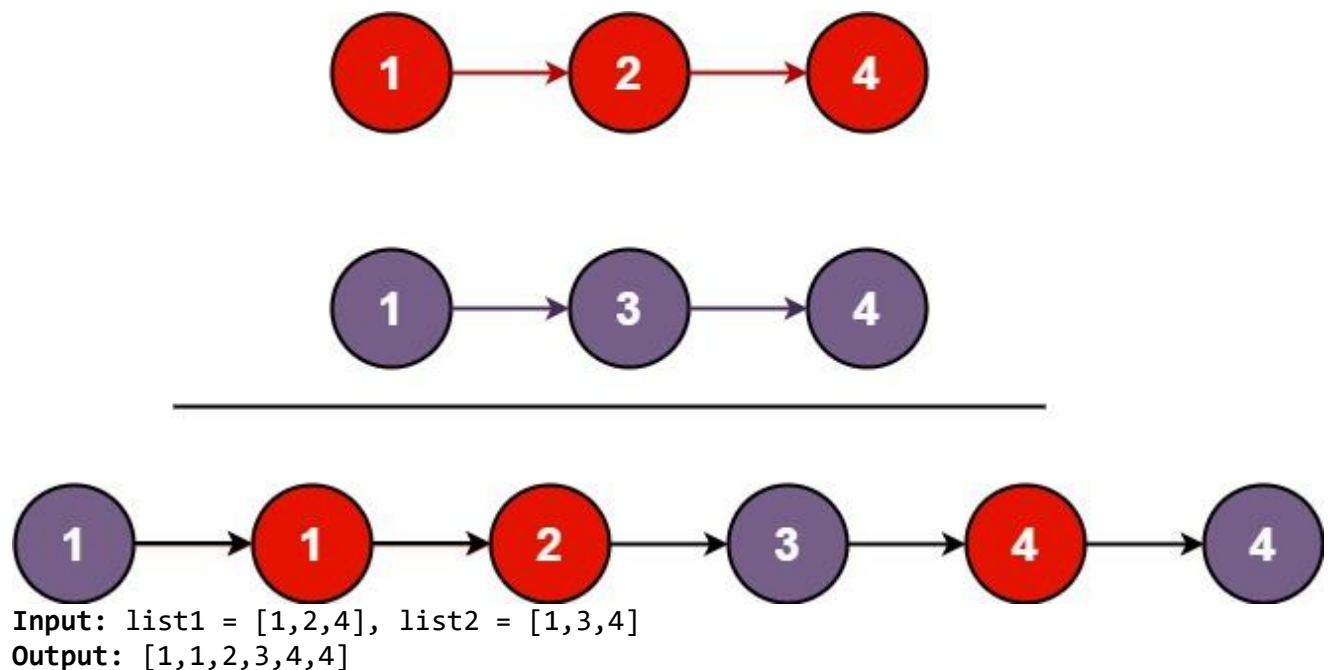
Easy

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists in a one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:



Example 2:

Input: `list1 = []`, `list2 = []`

Output: `[]`

Example 3:

Input: `list1 = []`, `list2 = [0]`

Output: `[0]`

Constraints:

- The number of nodes in both lists is in the range `[0, 50]`.

- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in **non-decreasing** order.

Solution:

```
/******
```

Following is the class structure of the Node class:

```
class LinkedListNode {  
    int data;  
    LinkedListNode next;  
  
    public LinkedListNode(int data) {  
        this.data = data;  
    }  
}
```

```
*****/
```

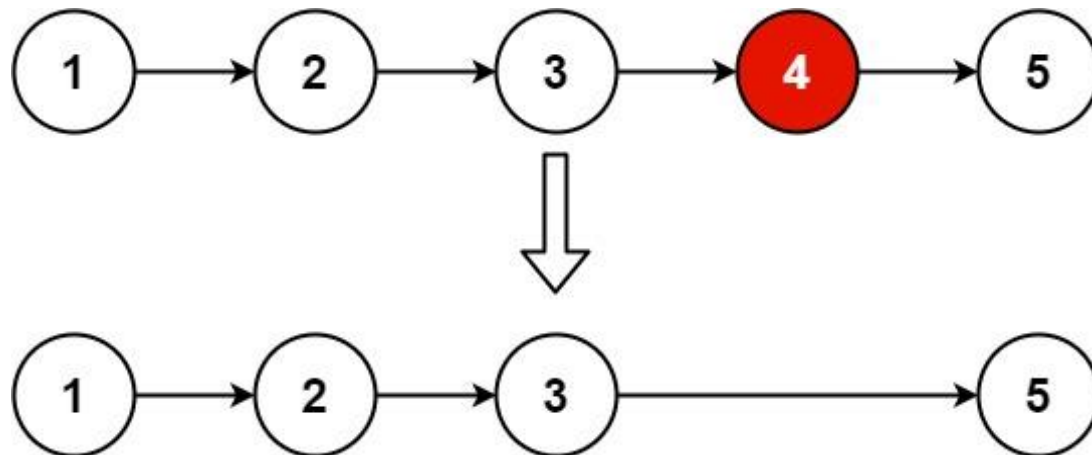
```
class Solution {  
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {  
        ListNode result = new ListNode(-1);  
        ListNode cur = result;  
        while (list1 != null && list2 != null) {  
            if (list1.val < list2.val) {  
                cur.next = list1;  
                list1 = list1.next;  
            } else {  
                cur.next = list2;  
                list2 = list2.next;  
            }  
            cur = cur.next;  
        }  
  
        ListNode rem = (list1 != null) ? list1 : list2;  
  
        while (rem != null) {  
            cur.next = rem;  
            cur = cur.next;  
            rem = rem.next;  
        }  
  
        return result.next;  
}
```

```
}  
}
```

Problem 29: 19. Remove Nth Node From End of List

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1

Output: []

Example 3:

Input: head = [1,2], n = 1

Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq \text{sz} \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq \text{sz}$

Solution:

```
import java.util.* ;
import java.io.*;
/*****
```

Following is the structure of the Singly Linked List.

```
class LinkedListNode<T> {
    T data;
    LinkedListNode<T> next;

    public LinkedListNode(T data) {
        this.data = data;
    }
}
```

```
*****/
```

```
public class Solution {
    public static LinkedListNode<Integer>
    removeKthNode(LinkedListNode<Integer> head, int K) {

        // 1 2 3 4 5 6 -1 //remove 6 th node.
        // 1 2 3 4 5 6
        // | |
        LinkedListNode<Integer> newHead = new LinkedListNode(-1);
        newHead.next = head;

        LinkedListNode slow = newHead;
        LinkedListNode fast = newHead;
        // move K points
        // 1 2 3 4 5 6
        // | | | | | |
        for (int i = 1; i <= K; i++) {
            fast = fast.next;
        }

        while (fast.next != null) {
            slow = slow.next;
            fast = fast.next;
        }
        if (slow.next != null)
            slow.next = slow.next.next;

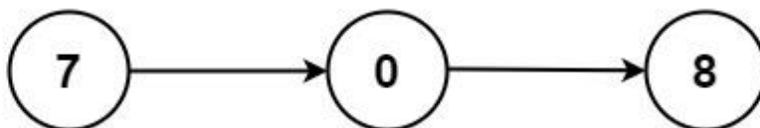
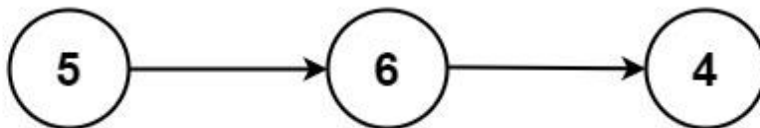
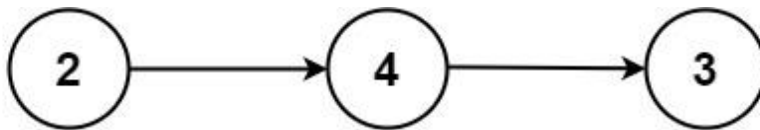
        return newHead.next;
    }
}
```

Problem 30: 2. Add Two Numbers

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$

- It is guaranteed that the list represents a number that does not have leading zeros.

Solution:

```
import java.util.* ;
import java.io.*;

public class Solution {
    static LinkedListNode addTwoNumbers(LinkedListNode head1,
    LinkedListNode head2) {
        // 5 6 3
        // 8 4 2
        // 14 0 5

        LinkedListNode res = null;
        LinkedListNode head = null;
        int sum = 0;
        int carry = 0;

        while (head1 != null || head2 != null) {
            int cur = 0;
            if (head1 != null) {
                cur += head1.data;
                head1 = head1.next;
            }
            if (head2 != null) {
                cur += head2.data;
                head2 = head2.next;
            }
            // 4+7 = 11/10
            int currentSum = carry + cur;
            sum = currentSum % 10;
            carry = currentSum / 10;

            if (res == null) {
                res = new LinkedListNode(sum);
                head = res;
            } else {
                res.next = new LinkedListNode(sum);
                res = res.next;
            }
        }

        if (carry > 0) {
            res.next = new LinkedListNode(carry);
            res = res.next;
        }
    }
}
```

```
        return head;
    }
}
```

Problem 31: 237. Delete Node in a Linked List

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will **not be given access** to the first node of head.

All the values of the linked list are **unique**, and it is guaranteed that the given node node is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before node should be in the same order.
- All the values after node should be in the same order.

Custom testing:

- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Input: head = [4,5,1,9], node = 5

Output: [4,1,9]

Explanation: You are given the second node with value 5, the linked list should become 4 -> 1 -> 9 after calling your function.

Solution:

```
import java.util.*;
import java.io.*;

public class Solution {
    public static void deleteNode(LinkedListNode<Integer> node) {
```

```
        // 1 ->2 ->3
        // p-> X ->next
        node.data = node.next.data;
        node.next = node.next.next;
    }
}
```

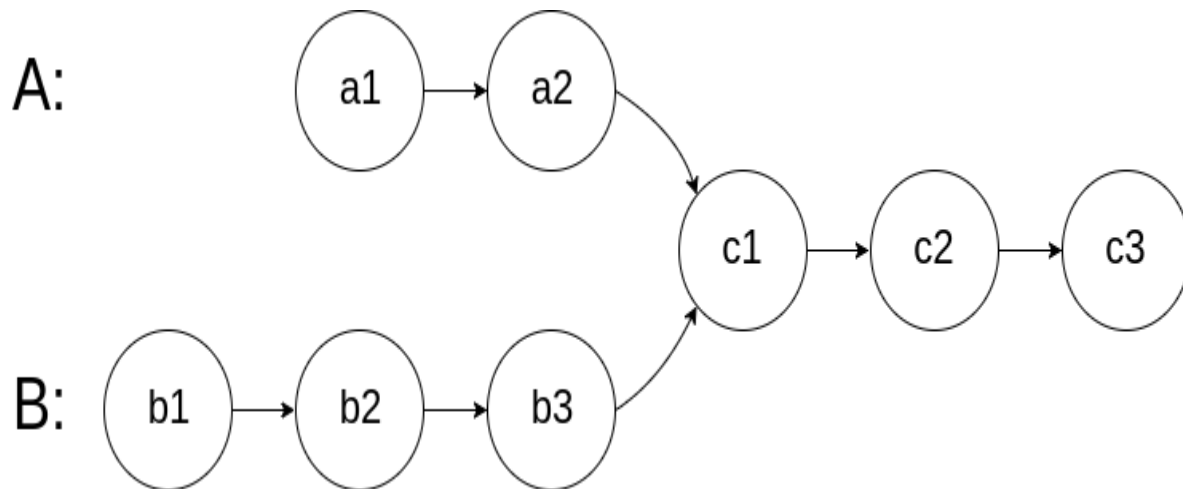
DAY 6: Linked List Part-II

Problem 32: 160. Intersection of Two Linked Lists**160. Intersection of Two Linked Lists**

Easy

Given the heads of two singly linked-lists `headA` and `headB`, return *the node at which the two lists intersect*. If the two linked lists have no intersection at all, return `null`.

For example, the following two linked lists begin to intersect at node `c1`:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must **retain their original structure** after the function returns.

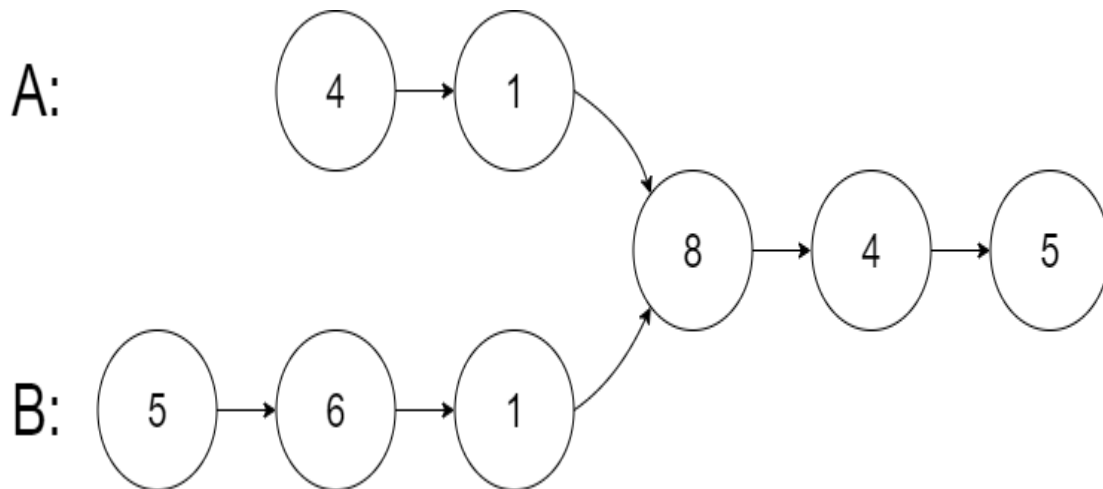
Custom Judge:

The inputs to the **judge** are given as follows (your program is **not** given these inputs):

- `intersectVal` - The value of the node where the intersection occurs. This is `0` if there is no intersected node.
- `listA` - The first linked list.
- `listB` - The second linked list.
- `skipA` - The number of nodes to skip ahead in `listA` (starting from the head) to get to the intersected node.
- `skipB` - The number of nodes to skip ahead in `listB` (starting from the head) to get to the intersected node.

The judge will then create the linked structure based on these inputs and pass the two heads, `headA` and `headB` to your program. If you correctly return the intersected node, then your solution will be **accepted**.

Example 1:



Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3

Output: Intersected at '8'

Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).

From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.

- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.

Solution:

```

import java.util.*;
import java.io.*;
public class Solution {

    public static int findIntersection(LinkedListNode<Integer> firstHead,
    LinkedListNode<Integer> secondHead) {

        LinkedListNode<Integer> a = firstHead;
        LinkedListNode<Integer> b = secondHead;

        while (a != b) {
            a = (a == null) ? secondHead : a.next;

```

```

        b = (b == null) ? firstHead : b.next;
    }
    if (a == null)
        return -1;
    return a.data;
}
}

```

Problem 33: 141. Linked List Cycle

141. Linked List Cycle

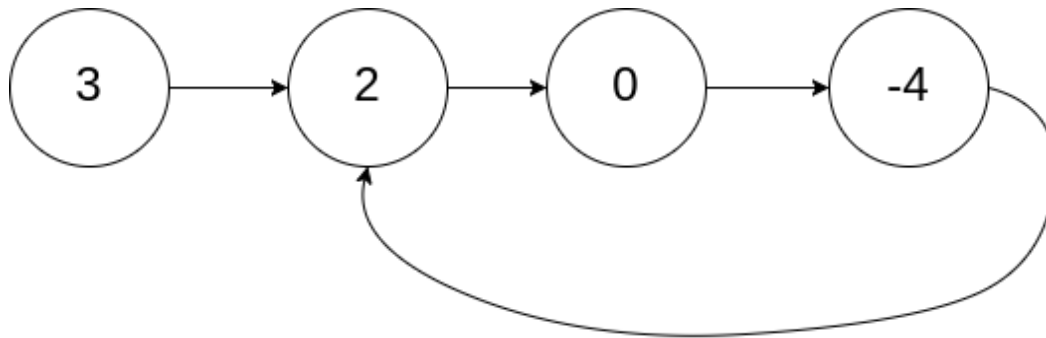
Easy

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. **Note that pos is not passed as a parameter.**

Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:



Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Solution:

```

public class Solution {

    public static boolean detectCycle(Node<Integer> head) {
        if (head == null)
            return false;
        if (head.next == null)
            return false;
        Node<Integer> slow = head;
    }
}

```

```

Node<Integer> fast = head.next;

while (fast != null && fast.next != null) {
    if (slow == fast)
        return true;
    slow = slow.next;
    fast = fast.next.next;
}

return false;
}
}

```

Problem 34: 25. Reverse Nodes in k-Group

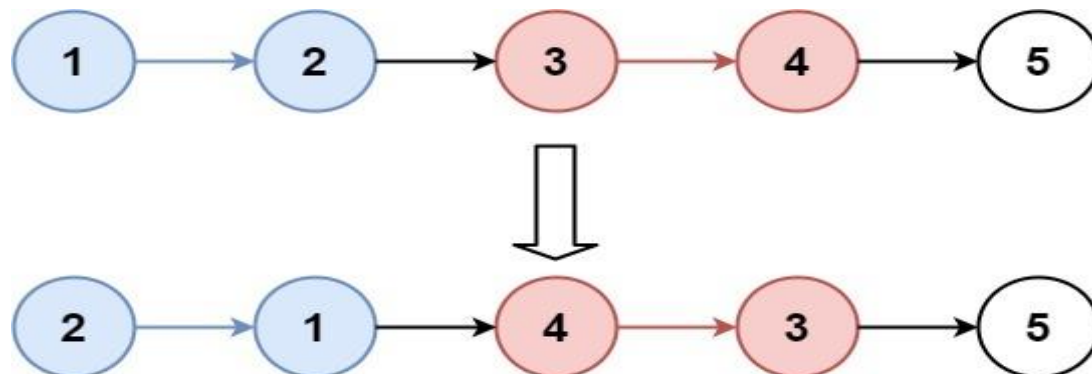
Hard

Given the head of a linked list, reverse the nodes of the list *k* at a time, and return *the modified list*.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

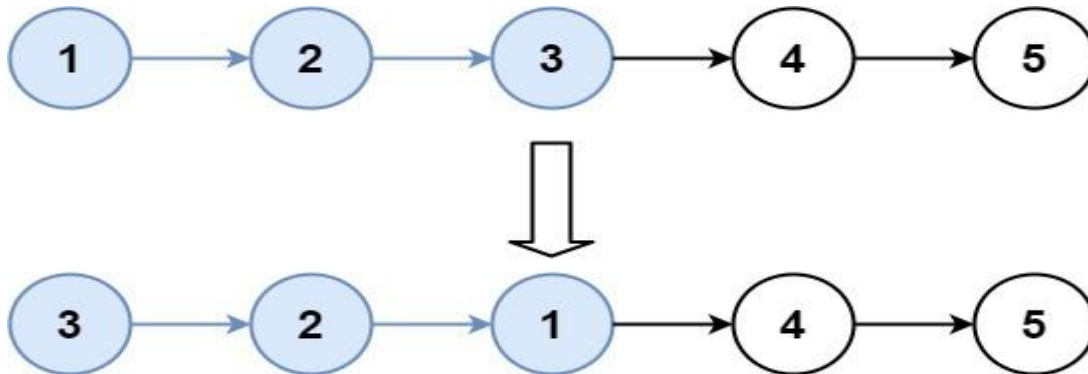
Example 1:



Input: head = [1,2,3,4,5], k = 2

Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3

Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n.
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$
-

Solution 1:

```

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {

        ListNode res = new ListNode(-1);
        ListNode resHead = res;

        ListNode slow = head;
        ListNode slowHead = slow;
        if (slow == null)
            return slow;
        if (slow.next == null) {
            return slow;
        }
        int count = 1;
        while (slow != null) {

```

```
        if (count == k) {
            ListNode next = slow.next;
            // split the node.
            slow.next = null;
            res.next = reverse(slowHead);

            while (res.next != null) {
                res = res.next;
            }
            // reset k
            count = 1;
            slow = next;
            slowHead = slow;
        }
        count++;
        if (slow != null) {
            slow = slow.next;
        }
    }

    res.next = slowHead;

    return resHead.next;
}

private ListNode reverse(ListNode head) {

    ListNode prev = null;
    ListNode cur = head;
    while (cur != null) {
        ListNode next = cur.next;
        cur.next = prev;
        prev = cur;
        cur = next;
    }

    return prev;
}
```

Solution 2(codeninjas):

```
import java.util.ArrayList;
import java.util.List;

public class Solution {
```

```

    public static Node getListAfterReverseOperation(Node head, int n, int
b[]) {

        // 1->2->3->4->5->6 {3,3}
        // 1->2->3
        // 3->2->1
        // ->4->5->6
        // 6->5->4
        // 3->2->1->6->5->4
        List<Integer> list = new ArrayList<>();
        for (int val : b) {
            if (val != 0) {
                list.add(val);
            }
        }
        Integer res[] = list.toArray(new Integer[0]);

        head = reverseKGroup(head, res, 0);
        print(head);
        return head;
    }

    public static Node reverseKGroup(Node head, Integer b[], int index) {
        Node cur = head;
        int count = 1;
        int n = b.length;
        if (index >= n)
            return head;
        while (cur != null && count < b[index]) { // find the k+1 node
            cur = cur.next;
            count++;
        }
        // no remaining
        if (cur == null)
            return reverse(head, null);

        // split in to 2
        // rem- remaining to traverse pass
        // prev - prev elements to reverse
        Node rem = cur.next;
        cur.next = null;
        Node prev = head;
        cur = reverseKGroup(rem, b, index + 1); // next batch size to
reverse
        prev = reverse(prev, cur);

        return prev;
    }

```

```
    }

    private static Node reverse(Node head, Node last) {
        Node prev = null;
        Node prevHead = null;

        // 1->2->3->4
        while (head != null) {
            Node next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }

        // set the head to reverse prev.
        prevHead = prev;

        // 1->2->3(prev) 4->5(last)
        // 3->2->1 (reversed prev)
        // traverse till end and last the remaining nodes
        while (prev != null && prev.next != null) {
            prev = prev.next;
        }
        // add remaining nodes.
        // 3->2->1->4->5
        if (prev != null)
            prev.next = last;

        return prevHead;
    }

}

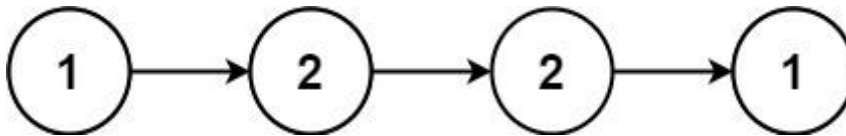
class Node {
    Node next;
    int data;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Problem 35: 234. Palindrome Linked List

Given the head of a singly linked list, return true *if it is a palindrome* or false *otherwise*.

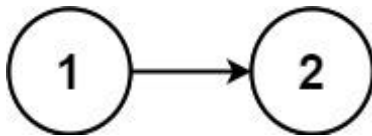
Example 1:



Input: head = [1,2,2,1]

Output: true

Example 2:



Input: head = [1,2]

Output: false

Solution:

```
class Solution {  
    public boolean isPalindrome(ListNode head) {  
        ListNode slow = head;  
        ListNode fast = head;  
        while (fast != null && fast.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
        }  
        if (fast != null && fast.next != null) {
```

```
        slow = slow.next;
    }
    ListNode rev = reverse(slow);
    ListNode first = head;

    while (first != null && rev != null) {
        if (rev.val != first.val)
            return false;
        first = first.next;
        rev = rev.next;
    }

    return true;
}

private ListNode reverse(ListNode head) {
    ListNode prev = null;
    ListNode cur = head;

    while (cur != null) {
        ListNode next = cur.next;
        cur.next = prev;
        prev = cur;
        cur = next;
    }

    return prev;
}
}
```

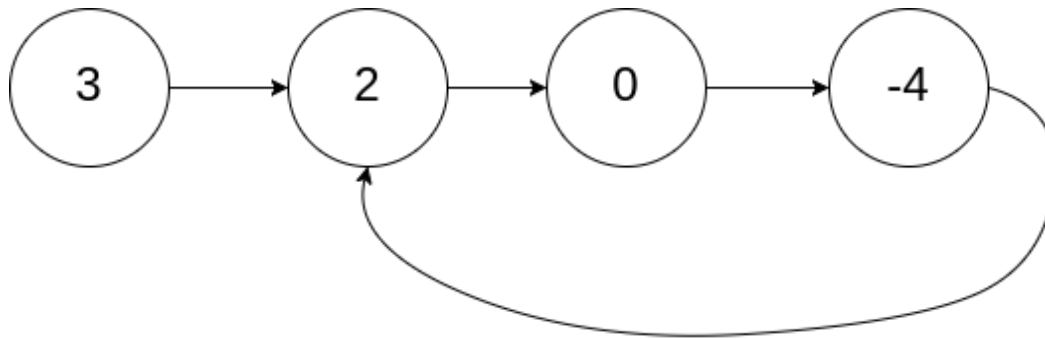
Problem 36: 142. Linked List Cycle II

Given the head of a linked list, return *the node where the cycle begins*. If there is no cycle, return null.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to (0-indexed). It is -1 if there is no cycle. **Note that pos is not passed as a parameter.**

Do not modify the linked list.

Example 1:



Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

Solution:

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast)
                break;
        }

        if (fast == null || (fast != null && fast.next == null)) {
```

```

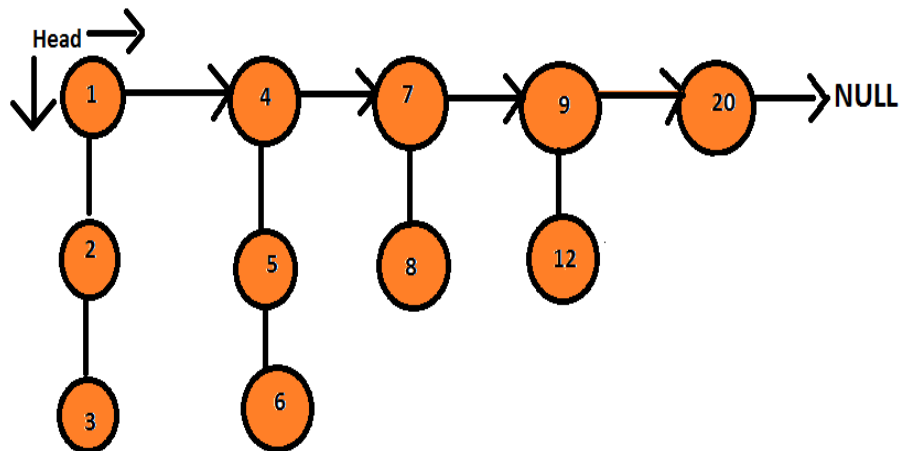
        return null;
    }

    fast = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }

    return fast;
}
}

```

Problem 37: Flatten A Linked List



Solution:

```

import java.util.*;
import java.io.*;

/*****
 *
 * Following is the class structure of the Node class:
 *
 * class Node { int data; Node next; Node child;

```

```
*
* public Node(int data) { this.data = data; this.next = null; this.child =
* null; } }
*
*****/
public class Solution {
    public static Node flattenLinkedList(Node start) {
        return flatten(start);
    }

    private static Node flatten(Node root) {
        if (root == null || (root.next == null))
            return root;

        root.next = flatten(root.next);
        root = merge(root, root.next);

        return root;
    }

    private static Node merge(Node a, Node b) {
        Node res = null;
        Node resHead = null;
        while (a != null && b != null) {
            if (a.data < b.data) {
                if (res == null) {
                    res = a;
                    resHead = res;
                    a = a.child;
                } else {
                    res.child = a;
                    res = res.child;
                    a = a.child;
                }
            } else {
                if (res == null) {
                    res = b;
                    resHead = res;
                    b = b.child;
                } else {
                    res.child = b;
                    res = res.child;
                    b = b.child;
                }
            }
        }
    }
}
```

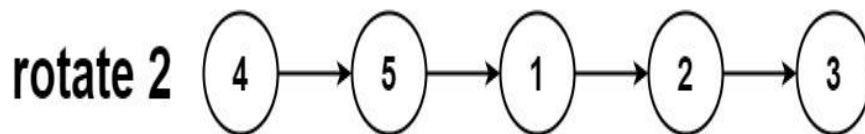
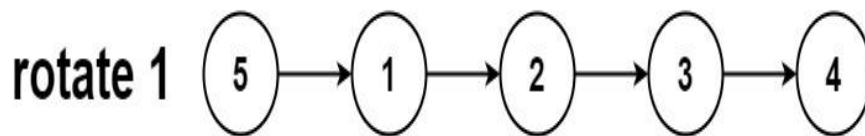
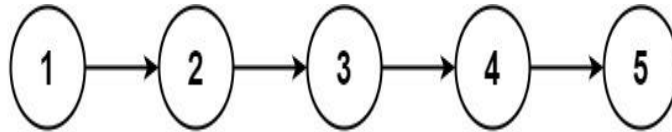
```
        }  
    }  
    if (a != null)  
        res.child = a;  
    else  
        res.child = b;  
    return resHead;  
}  
  
}
```

Day 7: Linked List and Arrays

Problem 38: 61. Rotate List

Given the head of a linked list, rotate the list to the right by k places.

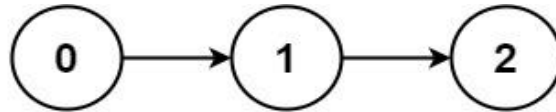
Example 1:



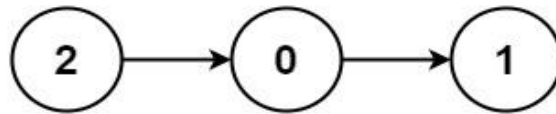
Input: head = [1,2,3,4,5], k = 2

Output: [4,5,1,2,3]

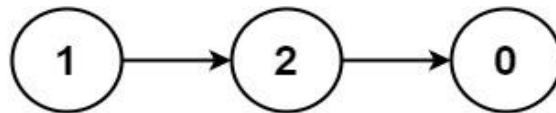
Example 2:



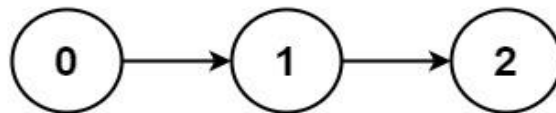
rotate 1



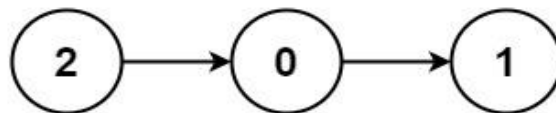
rotate 2



rotate 3



rotate 4



Input: head = [0,1,2], k = 4

Output: [2,0,1]

Solution:

```
/**
 * Definition for singly-linked list. public class ListNode { int val;
ListNode
 * next; ListNode() {} ListNode(int val) { this.val = val; } ListNode(int
val,
 * ListNode next) { this.val = val; this.next = next; } }
```

```
*/
class Solution {
    public ListNode rotateRight(ListNode head, int k) {

        ListNode fast = head;
        ListNode slow = head;

        if (fast == null || fast.next == null || k == 0) {
            return head;
        }

        int count = 1;
        while (fast.next != null) {
            fast = fast.next;
            count++;
        }

        int rem = k % count;
        for (int i = 1; i < count - rem; i++) {
            slow = slow.next;
        }
        ListNode newHead = null;
        if (slow.next != null) {
            newHead = slow.next;
            slow.next = null;
            fast.next = head;
        } else {
            newHead = head;
        }

        return newHead;
    }
}
```

Problem 39: 138. Copy List with Random Pointer

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or null.

Construct a [deep copy](#) of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its

corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list.**

For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, then for the corresponding two nodes x and y in the copied list, $x.random \rightarrow y$.

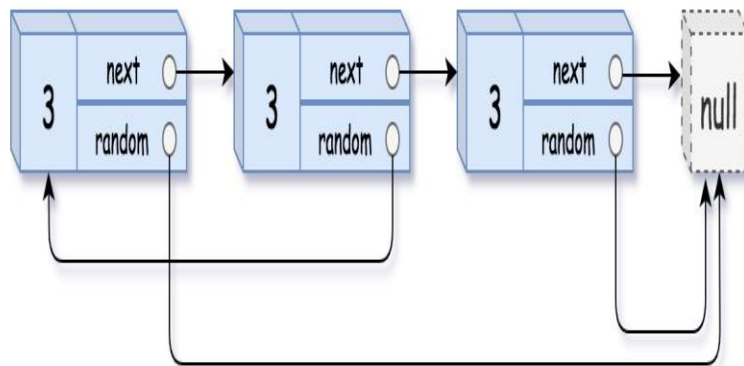
Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of [val, random_index] where:

- val: an integer representing Node.val
- random_index: the index of the node (range from 0 to n-1) that the random pointer points to, or null if it does not point to any node.

Your code will **only** be given the head of the original linked list.

Example 3:



Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]

Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

Solution:

```
/*
```

```
// Definition for a Node.
class Node {
    int val;
    Node next;
    Node random;

    public Node(int val) {
        this.val = val;
        this.next = null;
        this.random = null;
    }
}
*/

class Solution {
    public Node copyRandomList(Node head) {

        HashMap<Node, Node> map = new HashMap<>();
        return clone(head, map);
    }

    private Node clone(Node head, HashMap<Node, Node> map) {
        if (head == null)
            return head;

        if (map.get(head) != null) {
            return map.get(head);
        }

        Node cur = head;

        while (cur != null) {
            map.put(cur, new Node(cur.val));
            cur = cur.next;
        }
        cur = head;
        while (cur != null) {
            map.get(cur).next = map.get(cur.next);
            map.get(cur).random = map.get(cur.random);
            cur = cur.next;
        }

        return map.get(head);
    }
}
```

Problem 40: 15. 3Sum

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Solution:

```
class Solution {  
    public List<List<Integer>> threeSum(int[] nums) {  
        Arrays.sort(nums);  
  
        List<List<Integer>> result = new ArrayList<>();  
  
        return helper(nums);  
    }  
}
```

```
}

private List<List<Integer>> helper(int[] nums) {
    HashSet<List<Integer>> result = new HashSet<>();
    int n = nums.length;

    for (int i = 0; i < n; i++) {
        int low = i + 1;
        int high = n - 1;
        while (low < high && low < n) {
            // int c = nums[low];
            // [-1,0,1,2,-1,-4]
            // -4 -1 -1 0 1 2 4
            // -5 -1 == -4
            int a = nums[i];
            int b = nums[low];
            int c = nums[high];
            int sum = a + b + c;

            if (sum == 0) {
                List<Integer> list = new ArrayList<>();
                list.add(a);
                list.add(b);
                list.add(c);

                result.add(list);
            }
            if (sum < 0) {
                low++;
            } else {
                high--;
            }
        }
    }

    return new ArrayList<>(result);
}
```

Problem 41: 42. Trapping Rain Water**42. Trapping Rain Water****Hard**

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

Solution:

```
import java.util.*;
import java.io.*;

public class Solution {
    public static long getTrappedWater(long[] arr, int n) {
        long lmax = 0;
        long rmax = 0;
        int i = 0;
        int j = n - 1;
        long sum = 0;
        while (i < j) {
            lmax = Math.max(arr[i], lmax);
            rmax = Math.max(arr[j], rmax);
            if (lmax <= rmax) {
                sum += lmax - (arr[i++]);
            } else {
                sum += rmax - (arr[j--]);
            }
        }

        return sum;
    }
}
```

Problem 42: 26. Remove Duplicates from Sorted Array

26. Remove Duplicates from Sorted Array

Easy

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**. Then return *the number of unique elements in `nums`*.

Consider the number of unique elements of `nums` to be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the unique elements in the order they were present in `nums` initially. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:


```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}

```

If all assertions pass, then your solution will be **accepted**.

Solution:

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int n = nums.length;
        int j = 0;
        for (int i = 0; i < n; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }
            nums[j++] = nums[i];
        }
        return j;
    }
}
```

Problem 43: 485. Max Consecutive Ones

485. Max Consecutive Ones

Easy

Given a binary array *nums*, return *the maximum number of consecutive 1's in the array*.

Example 1:

Input: *nums* = [1,1,0,1,1,1]

Output: 3

Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

Solution:

```
class Solution {
```

```
public int findMaxConsecutiveOnes(int[] nums) {  
    int n = nums.length;  
  
    int max = 0;  
    int count = 0;  
    for (int i = 0; i < n; i++) {  
        int cur = nums[i];  
        if (cur == 1) {  
            count++;  
        } else {  
            max = Math.max(count, max);  
            count = 0;  
        }  
    }  
    max = Math.max(count, max);  
  
    return max;  
}
```

```
}
```

Day 8: Greedy Algorithm

Problem 44: N meetings in one room

6
1 3 0 5 8 5
2 4 6 7 9 9
Sample Output 1:

4
Explanation For Sample Input 1:

You can organize a maximum of 4 meetings. Meeting number 1 from 1 to 2, Meeting number 2 from 3 to 4, Meeting number 4 from 5 to 7, and Meeting number 5 from 8 to 9.

Solution:

```
import java.util.*;
import java.io.*;

public class Solution {
    public static List<Integer> maximumMeetings(int[] start, int[] end) {
        int n = start.length;
        List<Node> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new Node(start[i], end[i], i + 1));
        }
        Collections.sort(list, (a, b) -> {
            if (a.end == b.end) {
                return Integer.compare(a.index, b.index);
            } else {
                return Integer.compare(a.end, b.end);
            }
        });

        List<Integer> result = new ArrayList<>();
        Node prev = list.get(0);
        result.add(prev.index);
        for (int i = 1; i < n; i++) {
            Node cur = list.get(i);
            if (cur.start > prev.end) {
                result.add(cur.index);
                prev = cur;
            }
        }

        return result;
    }
}
```

```
}  
  
class Node {  
    int start, end, index;  
  
    Node(int start, int end, int index) {  
        this.start = start;  
        this.end = end;  
        this.index = index;  
    }  
}
```

Problem 45: Minimum Number of Platforms

```
2  
6  
900 940 950 1100 1500 1800  
910 1200 1120 1130 1900 2000  
4  
100 200 300 400  
200 300 400 500
```

Sample Output 1:

```
3  
2
```

Explanation of the Sample Output 1:

In test case 1, For the given input, the following will be the schedule of the trains:

Train 1 arrived at 900 on platform 1.
Train 1 departed at 910 from platform 1.
Train 2 arrived at 940 on platform 1.
Train 3 arrived at 950 on platform 2 (since platform 1 was already occupied by train 1).
Train 4 arrived at 1100 on platform 3 (since both platforms 1 and 2 were occupied by trains 2 and 3 respectively).
Train 3 departed at 1120 from platform 2 (platform 2 becomes vacant).
Train 4 departed at 1130 from platform 3 (platform 3 also becomes vacant).
Train 2 departed at 1200 from platform 1 (platform 1 also becomes vacant).
Train 5 arrived at 1500 on platform 1.
Train 6 arrived at 1800 on platform 2.
Train 5 departed at 1900 from platform 1.
Train 6 departed at 2000 from platform 2.

Thus, minimum 3 platforms are needed for the given input.

In test case 2, For the given input, the following will be the schedule of the trains:

Train 1 arrived at 100 on platform 1.
Train 2 arrived at 200 from platform 2 (as platform 1 is occupied by train 1).
Train 1 departed at 200 from platform 1.
Train 3 arrived at 300 on platform 1.
Train 2 departed at 300 from platform 2.
Train 4 arrived at 400 on platform 2.
Train 3 departed at 400 from platform 1.
Train 4 departed at 500 from platform 2.

Thus, 2 platforms are needed for the given input

Solution

```
import java.util.*;

public class Solution {
    public static int calculateMinPlatforms(int at[], int dt[], int n) {
        // 900 940 950 1100 1500 1800
        // 910 1200 1120 1130 1900 2000

        Arrays.sort(at);
        Arrays.sort(dt);
        int maxPlatformReq = 1;
        int platformReq = 1;

        int i = 1, j = 0;

        while (i < n && j < n) {
            if (at[i] <= dt[j]) {
                platformReq++;
                i++;
            } else {
                maxPlatformReq = Math.max(maxPlatformReq,
platformReq);
                platformReq--;
                j++;
            }
        }
        maxPlatformReq = Math.max(maxPlatformReq, platformReq);

        return maxPlatformReq;
    }
}
```

```
}  
  
class Node {  
    int x, y, index;  
  
    Node(int x, int y, int index) {  
        this.x = x;  
        this.y = y;  
        this.index = index;  
    }  
}
```

Problem 46: Job Sequencing Problem

```
2  
4  
2 2 1 1  
30 40 10 10  
5  
2 4 1 3 5  
25 40 5 50 60  
Sample Output 1 :
```

```
70  
180
```

Explanation For Sample Input 1 :

For the first test case, the first and second jobs can be completed within the deadlines, and we earn a profit of 70 by doing so.

For the second test case, all the jobs have different deadlines. So we complete all the jobs.

Solution

```
import java.util.*;  
import java.lang.*;  
  
public class Solution {  
    public static int jobScheduling(int[][] jobs) {  
  
        // [2,40]  
        // [2,20]  
        // [1,10]  
        // sort  
        ArrayList<Job> list = new ArrayList<>();
```

```

int max = 0;

int index = 0;
for (int job[] : jobs) {
    int deadline = job[0];
    int curprofit = job[1];
    max = Math.max(deadline, max);
    list.add(new Job(deadline, curprofit));
}

Collections.sort(list, (a, b) -> (b.profit - a.profit));

// Step 1: sort by profit.
// Step 2: found the min-max.
// Step 3: create a arr based on max deadline.
// step 4: traverse entire array , set the arr value to
// set first maxprofit for each deadline (min - max index).
// Step 5: fill the arr index based on the deadline priorit
// deadline arr is utilized break.
// Step 6: if could not array check in the other available
// deadline index
// : of lower index.

// 2 4 1 3 5
// 25 40 5 50 60
// 1 2 3 4 5

// i dead profit
// 5 5 60
// 4 3 50
// 2 4 40
// 1 2 25
// 3 1 5

// min| max| profit
// ---|----|-----
// 60 | 60 |since deadline is 5 for 60
// 50 | 50 |//110 (total -60+50)
// 40 | 40 |//150 (110+40)
// 25 | 25 |//175(150+25)
// 5 | 5 |//180
int profit[] = new int[max + 1];
Arrays.fill(profit, -1);
int n = jobs.length;

int profitSum = 0;
for (int i = 0; i < n; i++) {// iterate over array
    Job job = list.get(i);

```



```
        int deadline = job.deadline;
        int curprofit = job.profit;
        // iterate from min to max
        for (int j = deadline; j > 0; j--) {
            if (profit[j] == -1) {
                profit[j] = i;
                profitSum += curprofit;
                // break i since found first max deadline with
                // maxprofit
                break;
            }
        }
    }

    return profitSum;
}

class Job {
    int deadline;
    int profit;

    Job(int deadline, int profit) {
        this.deadline = deadline;
        this.profit = profit;
    }
}
```

Problem 47: Fractional Knapsack

Sample Input 1:

```
1
6 200
50 40 90 120 10 200
40 50 25 100 30 45
```

Sample output 1:

204.00

Explanation of Sample output 1:

The most optimal way to fill the knapsack is to choose full items with weight 10 and value 30, weight 40 and value 50, weight 120 and value 100. Then take weight 30 from the item with weight 50 and value 40.

The total value = $30 + 50 + 100 + (30/50)*(40) = 204.00$

Solution

```
import java.util.* ;
import java.io.*;
/*****
```

Following is the class structure of the Pair class:

```
class Pair
{
    int weight;
    int value;
    Pair(int weight, int value)
    {
        this.weight = weight;
        this.value = value;
    }
}

*****/

public class Solution {
    public static double maximumValue(Pair[] items, int n, int w) {
        Arrays.sort(items, (a, b) -> {

            double x = (double) ((double) (a.value) / (double)
a.weight);
            double y = (double) ((double) (b.value) / (double)
b.weight);

            return Double.compare(y, x);
        });

        double max = 0;
        int curWeight = 0;
        for (Pair cur : items) {
            if (curWeight + cur.weight <= w) {
                curWeight += cur.weight;
                max += cur.value;
            } else {
                int remWeight = w - curWeight;
                // calculate fraction weight if whole weight can be
added.

                // diff is the weight which it can add.
                double fractionweightValue = (double) ((double)
cur.value /
                (double) cur.weight) *
remWeight;
```

```
        max += fractionweightValue;
        break;
    }
}
return max;
}
```

Problem 48: Maximum activities

Sample Output 1:

3
1

Explanation For Sample Input 1:

For test case 1:

A person can perform maximum of 3 activities, by performing the activities in the given order 1 - > 3 -> 2.

For test case 2:

As the starting of all the activities is the same, a person can perform a maximum of 1 activity.

Sample Input 2:

2
4
1 3 2 5
2 4 3 6
2
1 2
6 3

Sample Output 2:

4
1

Solution:

```
import java.util.*;

public class Solution {
    public static int maximumActivities(List<Integer> start, List<Integer>
end) {
        // Write your code here
```

```
List<Node> list = new ArrayList<>();
int n = start.size();
for (int i = 0; i < n; i++) {
    list.add(new Node(start.get(i), end.get(i)));
}
Collections.sort(list, (a, b) -> {
    return Integer.compare(a.y, b.y);
});

// 3 2 0 0 9 6
// 18 19 18 5 18 8

// 0 0 2 3 6 9
// 5 8 18 18 18 19
Node prev = list.get(0);
int count = 1;
for (int i = 1; i < n; i++) {
    Node cur = list.get(i);
    if (prev.y <= cur.x) {
        count++;
        prev = cur;
    }
}

return count;
}

}

class Node {
    int x, y;

    Node(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

DAY 9: RECURSION

Problem:49: Subset Sum**Sample Input 1 :**

3
1 2 3

Sample output 1 :

0 1 2 3 3 4 5 6

Explanation For Sample Output 1:

For the first test case,
Following are the subset sums:
0 (by considering empty subset)
1
2
1+2 = 3
3
1+3 = 4
2+3 = 5
1+2+3 = 6
So, subset-sums are [0,1,2,3,3,4,5,6]

Sample Input 2 :

2
4 5

Sample output 2 :

0 4 5 9

Solution:

```
import java.util.*;

public class Solution {
    public static ArrayList<Integer> subsetSum(int num[]) {
```

```

        ArrayList<Integer> result = new ArrayList<>();
        Arrays.sort(num);
        result.add(0);
        helper(result, num, 0, 0);
        Collections.sort(result);
        return result;
    }

    private static void helper(ArrayList<Integer> result, int num[], int
index, int sum) {
        int n = num.length;
        if (index >= n) {

            return;
        }

        sum = sum + num[index];
        result.add(sum);
        helper(result, num, index + 1, sum);
        sum = sum - num[index];
        helper(result, num, index + 1, sum);
    }
}

```

Problem 50: 90. Subsets II

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: `nums = [1,2,2]`

Output: `[[],[1],[1,2],[1,2,2],[2],[2,2]]`

Example 2:

Input: `nums = [0]`

Output: `[[],[0]]`

Solution:

```

class Solution {
// Runtime: 3 ms, faster than 46.96% of Java online submissions for
Subsets II.

```

// Memory Usage: 44 MB, less than 64.18% of Java online submissions for Subsets II.

```

    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);

        //          [1,2,2] //s
        //          / \   \ (duplicate ignore)
        //          [1]  [2] [2]    //size 1
        //          /      \
        //          [1,2]    [1,2]  //size 2
        //          /
        //          [1,2,2]        //size 3

        List<List<Integer>> result = new ArrayList<>();
        List<Integer> list = new ArrayList<>();

        helper(nums, list, result, 0);

        return result;
    }

    private void helper(int nums[], List<Integer> list, List<List<Integer>>
result, int index) {

        result.add(new ArrayList<>(list));
        int n = nums.length;
        if (index == n) {

            return;
        }

        for (int i = index; i < n; i++) {
            if (i != index && nums[i] == nums[i - 1])
                continue;
            list.add(nums[i]);
            helper(nums, list, result, i + 1);
            list.remove(list.size() - 1);
        }
    }
}

```

Problem 51: 39. Combination Sum

Given an array of **distinct** integers candidates and a target integer target, return a list of all **unique combinations** of candidates where the chosen numbers sum to target. You may return the combinations in **any order**.

The **same** number may be chosen from candidates an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1

Output: []

Solution:

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target)
    {
        List<List<Integer>> result = new ArrayList<>();

        Arrays.sort(candidates);
        List<Integer> list = new ArrayList<>();

        //          2 ,3,6,7
        //          /  \
        //         [2] [3]
        //         /    \
```

```

//          [2,2]  [2,6]
//          /      \
//          [2,2,3]  X

helper(candidates, list, result, 0, target);

return result;
}

private void helper(int[] candidates, List<Integer> list,
List<List<Integer>> result, int index, int target) {
    int n = candidates.length;
    if (index == n) {
        if (target == 0) {
            result.add(new ArrayList<>(list));
        }
        return;
    }

    // Pick or Dont pick pattern
    if (target < 0)
        return;
    // target >=0 to add to combination list
    // either add target and dont dec index
    // or inc index to avoid stack overflow.
    // in base case add 2 conditions either index==n
    // or target reaches 0 so that whether take or nottake
    // will make sure stack overflow wont happen.
    int cur = target - candidates[index];
    list.add(candidates[index]); // >=0 so add to result list
    helper(candidates, list, result, index, cur);
    list.remove(list.size() - 1);

    helper(candidates, list, result, index + 1, target);
}
}

```

Problem 52: 40. Combination Sum II

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

Input: candidates = [10,1,2,7,6,1,5], target = 8

Output:

```
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

```
[
  [1,2,2],
  [5]
]
```

Solution:

```
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target)
    {

        Arrays.sort(candidates);
        //sorted because if we pick 2 and look for rem only
        //pickup next higher in order.
        //otherwise end up pickup multiple duplicates.

        List<Integer> list = new ArrayList<>();
        List<List<Integer>> result = new ArrayList<>();

        //[10,1,2,7,6,1,5], target =8

        //[1,1,6]( no same index allowed)
        //[1,2,5]
```

```

// [1,7]
// [2,6]

// [10      1      ,2      ,7,      6,      1,      5 ]//8
// \      \      \      \      \      \      \
// \      \      \      \      \      \      \
//  x      [1,{7}]      [2,{6}] [7,{1}] [6,{2}] [1,{7}] [5,{3}]
//      /..|... \
//      [1,2{5}] | [1,5{3}]
//      |
//      [1,7{0}] for each there will be n tree to pickup
//
//Note : apply pick or not-pick pattern not workout
// because we cant compare previous index also
// we can never pickup duplicates.
// even if we use pick or not-pick pattern we need to store in hashset.
// then transfer to list which will be again take logn
// so complexity becomes  $2^n * k * \log n$  -> k for add to different result hashset.
// logn for transfer or conver to List<List<Integer>>

//add a extra layer like below for loop

//if pick or not pick we cant do pruning to avoid some duplicate data, all we have
current index to iterate.
//so we have extra loop for the no of option (decision tree) and prune to avoid
recursion to happen
//we try to reduce the no of recursive call by adding condition and break

    helper(candidates,list,result,target,0);
    return result;
}

```

```

private void helper(int candidates[], List<Integer> list, List<List<Integer>>
result, int target, int index) {
    int n = candidates.length;
    if (target == 0) {
        result.add(new ArrayList<Integer>(list));
        return;
    }
    if (target < 0)
        return;

    for (int i = index; i < n; i++) {

```

```

// pruning.
// we actually if ignore current index to be added again.
// but since sorted let and we never want to visit
// previous visited index.

// it also work.
// if(i!=index && candidates[i] ==candidates[i-1] )continue;

// ignore all previous visited index.
if (i > index && candidates[i] == candidates[i - 1])
    continue;

// since sort we know there is nothing in current index or after
which can make
// target less than =0
// since all element from current index are high value , so no
need to try the
// remaining searchspace.
if (candidates[i] > target)
    break;
list.add(candidates[i]);
helper(candidates, list, result, target - candidates[i], i + 1);
list.remove(list.size() - 1);
    }
}
}

```

Problem 53: 131. Palindrome Partitioning

Given a string *s*, partition *s* such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of s*.

Example 1:

Input: *s* = "aab"

Output: [["a","a","b"],["aa","b"]]

Example 2:

Input: *s* = "a"

Output: [["a"]]

Constraints:

- `1 <= s.length <= 16`
- `s` contains only lowercase English letters.

Solution:

```
class Solution {
    public List<List<String>> partition(String s) {
        //      "aab"
        //      /  \
        //  [a]      [a]2 a
        //  / \      /  \
        // [a] [ab] [b]  []
        // /
        // b[1,["a","a","b"]]

        List<List<String>> result = new ArrayList<>();
        List<String> list = new ArrayList<>();

        helper(s, list, result, 0);

        return result;
    }

    private void helper(String s, List<String> list,
        List<List<String>> result, int index) {
        int n = s.length();
        if (index == n) {
            result.add(new ArrayList<String>(list));
            return;
        }
    }
}
```

```
        for (int i = index; i < n; i++) {
            String cur = s.substring(index, i + 1);
            if (cur.length() == 0)
                System.out.println(cur.length());
            if (isPalindrome(cur)) {
                list.add(cur);
                helper(s, list, result, i + 1);
                list.remove(list.size() - 1);
            }
        }
    }

    private boolean isPalindrome(String cur) {
        char arr[] = cur.toCharArray();
        int low = 0;
        int high = arr.length - 1;
        while (low <= high) {
            if (arr[low] != arr[high])
                return false;

            low++;
            high--;
        }

        return true;
    }
}
```

Problem 53: 60. Permutation Sequence

The set [1, 2, 3, ..., n] contains a total of $n!$ unique permutations.

By listing and labeling all of the permutations in order, we get the following sequence for $n = 3$:

1. "123"
2. "132"
3. "213"
4. "231"
5. "312"
6. "321"

Given n and k , return the k^{th} permutation sequence.

Example 1:**Input:** n = 3, k = 3**Output:** "213"**Example 2:****Input:** n = 4, k = 9**Output:** "2314"**Example 3:****Input:** n = 3, k = 1**Output:** "123"**Constraints:**

- $1 \leq n \leq 9$
- $1 \leq k \leq n!$

Solution:

```
class Solution {
// Runtime: 2 ms, faster than 92.00% of Java online submissions for
Permutation Sequence.
// Memory Usage: 42.5 MB, less than 11.38% of Java online submissions for
Permutation Sequence.

    public String getPermutation(int n, int k) {
        //if we do permutations for n then it will be 2!
        // so recursion wont work.

        // n=k, k=3,
        // output:"213"
        //[1 2 3] ->6 (3!)

        //6 combinations are 2 bucket for each index
        //start with 1 (2),2(2) start with 3 (2)permutations

        //0    -[2,3] (0-1)
        //    -[3,2]
```



```
//1    -[1,3]  (2-3)
//      -[3,1]

//2    -[1,3]  (4-5)
//      -[3,1]

//to get the 3
// k = 3
// n = total elements (3)
// k-1 since index start from 0.
// no of permutations= n*(n-1)

// k-1/(n*(n-1))
//

//Start 1:
//      n=3,k=3-1, partitions = 6
//      2%6 =2 //remove 2
//REMOVE : 2
//      remaining n=2,

// after remove 2 [1,3]

//      1 -[3] -partition 0

//      3 -[1] - partition 1

//Step 2:
//k becomes 2 because if we partition [1,3] we have 2 partition
sequence.
//      n=2,k=2-1=1 ,partition = 4.
//      1%4 =1(k%partition)
//REMOVE : 1
//      [3]

//Step 3"
//k becomes 1 because if we partition 1 we have 1 partition sequence.
//      n=1, k=1-1=0 ,partition=1
// it becomes 0 so add last element 3

int fact[] = new int[n+1];
int sum =1;
fact[0]=1;
```

```
List<Integer> numbers = new ArrayList<>();
for(int i=1;i<=n;i++){
    sum*=i;
    fact[i]=sum;
    numbers.add(i);
}

//numbers:1 2 3
//      0 1 2
//fact   [1, 1, 2, 6]
//      0 1 2 3
k =k-1;//0 based

StringBuffer buffer = new StringBuffer();
for(int i=1;i<=n;i++){
    //find index
    //3/fact[2] = 2/2 =1
    int index = (k/fact[n-i]); // 2/fact[3]
    buffer.append(String.valueOf(numbers.get(index)));
    numbers.remove(index); //remove index.
    //reduce k
    k=k- index*fact[n-i];
}

return buffer.toString();
}
}
```

Day 10: RECURSION AND BACKTRACKING

Problem 54: 46. Permutations

Given an array `nums` of distinct integers, return *all the possible permutations*. You can return the answer in **any order**.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2:

Input: `nums = [0,1]`

Output: `[[0,1],[1,0]]`

Example 3:

Input: `nums = [1]`

Output: `[[1]]`

Solution:

```
import java.util.ArrayList;
import java.util.List;

class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        helper(nums, 0, result);
        return result;
    }
}
```

```
private void helper(int nums[], int index, List<List<Integer>> result)
{
    int n = nums.length;
    if (index == n - 1) {
        addToList(nums, result);
        return;
    }

    for (int i = index; i < n; i++) {
        //note : swapping function
        swap(nums, i, index);
        //note : function index passed not local variable i
        helper(nums, index + 1, result);
        swap(nums, i, index);
    }
}

private void addToList(int nums[], List<List<Integer>> result) {
    List<Integer> list = new ArrayList<>();
    for (int num : nums) {
        list.add(num);
    }

    result.add(list);
}

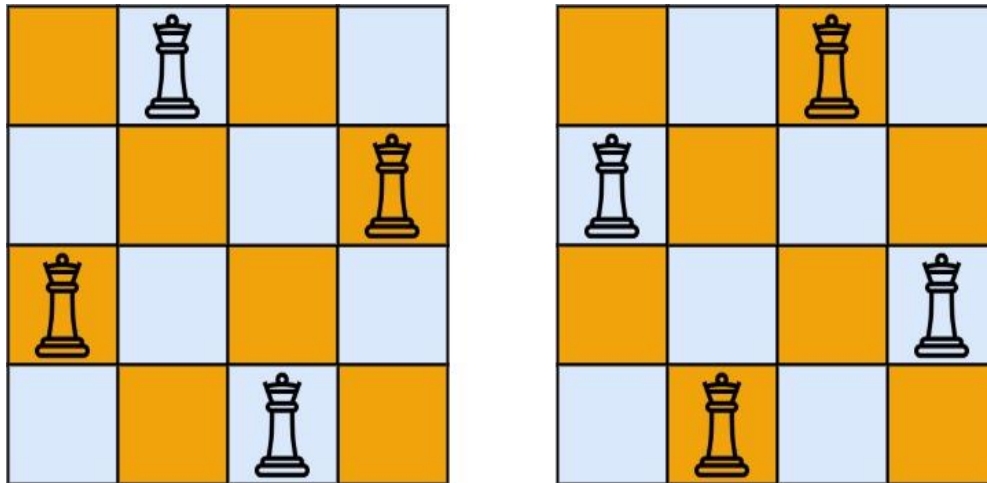
private void swap(int nums[], int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
}
```

Problem 55: 51. N-Queens

The **n-queens** puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other.

Given an integer n , return *all distinct solutions to the n-queens puzzle*. You may return the answer in **any order**.

Each solution contains a distinct board configuration of the n -queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:**Input:** `n = 4`**Output:** `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`**Explanation:** There exist two distinct solutions to the 4-queens puzzle as shown above**Solution:**

```

class Solution {
    public List<List<String>> solveNQueens(int n) {

        List<List<String>> result = new ArrayList<>();
        char matrix[][] = new char[n][n];

        fillInMatrix(matrix);

        solve(matrix, result, 0);

        return result;
    }

    private void solve(char matrix[][], List<List<String>> result, int col)
    {

        int n = matrix.length;

        if (col == n) {
            result.add(construct(matrix));
            return;
        }
    }

```

```

        for (int i = 0; i < n; i++) {
            if (isValid(matrix, i, col)) {
                matrix[i][col] = 'Q';
                solve(matrix, result, col + 1);
                matrix[i][col] = '.';
            }
        }
    }

private boolean isValid(char matrix[][], int row, int col) {
    int n = matrix.length;

    // 00 01 02 03
    // 10 11 12 13
    // 20 21 22 23
    // 30 31 32 33
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == 'Q' && (row == i || col == j
                || Math.abs(row - i) == Math.abs(col - j)))
                return false;
        }
    }

    return true;
}

private List<String> construct(char matrix[][]) {
    int n = matrix.length;
    List<String> list = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        String cur = new String(matrix[i]);
        list.add(cur);
    }

    return list;
}

private void fillInMatrix(char matrix[][]) {
    int n = matrix.length;
    for (int i = 0; i < n; i++) {
        Arrays.fill(matrix[i], '.');
    }
}
}

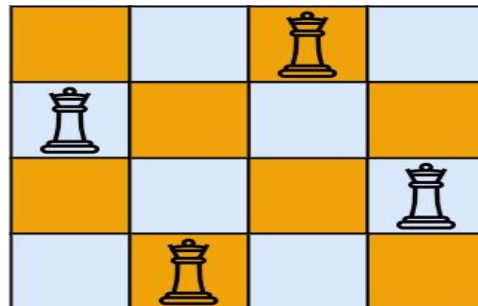
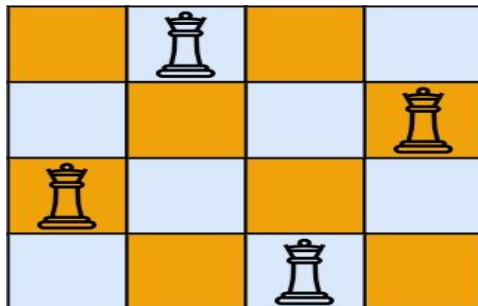
```

Problem 55-II: 52. N-Queens II

The **n-queens** puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other.

Given an integer n , return *the number of distinct solutions to the **n-queens** puzzle*.

Example 1:



Input: $n = 4$

Output: 2

Explanation: There are two distinct solutions to the 4-queens puzzle as shown.

Example 2:

Input: $n = 1$

Output: 1

Solution:

```
public class Solution {
    int count = 0;
    public int totalNQueens(int n) {
        boolean[] cols = new boolean[n];           // columns   |
        boolean[] d1 = new boolean[2 * n];         // diagonals \
        boolean[] d2 = new boolean[2 * n];         // diagonals /
        backtracking(0, cols, d1, d2, n);
        return count;
    }

    public void backtracking(int row, boolean[] cols,
        boolean[] d1, boolean []d2, int n) {
        if(row == n) count++;

        for(int col = 0; col < n; col++) {
            int id1 = col - row + n;
            int id2 = col + row;

```



```

        if(cols[col] || d1[id1] || d2[id2]) continue;

        cols[col] = true; d1[id1] = true; d2[id2] = true;
        backtracking(row + 1, cols, d1, d2, n);
        cols[col] = false; d1[id1] = false; d2[id2] = false;
    }
}
}

```

Problem 56: 37. Sudoku Solver

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy **all of the following rules**:

1. Each of the digits 1-9 must occur exactly once in each row.
2. Each of the digits 1-9 must occur exactly once in each column.
3. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

5	3	.	.	7	.	.	.	.
6	.	.	1	9	5	.	.	.
.	9	8	.	.	.	.	6	.
8	.	.	.	6	.	.	.	3
4	.	.	8	.	3	.	.	1
7	.	.	.	2	.	.	.	6
.	6	.	.	.	.	2	8	.
.	.	.	4	1	9	.	.	5
.	.	.	.	8	.	.	7	9

Input: board =

```

[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],
[[".","9","8",".",".",".","6","."],["8",".",".","6",".",".","3"],[
"4",".",".","8",".","3",".",".","1"],["7",".",".","2",".",".","6"],[
","6",".",".",".","2","8","."],[".",".","4","1","9",".",".","5"],[
",".",".","8",".",".","7","9"]]

```

Output:

```

[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],
["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],[

```

```
"4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]
```

Explanation: The input board is shown above and the only valid solution is shown below:

Solution:

```
class Solution {
    public void solveSudoku(char[][] board) {
        solve(board);
    }

    private boolean solve(char[][] board) {
        int r = board.length;
        int c = board[0].length;

        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                if (board[i][j] != '.')
                    continue;
                // set value from 1 to 9
                for (char k = '1'; k <= '9'; k++) {
                    if (isValid(board, i, j, k)) {
                        board[i][j] = k;
                        if (solve(board))
                            return true;
                        board[i][j] = '.';
                    }
                }
            }
        }
    }
}
```

```
        // 1 to 9 set still could not form sudoku
        return false;
    }
}

return true;
}
```

```
private boolean isValid(char board[][], int row, int col, int cellvalue) {
    // since fixed 9X9 matrix
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == cellvalue)
            return false;
        if (board[i][col] == cellvalue)
            return false;
    }

    // if (9,9)
    // to convert to row, col to 3X3
    // 9/3 = 3
    // if row 1 1/3 = 0
    // if row 3 3/3 = 1
    // if row >6 6/3 = 2
    int boxRow = (row / 3) * 3; // since we need to convert to 3 out
of 9
    int boxCol = (col / 3) * 3;

    // current box row + 3 since we iterate 3X3
    for (int i = boxRow; i < boxRow + 3; i++) {
```

```
        for (int j = boxCol; j < boxCol + 3; j++) {
            if (board[i][j] == cellvalue)
                return false;
        }
    }

    return true;
}
}
```

Problem 57: M-Coloring Problem

Sample Input 1:

```
3 2
0 1 0
1 0 1
0 1 0
```

Sample Output 1:

YES

Explanation of Input 1:

The adjacency matrix tells us that 1 is connected to 2, and 2 is connected to 3. We can see that a minimum of 2 colors would be needed to color the graph. So it is possible to color the graph in this case.

Sample Input 2:

```
3 1
0 1 0
1 0 1
0 1 0
```

Sample Output 2

NO

Solution

```
import java.util.*;

public class Solution {
    public static String graphColoring(int[][] mat, int m) {

        // 3 3
        // 0 1 0 -0
        // 1 0 1 -1
        // 0 1 0 -2

        // A -B
        // c
        int r = mat.length;
        if (r == 0)
            return "YES";
    }
}
```

```

        boolean adjList[][] = new boolean[r][r];
        int c = mat[0].length;

        for (int i = 0; i < r; i++) {
            for (int j = 0; j < r; j++) {
                if (mat[i][j] == 1) {
                    adjList[i][j] = true;
                    adjList[j][i] = true;
                }
            }
        }

        int color[] = new int[r];
        Arrays.fill(color, 0);
        return helper(adjList, color, 0, m) ? "YES" : "NO";
    }

    private static boolean helper(boolean adjList[][],
                                   int color[], int curNode, int noOfColors){
        int noOfNodes = adjList.length;

        if(curNode == noOfNodes){
            return true;
        }
        for(int i=1;i<=noOfColors;i++){
            if(isSafe(adjList,color,curNode,i,noOfColors)){
                color[curNode]=i;
                if(helper(adjList,color,curNode+1,noOfColors))return true;
                color[curNode]=0;
            }
        }

        return false;
    }

    private static boolean isSafe(boolean adjList[][], int color[], int curNode,
    int curColor, int noOfColors) {
        int noOfNodes = adjList.length;

        for (int i = 0; i < noOfNodes; i++) {
            if (adjList[curNode][i] && color[i] == curColor) {
                return false;
            }
        }

        return true;
    }

```

```
}  
  
class Node {  
    int x, y;  
  
    Node(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Problem 58: Rat In a Maze All Paths

Sample Input 1 :

```
3  
1 0 1  
1 0 1  
1 1 1
```

Sample Output 1 :

```
1 0 0 1 0 0 1 1 1
```

Explanation for Sample Output 1:

Only 1 path is possible which contains coordinate < (1,1), (2,1), (3,1), (3,2) and (3,3) >

So our path matrix will look like this:

```
1 0 0  
1 0 0  
1 1 1
```

Which is returned from left to right and then top to bottom in one line.

Sample Input 2 :

```
2  
1 0  
0 1
```

Sample Output 2 :

[Blank]

Explanation for Sample Output 2:

As no path is possible to the last cell, a blank vector will be returned and nothing is print

Solution:

```
import java.util.ArrayList;
```

```
public class Solution {
    private static void insertCurrentState(int[][] solution,
ArrayList<ArrayList<Integer>> ans, int n) {
        // Insert the solution matrix element by element in the ans.
        ArrayList<Integer> currentState = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                currentState.add(solution[i][j]);
            }
        }
        ans.add(currentState);
    }
}
```

```
private static void solveMaze(int[][] maze, int[][] solution,
ArrayList<ArrayList<Integer>> ans, int i, int j,
    int n) {
    // Base case that we reach our destination.
    if (i == n - 1 && j == n - 1) {
        solution[i][j] = 1;
        // Call to add the updated solution matrix in 'ANS'.
        insertCurrentState(solution, ans, n);
        return;
    }
    // Condition of out of boundary of the maze.
    if (i >= n || i < 0 || j >= n || j < 0) {
        return;
    }
}
```

```

        /*
        * Condition for 'MAZE[x][y]==0' - if that particular cell is
        block.
        * 'SOLUTION[x][y]' == 1 - if it is already visited or already
        we go through
        * it.
        */
        if (maze[i][j] == 0 || solution[i][j] == 1) {
            return;
        }

        // No problem comes in visiting this cell so visit it.
        solution[i][j] = 1;

        // Recursive calls to all directions(call to function having same
        name with diff
        // value of params).
        // Up move.
        solveMaze(maze, solution, ans, i + 1, j, n);
        solveMaze(maze, solution, ans, i - 1, j, n);
        solveMaze(maze, solution, ans, i, j - 1, n);
        solveMaze(maze, solution, ans, i, j + 1, n);
        solution[i][j] = 0;
    }

    public static ArrayList<ArrayList<Integer>> ratInAMaze(int[][] maze,
    int n) {
        // Initialize the 'SOLUTION' matrix by all 0s.
        int[][] solution = new int[n][n];

        // Vector used to store all the paths.
        ArrayList<ArrayList<Integer>> ans = new ArrayList<>();

        // Final call to function to print the solutions.
        solveMaze(maze, solution, ans, 0, 0, n);

        // Return the updated ans.
        return ans;
    }
}

```

Problem 59: 140. Word Break II

Given a string *s* and a dictionary of strings *wordDict*, add spaces in *s* to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in **any order**.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]

Output: ["cats and dog","cat sand dog"]

Example 2:

Input: s = "pineapplepenapple", wordDict =

["apple","pen","applepen","pine","pineapple"]

Output: ["pine apple pen apple","pineapple pen apple","pine applepen apple"]

Explanation: Note that you are allowed to reuse a dictionary word.

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]

Output: []

Solution:

```
class Solution {  
  
    public List<String> wordBreak(String s, List<String> wordDict) {  
        List<String> list = new ArrayList<>();  
  
        String arr[] = new String[21];  
        helper(s, wordDict, list, arr, 0, 0);  
  
        return list;  
    }  
  
    private String constructWord(String arr[], int end) {  
        StringBuffer buffer = new StringBuffer();  
        for (int i = 0; i <= end; i++) {  
            buffer.append(arr[i] + " ");  
        }  
  
        return buffer.toString();  
    }  
  
    private void helper(String s, List<String> wordDict, List<String> list,  
        String arr[], int arrIndex, int index) {  
        int n = s.length();  
        if (index == n) {  
  
            String res = constructWord(arr, arrIndex - 1);  
            res = res.substring(0, res.length() - 1);  
        }  
    }  
}
```

```
        list.add(res);

        return;
    }

    for (int i = index; i < n; i++) {
        String cur = s.substring(index, i + 1);
        if (wordDict.contains(cur)) {
            arr[arrIndex] = cur;
            helper(s, wordDict, list, arr, arrIndex + 1, i + 1);
        }
    }
}

}
```

Problem 59-I:200. Number of Islands

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

Input: `grid = [`
 `["1","1","1","1","0"],`
 `["1","1","0","1","0"],`
 `["1","1","0","0","0"],`
 `["0","0","0","0","0"]`
 `]`

Output: 1

Solution:

```
import java.util.*;

public class Solution {
    public static int getTotalIslands(int[][] grid) {
```

```

    int r = grid.length;
    if (grid == null || (grid != null && r == 0))
        return 0;
    int c = grid[0].length;
    int count = 0;
    // once visited any i,j can consider
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            if (grid[i][j] == 1) {
                count++;
                bfs(grid, i, j);
            }
        }
    }
    return count;
}

private static void bfs(int grid[][], int i, int j) {

    int r = grid.length;
    int c = grid[0].length;
    LinkedList<Node> queue = new LinkedList<>();
    queue.offer(new Node(i, j));

    int dirs[][] = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 },
                     { -1, -1 }, { -1, 1 }, { 0, 0 }, { 1, 1 },
                     { 1, -1 } };
    while (!queue.isEmpty()) {
        Node cur = queue.poll();

        for (int dir[] : dirs) {
            int x = cur.x + dir[0];
            int y = cur.y + dir[1];

            if (x < 0 || y < 0 || x >= r || y >= c || grid[x][y]
== 0)
                continue;
            grid[x][y] = 0;
            queue.offer(new Node(x, y));
        }
    }
}
}

```

```
class Node {  
    int x, y;  
  
    Node(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

DAY 11: BINARY SEARCH

m

Problem 60: Find Nth Root Of M**Sample Input 1:**

3 27

Sample Output 1:

3

Explanation For Sample Input 1:3rd Root of 27 is 3, as $(3)^3$ equals 27.**Sample Input 2:**

4 69

Sample Output 2:

-1

Solution:

```
public class Solution {
    public static double findNthRootOfM(int n, long m) {
        // Initialize the variable 'low'.
        double low = 0;
        // Initialize the variable 'high'.
        double high = m;
        // Initialize the variable 'eps'.
        double eps = 1e-7;

        // Binary Search algorithm.
        while ((high - low) > eps) {

            // Initialize the variable 'middle'.
            double middle = (low + high) / 2;
            // Initialize the variable 'exp'.
```

```

        double exp = power(middle, n);
        // Check if 'exp' is equal to 'm'.
        if (exp < m) {
            low = middle;
        } else {
            high = middle;
        }
    }

    return low;
}

static double power(double m, int n) {

    // Initialize the variable 'ans'.
    double ans = 1;
    // Binary Exponentiation.
    while (n > 0) {
        // Checking if 'n' is odd.
        if (n % 2 == 1) {
            // Multiply 'm' to 'ans'.
            ans *= m;
            // Decrement 'n' by one.
            n--;
        }

        // Assign 'm' the square of 'm'.
        m *= m;
        // Divide 'n' by 2.
        n /= 2;
    }

    return ans;
}
}

```

Problem 61: Matrix Median

Sample Input 1:

```

2
1 3
1 2 3
3 3
2 6 9
1 5 11
3 7 8

```

Sample Output 1:

```

2

```

6

Explanation of sample input 1:

In the first test case, the overall median of the matrix is 2.

In the second test case, the overall median of the matrix is 6.

Sample Input 2:

```
2
3 3
2 6 8
1 4 7
6 8 9
3 5
1 2 6 6 10
2 4 4 5 7
2 5 5 6 6
```

Sample Output 2:

```
2
5
```

Explanation for sample input 2:

In the first test case, the overall median of the matrix is 2.

In the second test case, the overall median of the matrix is 5.

Solution:

```
import java.util.*;

public class Solution {
    public static int getMedian(ArrayList<ArrayList<Integer>> matrix) {
        // 3 3
        // 2 6 9
        // 1 5 11
        // 3 7 8

        // 1 2 3 5 6 7 8 9 11

        int min = 0;
        int max = 0;
        int r = matrix.size();
        int c = matrix.get(0).size();
        for (int i = 0; i < r; i++) {
            min = Math.min(matrix.get(i).get(0), min);
            max = Math.max(matrix.get(i).get(c - 1), max);
        }
    }
}
```



```

    }

    // search in space (min, max)
    // min =1;
    // max = 11
    // mid = 12/2 = 6
    // row 0 - 2 6 9 (found)
    //
    int target = (r * c + 1) / 2;

    while (min < max) {
        int mid = min + (max - min) / 2;
        int left = 0;
        // Find count of elements smaller than or equal
        // to mid
        for (int i = 0; i < r; ++i) {
            int index = binarySearch(matrix.get(i), mid);
            // index= ~index;//if not exist return n
            // if n={1,2,3,4,5,6} target=18 return 6(n)
            // search column wise
            left = left + index;
        }

        if (left < target)
            min = mid + 1;
        else
            max = mid;
    }
    return min;
}

private static int binarySearch(ArrayList<Integer> arr, int target) {

    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr.get(mid) > target) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return low;
}
}

```

Problem 62: 540. Single Element in a Sorted Array

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in $O(\log n)$ time and $O(1)$ space.

Example 1:

Input: nums = [1,1,2,3,3,4,4,8,8]

Output: 2

Example 2:

Input: nums = [3,3,7,7,10,11,11]

Output: 10

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^5$

Solution

```
class Solution {  
    // Runtime: 0 ms, faster than 100.00% of Java online submissions for  
    // Single Element in a Sorted Array.  
    // Memory Usage: 60.3 MB, less than 56.10% of Java online submissions for  
    // Single Element in a Sorted Array.
```

```
    public int singleNonDuplicate(int[] nums) {  
  
        int n = nums.length - 2;  
        int low = 0;  
        int high = n;  
  
        // observation is  
        // if 2 duplicates exist.  
  
        // even is 1 occurrence( 0,2,4 index).  
        // odd is 2 occurrence(1,3,5 index).  
  
        while (low <= high) {  
  
            int mid = low + (high - low) / 2;
```

```

        // check by xor.
        // a^b = ab(inverse)+ba(inverse)
        // if both input is 0 or 1 it becomes 0.
        // 421
        // 000 + 001 = 001 (1) even 0 changes 1.
        // 001 + 001 = 000 (0) odd 1 changes 0.
        // 010 + 001 = 011 (3) even 2 changes 3.
        // 011 + 001 = 010 (2) odd 3 changes 2.

        // if odd shrink to left.
        // if even move to right.
        // if both left and right are same we found first
duplicates
        if (nums[mid] == nums[mid ^ 1]) {
            low = mid + 1; // if cur and next are same then move
right.
        } else {
            high = mid - 1; // if cur and previous are same than
move left
        }
    }
    return nums[low];
}
}

```

Problem 63: 4. Median of Two Sorted Arrays

HARD

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log (m+n))$.

Example 1:

Input: nums1 = [1,3], nums2 = [2]

Output: 2.00000

Explanation: merged array = [1,2,3] and median is 2.

Example 2:

Input: nums1 = [1,2], nums2 = [3,4]

Output: 2.50000

Explanation: merged array = [1,2,3,4] and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- `0 <= m <= 1000`
- `0 <= n <= 1000`
- `1 <= m + n <= 2000`
- `-106 <= nums1[i], nums2[i] <= 106`

Solution

```
class Solution {
// Runtime: 5 ms, faster than 43.16% of Java online submissions for Median
// of Two Sorted Arrays.
// Memory Usage: 49.4 MB, less than 68.91% of Java online submissions for
// Median of Two Sorted Arrays.

    public double findMedianSortedArrays(int[] nums1, int[] nums2) {

        return helper(nums1,nums2);
    }

    private double helper(int nums1[], int nums2[]){
        int n1 = nums1.length;
        int n2 = nums2.length;

        //we keep left as small so that we can partition based on left arr
        count.
        //let say 11 elements left has (5) and right has 6 then if we set left
        as small before partition.
        //so it will be easy for partition that we can assume that right more
        than or equal to 5.
        //so we partition start from 5 elements as base and we can increase
        the partition or decrease.
        //below we have 10 so partition 5 (left) 5 right initially since we
        know right will have atleast 5 elements.
```

```

//partitionx vs partition y
//2 3 4(lx) |(rx) 7 8
// 1 5(ly) |(ry) 6 9 10

if(n1>n2){
    return helper(nums2,nums1);
}

int low = 0;
int high= n1;

while (low <= high) {

    // median formula:
    // if odd = (n1)/2
    // if even = ((n1-1)/2 +(n+1)/2)/2;
    int partitionx = low + (high - low) / 2;
    int partitiony = (n1 + n2 + 1) / 2 - partitionx;

    //we expect 5 elements in left and 5 in right.
    // 2 3 4 7 8 | 1 5 6 9 10
    // nums1      | nums2

    //partitionx vs partition y
    //2 3 4(lx) |(rx) 7 8
    // 1 5(ly) |(ry) 6 9 10

    //2 3 (4) | (7) 8
    // 1 (5) | (6) 9 10
    //here 4<6 and 5<7
    // 4+7 = 11 = 5.5 (correct ans verified)

    //initialy

    //      2(lx) |(rx) 3 4 7 8
    // 1 5 6 9(ly) |(ry) 10

    //ly>rx
    int maxLeftX = (partitionx == 0) ? Integer.MIN_VALUE :
nums1[partitionx - 1];
    int minRightX = (partitionx == n1) ? Integer.MAX_VALUE :
nums1[partitionx];
    int maxLeftY = (partitiony == 0) ?
Integer.MIN_VALUE : nums2[partitiony - 1];
    int minRightY =
(partitiony == n2) ? Integer.MAX_VALUE : nums2[partitiony];
    if (maxLeftX <= minRightY && maxLeftY <= minRightX) {

```

```
        if ((n1 + n2) % 2 == 0) {
            double l = (double) Math.max(maxLeftX,
maxLeftY);
            double r = (double) Math.min(minRightX,
minRightY);

            return (l + r) / 2;
        } else {
            return Math.max(maxLeftX, maxLeftY);
        }
    } else if (maxLeftX > minRightY) {
        high = partitionx - 1;
    } else {
        low = partitionx + 1;
    }
}
return -1;
}
}
```

Problem 64: 33. Search in Rotated Sorted Array

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly rotated** at an unknown pivot index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3

Output: -1

Example 3:

Input: nums = [1], target = 0

Output: -1

Solution

```
class Solution {
    // Runtime: 1 ms, faster than 71.46% of Java online submissions for
    // Search in Rotated Sorted Array.
    // Memory Usage: 42.9 MB, less than 30.75% of Java online submissions for
    // Search in Rotated Sorted Array.
    // Next challenges:

    public int search(int[] nums, int target) {

        return helper(nums, target);
    }

    private int helper(int nums[], int target) {

        int low = 0;
        int high = nums.length - 1;

        while (low <= high) {
            int mid = (low + high) >> 1;
            if (nums[mid] == target)
                return mid;

            // search in left and check whether target lies between
left & mid
            if (nums[low] <= nums[mid]) {
                if (target >= nums[low] && target <= nums[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            } else {
                if (target >= nums[mid] && target <= nums[high]) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
        }
    }
}
```

```
        return -1;
    }
}
```

Problem 65: Kth Element of Two Sorted Arrays

Sample Input 1 :

```
2
5 4 3
3 11 23 45 52
4 12 14 18
1 1 2
1
2
```

Sample Output 1 :

```
11
2
```

Explanation for Sample Output 1 :

For sample test case 1:

1'st person will get 3 ladoos i.e a minimum of 3 and 4. Now 'ROW1' : [11, 23, 45, 52] and 'ROW2' : [4, 12, 14, 18].

2'nd person will get 4 ladoos i.e minimum of 11 and 4. Now 'ROW1' : [11, 23, 45, 52] and 'ROW2' : [12, 14, 18].

3'rd person will get 11 ladoos i.e minimum of 11 and 12.

For sample test case 2:

1'st person will get 1 ladoos i.e a minimum of 1 and 2. Now 'ROW1' : [] and 'ROW2' : [2].

2'st person will get 2 ladoos because we have only one element left in ROW2 . Now 'ROW1' : [] and 'ROW2' : [].

Sample Input 2 :

```
2
5 3
1 3 6 7 10
3 5 5 7
3 3 2
10 20 20
1 2 3
```

Sample Output 2 :

3
2

Solution

```
public class Solution {
    static long maxN = (long) 1e10; // the maximum value in the array
    possible.

    public static int ninjaAndLadoos(int row1[], int row2[], int m, int n,
    int k) {

        // [2,5,8,17]
        // [1,4,8,13,20]

        return (int) kthElement(row1, row2, m, n, k);
    }

    static int upperBound(int[] a, int low, int high, long element) {
        while (low < high) {
            int middle = low + (high - low) / 2;
            if (a[middle] > element)
                high = middle;
            else
                low = middle + 1;
        }
        return low;
    }

    static long kthElement(int arr1[], int arr2[], int n, int m, int k) {
        long left = 1, right = maxN; // The range of where ans can lie.
        long ans = (long) 1e15; // We have to find min of all
                                // the ans so take .

        // using binary search to check all possible values of
        // kth element
        while (left <= right) {
            long mid = (left + right) / 2;
            long up_cnt = upperBound(arr1, 0, n, mid);
            up_cnt += upperBound(arr2, 0, m, mid);

            if (up_cnt >= k) {
```

```

        ans = Math.min(ans, mid); // find the min of all
answers.
        right = mid - 1; // Try to find a smaller answer.
    } else
        left = mid + 1; // Current mid is too small so
                        // shift right.
    }

    return ans;
}
}

```

Problem 66: Allocate Books

Given an array of integers **A** of size **N** and an integer **B**.

The College library has **N** books. The **i**th book has **A[i]** number of pages.

You have to allocate books to **B** number of students so that the maximum number of pages allocated to a student is minimum.

1. A book will be allocated to exactly one student.
2. Each student has to be allocated at least one book.
3. Allotment should be in contiguous order, for example: A student cannot be allocated book 1 and book 3, skipping book 2.

Calculate and return that minimum possible number.

NOTE: Return -1 if a valid assignment is not possible.

Problem Constraints

$1 \leq N \leq 10^5$

$1 \leq A[i], B \leq 10^5$

Input Format

The first argument given is the integer array **A**. The second argument given is the integer **B**.

Output Format

Return that minimum possible number.

Example Input

Input 1:

A = [12, 34, 67, 90]

```
B = 2
Input 2:
A = [5, 17, 100, 11]
B = 4
```

Example Output

```
Output 1:
113
Output 2:
100
```

Solution

```
public class Solution {
    public int books(int[] A, int B) {

        int maxPages = 0;

        int n = A.length;
        // if no pages and no of students same then each student
        // reads each pages so minimum is min(all pages).

        int minPages = A[0];

        // if only one student then he reads all pages so maxPages
        for (int i = 0; i < n; i++) {
            int cur = A[i];
            minPages = Math.min(minPages, cur);
            maxPages += cur;
        }
        if (B > n)
            return -1;
        if (B == 1) {
            return maxPages;
        }

        // search space will be between min page to maxpage.

        int min = minPages;
        int max = maxPages;
        while (min <= max) {
            int mid = min + (max - min) / 2;
            if (isPossible(A, B, mid)) {
                max = mid - 1;
            } else {
                min = mid + 1;
            }
        }
    }
}
```

```
        return min;
    }

    boolean isPossible(int arr[], int noOfStudents, int availPages) {
        int n = arr.length;

        int low = 0;
        int high = n;

        int cur = 0;
        // [12, 34, 67, 90]
        int avail = noOfStudents - 1;
        for (int i = 0; i < n; i++) {
            if (cur + arr[i] <= availPages) {
                cur += arr[i];
            } else {
                cur = arr[i];
                if (cur > availPages)
                    return false;
                avail--; // allocate to the student.
                if (avail < 0)
                    return false;
            }
        }

        return true;
    }
}
```

Problem 67: Chess Tournament

Sample input 1 :

```
1
5 3
1 2 3 4 6
```

Sample output 1 :

```
2
```

Sample input 2 :

```
2
4 2
5 4 2 1
6 4
6 7 9 13 15 11
```

Sample output 2 :

4
2

Explanation for Sample Output 2:

In test case 1,
we only have to allocate rooms to 2 players so we can assign rooms that are first and last which are 1 and 5, so our answer is $5 - 1 = 4$.

In test case 2,
there is no way by which we can allocate rooms such that every player will have the 3 or more as its least distance to other players. So the answer is 2 and one possible allocation of rooms is as follows.

Player1 = 6
Player2 = 9
Player3 = 11
Player4 = 13

Solution:

```
import java.util.*;
public class Solution {

    public static int chessTournament(int[] positions, int n, int c) {
        int max = Integer.MIN_VALUE;
        Arrays.sort(positions);

        int minDistance = 1; // if positions are of same.
        int maxDistance = positions[n - 1] - positions[0];
        int len = positions.length;
        int cache[] = new int[len];
        Arrays.fill(cache, -1);
        // Runtime - N*log(n)
        // we need max distance so start from max if
        // max exist return no need to check less than that.

        // 1 2 3 4 6 //rooms
        //
        // if(n<c)return -1;
        int low = 0;
        int high = maxDistance;
        // search space is 0 to maxDistance.
        while (low <= high) {
            int mid = low + (high - low) / 2;
            // if high is possible return
            if (isPossible(positions, high, n, c)) {
                max = Math.max(max, high);
                return max;
            }
            // else if mid possible try to improve
        }
    }
}
```

```

        // by incrementing the players.
        // else mid -1.
        if (isPossible(positions, mid, n, c)) {
            max = Math.max(max, mid);
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return max;
}

// can assign maximum min distance between all players in n rooms
// to increase concentration of players
private static boolean isPossible(int[] positions, int distance, int
noOfRooms, int noOfChessPlayers) {
    int cur = positions[0];
    noOfChessPlayers--;
    for (int i = 1; i < noOfRooms; i++) {
        if (positions[i] - cur >= distance) {
            noOfChessPlayers--;
            if (noOfChessPlayers < 0)
                return false;
            cur = positions[i]; // update position
        }

        if (noOfChessPlayers == 0) {
            return true;
        }
    }
    return false;
}
}

```

Problem 67-I: 875. Koko Eating Bananas

Koko loves to eat bananas. There are n piles of bananas, the i^{th} pile has $\text{piles}[i]$ bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1:

Input: piles = [3,6,7,11], $h = 8$

Output: 4

Example 2:

Input: piles = [30,11,23,4,20], $h = 5$

Output: 30

Example 3:

Input: piles = [30,11,23,4,20], $h = 6$

Output: 23

Solution

```
class Solution {
    // Eating Bananas.
    // Memory Usage: 54.8 MB, less than 8.41% of Java online submissions
    for
        // Koko Eating Bananas.

    public int minEatingSpeed(int[] piles, int h) {

        // each 0 to h hours
        // hour0 - eats k bananas if pile[i] has less than k
        // wont eat further.

        // 3+6+7+11 = 26
        // 3 4
        // 8 *4 = 32

        int min = 0;
        int max = 0;
        int r = piles.length;

        for (int pile : piles) {
            min = Math.min(min, pile);
            max = Math.max(max, pile);
        }

        int low = 1;
        int high = 1000000000;

        // 3 +11 = 14 /2 = 7
        while (low <= high) {
            int mid = low + (high - low) / 2;
```

```
        // 1hr 7 bananas.
        if (isPossible(piles, mid, h)) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    return low;
}

private boolean isPossible(int piles[], int perHourtarget, int
remHours) {

    for (int curPile : piles) {
        remHours -= (curPile / perHourtarget);
        if (curPile % perHourtarget != 0)
            remHours--;
        if (remHours < 0)
            return false;
    }
    return true;
}
}
```

Problem 67-II: 1011. Capacity To Ship Packages Within D Days

A conveyor belt has packages that must be shipped from one port to another within days days.

The i^{th} package on the conveyor belt has a weight of $\text{weights}[i]$. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

Input: $\text{weights} = [1,2,3,4,5,6,7,8,9,10]$, $\text{days} = 5$

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5

2nd day: 6, 7

3rd day: 8

4th day: 9

5th day: 10

Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.

Solution

```
class Solution {  
    // Runtime: 12 ms, faster than 65.62% of Java online submissions for  
    Capacity To Ship Packages Within D Days.  
    // Memory Usage: 54.1 MB, less than 48.58% of Java online submissions for  
    Capacity To Ship Packages Within D Days.
```

```
    public int shipWithinDays(int[] weights, int days) {  
  
        // set search space.  
  
        int min = weights[0];  
        int total = 0;  
        int n = weights.length;  
  
        for (int weight : weights) {  
            total += weight;  
            min = Math.max(min, weight);  
        }  
  
        int max = total;  
  
        int low = min;  
        int high = max;  
  
        int minCapacity = max;  
  
        while (low < high) {  
            int mid = low + (high - low) / 2;  
            if (isPossible(weights, mid, days)) {  
                high = mid;  
            } else {  
                low = mid + 1;  
            }  
        }  
  
        return low;  
    }  
}
```

```
private boolean isPossible(int weights[], int maxCapacity, int days) {  
    int n = weights.length;  
  
    int curCapacity = 0;
```

```
    int availDays = days - 1;
    int result = 0;

    for (int i = 0; i < n; i++) {
        int cur = weights[i];
        if (cur + curCapacity <= maxCapacity) {
            curCapacity += cur;
        } else {
            result = Math.max(result, curCapacity);
            curCapacity = cur;
            if (curCapacity > maxCapacity) {
                return false;
            }
            availDays--;
            if (availDays < 0) {
                return false;
            }
        }
    }

    return true;
}
```

Day 12: HEAPS

Problem 68: MinHeap**Sample Input 1 :**

```
2
3
0 2
0 1
1
2
0 1
1
```

Sample Output 1 :

```
1
1
import java.util.*;
public class Solution {
    static int[] minHeap(int n, int[][] q) {
        // Write your code here.

        // 5
        // 0 5
        // 1
        // 0 43
        // 0 15
        // 0 5

        // 5,43,15,5,
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        List<Integer> list = new LinkedList<>();
        for (int cur[] : q) {
            int op = cur[0];
            if (op == 0) {
                pq.offer(cur[1]);
            } else {
                int temp = pq.poll();
                list.add(temp);
            }
        }

        int res[] = new int[list.size()];
        int count = 0;
        for (int val : list) {
            res[count++] = val;
        }
    }
}
```

```
        return res;
    }
}
```

Problem 69: 215. Kth Largest Element in an Array

Given an integer array *nums* and an integer *k*, return *the kth largest element in the array*.

Note that it is the *kth* largest element in the sorted order, not the *kth* distinct element.

Can you solve it without sorting?

Example 1:

Input: *nums* = [3,2,1,5,6,4], *k* = 2

Output: 5

Example 2:

Input: *nums* = [3,2,3,1,2,4,5,5,6], *k* = 4

Output: 4

Solution

```
import java.util.*;

public class Solution {
    public static ArrayList<Integer> kthSmallLarge(ArrayList<Integer> arr,
int n, int k) {
        PriorityQueue<Integer> min = new PriorityQueue<>();
        PriorityQueue<Integer> max = new PriorityQueue<>((a, b) -> (b -
a));

        for (int val : arr) {
            min.offer(val);
            max.offer(val);
        }
        int low = 0;
        int high = 0;
        // 4
        while (k-- > 0) {
            if (!min.isEmpty())
                low = min.poll();
            if (!max.isEmpty())
                high = max.poll();
        }
        ArrayList<Integer> result = new ArrayList<>();
        result.add(low);
    }
}
```

```
        result.add(high);  
        return result;  
    }  
}
```

Problem 70: K Max Sum Combinations

Sample Input 1 :

```
2  
3 2  
1 3 5  
6 4 2  
1 1  
3  
4
```

Sample Output 1 :

```
11 9  
7
```

Explanation of Sample Output 1 :

In test case 1, for the given arrays/lists, all the possible sum combinations are:

7(1 + 6), 5(1 + 4), 3(1 + 2), 9(3 + 6), 7(3 + 4), 5(3 + 2), 11(6 + 5), 9(5 + 4), 7(5 + 2).

The two maximum sum combinations from the above combinations are 11 and 9.

In test case 2, only one pair is possible with sum 7(3 + 4) from the given arrays/lists.

Sample Input 2 :

```
2  
2 2  
1 1  
1 1  
4 1  
1 2 3 4  
4 3 2 1
```

Sample Output 2 :

```
2 2  
8
```

Explanation of sample input 2 :

In test case 1, for the given arrays/lists, two possible sum combinations are : 2(1 + 1), 2(1 + 1).

In test case 2, for the given arrays/lists, one possible sum combination is: 8(4 + 4).

Solution

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.Objects;
import java.util.PriorityQueue;

class Pair {
    public Pair(int first, int second) {
        this.first = first;
        this.second = second;
    }

    int first;
    int second;

    @Override
    public boolean equals(Object o) {
        if (o == null) {
            return false;
        }
        if (!(o instanceof Pair)) {
            return false;
        }
        Pair obj = (Pair) o;
        return (first == obj.first && second == obj.second);
    }

    @Override
    public int hashCode() {
        return Objects.hash(first, second);
    }
}

class PairSum implements Comparable<PairSum> {
    public PairSum(int sum, int first, int second) {
        this.sum = sum;
        this.first = first;
        this.second = second;
    }

    int sum;
```

```
    int first;
    int second;

    @Override
    public int compareTo(PairSum o) {
        return Integer.compare(o.sum, sum);
    }
}

public class Solution {
    public static ArrayList<Integer> kMaxSumCombination(ArrayList<Integer> a,
        ArrayList<Integer> b, int n, int k) {
        // Sorting the arrays.
        Collections.sort(a);
        Collections.sort(b);
        // Using a max-heap.
        PriorityQueue<PairSum> maxHeap = new PriorityQueue<PairSum>();
        maxHeap.add(new PairSum((a.get(n - 1) + b.get(n - 1)), n - 1, n - 1));
        // Using a set.
        HashSet<Pair> mySet = new HashSet<Pair>();
        mySet.add(new Pair(n - 1, n - 1));
        // Output array to store the K max sum combinations.
        ArrayList<Integer> result = new ArrayList<Integer>();

        while (k > 0) {
            PairSum top = maxHeap.remove();

            int sum = top.sum;
            int x = top.first;
            int y = top.second;

            // Add the sum to the output array.
            result.add(sum);

            // Check if the indices (x-1, y) are present in the set.
            if (!mySet.contains(new Pair(x - 1, y)) && x > 0) {
                maxHeap.add(new PairSum((a.get(x - 1)
                    + b.get(y)), x - 1, y));
                mySet.add(new Pair(x - 1, y));
            }

            // Check if the indices (x, y-1) are present in the set.
            if (!mySet.contains(new Pair(x, y - 1)) && y > 0) {
                maxHeap.add(new PairSum(a.get(x)
                    + b.get(y - 1), x, y - 1));
                mySet.add(new Pair(x, y - 1));
            }
            k -= 1;
        }
        // Return the output array.
        return result;
    }
}
```



```

public static void main(String args[]) {
    Solution sol = new Solution();
    ArrayList<Integer> list1 = new ArrayList<>();
    list1.add(1);
    list1.add(3);
    list1.add(5);

    ArrayList<Integer> list2 = new ArrayList<>();
    list2.add(6);
    list2.add(4);
    list2.add(2);

    ArrayList<Integer> res = sol.kMaxSumCombination(list1, list2, 3, 2);
    // [11,9] = 7(1 + 6), 5(1 + 4), 3(1 + 2) | 9(3 + 6), 7(3 + 4), 5(3 + 2),
    // | 11(6 + 5), 9(5 + 4), 7(5 + 2).

    // The two maximum sum combinations from the above combinations
    // are 11 and 9.
    System.out.println("Solution.main()" + res);
}
}

```

Time Complexity: $O(N * \log(N))$

Space Complexity: $O(N)$

Problem 71: 295. Find Median from Data Stream

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input

```

["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]

```

Output

[null, null, null, 1.5, null, 2.0]

Explanation

```
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Solution:

```
import java.util.*;
import java.util.PriorityQueue;

public class Solution {
    public static void findMedian(int arr[]) {
        // 6
        // 6 2 1 3 7 5
        //
        int n = arr.length;
        PriorityQueue<Integer> min = new PriorityQueue<>();
        PriorityQueue<Integer> max = new PriorityQueue<>((a, b) -> (b -
a));

        for (int i = 0; i < n; i++) {
            int cur = arr[i];
            min.offer(cur);
            max.offer(min.poll());
            if (min.size() < max.size()) {
                min.offer(max.poll());
            }
            if (min.size() == max.size()) {
                int res = (min.peek() + max.peek()) / 2;
                System.out.print(res + " ");
                continue;
            }
            int res = min.peek();
            System.out.print(res + " ");
        }
    }
}
```

Problem 72: Merge K Sorted Arrays

Sample Input 1:

```
1
2
3
3 5 9
4
1 2 3 8
```

Sample Output 1:

```
1 2 3 3 5 8 9
```

Explanation of Sample Input 1:

After merging the two given arrays/lists [3, 5, 9] and [1, 2, 3, 8], the output sorted array will be [1, 2, 3, 3, 5, 8, 9].

Solution

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;
public class Solution {
    public static ArrayList<Integer>
mergeKSortedArrays(ArrayList<ArrayList<Integer>> kArrays, int k) {
    PriorityQueue<Node> pq = new PriorityQueue<>(k, (a, b) -> (a.val -
b.val));
    for (int i = 0; i < k; i++) {
        pq.offer(new Node(kArrays.get(i).get(0), i, 0));
    }
    ArrayList<Integer> result = new ArrayList<Integer>();

    while (!pq.isEmpty()) {
        Node cur = pq.poll();
        int row = cur.row;
        int col = cur.col;
        result.add(cur.val);

        int len = kArrays.get(row).size();
        if (col + 1 < len) {
            int val = kArrays.get(row).get(col + 1);
            pq.offer(new Node(val, row, col + 1));
        }
    }

    return result;
}
}

class Node {
    int val;
    int row;
```

```
int col;

Node(int val, int row, int col) {
    this.val = val;
    this.row = row;
    this.col = col;
}

}
```

Naive Approach:

iterate all arrays add to new Array and finally sort

Time Complexity: $O((N * K) * \log(N * K))$

Space Complexity: $O(N * K)$

Efficient Approach:

Time Complexity: $O((N * K) * \log(K))$ – size of input array

Space Complexity: $O(N * K)$ – Where N is the total number of elements in all the arrays, and K is the number of arrays.

Problem 73: 347. Top K Frequent Elements

347. Top K Frequent Elements

Medium

Given an integer array *nums* and an integer *k*, return *the k most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: *nums* = [1,1,1,2,2,3], *k* = 2

Output: [1,2]

Example 2:

Input: *nums* = [1], *k* = 1

Output: [1]

Solution

```
import java.util.*;

public class Solution {

    public static int[] KMostFrequent(int n, int k, int[] nums) {

        int bucket[] = new int[n];

        HashMap<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
    }
}
```

```
LinkedList[] adjList = new LinkedList[n + 1];

for (int key : map.keySet()) {
    int frequency = map.get(key);
    if (adjList[frequency] == null) {
        adjList[frequency] = new LinkedList<>();
    }
    adjList[frequency].add(key);
}

int res[] = new int[k];

int i = adjList.length - 1;

HashSet<Integer> st = new HashSet<>();

while (i >= 0 && list.size() != k) {
    LinkedList<Integer> cur = adjList[i];
    if (cur != null)
        list.add(cur);
    i--;
}

return list.stream().distinct().mapToInt(a -> a).toArray();
}
```

Efficient Approach:

Bucket sort-

Time Complexity: O(N)

Space Complexity: O(N)

Day 13: STACKS AND QUEUES

Problem 74:Maximum XOR With an Element From Array**Sample Input 1:**

```
2
5 2
0 1 2 3 4
1 3
5 6
1 1
1
1 0
```

Sample Output 1:

```
3 7
-1
```

Explanation of sample input 1:

In the first test case, the answer of query [1, 3] is 3 because $1^2 = 3$ and $2 \leq 3$, and the answer of query [5, 6] is 7 because $5^2 = 7$ and $2 \leq 6$.

In the second test case, no element is less than or equal to 0 in the given array 'ARR'.

Sample Input 2:

```
2
6 3
6 6 3 5 2 4
6 3
8 1
12 4
5 2
0 0 0 0 0
1 0
1 1
```

Sample Output 2:

```
5 -1 15
1 1
```

Solution

```
import java.util.ArrayList;

public class Solution {
    public static ArrayList<Integer> maxXorQueries(ArrayList<Integer> arr,
        ArrayList<ArrayList<Integer>> queries) {
        int n = arr.size();
        int qlen = queries.size();
```

```

        ArrayList<Integer> result = new ArrayList<>();
        for (int i = 0; i < qlen; i++) {
            result.add(-1);
        }

        // math.max(max,xi * { A[i] <= Ai} )
        for (int i = 0; i < qlen; i++) {
            int xi = queries.get(i).get(0);
            int Ai = queries.get(i).get(1);
            int max = -1;
            for (int j = 0; j < n; j++) { // arr
                if (arr.get(j) <= Ai) {
                    max = Math.max(max, xi ^ arr.get(j));
                }
            }
            result.set(i, max);
        }

        return result;
    }
}

```

Problem 75:Implement a Queue

```

1
7
1 17
1 23
1 11
2
2
2
2

```

Sample Output 1 :

```

17
23
11
-1

```


Explanation for Sample Output 1 :

The first three queries are of enqueue, so we will push 17, 23, and 11 into the queue.

The next four queries are of dequeue, so we will start removing elements from the queue, so the first element will be 17, then 23, and then 11. And after the third dequeue query, the queue is now empty so for the fourth query, we will return -1.

Solution
Double linked List:

```
public class Queue {
    singleLinkedList head = null;
    singleLinkedList tail = null;

    Queue() {
        // initially both head & tail is null;
    }

    /*----- Public Functions of Queue -----*/

    boolean isEmpty() {
        return head == null;
    }

    void enqueue(int data) {
        if (head == null) {
            head = tail;
        }
        if (tail == null) {
            // first element both head & tail are same.
            tail = new singleLinkedList(data);
            head = tail;
        } else {
            tail.next = new singleLinkedList(data);
            // only update if not new record.
            // else everytime it re-initialize tail to new object
            // old previous records will be cleared .
            tail = tail.next;
        }
    }
}
```

```
int dequeue() {
    if (head == null) {
        return -1;
    }

    singleLinkedList temp = head;
    if (temp == null) {
        return -1;
    }
    int res = temp.val;
    temp = temp.next;
    // update new head;
    // otherwise head remain isolated
    head = temp;

    // 1->2->3
    // | |
    // head tail

    // 2---->3
    // | |
    // head tail

    // ---->3
    // |
    // (head,tail)- same position.

    // 3(head,tail collide) - it is default referenced.

    // but if says all elements removed;

    // - ->3
    // | |
    // head tail(isolated invalid reference)
    // (null)

    // above head cleared to null

    // since we are working on head so if head is null.
    // need to sync tail to null else tail will still refer to
previous values.
    if (head == null) {
        tail = null;
    }

    // after remove check both head & tail are same.
    // if so sync.
```

```
        return res;
    }

    int front() {
        singleLinkedList temp = head;
        if (temp == null)
            return -1;
        return temp.val;
    }
}

class singleLinkedList {
    int val;
    singleLinkedList next;

    singleLinkedList(int val) {
        this.val = val;
    }
}
```

Arrays:

```
public class Queue {
    int arr[];
    int size;
    int front = 0;
    int tail = 0;

    Queue() {
        this.size = 0;
        this.arr = new int[5000];
    }

    /*----- Public Functions of Queue -----*/

    boolean isEmpty() {
        return size == 0;
    }

    void enqueue(int data) {
        // add element to queue
        // 1 2 3 4 5 <---- add(1,2,3,4,5)
        // ->2 3 4 5 (removed 1)

        // add at the end.
        arr[tail++] = data;
        size++;
    }
}
```

```
    }

    int dequeue() {
        if (isEmpty())
            return -1;
        int res = arr[front];
        // in a queue 1 records deque so move the front to next record
        front = front + 1;
        size--; // decrement used only for is empty.
        if (front == tail) {
            // after removing front == tail sync it say all removed.
            front = 0;
            tail = 0;
        }
        return res;
    }

    int front() {
        if (isEmpty())
            return -1;
        return arr[front];
    }
}
```

Problem 76: 225. Implement Stack using Queues

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- void push(int x) Pushes element x to the top of the stack.
- int pop() Removes the element on the top of the stack and returns it.
- int top() Returns the element on the top of the stack.
- boolean empty() Returns true if the stack is empty, false otherwise.

Notes:

- You must use **only** standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:**Input**

```
["MyStack", "push", "push", "top", "pop", "empty"]  
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 2, 2, false]
```

Explanation

```
MyStack myStack = new MyStack();  
myStack.push(1);  
myStack.push(2);  
myStack.top(); // return 2  
myStack.pop(); // return 2  
myStack.empty(); // return False
```

Solution

```
class MyStack {  
    // Runtime: 0 ms, faster than 100.00% of Java online submissions for  
    // Implement Stack using Queues.  
    // Memory Usage: 41.9 MB, less than 37.00% of Java online submissions for  
    // Implement Stack using Queues.
```

```
    LinkedList<Integer> queue;  
  
    public MyStack() {  
        queue = new LinkedList<>();  
    }  
  
    public void push(int x) {  
        // LILO -> LIFO  
  
        // 1  
        // 2  
        // 3
```

```
// 4

// 4
// 3 -> more than 1

// 3
// 4

// after all element insert example
// 1 -insert 1
// 2 -insert 2
// 3 -insert 3
// 4 - last n-1

// 4
// 3
// 2
// 1

queue.offer(x);

int n = queue.size();

// for(int i =0;i<n-1;i++) this also works just poll 1 less so
that last element
// becomes first.
// so that lastly inserted stay at the first.
// 1->2->3->(4 inserted)
// iterate from 1 to 3.
// 4->1->2->3
for (int i = 1; i < n; i++) {
    queue.offer(queue.poll());
}

}

public int pop() {
    if (empty())
        return -1;

    return queue.poll();
}

public int top() {
```

```
        if (empty())
            return -1;
        return queue.peek();
    }

    public boolean empty() {
        return queue.isEmpty();
    }
}
```

Problem 77: 232. Implement Queue using Stacks

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false
```

Solution

```
class MyQueue {
    Stack<Integer> input;
    Stack<Integer> output;

    public MyQueue() {
        input = new Stack<>();
        output = new Stack<>();
    }

    public void push(int x) {
        input.push(x);
    }

    public int pop() {
        if (!output.isEmpty())
            return output.pop();
        shuffle();

        return output.pop();
    }

    public int peek() {
        if (!output.isEmpty()) {
            return output.peek();
        }

        shuffle();
        return output.peek();
    }

    private void shuffle() {
        while (!input.isEmpty()) {
            output.push(input.pop());
        }
    }
}
```



```
        public boolean empty() {
            // initially we push and check for empty - then only data exist in
input.
            // if we push and peek() -all data in output. no data in input.
so that
            // time we have only output has data and input is null.
            return input.isEmpty() && output.isEmpty();
        }
    }

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = new MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * boolean param_4 = obj.empty();
 */
```

Problem 78: 20. Valid Parentheses

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "([]"

Output: false

Constraints:

- $1 \leq s.length \leq 10^4$
- *s* consists of parentheses only '()[]{}'

Solution

```
class Solution {
// Runtime: 3 ms, faster than 58.69% of Java online submissions for Valid
// Parentheses.
// Memory Usage: 42 MB, less than 58.77% of Java online submissions for Valid
// Parentheses.

    public boolean isValid(String s) {

        HashMap<Character, Character> map = new HashMap<>();

        map.put('(', ')');
        map.put('{', '}');
        map.put('[', ']');

        Stack<Character> st = new Stack<>();
        String strOpen = "{[(";

        for (char ch : s.toCharArray()) {

            if (strOpen.indexOf(ch) != -1) {
                st.push(ch);
            } else {
                if (st.isEmpty())
                    return false;
                char top = st.pop();
                if (ch != map.get(top)) {
                    return false;
                }
            }
        }

        if (!st.isEmpty())
            return false;

        return true;
    }
}
```

Problem:78-I. 1249. Minimum Remove to Make Valid Parentheses

Given a string s of '(', ')', and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting *parentheses string* is valid and return **any** valid string.

Formally, a *parentheses string* is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

Example 1:

Input: s = "lee(t(c)o)de)"

Output: "lee(t(c)o)de"

Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.

Example 2:

Input: s = "a)b(c)d"

Output: "ab(c)d"

Example 3:

Input: s = ")))((("

Output: ""

Explanation: An empty string is also valid.

Constraints:

- $1 \leq s.length \leq 10^5$
- s[i] is either '(', ')', or lowercase English letter.

Solution:

```
import java.util.Stack;
```

```
class Solution {
```

```
// Runtime: 9 ms, faster than 94.21% of Java online submissions for Minimum  
Remove to Make Valid Parentheses.
```

```
// Memory Usage: 39.5 MB, less than 86.47% of Java online submissions for  
Minimum Remove to Make Valid Parentheses.
```

```
    public String minRemoveToMakeValid(String s) {
```

```

//lee(t(c)o)de)
Stack<Character> st = new Stack<>();
char arr[] = s.toCharArray();
int n = s.length()-1;
int open = 0;
int close = 0;

int i = 0;

while(i<=n){
    if(arr[i]==')'){
        close++;
    }
    i++;
}

StringBuffer buffer = new StringBuffer();
i = -1;

while(++i<=n){
    char ch = arr[i];
    if(ch == '('){
        if(open== close)continue;
        open++;
    }else if(ch == '){
        close--;
        if(open == 0)continue;
        open--;
    }
    buffer.append(ch);
}

return buffer.toString();
}
}

```

Problem 79: 496. Next Greater Element I

The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array *ans* of length `nums1.length` such that `ans[i]` is the **next greater element** as described above.

Example 1:

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is -1.
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is -1.

Solution

```
class Solution {  
    // Runtime: 5 ms, faster than 68.98% of Java online submissions for Next  
    // Greater Element I.  
    // Memory Usage: 43.5 MB, less than 80.14% of Java online submissions for  
    // Next Greater Element I.
```

```
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {  
  
        int n2 = nums2.length;  
  
        Stack<Integer> st = new Stack<Integer>();  
  
        // step 1: add to stack all next greater element.  
        // step 2: if any element < cur exist remove all elements  
        // from current index to last index , remove from stack.  
        // step 3: finally add cur to stack to check whether it is next  
        // greater to next element reverse in index.  
  
        // 1 3 4 2  
        // -1  
  
        //  
        // 1 // 3  
        // 3 // 4  
        // 4 // -1  
        // 2 // -1  
  
        // st //next greater  
        // need to find greater in reverse order.  
        // since no wrap i.e for n no next greater element.
```

```

// and for 0 next greater element can be from 1 to n.

// if we go from 0 to n we need to find for 0 1 time
// 1 one more it is like n^2 time.

// if we traverse n-1 to 0 just 1 time.
int high[] = new int[n2];

for (int i = n2 - 1; i >= 0; i--) {
    int val = nums2[i];
    while (!st.isEmpty() && st.peek() < val) {
        // remove from stack elements <val.
        st.pop();
    }

    // add cur element to stack.
    if (!st.isEmpty()) {
        high[i] = st.peek();
    } else {
        high[i] = -1;
    }
    // add element current to stack to
    // make sure it can be a next greater
    // element for next coming index.
    st.push(val);
}
// System.out.println(Arrays.toString(high));

Map<Integer, Integer> indexMap = new HashMap<>();
int i = 0;
for (int val : nums2) {
    indexMap.put(val, i++);
}
int res[] = new int[nums1.length];
i = 0;
for (int val : nums1) {
    int index = indexMap.get(val);
    res[i++] = high[index];
}

return res;
}
}

```

Problem 80: Sort a Stack

1
5

5 -2 9 -7 3

Sample Output 1:

9 5 3 -2 -7

Explanation of Sample Input 1:

9 Is the largest element, hence it's present at the top. Similarly 5>3, 3>-2 and -7 being the smallest element is present at the last.

Sample Input 2:

1

5

-3 14 18 -5 30

Sample Output 2:

30 18 14 -3 -5

Explanation of Sample Input 2:

30 is the largest element, hence it's present at the top. Similarly, 18>14, 14>-3 and -5 being the smallest element is present at the last.

Solution:Recursive

```
import java.util.*;
```

```
public class Solution {
```

```
    public static void sortStack(Stack<Integer> st) {
```

```
        if (st.isEmpty())  
            return;
```

```
        helper(st);
```

```
    }
```

```
    // 3,-7,9,-2,5
```

```
    // [3,-7,9,-2,5]
```

```
    // /
```

```
    // 3-helper([-7,9,-2,5])
```

```
    // /
```

```
    // -7 -helper([9,-2,5])
```

```
    // /
```

```
    // 9 -helper([-2,5])
```

```
    // /
```

```
    // -2 -helper([5]) - [5]
```

```
    // /
```

```
    // 5- sortInStack([],5)-return [5]
```

```
    //
```

```
    // separate flow
```

```

// sortInStack([],5) ->[5]
// /
// sortInStack([5],-2) ->[5]
// /
// sortInStack([5],-2) ->[5] -> sortInStack([],-2)->sortInStack([-2,5])
//
//
//
//
//

```

```

private static void helper(Stack<Integer> st) {
    // base case
    if (st.isEmpty())
        return;
    // hold the previous val in this variable.
    int cur = st.pop();
    helper(st);
    sortInStack(st, cur);
}

```

```

private static void sortInStack(Stack<Integer> st, int cur) {
    // base cases

    if (st.isEmpty() || (!st.isEmpty() && cur > st.peek())) {
        st.push(cur);
        return;
    }

    // if cur not greater than currently in stack.

    // 1<-- incoming
    // 4
    // 3
    // 2
    int temp = st.pop();// 4
    // do till elements < 4 like 3 gets add to stack
    sortInStack(st, cur);
    st.push(temp);// now add 4
}

```

```

public static void main(String args[]) {
    Stack<Integer> st = new Stack<>();
    int arr[] = { 5, -2, 9, -7, 3 };
    for (int val : arr) {

```



```

        st.push(val);
    }
    sortStack(st);
    printStack(st);
}

private static void printStack(Stack<Integer> st) {
    while (!st.isEmpty()) {
        System.out.println(st.pop());
    }
}
}

```

Solution:iterative

```

import java.util.*;
public class Solution {

    public static void sortStack(Stack<Integer> input) {

        Stack<Integer> output = new Stack<Integer>();

        //5 -2 9 -7 3
        //3 -7 9 -2 5//order of flowing.
        while(!input.isEmpty()){
            int cur = input.pop();

            while( (!output.isEmpty() && output.peek()> cur)){
                input.push(output.pop());
            }
            output.push(cur);//[3]

            //1:input => [-7,9,-2,5] removed 3 | output =>[3]
            //2:input=>[9,-2,5] removed -7 | output =>[3]
            //since output has greater > -7
            // add back to input becomes
            //final: input=> [3,9,-2,5] | output[-7]
            //3:input=>[9,-2,5] removed 3 | output=>[-7]
            //since 3 >-7 add to output
            //final : input=>[9,-2,5] | output=>[3,-7]
        }

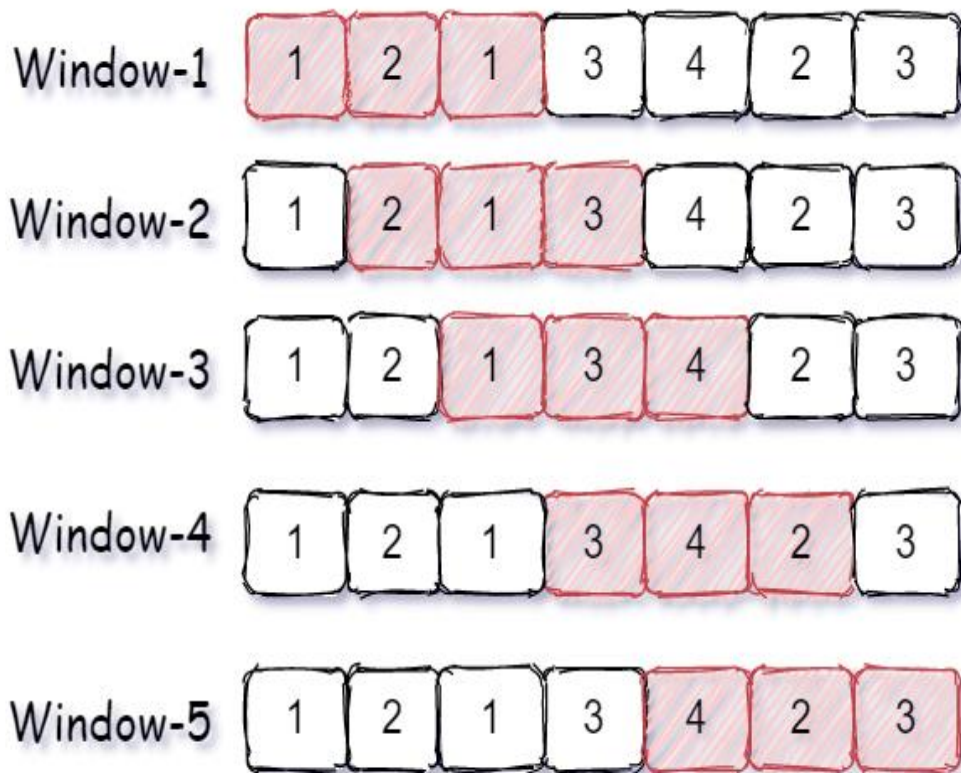
        //finally set output to input;
    }
}

```

```
Stack<Integer> temp = new Stack<Integer>();  
while(!output.isEmpty()){  
    temp.push(output.pop());  
}  
while(!temp.isEmpty()){  
    input.push(temp.pop());  
}  
}
```

Problem 81-Count Distinct Element in Every K Size Window

$K = 3$



```
7 4
1 2 1 3 4 2 3
5 3
1 1 2 1 3
```

Sample Output 1:

```
3 4 4 3
2 2 3
```

Explanation of sample input 1:

Test Case 1:

Window-1 has four elements { 1, 2, 1, 3 } and only three elements { 1, 2, 3 } are distinct because 1 is repeating two times.

Window-2 has four elements { 2, 1, 3, 4 } and all four elements { 2, 1, 3, 4 } are distinct.

Window-3 has four element { 1, 3, 4, 2 } and all four elements { 1, 3, 4, 2 } are distinct.

Window-4 has four element { 3, 4, 2, 3 } and only three elements { 3, 4, 2 } are distinct because 3 is repeating two times.

Hence, the count of distinct elements in all windows is { 3, 4, 4, 3}.

Test case 2:

Window-1 has three elements { 1, 1, 2 } and only two elements { 1, 2 } are distinct because 1 is repeating two times.

Window-2 has three elements { 1, 2, 1 } and only two elements { 2, 1 } are distinct.

Window-3 has three elements { 2, 1, 3 } and all three elements { 2, 1, 3 } are distinct.

Hence, the count of distinct elements in all windows is { 2, 2, 3 }.

Solution

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;

public class Solution {

    public static ArrayList<Integer>
countDistinctElements(ArrayList<Integer> arr, int k) {
    ArrayList<Integer> ans = new ArrayList<>();
    HashMap<Integer, Integer> map = new HashMap<>();
    int n = arr.size();
    int j = 0, i = 0;
    while (j < n) {
        int cur = arr.get(j);
        map.put(cur, map.getOrDefault(cur, 0) + 1);
        if (j - i + 1 == k) {
```

```
        int next = arr.get(i);
        ans.add(map.size());
        map.put(next, map.get(next) - 1);
        if (map.get(next) == 0)
            map.remove(next);
        i++;
    }
    j++;
}
return ans;
}
}
```

Day 14: STACKS AND QUEUE-II

Problem 82- Next Smaller Element

Next Smaller Element

For Example

Input 1:

A = [4, 5, 2, 10, 8]

Output 1:

G = [-1, 4, -1, 2, 2]

Explanation 1:

index 1: No element less than 4 in left of 4, G[1] = -1

index 2: A[1] is only element less than A[2], G[2] = A[1]

index 3: No element less than 2 in left of 2, G[3] = -1

index 4: A[3] is nearest element which is less than A[4], G[4] = A[3]

index 4: A[3] is nearest element which is less than A[5], G[5] = A[3]

Input 2:

A = [3, 2, 1]

Output 2:

[-1, -1, -1]

Explanation 2:

index 1: No element less than 3 in left of 3, G[1] = -1

index 2: No element less than 2 in left of 2, G[2] = -1

index 3: No element less than 1 in left of 1, G[3] = -1

Solution

```

public class Solution {
    public ArrayList<Integer> prevSmaller(ArrayList<Integer> A) {
        ArrayList<Integer> result = new ArrayList<Integer>();

        return helper(A);
    }

    private ArrayList<Integer> helper(ArrayList<Integer> list) {
        Stack<Integer> st = new Stack<>();
        int n = list.size();
        ArrayList<Integer> smallerList = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int cur = list.get(i);
            while (!st.isEmpty() && st.peek() >= cur) {
                st.pop();
            }

            if (!st.isEmpty()) {
                smallerList.add(st.peek());
            } else {

```

```

        smallerList.add(-1);
    }
    st.push(cur);
}
return smallerList;
}
}

```

Problem 83-I:146. LRU Cache

Design a data structure that follows the constraints of a [Least Recently Used \(LRU\) cache](#).

Implement the LRUcache class:

- LRUcache(int capacity) Initialize the LRU cache with **positive** size capacity.
- int get(int key) Return the value of the key if the key exists, otherwise return -1.
- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, **evict** the least recently used key.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```

LRUCache lruCache = new LRUCache(2);
lruCache.put(1, 1); // cache is {1=1}
lruCache.put(2, 2); // cache is {1=1, 2=2}
lruCache.get(1);    // return 1
lruCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lruCache.get(2);    // returns -1 (not found)
lruCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}

```

```
lRUCache.get(1);    // return -1 (not found)
lRUCache.get(3);    // return 3
lRUCache.get(4);    // return 4
```

Solution

```
class LRUCache {
// Runtime: 12 ms, faster than 97.93% of Java online submissions for LRU Cache.
// Memory Usage: 47.3 MB, less than 56.24% of Java online submissions for LRU Cache.
    Map<Integer,DNode> map = new HashMap<>();
    int capacity;
    DNode head;
    DNode tail;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.head=this.tail =null;
    }
    public int get(int key) {
        if(map.get(key)==null)return -1;
        DNode cur = map.get(key);
        removeFromFront(cur);
        addToLast(cur);

        return cur.value;
    }

    public void put(int key, int value) {
        if(map.get(key)!=null){
            DNode cur = map.get(key);
            cur.value = value;
            removeFromFront(cur);
            addToLast(cur);

            return;
        }
        //if node not exists and exceeds capacity.

        if(map.size())>=this.capacity){
            //since head is oldest accessed key remove it.
            map.remove(this.head.key);
            removeFromFront(this.head);
        }

        //common add to Last i.e if exceeds or not exceeds need to add to Last
    }
}
```

```

        //to make it as very recent accessed element.

        DNode cur = new DNode(key,value);
        //update the map
        map.put(key, cur);
        addToLast(cur);
    }

    //Two utility methods

    //
    //      ++++++      ++++++      ++++++
    // ---> +   + -----> +   + -----> +   + --->
    // <--- +   + <----- +   + <----- +   + <---
    //      ++++++      ++++++      ++++++
    //      HEAD          CURRENT          TAIL

    //
    //      ++++++
    // ---> +   + ----->
    // <--- +   + <-----
    //      ++++++
    //      To ADD

    //1.tail is not null

    //
    //      ++++++      ++++++      ++++++      ++++++
    // ---> +   + -----> +   + -----> +   + ---> +   + --->
    // <--- +   + <----- +   + <----- +   + <--- +   + <---
    //      ++++++      ++++++      ++++++      ++++++
    //      HEAD          CURRENT          PREV TAIL      (CUR NODE -NEW
TAIL)

    //1.tail is null
    //
    //      ++++++
    // ---> +   + ----->
    // <--- +   + <-----
    //      ++++++
    //      (CUR NODE -NEW TAIL)

    public void addToLast(DNode cur){
        if(this.tail != null){
            this.tail.next = cur;
        }
    }

```

```

//if tail is null, then make cur as tail.
cur.prev = tail;
cur.next = null;
this.tail = cur;

//if head is null, then it is first node to add
//then assign both head & tail to cur node.

if(head == null)head= tail;
}

```

```

//          ++++++          ++++++          ++++++
// ---> +    + -----> +    + -----> +    + --->
// <--- +    + <----- +    + <----- +    + <---
//          ++++++          ++++++          ++++++
//          HEAD           CURRENT          TAIL

```

//1. if cur node to add has prev node

```

//          ++++++          ++++++          ++++++
// ---> +    + -----> +    + -----> +    + --->
// <--- +    + <----- +    + <----- +    + <---
//          ++++++          ++++++          ++++++
//          HEAD           CURRENT          TAIL

```

// (TO REMOVE)

//After removing current Node

```

//          ++++++          ++++++
// ---> +    + -----> +    + --->
// <--- +    + <----- +    + <---
//          ++++++          ++++++
//          HEAD           TAIL

```

//2. if cur node to add is only node.

```

//          ++++++
// ---> +    + ----->
// <--- +    + <-----
//          ++++++
//          CURRENT(MAKE AS TAIL)
//

```

```
public void removeFromFront(DNode cur){

    //Step 1: handle prev reference from prev node.
    //if prev is not null then prev.next to cur.next then (cur) got removed.
    if(cur.prev!=null){
        cur.prev.next = cur.next;
        //NOTE: prev reference does not matter since we remove it
        //automatically garbage collected.
    }else{
        //if no previous node then make cur.next as head since
        //we remove (cur);
        this.head = cur.next;
    }

    //Step 2: handle next reference from cur node.
    if(cur.next!=null){
        cur.next.prev = cur.prev;
    }else{
        //if only one node exist then make it as tail node.
        //always assign partial object so that whole reference wont change.
        this.tail = cur.prev;
    }
}

}

class DNode{

    DNode prev;
    DNode next;

    int key,value;
    DNode(int key, int value){
        this.key = key;
        this.value = value;
    }
}
```

Solution 2- object oriented

DLLHelper utility same for both LRU and LFU – only difference frequency is Maintain count for each Node frequency of visited pages.

```
class LRUCache{

    Map<Integer,DNode> map = new HashMap<>();
    int capacity ;
    DLLHelper helper;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        helper = new DLLHelper();
    }
}
```

```
public int get(int key) {
    DNode cur = map.get(key);
    if(cur==null){
        return -1;
    }
    //update the LRU or accessed item by
    //remove from front and add to end.

    helper.removeFromFront(cur);
    helper.addToTail(cur);

    return cur.val;
}

public void put(int key, int value) {
    DNode cur = map.get(key);
    if(cur!=null){
        cur.val=value;
        helper.removeFromFront(cur);
        helper.addToTail(cur);

        return ;
    }

    //exceeds capacity remove one element from head.
    if(map.size()>=this.capacity){
        map.remove(this.helper.head.key);
        helper.removeFromFront(this.helper.head);
    }
    cur = new DNode(key,value);
    map.put(key,cur);
    helper.addToTail(cur);
}
}
```

```

class DLLHelper{
    DNode head;
    DNode tail;

    DLLHelper(){

    }
    //add element to Tail
    public void addToTail(DNode cur){
        //1->2->3
        //      (add)

        if(this.tail!=null){
            this.tail.next = cur;
        }
        //set cur node prev to previous tail (last node)
        cur.prev = this.tail;
        cur.next = null; //important to set next node to null.
        this.tail = cur; //set cur as tail node.
        //if add node is first node
        //since null object cannot refer and track other object
        //sync first and point both (head==>tail=cur)
        if(head==null){
            this.head = this.tail;
        }
    }

    public void removeFromFront(DNode cur){
        //1->2->3
        //  X(remove)
        //if node to remove is in middle.

        //to remove middle node
        //1. first remove prev ref and next reference to cur node(to be removed)
        if(cur.prev!=null){
            cur.prev.next = cur.next;
        }else{
            this.head = cur.next;
        }

        if(cur.next!=null){
            cur.next.prev = cur.prev;
        }else{
            this.tail = cur.prev;
        }

        //if suppose we remove only one existing node.
        //sync tail to null.
        if(this.head==null){
            this.tail = null;
        }
    }
}

```

```
class DNode{

    DNode prev=null;
    DNode next=null;
    int key;
    int val;

    DNode(int key, int val){
        this.key = key;
        this.val = val;
    }

}

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
```

Problem 83-II : 460. LFU Cache

460. LFU Cache

Hard

Design and implement a data structure for a [Least Frequently Used \(LFU\)](#) cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the **least recently used** key would be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest **use counter** is the least frequently used key.

When a key is first inserted into the cache, its **use counter** is set to 1 (due to the put operation). The **use counter** for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get",  
"get"]
```

```
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]
```

Explanation

```
// cnt(x) = the use counter for key x  
// cache=[] will show the last used order for tiebreakers (leftmost element  
is most recent)  
LFUCache lfu = new LFUCache(2);  
lfu.put(1, 1); // cache=[1,_], cnt(1)=1  
lfu.put(2, 2); // cache=[2,1], cnt(2)=1, cnt(1)=1  
lfu.get(1);    // return 1  
               // cache=[1,2], cnt(2)=1, cnt(1)=2  
lfu.put(3, 3); // 2 is the LFU key because cnt(2)=1 is the smallest,  
invalidate 2.  
               // cache=[3,1], cnt(3)=1, cnt(1)=2  
lfu.get(2);    // return -1 (not found)  
lfu.get(3);    // return 3  
               // cache=[3,1], cnt(3)=2, cnt(1)=2  
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU, invalidate  
1.  
               // cache=[4,3], cnt(4)=1, cnt(3)=2  
lfu.get(1);    // return -1 (not found)  
lfu.get(3);    // return 3  
               // cache=[3,4], cnt(4)=1, cnt(3)=3  
lfu.get(4);    // return 4  
               // cache=[4,3], cnt(4)=2, cnt(3)=3
```

Constraints:

- $1 \leq \text{capacity} \leq 10^4$

- $0 \leq \text{key} \leq 10^5$
- $0 \leq \text{value} \leq 10^9$
- At most $2 * 10^5$ calls will be made to get and put.

Solution:

```
import java.util.HashMap;
import java.util.LinkedHashSet;
import java.util.Set;

class LFUCache {

    int capacity = 0;
    HashMap<Integer, Node> map;

    FrequencyHelper frequencyHelper;
    DLHelper dlhelper;

    //capacity put      put      get      put      get      get      put      get      get      get
    // [[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
    // [null, null, null, 1, null, -1, 3, null, -1, 3, 4]

    // frequency map - [{1 -3},{2-2},{3-3},{4 - 2}]
    // map [ {1,1},{2,2},{3,3},{4,4}] ,

    //map          - > stores key value.
    //countmap      -> stores {key, frequency} - update during both put & set operation
    //freqMap       -> stores {frequency, [list of key in that frequency]}

    //min frequency to look & remove.
    //min          -> minfrequency to remove

    public LFUCache(int capacity) {
        this.map = new HashMap<>();
        this.capacity = capacity;
        this.frequencyHelper = new FrequencyHelper();
        this.dlhelper = new DLHelper();
    }
}
```



```
public int get(int key) {
    Node cur = map.get(key);
    if (cur == null) {
        return -1;
    }
    this.frequencyHelper.updateFrequency(map, key);
    this.dlhelper.removeFromFront(cur);
    this.dlhelper.addToTail(cur);

    return map.get(key).val;
}

public void put(int key, int value) {
    if (this.capacity <= 0)
        return;
    Node cur = map.get(key);
    if (cur != null) {
        cur.val = value;
        this.frequencyHelper.updateFrequency(map, key);
        this.dlhelper.removeFromFront(cur);
        this.dlhelper.addToTail(cur);
        return;
    }
    if (map.size() >= this.capacity) {
        this.frequencyHelper.removeFrequency(map);
        this.dlhelper.removeFromFront(this.dlhelper.head);
    }
    this.frequencyHelper.addFrequency(map, key);
    cur = new Node(key, value, 1);
    this.dlhelper.addToTail(cur);
    map.put(key, cur);
}
```

```

    public static void main(String args[]) {
        // ["LFUCache", "put", "put", "get", "put", "get", "get", "put",
"get",
        // "get", "get"]
        // [[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3],
[4]]

        LFUCache sol = new LFUCache(2);
        System.out.println("LFUCache.main():Initial:null");
        sol.put(1, 1);
        System.out.println("LFUCache.main():null");
        sol.put(2, 2);
        System.out.println("LFUCache.main():null");
        System.out.println("sol.get:" + sol.get(1)); // 1[2],2[1]
        sol.put(3, 3); // 1[2],3[1]
        System.out.println("LFUCache.main():null");
        System.out.println("sol.get 2:" + sol.get(2)); // -1
        System.out.println("sol.get 3:" + sol.get(3)); // 1[2],3[2]
        sol.put(4, 4);
        ;// 3[2],4[4]
        System.out.println("LFUCache.main():null");
        System.out.println("sol.get 4:" + sol.get(1));
        System.out.println("sol.get 5:" + sol.get(3));
        System.out.println("sol.get 6:" + sol.get(4));

    }
}

/**
 *
 * Frequency related operation.
 *
 */
class FrequencyHelper {

    HashMap<Integer, Set<Integer>> freqMap;
    //maintain global minCount Frequency maintains range of
    //frequency (min,max)
    int minCountFrequency = 1;

    FrequencyHelper() {
        freqMap = new HashMap<>();
        freqMap.put(1, new LinkedHashSet<>());
    }
}

```

```
public void addFrequency(HashMap<Integer, Node> map, int key) {
    //new node first time visited - update global min to 1
    minCountFrequency = 1;
    freqMap.get(minCountFrequency).add(key);
}

public void removeFrequency(HashMap<Integer, Node> map) {
    int remove_index =
freqMap.get(minCountFrequency).iterator().next();
    freqMap.get(minCountFrequency).remove(remove_index);
    map.remove(remove_index);
}

public void updateFrequency(HashMap<Integer, Node> map, int key) {
    Node cur = map.get(key);
    int count = cur.count;
    freqMap.get(count).remove(key);
    //if count = global min and all records in global min range
    // is zero ,means no node with minfrequency exists,hence update
    //to global min to next as min Range.
    if (count == minCountFrequency && freqMap.get(count).size() == 0)
        minCountFrequency++;

    // update to next frequency.
    freqMap.putIfAbsent(count + 1, new LinkedHashSet<>());
    freqMap.get(count + 1).add(key);
    cur.count++;
}

}

class Node {
    int key, val, count;
    Node next;
    Node prev;

    Node(int key, int val, int count) {
        this.key = key;
        this.val = val;
        this.count = count;
    }
}
```

```
/**
 *
 * Double linked list helper - same as LRU Cache
 *
 */
class DLHelper {
    Node head;
    Node tail;

    DLHelper() {

    }

    public void addToTail(Node cur) {
        // 1->2->3
        // (add)

        if (this.tail != null) {
            this.tail.next = cur;
        }
        // set cur node prev to previous tail (last node)
        cur.prev = this.tail;
        cur.next = null; // important to set next node to null.
        this.tail = cur; // set cur as tail node.

        // if add node is first node
        // since null object cannot refer and track other object
        // sync first and point both (head==>tail=cur)
        if (head == null) {
            this.head = this.tail;
        }
    }

    public void removeFromFront(Node cur) {
        // 1->2->3
        // X(remove)
        // if node to remove is in middle.

        // to remove middle node
        // 1. first remove prev ref and next reference to cur node(to be
        // removed)
        if (cur.prev != null) {
            cur.prev.next = cur.next;
        } else {
            this.head = cur.next;
        }

        if (cur.next != null) {
            cur.next.prev = cur.prev;
        }
    }
}
```

```

    } else {
        this.tail = cur.prev;
    }

    // if suppose we remove only one existing node.
    // sync tail to null.
    if (this.head == null) {
        this.tail = null;
    }
}
}

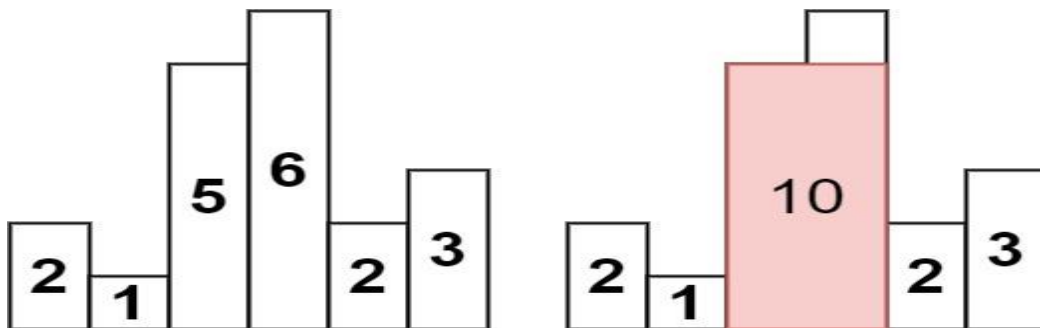
```

Problem 84. 84. Largest Rectangle in Histogram

Hard

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram.*

Example 1:

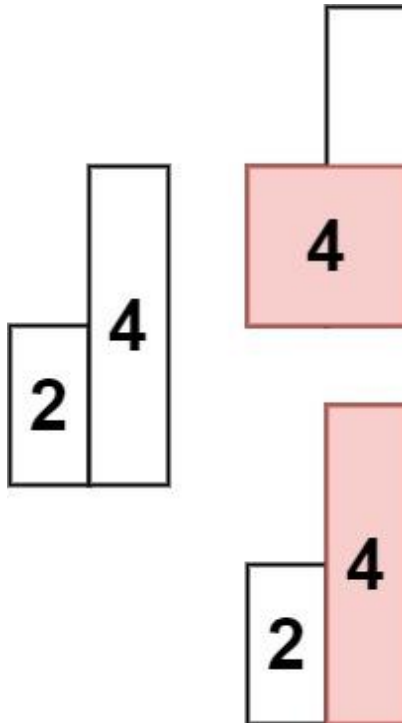


Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1. The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 10^5$
- $0 \leq \text{heights}[i] \leq 10^4$

Solution

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        int n = heights.length;
        // make sure add elements >= cur
        Stack<Integer> st = new Stack<>();
        int maxArea = 0;
        int i = 0;
        while (i < n) {
            int cur = heights[i];
            if (!st.isEmpty() && cur < heights[st.peek()]) {
```

```

        // for 1,5,6 -> calculate for 1to 6 min is (1)*3 =3
        // then poll(1) then left 5,6
        maxArea = Math.max(maxArea, getArea(heights, st, i));
        continue;
    }
    st.push(i++);
}
while (!st.isEmpty()) {
    maxArea = Math.max(maxArea, getArea(heights, st, i));
}

return maxArea;
}

private int getArea(int heights[], Stack<Integer> st, int endIndex) {
    int startIndex = st.pop();
    // since condition cur>previous violated at endIndex,
    // so 1 index less

    // | |
    // | | |
    // |1|5|6|2
    // |-|-|-|-
    // |1|2|3|4

    // width = (endIndex-1)- (startIndex)= (4-1)-0 =

    // when calculate 1 to 2 max width since 2 reducing.
    // so width will be
    // since for e.g [2,1,5,6,2,3]
    // 2 and 1 cur> prev violates so only 1 added.
    // start =0 and end =0 but 1 rectangle is there
    // width should be 1 not 0.

    int width = st.isEmpty() ? endIndex : (endIndex - 1 - st.peek());

    return width * heights[startIndex];
}

}

class Node {

    int x, y, height;

    Node(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

```

    }
}

```

Problem 85. 239. Sliding Window Maximum

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window*.

Example 1:

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

Window position	Max
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $1 \leq k \leq \text{nums.length}$

Sling window maximum Formula: $(n-k)+1 = 8-3+1 = 5$

First Sling Window : $k-1$

Window for I'th index : $(i-k)+1$ i.e First $\Rightarrow (2-3)+1 = 0$
 Second $\Rightarrow (3-3)+1 = 1$
 Third $\Rightarrow (4-3)+1 = 2$

Solution

```

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        //0  1  2 3  4 5  6  7
        //1  3  -1 -3 5 3  6  7
        //|      |

```



```
//      |       |  
//          |   |  
//              | |  
//                  | |  
//                      |  
//                          <-window-->  
int n = nums.length;  
//n-k+1 -> if you see above window you  
// have last window at 5th index which is  
//n=8 => n-k= 5 since array start at 0 n-k+1.  
int res[] = new int[n-k+1];  
//similary to get the total sliding window size for any num  
//i=3 , window size = ; i-k+1 =3-3+1 //total window 1  
//i=n , windows size= k ; n+k  
int window = 0;  
LinkedList<Integer> deque = new LinkedList<>();  
for(int i =0;i<n;i++){  
    //remove all index less than current index.  
    //if i=3 , we dont need index 0  
    //if i=4 ,we dont need index1  
    //if i=5 , we dont need index2  
    while(!deque.isEmpty() && deque.peek()<(i-k)+1){  
        deque.poll();  
    }  
    //maintain decreasing order so that max at the first index  
    // [1,3,-1,-3,5,3,6,7]  
    //  0 1 2   3 4 5 6 7  
    //  | | |  
  
    // [1][added]  
    // [3] //added removed 1  
    // [3,-1] //added window = 1,3  
    // [3,-1,-3]//added  
    while(!deque.isEmpty() && nums[i]> nums[deque.peekLast()] ){  
        //found cur max so delete all previous max.  
        deque.pollLast();  
    }  
  
    //initially first empty add  
    //or remove previous small elements and add at the front.  
    deque.offer(i);  
  
    //i=2,3,4,5,6,7 (minimum window 2)  
    if(i>=k-1){  
        //since we maintain decreasing sequence.  
        //first will be max  
        res>window++]=nums[deque.peek()];  
    }  
}
```

```
        }  
        return res;  
    }  
}
```

Problem 86: 155. Min Stack

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack","push","push","push","getMin","pop","top","getMin"]  
[[],[-2],[0],[-3],[],[],[],[ ]]
```

Output

```
[null,null,null,null,-3,null,0,-2]
```

Explanation

```
MinStack minStack = new MinStack();  
minStack.push(-2);  
minStack.push(0);  
minStack.push(-3);  
minStack.getMin(); // return -3  
minStack.pop();  
minStack.top();    // return 0  
minStack.getMin(); // return -2
```

Constraints:

- $-2^{31} \leq \text{val} \leq 2^{31} - 1$

- Methods pop, top and getMin operations will always be called on **non-empty** stacks.
- At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

Solution

```
class MinStack {
// Runtime: 7 ms, faster than 65.70% of Java online submissions for Min
// Stack.
// Memory Usage: 48.8 MB, less than 26.86% of Java online submissions for Min
// Stack.

    Stack<Integer> st = null;
    Stack<Integer> min = null;

    public MinStack() {
        st = new Stack<>();
        min = new Stack<>();
    }

    public void push(int val) {
        //always add to main stack.
        st.push(val);
        if(min.isEmpty()){
            min.push(val);
            return;
        }
        //main min stack to smaller elements
        if(!min.isEmpty() && min.peek()>=val){
            min.push(val);
        }
    }

    public void pop() {
        if(st.isEmpty()) return;

        int val = st.pop();
        //after removing from main stack.
        //sync to min stack also
        if(!min.isEmpty() && min.peek() == val){
            min.pop();
        }
    }

    public int top() {
        if(st.isEmpty()) return -1;
        //just return from main stack not from min stack
    }
}
```

```
        return st.peek();
    }

    public int getMin() {
        //O(1) operation always retrieve from minstack.

        if(min.isEmpty())return -1;

        return min.peek();
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * int param_3 = obj.top();
 * int param_4 = obj.getMin();
 */
```

Problem 87 : 994. Rotting Oranges

994. Rotting Oranges

Medium

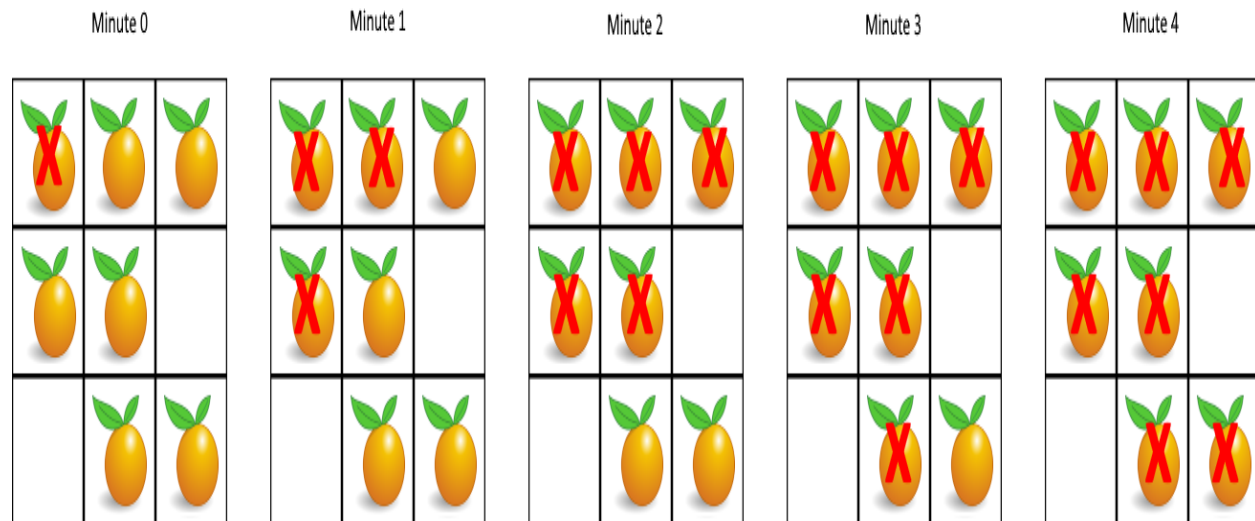
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]

Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]

Output: -1

Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = [[0,2]]

Output: 0

Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Solution

```
import java.util.LinkedList;
class Solution {
    public int orangesRotting(int[][] grid) {
        int r = grid.length;
        int c = grid[0].length;
        // empty, fresh orange, rotten.
        LinkedList<Node> queue = new LinkedList<>();
        int rotten = 0;
        int fresh = 0;
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                if (grid[i][j] == 2) {
                    queue.offer(new Node(i, j, 0));
                    rotten++;
                } else if (grid[i][j] == 1) {

```

```

        fresh++;
    }
}

// no rotten orange 0 minutes since no need to count further
if (fresh == 0)
    return 0;
// if rotten no ways we can convert to rotten.
if (rotten == 0)
    return -1;

return helper(grid, fresh, queue);
}

private int helper(int grid[][], int fresh, LinkedList<Node> queue) {
    int r = grid.length;
    int c = grid[0].length;
    int dirs[][] = { { -1, 0 }, { 1, 0 }, { 0, 1 }, { 0, -1 } };
    boolean visited[][] = new boolean[r][c];

    while (!queue.isEmpty()) {
        Node cur = queue.poll();
        int i = cur.x;
        int j = cur.y;
        int count = cur.count;

        for (int dir[] : dirs) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x < 0 || y < 0 || x >= r || y >= c ||
visited[x][y] || grid[x][y] != 1)
                continue;

            fresh--;
            // all made to rotten no ne
            if (fresh == 0) {
                return count;
            }

            visited[x][y] = true;
            queue.offer(new Node(x, y, count + 1));
        }
    }

    return -1;
}
}

```

```
class Node {
    int x, y, count;

    Node(int x, int y, int count) {
        this.x = x;
        this.y = y;
        this.count = count;
    }
}
```

Problem 88: 901. Online Stock Span

Design an algorithm that collects daily price quotes for some stock and returns the **span** of that stock's price for the current day.

The **span** of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the **span** of the stock's price given that today's price is price.

Example 1:

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
```

```
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4 prices (including
today's price of 75) were less than or equal to today's price.
stockSpanner.next(85); // return 6
```

Solution

```
class StockSpanner {
// Runtime: 58 ms, faster than 63.81% of Java online submissions for
Online Stock Span.
// Memory Usage: 75.3 MB, less than 15.61% of Java online submissions for
Online Stock Span.
```

```
Stack<int[]> st;
public StockSpanner() {
    st= new Stack<>();
}

public int next(int price) {
    int res =1;

    //if price is increasing.
    while(!st.isEmpty() && price>=st.peek()[0]){
        res+=st.pop()[1];
    }

    st.push(new int[]{price,res});
    return res;
}
}
```

Problem 89: Maximum of minimum for every window size

Sample Input 1:

```
2
4
1 2 3 4
```


5
3 3 4 2 4
Sample Output 1:

4 3 2 1
4 3 3 2 2

Explanation for sample input 1:

Test case 1:
Already explained in the question.

Test case 2:
Minimums of window size 1 = $\min(3), \min(3), \min(4), \min(2), \min(4) = 3, 3, 4, 2, 4$
Maximum among (3, 3, 4, 2, 4) is 4

Minimums of window size 2 = $\min(3,3), \min(3,4), \min(4,2), \min(2,4) = 3, 3, 2, 2$
Maximum among (3, 3, 2, 2) is 3

Minimums of window size 3 = $\min(3,3,4), \min(3,4,2), \min(4,2,4) = 3, 2, 2$
Maximum among (3, 2, 2) is 3

Minimums of window size 4 = $\min(3,3,4,2), \min(3,4,2,4) = 2, 2$
Maximum among (2, 2) is 2

Minimums of window size 4 = $\min(3,3,4,2,4) = 2$
Maximum among them is 2
The output array should be [4,3,3,2,2].

Sample Input 2:

2
5
45 -2 42 5 -11
6
-2 12 -1 1 20 1

Sample Output 2:

45 5 -2 -2 -11
20 1 1 -1 -1 -2

Solution

```
import java.util.Arrays;
import java.util.Stack;
public class Solution {
    public static int[] maxMinWindow(int[] arr, int n) {
```

```

    int next[] = getNextSmallerElement(arr, n);
    // [1,2,3,4] => [1, 2, 3, 4]
    int prev[] = getPrevSmallerElement(arr, n);
    // [1,2,3,4] => [ -1 ,1 ,-1,-1]

    int res[] = new int[n];
    Arrays.fill(res, Integer.MIN_VALUE);
    for (int i = 0; i < n; i++) {
        // --> <-- diff of next and prev is window size
        int window = next[i] - prev[i] - 1;
        res[window - 1] = Math.max(res[window - 1], arr[i]);
    }
    // [4 3 2 1]
    for (int i = n - 2; i >= 0; i--) {
        res[i] = Math.max(res[i], res[i + 1]);
    }

    return res;
}

private static int[] getPrevSmallerElement(int arr[], int n) {
    int prev[] = new int[n];

    Stack<Integer> st = new Stack<>();

    // 2,1
    // 1,2
    // from i previous element is higher pop
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && arr[i] <= arr[st.peek()]) {
            st.pop();
        }

        if (st.isEmpty()) {
            prev[i] = -1;
        } else {
            prev[i] = st.peek();
        }

        st.push(i);
    }

    return prev;
}

/**

```

```

    * To get the next smaller Element for ith index.
    *
    * @param arr
    * @param n
    * @return
    */
private static int[] getNextSmallerElement(int arr[], int n) {
    int next[] = new int[n];
    Stack<Integer> st = new Stack<>();

    // 1,2
    // 2 1
    // from i next element is higher pop
    for (int i = n - 1; i >= 0; i--) {
        while (!st.isEmpty() && arr[i] <= arr[st.peek()]) {
            st.pop();
        }

        if (st.isEmpty()) {
            next[i] = n;
        } else {
            next[i] = st.peek();
        }

        st.push(i);
    }

    return next;
}
}

```

Problem 89-I: The Celebrity Problem

Sample Input 1:

```

1
2
Call function 'knows(0, 1)' // returns false
Call function 'knows(1, 0)' // returns true

```

Sample Output 1:

```
0
```

Explanation For Sample Input 1:

In the first test case, there are 2 people at the party. When we call function knows(0,1), it returns false. That means the person having id '0' does not know a person having id '1'. Similarly, the person having id '1' knows a person having id '0' as knows(1,0) returns true. Thus a person having id '0' is a celebrity because he is known to everyone at the party but doesn't know anyone.

Solution

```
/*
    This is signature of helper function 'knows'.
    You should not implement it, or speculate about its implementation.
    boolean knows(int A, int B);
    Function 'knows(A, B)' will returns "true" if the person having
    id 'A' know the person having id 'B' in the party, "false" otherwise.
    Use it as Runner.knows(A, B);
*/

public class Solution {
    public static int findCelebrity(int n) {
        //celebrity - known to n person
        //          - does not know any of n person.

        // 0    1    2    3
        // \    |    /    /
        //  celebrity
        //if(0,)
        int celebrity = 0;
        for(int i=1;i<n;i++){
            if(Runner.knows(celebrity,i)){
                //if celebrity knows i then celebrity no longer can be
celebrity.
                //so i can be celebrity.
                celebrity =i;
            }
        }
        //if A knows B
        //B does not know A

        // A,B,C and if celebrity is A.
        // B knows A
        // C Knows A.

        // A does not know B,C,A
        // B knows A and A does not know B
        // C knows A and A does not know C.

        for(int i=0;i<n;i++){
            //i is not celebrity

            //All knows celebrity and celebrity does not know no one.
            //if current is not celebrity
            //celebrity knows cur - not a celebrity
        }
    }
}
```

```
        //we cannot validate like i==celebrity since it becomes compare
i with i.
        //validate reverse i not a celebrity
        //and celebrity knows i - means no celebrity exist.
        //or i does not celebrity.
        if(i!=celebrity && (Runner.knows(celebrity,i)
||!Runner.knows(i,celebrity)) )
        {
            return -1;
        }
    }

    return celebrity;
}
```

DAY 15: String

Problem 90: 151. Reverse Words in a String

Given an input string *s*, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in *s* will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space*.

Note that *s* may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: *s* = "the sky is blue"

Output: "blue is sky the"

Example 2:

Input: *s* = " hello world "

Output: "world hello"

Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: *s* = "a good example"

Output: "example good a"

Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Solution

```
class Solution {  
    public String reverseWords(String s) {  
  
        s = s.trim();  
        char arr[] = s.toCharArray();  
  
        // the sky is blue.
```

```

    // eulb si yks the
    // 012 456 78 9123
    int n = arr.length;
    int start = 0;
    int end = n;

    reverse(arr, start, end - 1);

    int startWordPos = 0;
    while (start < end) {
        // do reverse from ' ' as delimiter
        if (arr[start] == ' ') {
            reverse(arr, startWordPos, start - 1);
            startWordPos = start + 1; // inc next word start
        }
        start++;
    }
    reverse(arr, startWordPos, end - 1);
    String res = cleanSpace(arr);

    return res;
}
private String cleanSpace(char arr[]) {
    int n = arr.length;
    int left = 0; // only non-white,
    int right = 0; // (white+non-white)forward pointer;
    while (right < n) {
        while (right < n && arr[right] != ' ') {
            // since j moves forward for even space.
            // left wont move so we overwrite left with right
values
            arr[left++] = arr[right++];
        }
        while (right < n && arr[right] == ' ') {
            right++;
        }

        if (right < n) {
            arr[left++] = ' ';
        }
    }
    // all non -white will be moved after i.
    // so truncate.
    return new String(arr, 0, left);
}

private void reverse(char arr[], int start, int end) {

```



```
        while (start < end) {
            char temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }

    public static void main(String args[]) {
        String res = new Solution().reverseWords("a good
example");
        System.out.println("Solution.main()" + res);
    }
}
```

Problem 91: 5. Longest Palindromic Substring

Medium

Given a string *s*, return *the longest palindromic substring* in *s*.

Example 1:

Input: *s* = "babad"

Output: "bab"

Explanation: "aba" is also a valid answer.

Example 2:

Input: *s* = "cbabd"

Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- *s* consist of only digits and English letters.

Solution

```
class Solution {
    public String longestPalindrome(String s) {
        // babad
        int n = s.length();
        String max = "";
        for (int i = 0; i < n; i++) {
```

```

        // "a b a" //"b a a b"
        // -->|<-- // -->| |<--
        // odd //
        // aba
        max = moveFromCenter(s, i, i, max); // odd
        max = moveFromCenter(s, i, i + 1, max); // odd
    }

    return max;
}

private String moveFromCenter(String s, int start, int end, String max)
{
    char arr[] = s.toCharArray();
    int n = arr.length;
    // babad
    while (start >= 0 && end < n) {
        if (arr[start] != arr[end]) {
            break;
        } else {
            start--;
            end++;
        }
    }

    // handle single character so >n not >= and anyway substring end
    // be n+1.
    if (start + 1 < 0 || end > n)
        return max;

    String res = s.substring(start + 1, end);
    if (max.length() < res.length()) {
        max = res;
    }

    return max;
}
}

```

Problem 92: 13. Roman to Integer

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
--------	-------

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: s = "III"

Output: 3

Explanation: III = 3.

Example 2:

Input: s = "LVIII"

Output: 58

Explanation: L = 50, V = 5, III = 3.

Example 3:

Input: s = "MCMXCIV"

Output: 1994

Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Solution

```
import java.util.HashMap;  
import java.util.Map;
```

```
class Solution {
```

```
public int romanToInt(String s) {

    Map<Character, Integer> map = new HashMap<Character, Integer>();
    map.put('I', 1);
    map.put('V', 5);
    map.put('X', 10);
    map.put('L', 50);
    map.put('C', 100);
    map.put('D', 500);
    map.put('M', 1000);

    int r = 0;
    int prev = 0;

    for (char c : s.toCharArray()) {
        // calculate
        int v = map.get(c);
        if (v > prev) {
            r -= prev;
            r += v - prev;
        } else {
            r += v;
        }
        prev = v;
    }
    return r;
}
```

Problem 93: 8. String to Integer (atoi)

Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer (similar to C/C++'s atoi function).

The algorithm for myAtoi(string s) is as follows:

1. Read in and ignore any leading whitespace.
2. Check if the next character (if not already at the end of the string) is '-' or '+'. Read this character in if it is either. This determines if the final result is negative or positive respectively. Assume the result is positive if neither is present.
3. Read in next the characters until the next non-digit character or the end of the input is reached. The rest of the string is ignored.
4. Convert these digits into an integer (i.e. "123" -> 123, "0032" -> 32). If no digits were read, then the integer is 0. Change the sign as necessary (from step 2).
5. If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then clamp the integer so that it remains in the range. Specifically, integers less than -2^{31} should be clamped to -2^{31} , and integers greater than $2^{31} - 1$ should be clamped to $2^{31} - 1$.
6. Return the integer as the final result.

Note:

- Only the space character ' ' is considered a whitespace character.
- **Do not ignore** any characters other than the leading whitespace or the rest of the string after the digits.

Example 1:**Input:** s = "42"**Output:** 42**Explanation:** The underlined characters are what is read in, the caret is the current reader position.Step 1: "42" (no characters read because there is no leading whitespace)
 ^Step 2: "42" (no characters read because there is neither a '-' nor '+')
 ^Step 3: "42" ("42" is read in)
 ^

The parsed integer is 42.

Since 42 is in the range $[-2^{31}, 2^{31} - 1]$, the final result is 42.**Solution**

```
class Solution {
    public int myAtoi(String s) {
        // (a-z A-Z)
        // digits (0-9),
        // ' ', '+', '-', and '.'.
```

```
s = s.trim();
char arr[] = s.split(" ")[0].toCharArray();
int n = s.length();
StringBuffer buffer = new StringBuffer();

char sign = '#';
n = arr.length;
for (int i = 0; i < n; i++) {
    char ch = arr[i];
    if (ch == ' ')
        continue;
    if (ch == '+' || ch == '-') {
        if (i > 0)
            break;
        sign = ch;
        continue;
    }
    if (!Character.isDigit(ch)) {
        break;
    }
    buffer.append(arr[i]);
}

String res = buffer.toString();
res = trimZeroPrefix(res);

if (res.length() == 0)
    return 0;

return getResult(res, sign);
}

private String trimZeroPrefix(String res) {
    int start = 0, end = res.length();
    int count = 0;
    while (start < end) {
        if (res.charAt(start) == '0') {
            count++;
            start++;
        } else {
            break;
        }
    }
    res = res.substring(count, end);
    return res;
}

private int getResult(String res, char sign) {
```

```

    int n = res.length();
    int i = 0;
    char arr[] = res.toCharArray();
    double sum = 0;
    // 12345
    // 1 (0*10)
    // 10+2=12
    // 120+3=123
    // 123*10+4 =1234
    // 1234*10+5 = 12345
    while (i < n) {
        int cur = arr[i] - '0';
        sum = (sum * 10) + cur;
        i++;
    }

    if (sum > Integer.MAX_VALUE) {
        if (sign == '-')
            return Integer.MIN_VALUE;
        else
            return Integer.MAX_VALUE;
    }

    if (sign != '#')
        res = sign + res;

    return Integer.parseInt(res);
}

public static void main(String[] args) {
    int res = new Solution().myAtoi("-13+8");
    System.out.println("Solution.res" + res);
}
}

```

Problem 94: 14. Longest Common Prefix

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:**Input:** strs = ["flower","flow","flight"]**Output:** "fl"**Example 2:****Input:** strs = ["dog","racecar","car"]**Output:** ""**Explanation:** There is no common prefix among the input strings.**Constraints:**

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- `strs[i]` consists of only lowercase English letters.

Solution

```
class Solution {  
    // Runtime: 2 ms, faster than 49.32% of Java online submissions for  
    // Longest Common Prefix.  
    // Memory Usage: 43.1 MB, less than 12.36% of Java online submissions for  
    // Longest Common Prefix.  
  
    public String longestCommonPrefix(String[] strs) {  
        int n = strs.length;  
        String res = "";  
        if (n == 1)  
            return strs[0];  
        Trie trie = new Trie();  
  
        for (String cur : strs) {  
            trie.addToTrie(cur);  
        }  
  
        // if common prefix all words in arr should match .  
        // ["flower","flow","flight"]  
    }  
}
```



```

        // e.g taken first word flower -> f count should min 3
        // -> l count should min 3 times
        // -> o count should min 3 times - not matching
        // hence fl is longest common substring.
        return trie.search(strs[0], n);
    }
}

class Trie {
    TrieNode node;

    Trie() {
        node = new TrieNode();
    }

    public void addToTrie(String word) {
        TrieNode cur = node;
        int n = word.length();
        char arr[] = word.toCharArray();

        for (int i = 0; i < n; i++) {
            int index = arr[i] - 'a';
            if (cur.words[index] == null) {
                cur.words[index] = new TrieNode();
            }
            cur = cur.words[index];

            cur.count++;
        }
    }
}

// check by taking first index of arr checking those words matching
with
// remaining words in index
// check match by count of that word in trie.
// min count of a word = len(words arr)
// ["flower","flow","flight"]
// flower = >
// f(>4)
// /
// l(>4)
public String search(String word, int len) {
    TrieNode cur = node;
    int n = word.length();
    char arr[] = word.toCharArray();

```

```
        StringBuffer buffer = new StringBuffer();

        for (int i = 0; i < n; i++) {
            int index = arr[i] - 'a';
            cur = cur.words[index];
            if (cur.count >= len) {
                buffer.append(arr[i]);
            }
        }

        return buffer.toString();
    }
}

class TrieNode {

    // a-z based on constraints
    TrieNode words[] = new TrieNode[26];
    int count = 0;

    TrieNode() {

    }

}
```

Problem 95: 686. Repeated String Match

Given two strings a and b, return *the minimum number of times you should repeat string a so that string b is a substring of it*. If it is impossible for b to be a substring of a after repeating it, return -1.

Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcabc".

Example 1:

Input: a = "abcd", b = "cdabcdab"

Output: 3

Explanation: We return 3 because by repeating a three times "abcdabcdabcd", b is a substring of it.

Example 2:

Input: a = "a", b = "aa"

Output: 2

Constraints:

- $1 \leq a.length, b.length \leq 10^4$
- a and b consist of lowercase English letters.

Solution

```
class Solution {  
    // Runtime: 342 ms, faster than 45.06% of Java online submissions for  
    // Repeated String Match.  
    // Memory Usage: 42.9 MB, less than 56.54% of Java online submissions for  
    // Repeated String Match.  
  
    public int repeatedStringMatch(String a, String b) {  
  
        int i = 0;  
        int n = b.length();  
  
        StringBuffer buffer = new StringBuffer();  
        int count = 0;  
        while (i < n) {  
            buffer.append(a);  
            i = buffer.length();  
            count++;  
        }  
  
        if (buffer.toString().indexOf(b) != -1)  
            return count;  
        if (buffer.append(a).toString().indexOf(b) != -1)  
            return count + 1;  
  
        return -1;  
    }  
}
```

Day 16: String Part-II

Problem 96,97:28. Find the Index of the First Occurrence in a String

Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: haystack = "leetcode", needle = "leeto"

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

Solution

```
class Solution {  
    public int strStr(String haystack, String needle) {  
        return helper(haystack,needle);  
    }  
}
```

```
private int helper(String s1, String s2){  
    char s[] = s1.toCharArray();  
    char t[] = s2.toCharArray();  
    int tlen =t.length;  
    int slen = s.length;  
    int index = -1;  
    int count = 0;  
    int i =0,j=0;
```

```

//mississippi ==> issip
//      i | m | i | s | s | i | s | s | i | p | p | i | comments
//      ---|---|---|---|---|---|---|---|---|---|---|
//      j | i | s | s | i | p |   |   |   |   |   |   | i don't match inc i
//      j |   | i | s | s | i |   |   |   |   |   |   | both match inc i ,j
//      j |   |   | s | s | i | p |   |   |   |   |   |   | both match inc i ,j
//      j |   |   | i | s | i |   |   |   |   |   |   | both match inc i ,j
//      j |   |   | i | s | i |   |   |   |   |   |   | both match inc i ,j
//
//i      j
//      //m i s s i s s i p p i | i s s i p
//
//      |
//      |
//      |
//      |
//      |
//
//      |
//      |
//      |
//      |
//      |
//
while(i<slen && j<tlen){
    if(s[i]==t[j] ){
        if(index==-1) index = i;
        j++;
        i++;
        count++;
    }else{
        if(index>=0){
            i=index+1;
            j=0;
            index =-1;
            count = 0;
        }else{
            i++;
        }
    }
    if(count == tlen){
        return index;
    }
}

return -1;
}
}

```

Problem 98: Minimum Characters For Palindrome

Sample output 1 :

3
0

Explanation of sample input 1 :

For test case 1 :

Minimum characters to be added at front to make it a palindrome are 3 i.e. “dcb” which makes the string “dcbabcb”.

For test case 2 :

The string is already a palindrome, we do not need to add any character.

Sample input 2 :

```
2
xxaxxa
abb
```

Sample output 2 :

```
1
2
```

Solution

```
public class Solution {
    public static int minCharsforPalindrome(String str) {
        char arr[] = str.toCharArray();
        int n = arr.length;
        int dp[][] = new int[n + 1][n + 1];
        // 1 2 3
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                // 0 == 3, 2, 3-1-1=1, 3-1-2 0
                // 3-1=2, 3-2=1, 3-3=0
                // 3-1=2 , 3-2=1 3-3=0
                if (arr[i - 1] == arr[n - j]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return n - dp[n][n];
    }
}
```

Problem 99: 242. Valid Anagram

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Solution

```
class Solution {
// Runtime: 7 ms, faster than 53.16% of Java online submissions for Valid
// Anagram.
// Memory Usage: 44.3 MB, less than 46.26% of Java online submissions for
// Valid Anagram.

    public boolean isAnagram(String s, String t) {

        int res[] = new int[256];
        if (s.length() > t.length()) {
            return isAnagram(t, s);
        }

        for (char ch : s.toCharArray()) {
            res[ch - 'a']++;
        }

        for (char ch : t.toCharArray()) {
            res[ch - 'a']--;
        }

        for (char ch : t.toCharArray()) {
            if (res[ch - 'a'] != 0) {
                return false;
            }
        }

        return true;
    }
}
```


Problem 100: 38. Count and Say

The **count-and-say** sequence is a sequence of digit strings defined by the recursive formula:

- `countAndSay(1) = "1"`
- `countAndSay(n)` is the way you would "say" the digit string from `countAndSay(n-1)`, which is then converted into a different digit string.

To determine how you "say" a digit string, split it into the **minimal** number of substrings such that each substring contains exactly **one** unique digit. Then for each substring, say the number of digits, then say the digit. Finally, concatenate every said digit.

For example, the saying and conversion for digit string "3322251":

"3322251"
 two 3's, three 2's, one 5, and one 1
 2 3 + 3 2 + 1 5 + 1 1
 "23321511"

Given a positive integer n , return the n^{th} term of the **count-and-say** sequence.

Example 1:

Input: $n = 1$

Output: "1"

Explanation: This is the base case.

Example 2:

Input: $n = 4$

Output: "1211"

Explanation:

`countAndSay(1) = "1"`

`countAndSay(2) = say "1" = one 1 = "11"`

`countAndSay(3) = say "11" = two 1's = "21"`

`countAndSay(4) = say "21" = one 2 + one 1 = "12" + "11" = "1211"`

Solution

```
class Solution {
    public String countAndSay(int n) {

        // take previous row 1 and current val

        String res = "1";
        for (int i = 2; i <= n; i++) {
            res = helper(res);
        }

        return res;
    }
    private String helper(String val) {
        int n = val.length();
        char arr[] = val.toCharArray();
        char prev = arr[0]; // previous row

        int count = 1;

        StringBuffer buffer = new StringBuffer();
        for (int i = 1; i < n; i++) {
            char cur = arr[i];
            // previous row first val = current
            if (prev == cur) {
                count++;
            } else {
                // if no match reset count =1;
                buffer.append(count).append(prev);
                count = 1;
                // set cur as prev row val
                prev = arr[i];
            }
        }

        buffer.append(count).append(prev);

        return buffer.toString();
    }
}
```

Problem:101. 165. Compare Version Numbers

Given two version numbers, version1 and version2, compare them.

Version numbers consist of **one or more revisions** joined by a dot '.'. Each revision consists of **digits** and may contain leading **zeros**. Every revision contains **at least one character**. Revisions are **0-indexed from left to right**, with the leftmost revision being revision 0, the next revision being revision 1, and so on. For example 2.5.33 and 0.1 are valid version numbers.

To compare version numbers, compare their revisions in **left-to-right order**. Revisions are compared using their **integer value ignoring any leading zeros**. This means that revisions 1 and 001 are considered **equal**. If a version number does not specify a revision at an index, then **treat the revision as 0**. For example, version 1.0 is less than version 1.1 because their revision 0s are the same, but their revision 1s are 0 and 1 respectively, and $0 < 1$.

Return the following:

- If version1 < version2, return -1.
- If version1 > version2, return 1.
- Otherwise, return 0.

Example 1:

Input: version1 = "1.01", version2 = "1.001"

Output: 0

Explanation: Ignoring leading zeroes, both "01" and "001" represent the same integer "1".

Example 2:

Input: version1 = "1.0", version2 = "1.0.0"

Output: 0

Explanation: version1 does not specify revision 2, which means it is treated as "0".

Example 3:

Input: version1 = "0.1", version2 = "1.1"

Output: -1

Explanation: version1's revision 0 is "0", while version2's revision 0 is "1". $0 < 1$, so version1 < version2.

Constraints:

- $1 \leq \text{version1.length}, \text{version2.length} \leq 500$
- version1 and version2 only contain digits and '.'.

- version1 and version2 are valid version numbers.
- All the given revisions in version1 and version2 can be stored in a 32-bit integer.

Solution

```
class Solution {
    public int compareVersion(String version1, String version2) {
        // if before . check the value.
        // check all the values in the .
        String arr1[] = version1.split("\\.");
        String arr2[] = version2.split("\\.");

        return helper(arr1, arr2);
    }

    /**
     * To compare 2 string based on first higher bit
     *
     * @param arr1
     * @param arr2
     * @return if arr1 has first higher bit return 1 else return -1
     */
    private int helper(String arr1[], String arr2[]) {

        int n1 = arr1.length;
        int n2 = arr2.length;

        int i = 0, j = 0;
        while (i < n1 && j < n2) {
            String version1 = arr1[i];
            String version2 = arr2[j];

            version1 = trimLeadingZero(version1);
            version2 = trimLeadingZero(version2);
            int res = compare(version1, version2);
            if (res != 0)
                return res;

            i++;
            j++;
        }

        int res = 0;
        if (i < n1) {
            res = getHigherBit(arr1, i, n1);
            return Integer.compare(res, 0);
        }
    }
}
```

```
        if (j < n2) {
            res = getHigherBit(arr2, j, n2);
            return Integer.compare(0, res);
        }

        return res;
    }

    /**
     * to get the first higher bit in both string .
     *
     * @param arr
     * @param i
     * @param n
     * @return - val=1(val>0), else return -1.
     */
    private int getHigherBit(String[] arr, int i, int n) {
        String res = null;
        while (i < n) {
            res = trimLeadingZero(arr[i++]);
            int val = Integer.parseInt(res);
            if (val > 0)
                return 1;
            else if (val < 0)
                return -1;
        }

        return 0;
    }

    private int compare(String version1, String version2) {
        return Integer.compare(Integer.parseInt(version1),
Integer.parseInt(version2));
    }
    private String trimLeadingZero(String value) {

        int n = value.length();

        int count = 0;
        if (n == 1)
            return value;
        // 0001
        for (int i = 0; i < n; i++) {
            if (value.charAt(i) == '0') {
                count++;
            } else {
                break;
            }
        }
    }
}
```

```
        }
    }
    // "1.00.1"
    // "1.0.2"
    // 0000 -> if more than 1 zero it should return 0.
    if (n == count)
        return "0";

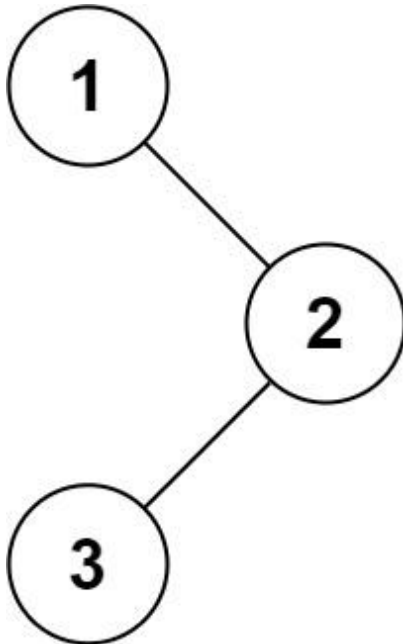
    return value.substring(count);
}
public static void main(String args[]) {
    int res = new Solution().compareVersion("1.1", "1.10");
    System.out.println("Solution.main()" + res);
}
}
```

Day 17: Binary Tree-I

Problem:101: 94. Binary Tree Inorder Traversal

Given the root of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:



Input: root = [1,null,2,3]

Output: [1,3,2]

Example 2:

Input: root = []

Output: []

Example 3:

Input: root = [1]

Output: [1]

Solution

```
import java.util.ArrayList;  
import java.util.List;
```



```
import java.util.Stack;

import javax.swing.tree.TreeNode;

class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        Stack<TreeNode> st = new Stack<>();

        while (!st.isEmpty() || root != null) {

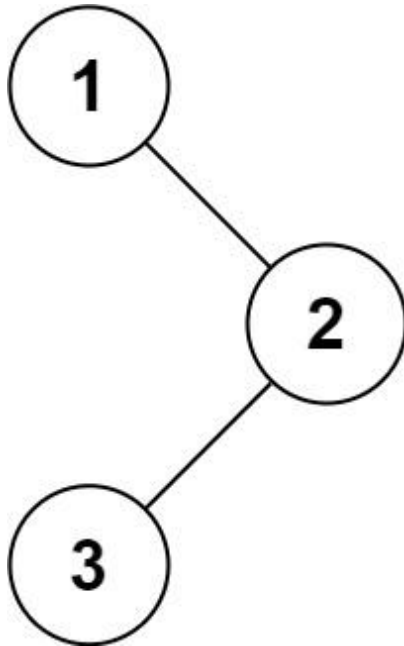
            while (root != null) {
                st.push(root);
                root = root.left;
            }
            TreeNode temp = st.pop();
            result.add(temp.val);
            root = temp.right;
        }

        return result;
    }
}
```

Problem:102: 144. Binary Tree Preorder Traversal

Given the root of a binary tree, return *the preorder traversal of its nodes' values*.

Example 1:



Input: root = [1,null,2,3]

Output: [1,2,3]

Example 2:

Input: root = []

Output: []

Solution

```
import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

import javax.swing.tree.TreeNode;

/**
 * Definition for a binary tree node. public class TreeNode { int val;
TreeNode
 * left; TreeNode right; TreeNode(int x) { val = x; } }
```

```
*/
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {

        List<Integer> result = new ArrayList<Integer>();

        helper(root, result);
        return result;
    }

    private void helper(TreeNode root, List<Integer> list) {

        if (root == null)
            return;
        Stack<TreeNode> st = new Stack<TreeNode>();

        st.push(root);

        // root,left,right

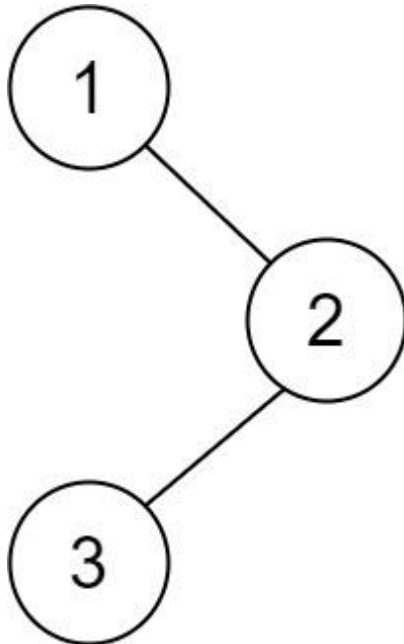
        while (!st.isEmpty()) {
            TreeNode cur = st.pop();
            list.add(cur.val);
            if (cur.right != null) {
                st.push(cur.right);
            }
            if (cur.left != null) {
                st.push(cur.left);
            }
        }

    }
}
```

Problem:103: 145. Binary Tree Postorder Traversal

Given the root of a binary tree, return *the postorder traversal of its nodes' values*.

Example 1:



Input: root = [1,null,2,3]

Output: [3,2,1]

Example 2:

Input: root = []

Output: []

Solution

```
import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<Integer>();

        if (root == null) {
            return res;
        }

        Stack<TreeNode> stack = new Stack<TreeNode>();
        stack.push(root);
```

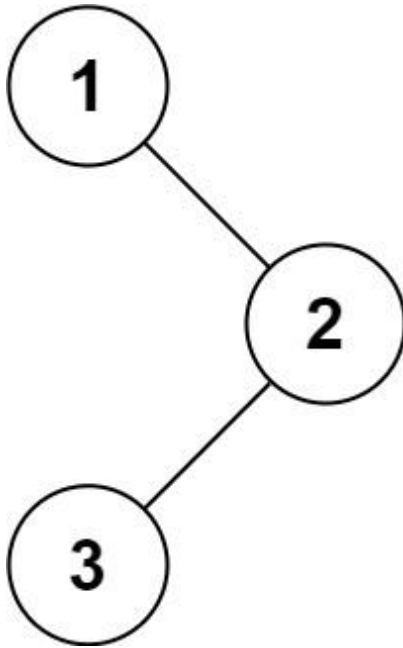
```
while (!stack.isEmpty()) {  
    TreeNode temp = stack.peek();  
    if (temp.left == null && temp.right == null) {  
        TreeNode pop = stack.pop();  
        res.add(pop.val);  
    } else {  
        if (temp.right != null) {  
            stack.push(temp.right);  
            temp.right = null;  
        }  
  
        if (temp.left != null) {  
            stack.push(temp.left);  
            temp.left = null;  
        }  
    }  
}  
  
return res;  
}
```

Problem:104: 145. Morris Inorder Traversal

94. Binary Tree Inorder Traversal

Given the root of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:



Input: root = [1,null,2,3]

Output: [1,3,2]

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
```

```

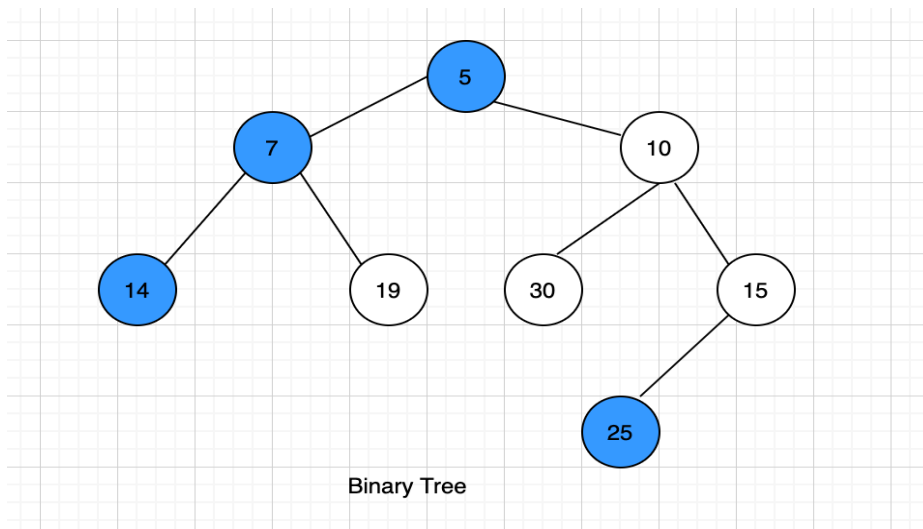
List<Integer> inorder = new ArrayList<Integer>();

TreeNode cur = root;
while (cur != null) {
    if (cur.left == null) {
        inorder.add(cur.val);
        cur = cur.right;
    } else {
        TreeNode prev = cur.left;
        while (prev.right != null && prev.right != cur) {
            prev = prev.right;
        }

        if (prev.right == null) {
            prev.right = cur;
            cur = cur.left;
        } else {
            prev.right = null;
            inorder.add(cur.val);
            cur = cur.right;
        }
    }
}
return inorder;
}
}

```

Problem:105: Left View Of a Binary Tree



Sample Input 1:

```

2
3 4 -1 -1 -1
2 8 7 -1 5 -1 -1 1 -1 -1 -1

```

Sample Output 1:

3 4
2 8 5 1

Explanation of Sample Input 1:

For the first test case, node 3 and node 4 are visible when the binary tree is viewed from the left.

For the second test case, nodes 2, 8, 5, 1 are visible when the binary tree is viewed from the left.

Solution

```
/******
```

Following is the `TreeNode` class structure

```
class TreeNode<T>
{
    public:
        T data;
        TreeNode<T> left;
        TreeNode<T> right;

        TreeNode(T data)
        {
            this.data = data;
            left = null;
            right = null;
        }
};
```

```
*****/
```

```
import java.util.*;
public class Solution {
    public static ArrayList<Integer> getLeftView(TreeNode<Integer> root) {
        LinkedList<TreeNode<Integer>> queue = new LinkedList<>();
        queue.offer(root);
        ArrayList<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int len = queue.size();
            for (int i = 0; i < len; i++) {
                TreeNode<Integer> cur = queue.poll();

                if (cur == null)
                    continue;
                if (i == 0) {
                    result.add(cur.data);
                }
                if (cur.left != null) {
                    queue.offer(cur.left);
                }
            }
        }
    }
}
```



```

    }

    if (cur.right != null) {
        queue.offer(cur.right);
    }
}

return result;
}
}

```

Problem:106: Bottom View Of Binary Tree

Sample input 1 :

```

2
1 2 3 -1 -1 5 6 7 8 -1 -1 -1 -1 -1 -1
20 8 22 5 3 -1 25 -1 -1 10 14 -1 -1 -1 -1 -1

```

Sample output 1 :

```

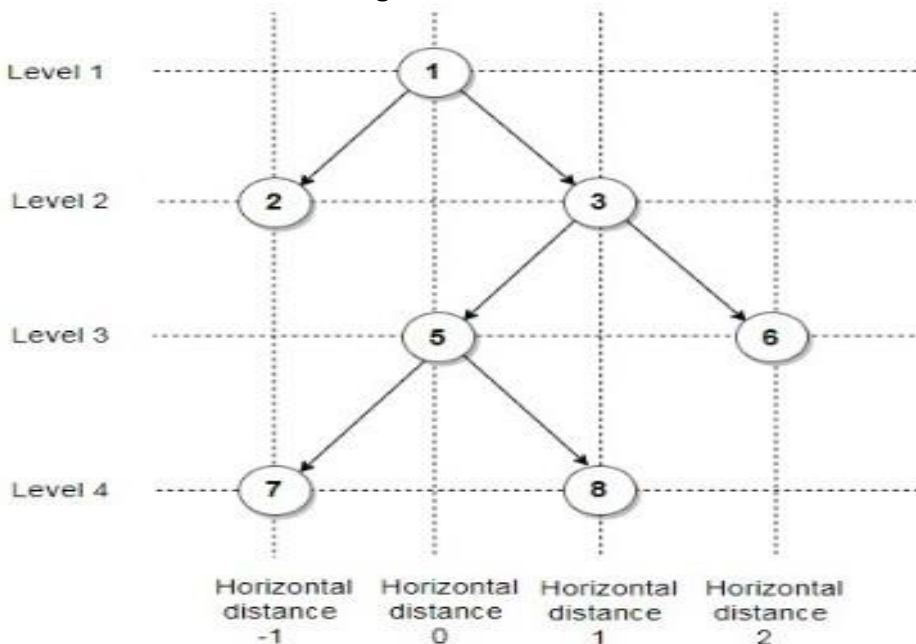
7 5 8 6
5 10 3 14 25

```

Explanation of sample input 1 :

Test case 1:

As shown in the above figure,



1 is the root node, so its horizontal distance = 0.

Since 2 lies to the left of 0, its horizontal distance = $0-1 = -1$

3 lies to the right of 0, its horizontal distance = $0+1 = 1$

Similarly, horizontal distance of 5 = Horizontal distance of 3 - 1 = 1-1 = 0
 Horizontal distance of 6 = Horizontal distance of 3 + 1 = 1+1 = 2
 Horizontal distance of 7 = 0-1 = -1
 Horizontal distance of 8 = 0+1 = 1

The bottom-most node at a horizontal distance of -1 is 7.
 The bottom-most node at a horizontal distance of 0 is 5.
 The bottom-most node at a horizontal distance of 1 is 8.
 The bottom-most node at a horizontal distance of 2 is 6.

Hence, the bottom view would be 7 5 8 6.

Test case 2:

20 is the root node, so its horizontal distance = 0.
 Since 8 lies to the left of 20, its horizontal distance = 0-1 = -1
 22 lies to the right of 20, its horizontal distance = 0+1 = 1
 Similarly, horizontal distance of 5 = Horizontal distance of 8 - 1 = -1-1 = -2
 Horizontal distance of 3 = Horizontal distance of 8 + 1 = -1+1 = 0
 Horizontal distance of 25 = 1+1 = 2
 Horizontal distance of 10 = 0-1 = -1
 Horizontal distance of 14 = 0+1 = 1

The bottom-most node at a horizontal distance of -2 is 5.
 The bottom-most node at a horizontal distance of -1 is 10.
 The bottom-most node at a horizontal distance of 0 is 3.
 The bottom-most node at a horizontal distance of 1 is 14.
 The bottom-most node at a horizontal distance of 2 is 25.

Hence, the bottom view would be 5 10 3 14 25

Solution

/*****

```
class BinaryTreeNode {
    int val;
    BinaryTreeNode left;
    BinaryTreeNode right;

    BinaryTreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
```

*****/

```
import java.util.*;
public class Solution {
    public static ArrayList<Integer> bottomView(BinaryTreeNode root) {
```

```

        LinkedList<BinaryTreeNode> queue = new LinkedList<>();

        Map<Integer,Node> map = new HashMap<>();
        helper(root,map,0,0);
        ArrayList<Integer> result = new ArrayList<Integer>();
        Set<Integer> st = map.keySet();
        Iterator<Integer> it = st.iterator();
        while(it.hasNext()){
            int key = it.next();
            Node node = map.get(key);
            result.add(node.val);
        }

        return result;
    }

    private static void helper(BinaryTreeNode root, Map<Integer,Node> map,
                               int rootVal, int level){
        if(root == null) return;
        Node cur = new Node(rootVal, level,root.val);
        if(map.get(rootVal)!=null ){
            //only update if level is less
            //since we want bottom most.
            if(map.get(rootVal).level <= level)
                map.put(rootVal,cur);
        } else{
            map.put(rootVal,cur);
        }
        helper(root.left, map, rootVal-1,level+1);
        helper(root.right,map, rootVal+1,level+1);
    }
}

class Node{
    int key;
    int level;
    int val;
    Node(int key,int level,int val){
        this.key = key;
        this.level = level;
        this.val = val;
    }
}

```

Problem:107: Top View Of Binary Tree

Sample Input 1:

```

2
1 2 3 4 5 6 7 8 9 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
1 2 3 -1 -1 -1 -1

```

Sample Output 1:

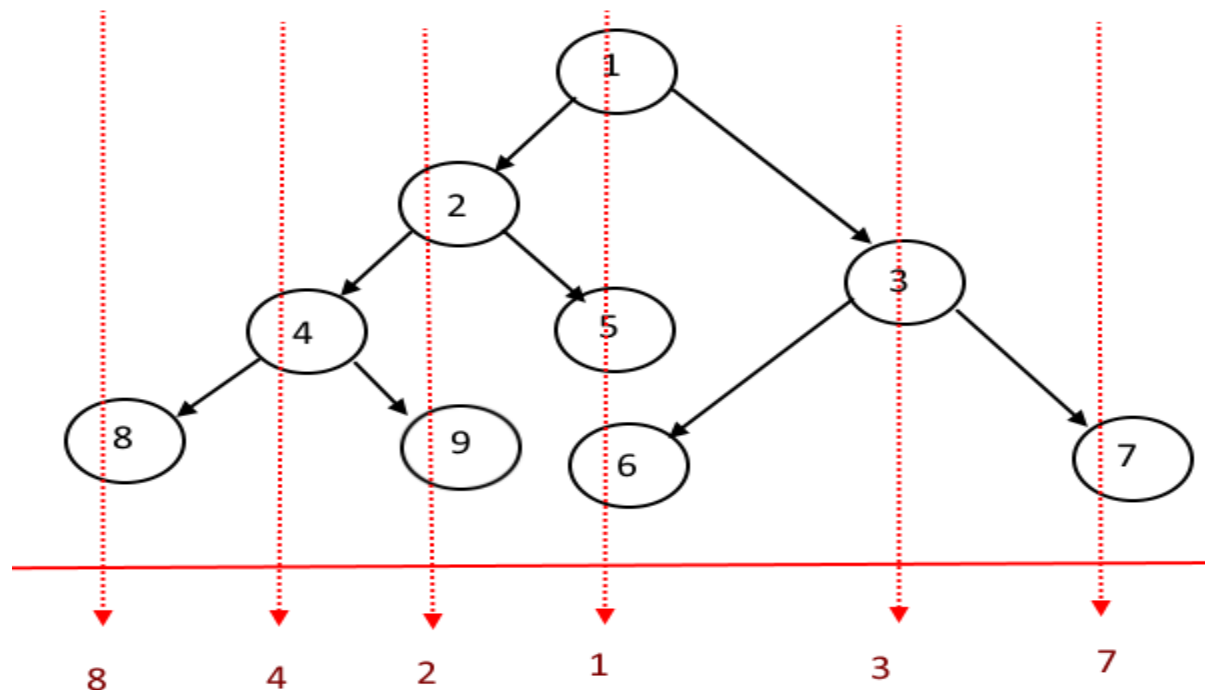
```

8 4 2 1 3 7
2 1 3

```

Explanation of Sample Output 1:

In test case 1,



Top View of a Binary Tree

From left to right, the top view of the tree will be {8,4,2,1,3,7}. Where 9,5 and 6 will be hidden when we see from the top of the tree.

In test case 2, from left to right, the top view of the tree will be {2,1,3} as there are only three nodes. There is nothing to hide.

Sample Input 2:

```

2
1 2 3 4 -1 5 6 -1 7 -1 -1 -1 -1 -1 -1
5 -1 -1

```

Sample Output 2:

4 2 1 3 6
5

Solution

```

/*****
Following is the TreeNode class structure:
class BinaryTreeNode {
    int val;
    BinaryTreeNode left;
    BinaryTreeNode right;
    BinaryTreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
*****/
import java.util.*;

public class Solution {
    public static ArrayList<Integer> getTopView(BinaryTreeNode root) {
        LinkedList<BinaryTreeNode> queue = new LinkedList<>();
        Map<Integer, Node> map = new TreeMap<>();
        helper(root, map, 0, 0);
        ArrayList<Integer> result = new ArrayList<Integer>();
        Set<Integer> st = map.keySet();
        Iterator<Integer> it = st.iterator();
        while (it.hasNext()) {
            int key = it.next();
            Node node = map.get(key);
            result.add(node.val);
        }
        return result;
    }

    private static void helper(BinaryTreeNode root, Map<Integer, Node> map,
int rootVal, int level) {
        if (root == null)
            return;
        Node cur = new Node(rootVal, level, root.val);
        if (map.get(rootVal) != null) {
            // only update if level is less
            // since we want bottom most.
            if (map.get(rootVal).level >= level)

```

```
        map.put(rootVal, cur);
    } else {
        map.put(rootVal, cur);
    }
    helper(root.left, map, rootVal - 1, level + 1);
    helper(root.right, map, rootVal + 1, level + 1);
}
}
```

```
class Node {
    int key;
    int level;
    int val;

    Node(int key, int level, int val) {
        this.key = key;
        this.level = level;
        this.val = val;
    }
}
```

Problem:108: Tree Traversals-Preorder inorder postorder in a single traversal

Sample Input 1 :

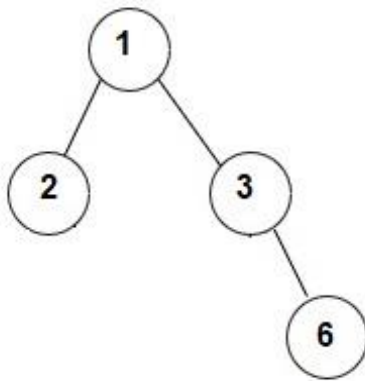
```
2
1 2 3 -1 -1 -1 6 -1 -1
1 2 3 -1 -1 -1 -1
```

Sample Output 1 :

```
2 1 3 6
1 2 3 6
2 6 3 1
2 1 3
1 2 3
2 3 1
```

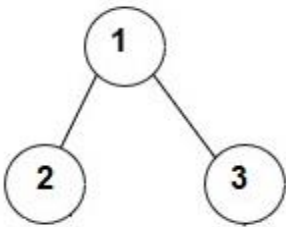
Explanation of Sample Output 1 :

In test case 1, the given binary tree is shown below:



Inorder traversal of given tree = [2, 1, 3, 6]
Preorder traversal of given tree = [1, 2, 3, 6]
Postorder traversal of given tree = [2, 6, 3, 1]

In test case 2, the given binary tree is shown below:



Inorder traversal of given tree = [2, 1, 3]
Preorder traversal of given tree = [1, 2, 3]
Postorder traversal of given tree = [2, 3, 1]

Solution

```
/******
```

Following is the Binary Tree node structure:

```
class BinaryTreeNode<T> {  
    T data;  
    BinaryTreeNode<T> left;  
    BinaryTreeNode<T> right;  
  
    public BinaryTreeNode(T data) {  
        this.data = data;  
    }  
}
```

```

*****/
import java.util.*;

public class Solution {
    public static List<List<Integer>> getTreeTraversal(BinaryTreeNode<Integer> root) {

        List<List<Integer>> result = new ArrayList<>();
        List<Integer> list1 = new ArrayList<>();
        List<Integer> list2 = new ArrayList<>();
        List<Integer> list3 = new ArrayList<>();
        inorder(root, list1);
        preorder(root, list2);
        postorder(root, list3);
        // Inorder - LRoot,Root, Right
        // Preorder - Root, Left,Right.
        // PostOrder - Left,Right, Root.

        result.add(list1);
        result.add(list2);
        result.add(list3);

        return result;
    }
    private static void inorder(BinaryTreeNode<Integer> root, List<Integer> list) {
        if (root == null)
            return;
        inorder(root.left, list);
        list.add(root.data);
        inorder(root.right, list);
    }

    private static void preorder(BinaryTreeNode<Integer> root, List<Integer> list) {
        if (root == null)
            return;
        list.add(root.data);
        preorder(root.left, list);
        preorder(root.right, list);
    }

    private static void postorder(BinaryTreeNode<Integer> root, List<Integer> list) {
        if (root == null)
            return;
        postorder(root.left, list);
        postorder(root.right, list);
        list.add(root.data);
    }
}

```

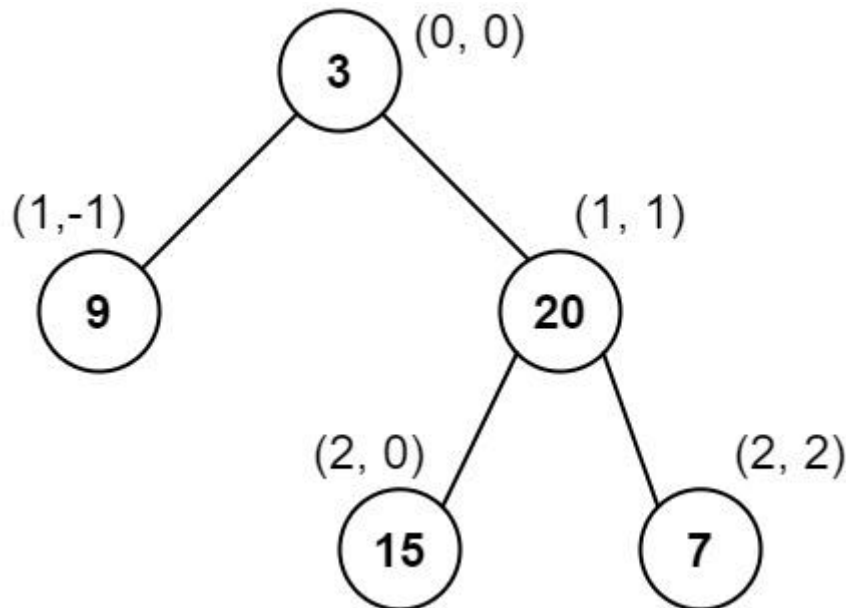
Problem:109: 987. Vertical Order Traversal of a Binary Tree

Given the root of a binary tree, calculate the vertical order traversal of the binary tree.

For each node at position (row, col), its left and right children will be at positions (row + 1, col - 1) and (row + 1, col + 1) respectively. The root of the tree is at (0, 0).

The **vertical order traversal** of a binary tree is a list of top-to-bottom orderings for each column index starting from the leftmost column and ending on the rightmost column. There may be multiple nodes in the same row and same column. In such a case, sort these nodes by their values.

Return the *vertical order traversal* of the binary tree.



Input: root = [3,9,20,null,null,15,7]

Output: [[9],[3,15],[20],[7]]

Explanation:

Column -1: Only node 9 is in this column.

Column 0: Nodes 3 and 15 are in this column in that order from top to bottom.

Column 1: Only node 20 is in this column.

Column 2: Only node 7 is in this column.

Solution

```
import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.TreeMap;

class Solution {
    public List<List<Integer>> verticalTraversal(TreeNode root) {
        TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map =
new TreeMap<>();
        dfs(root, 0, 0, map);
        List<List<Integer>> result = new ArrayList<>();

        for (TreeMap<Integer, PriorityQueue<Integer>> ys : map.values())
        {
            List<Integer> list = new ArrayList<>();
            for (PriorityQueue<Integer> nodes : ys.values()) {
                while (!nodes.isEmpty()) {
                    list.add(nodes.poll());
                }
            }
            result.add(list);
        }
        return result;
    }

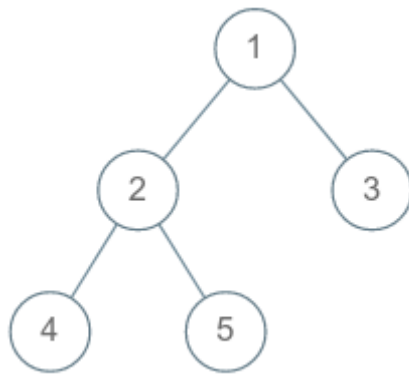
    // -1{9}, 0={3,15} , 1={20}, 2={7}

    private void dfs(TreeNode root, int depth, int level,
        TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>>
map) {
        if (root == null) {
```

```
        return;
    }
    map.putIfAbsent(depth, new TreeMap<>());
    map.get(depth).putIfAbsent(level, new PriorityQueue<>());
    map.get(depth).get(level).offer(root.val);

    dfs(root.left, depth - 1, level + 1, map);
    dfs(root.right, depth + 1, level + 1, map);
}
}
```

Problem:109: All Root to Leaf Paths In Binary Tree.



All Leaf Paths are

1 2 4

1 2 5

1 3

Sample Input 1 :

1

5

1 2 3 4 5 -1 -1 -1 -1 -1 -1

Sample Output 1 :

1 2 4

1 2 5

1 3

Sample Input 2 :

1

7

1 2 3 4 -1 5 6 -1 7 -1 -1 -1 -1 -1

Sample Output 2 :

1 3 5

1 3 6

1 2 4 7

Solution

```

import java.util.*;
import java.io.*;
/*****
Following is the class structure of the Node class:
class BinaryTreeNode {
  
```

```

        int data;
        BinaryTreeNode left;
        BinaryTreeNode right;

        BinaryTreeNode(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    };
    *****/
import java.util.ArrayList;
public class Solution {
    static void dfs(BinaryTreeNode root, ArrayList<String> result, String
curr) {
        // Base Case.
        if (root == null) {
            return;
        }
        // Current string consists of data at root node.
        curr += root.data + " ";

        // If both child of root is NULL, then pushing the path in result
array.
        if (root.left == null && root.right == null) {
            result.add(curr.substring(0, curr.length() - 1));
        }
        // If left child is not NULL, traverse on the left side.
        if (root.left != null) {
            dfs(root.left, result, curr);
        }
        // If right child is not NULL, traverse on the right side.
        if (root.right != null) {
            dfs(root.right, result, curr);
        }

        return;
    }

    public static ArrayList<String> allRootToLeaf(BinaryTreeNode root) {

        // It stores all the paths from root to leaf.
        ArrayList<String> result = new ArrayList<>();

        // It is used for traversal.
        dfs(root, result, "");
        return result;
    }
}

```

```
}
```

Problem:110: 662. Maximum Width of Binary Tree

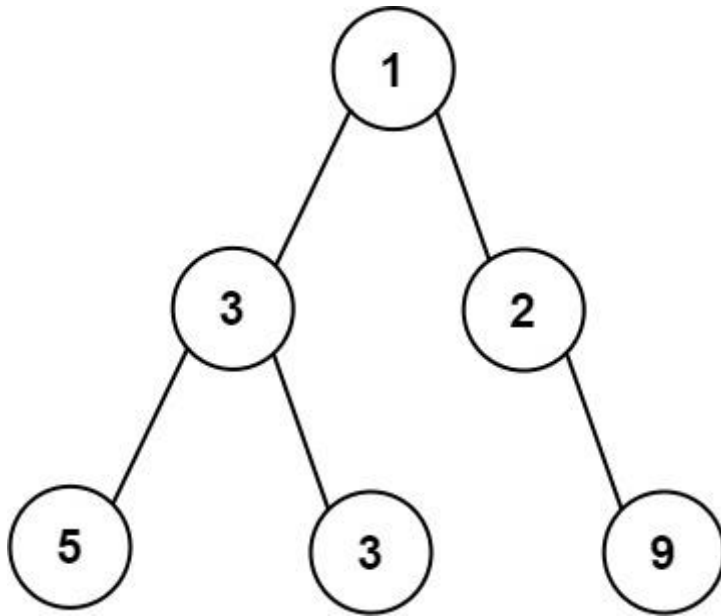
Given the root of a binary tree, return *the maximum width of the given tree*.

The **maximum width** of a tree is the maximum **width** among all levels.

The **width** of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is **guaranteed** that the answer will in the range of a **32-bit** signed integer.

Example 1:



Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Solution

```
/******
```

Following is the `TreeNode` class structure

```
class TreeNode {  
    int val;  
    TreeNode left;  
    TreeNode right;  
  
    TreeNode(int val) {  
        this.val = val;  
        this.left = null;  
        this.right = null;  
    }  
}
```

```
*****/
```

```
import java.util.*;
```

```
public class Solution {  
    public static int getMaxWidth(TreeNode root) {  
  
        LinkedList<TreeNode> queue = new LinkedList<>();
```

```
queue.offer(root);

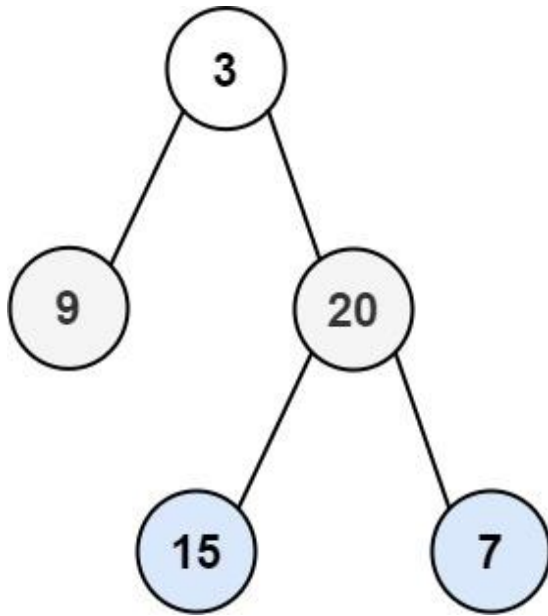
if (root == null)
    return 0;
int max = 0;
while (!queue.isEmpty()) {
    int len = queue.size();
    max = Math.max(max, len);

    for (int i = 0; i < len; i++) {
        TreeNode cur = queue.poll();
        if (cur == null)
            continue;
        if (cur.left != null) {
            queue.offer(cur.left);
        }
        if (cur.right != null) {
            queue.offer(cur.right);
        }
    }
}
return max;
}
```


Day 18: Binary Tree part-II

Problem:111: 102. Binary Tree Level Order Traversal

Given the root of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).



Input: root = [3,9,20,null,null,15,7]

Output: [[3],[9,20],[15,7]]

Solution

/*

Following is the structure used to represent the Binary Tree Node

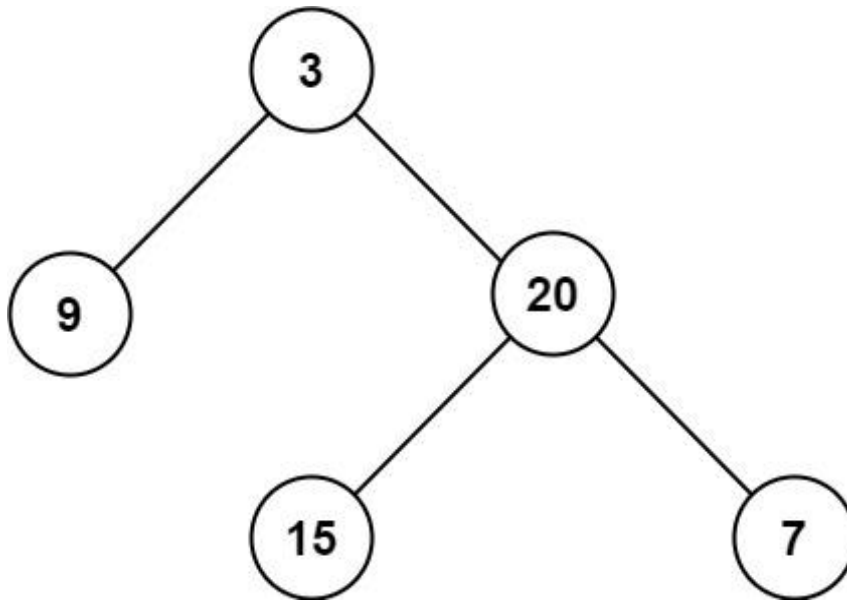
```
class BinaryTreeNode<T> {  
    T val;  
    BinaryTreeNode<T> left;  
    BinaryTreeNode<T> right;  
  
    public BinaryTreeNode(T val) {  
        this.val = val;  
        this.left = null;  
        this.right = null;  
    }  
}
```

```
*/  
  
import java.util.*;  
  
public class Solution {  
    public static ArrayList<Integer> getLevelOrder(BinaryTreeNode root) {  
        ArrayList<Integer> result = new ArrayList<Integer>();  
  
        LinkedList<BinaryTreeNode> queue = new LinkedList<>();  
        queue.offer(root);  
        while (!queue.isEmpty()) {  
            int len = queue.size();  
            for (int i = 0; i < len; i++) {  
                BinaryTreeNode cur = queue.poll();  
                if (cur == null)  
                    continue;  
                result.add(cur.val);  
                if (cur.left != null) {  
                    queue.offer(cur.left);  
                }  
  
                if (cur.right != null) {  
                    queue.offer(cur.right);  
                }  
            }  
        }  
  
        return result;  
    }  
}
```

Problem:112: 104. Maximum Depth of Binary Tree

Given the root of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.



Input: root = [3,9,20,null,null,15,7]

Output: 3

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
```

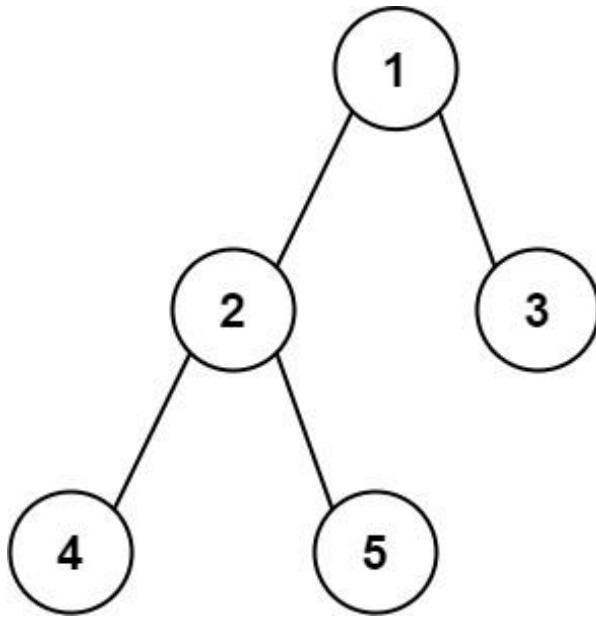
```
class Solution {  
    // Runtime: 0 ms, faster than 100.00% of Java online submissions for Maximum  
    // Depth of Binary Tree.  
    // Memory Usage: 38.7 MB, less than 93.51% of Java online submissions for  
    // Maximum Depth of Binary Tree.  
  
    public int maxDepth(TreeNode root) {  
        if (root == null)  
            return 0;  
  
        return helper(root);  
    }  
  
    private int helper(TreeNode root) {  
        if (root == null)  
            return 0;  
  
        int left = 0, right = 0;  
  
        left = helper(root.left);  
        right = helper(root.right);  
        left = left + 1;  
        right = right + 1;  
  
        return Math.max(left, right);  
    }  
}
```

Problem:113: 543. Diameter of Binary Tree

Given the root of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The **length** of a path between two nodes is represented by the number of edges between them.



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Solution

```
import java.util.*;  
/*****
```

Following is the Binary Tree Node structure:

```
class TreeNode<T> {  
    public T data;  
    public BinaryTreeNode<T> left;  
    public BinaryTreeNode<T> right;  
  
    TreeNode(T data) {  
        this.data = data;  
        left = null;  
    }  
}
```

```

        right = null;
    }
}

*****/

public class Solution {

    public static int diameterOfBinaryTree(TreeNode<Integer> root) {

        HashMap<Integer, Integer> map = new HashMap<>();

        if (root == null)
            return 0;
        int res[] = new int[1];
        res[0] = 0;
        helper(root, res);
        return res[0];
    }

    private static int helper(TreeNode root, int res[]) {
        if (root == null)
            return 0;

        int diameter = 0;
        int left = helper(root.left, res);
        int right = helper(root.right, res);
        diameter = Math.max(diameter, left + right);
        res[0] = Math.max(res[0], diameter);

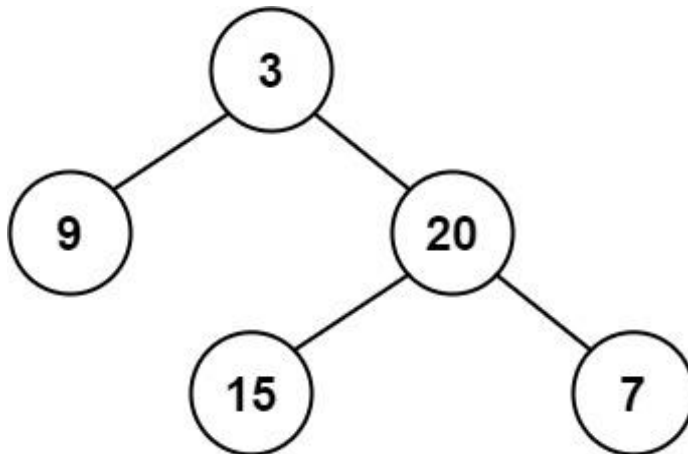
        return Math.max(left, right) + 1;
    }
}

```

Problem:114: 110. Balanced Binary Tree

Given a binary tree, determine if it is height-balanced.

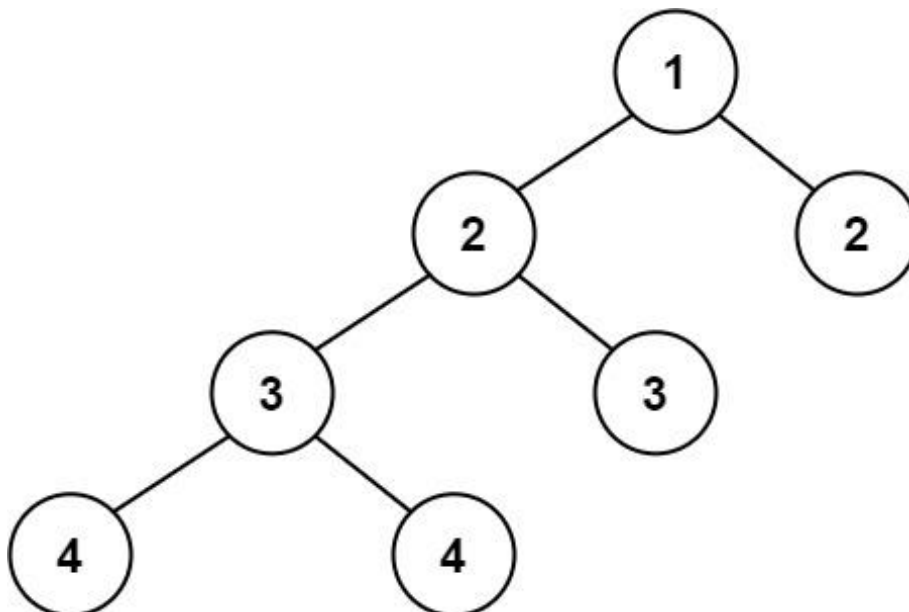
Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]

Output: false

Example 3:

Input: root = []

Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].

- $-10^4 \leq \text{Node.val} \leq 10^4$

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public boolean isBalanced(TreeNode root) {

        if (root == null) {
            return true;
        }

        return height(root) == -1 ? false : true;
    }

    private int height(TreeNode root) {

        if (root == null) {
            return 0;
        }

        int left = height(root.left);
        if (left == -1)
            return -1;
        int right = height(root.right);

        if (right == -1)
            return -1;
        if (Math.abs(left - right) > 1)
            return -1;

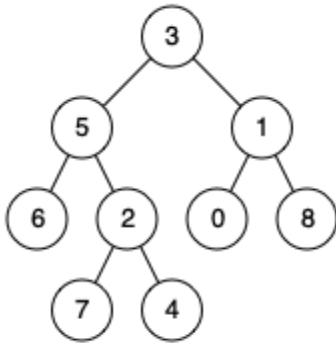
        return 1 + Math.max(left, right);
    }
}
```

Problem:115: 236. Lowest Common Ancestor of a Binary Tree

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Solution

```

/*****

```

Following is the TreeNode class structure

```

class TreeNode<T>

```

```

{
    public:
        T data;
        TreeNode<T> left;
        TreeNode<T> right;

        TreeNode(T data)
        {
            this.data = data;
            left = null;
            right = null;
        }
};

*****/

public class Solution
{
    public static int lowestCommonAncestor(TreeNode<Integer> root, int x, int y)
    {
        TreeNode<Integer> res = helper(root,x,y);

        return res.data;
    }

    private static TreeNode<Integer> helper(TreeNode<Integer> root, int p,int q){
        if(root==null || root.data ==p ||root.data ==q)return root;

        TreeNode<Integer> left = helper(root.left, p, q);
        TreeNode<Integer> right = helper(root.right, p, q);

        if(left!=null && right!=null) return root;
        else
            return (left==null)?right:left;
    }
}

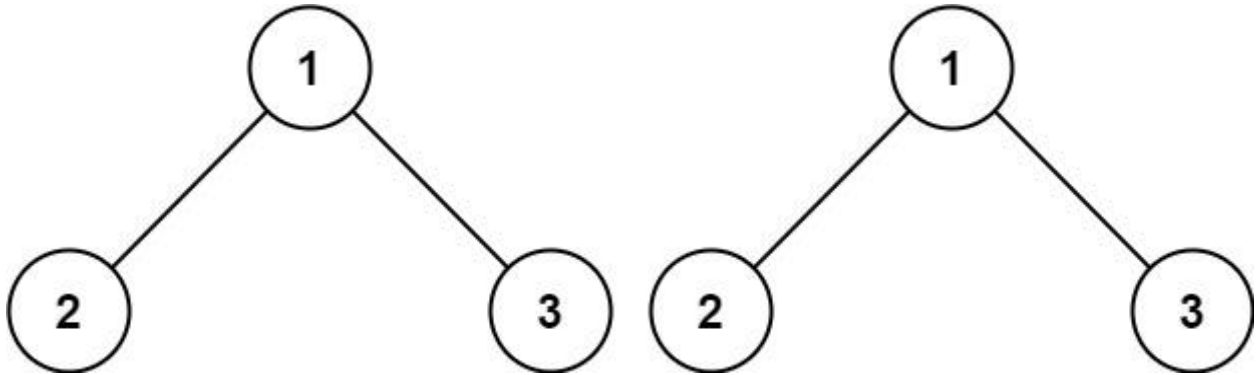
```

Problem:116: 100. Same Tree

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

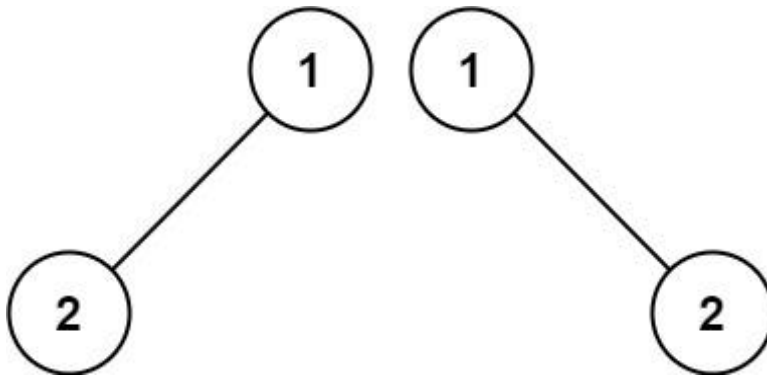
Example 1:



Input: $p = [1, 2, 3]$, $q = [1, 2, 3]$

Output: true

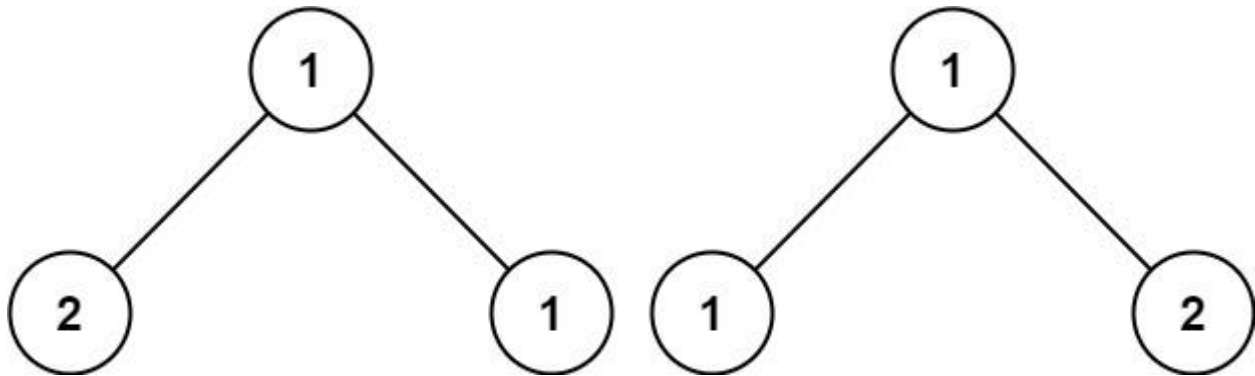
Example 2:



Input: $p = [1, 2]$, $q = [1, \text{null}, 2]$

Output: false

Example 3:



Input: p = [1,2,1], q = [1,1,2]

Output: false

Solution

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null)
            return true;
        if (p == null || q == null)
            return false;
        if (p.val != q.val)
            return false;

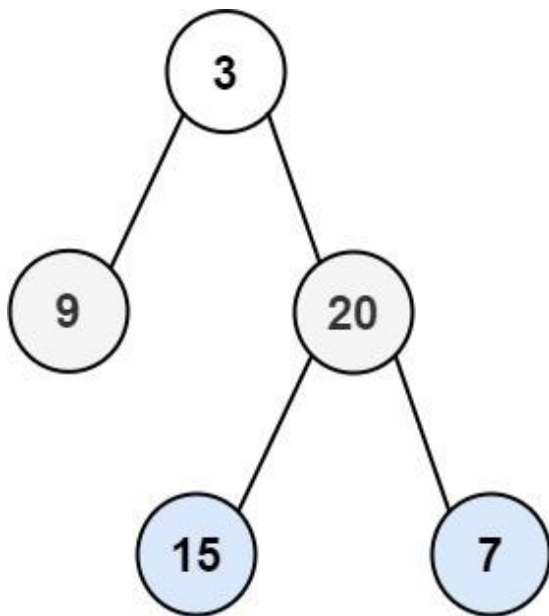
        return isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
    }
}

```

Problem:117: 103. Binary Tree Zigzag Level Order Traversal

Given the root of a binary tree, return *the zigzag level order traversal of its nodes' values*. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]

Output: [[1]]

Example 3:

Input: root = []

Output: []

Solution

```
/*  
    Following is the class used to represent the object/node of the Binary  
    Tree  
    class BinaryTreeNode<T> {
```

```

        T data;
        BinaryTreeNode<T> left;
        BinaryTreeNode<T> right;

        public BinaryTreeNode(T data) {
            this.data = data;
        }
    }
}

import java.util.*;

public class Solution {
    public static List<Integer> zigZagTraversal(BinaryTreeNode<Integer>
root) {

        List<Integer> result = new ArrayList<>();
        LinkedList<BinaryTreeNode<Integer>> queue = new LinkedList<>();
        queue.offer(root);
        int layer = 0;

        while (!queue.isEmpty()) {
            List<Integer> list = new ArrayList<>();
            int len = queue.size();

            for (int i = 0; i < len; i++) {
                BinaryTreeNode<Integer> cur = queue.poll();
                if (cur == null)
                    continue;

                if (layer % 2 != 0) {
                    list.add(0, cur.data);
                } else {
                    list.add(cur.data);
                }

                if (cur.left != null) {
                    queue.offer(cur.left);
                }
                if (cur.right != null) {
                    queue.offer(cur.right);
                }
            }
            result.addAll(list);
            layer++;
        }

        return result;
    }
}

```

}

DAY 19: BINARY TREE-III

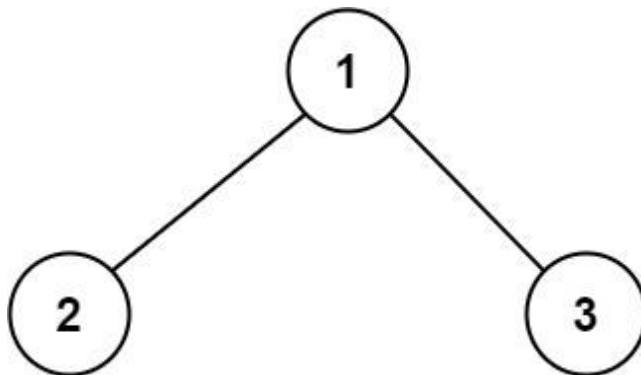
Problem:118: 124. Binary Tree Maximum Path Sum

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the root of a binary tree, return *the maximum path sum of any non-empty path*.

Example 1:

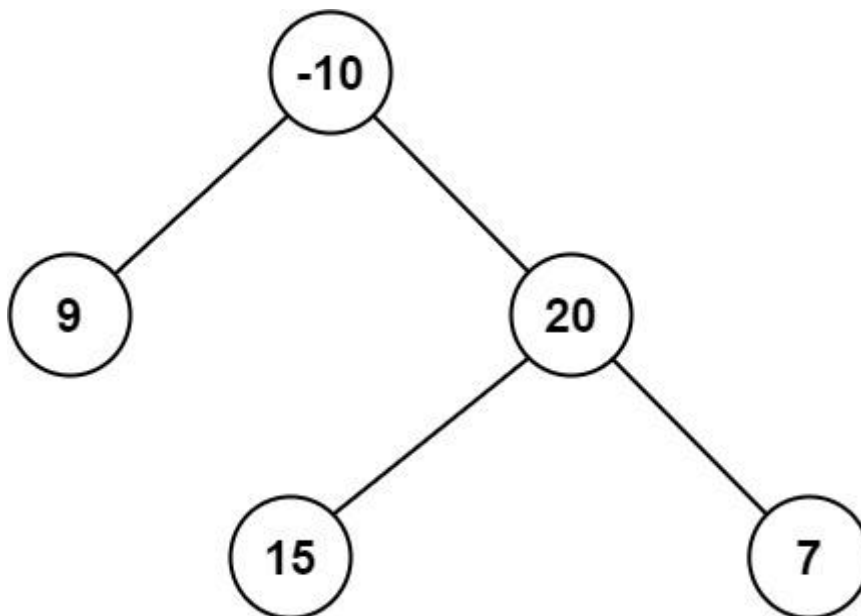


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int maxPathSum(TreeNode root) {
        if(root==null)return 0;

        int res[] = new int[1];
        res[0]=Integer.MIN_VALUE;
        helper(root,0,res);
        return res[0];
    }

    private int helper(TreeNode root,int val, int res[]){
        if(root==null)return 0;

        int left =0,right =0;
        left+=helper(root.left,val,res);
        right+=helper(root.right,val,res);

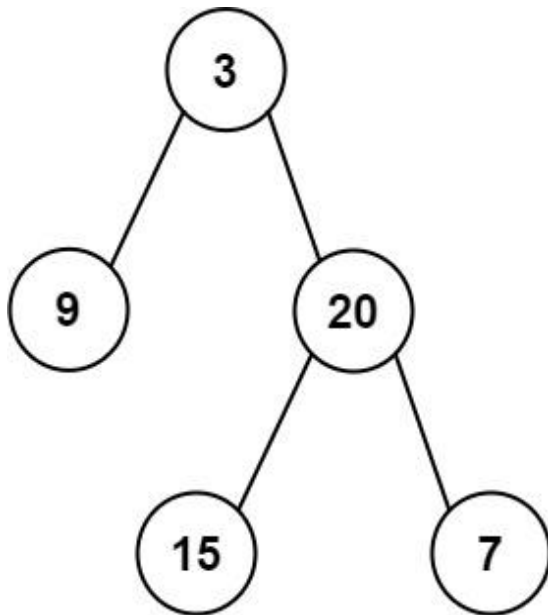
        res[0]= Math.max(res[0],left+right+root.val);

        int result =Math.max(left,right)+root.val;
        return result>=0?result:0;
    }
}
```

Problem:119: 105. Construct Binary Tree from Preorder and Inorder Traversal

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]

Output: [3,9,20,null,null,15,7]

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
```

```

*   TreeNode() {}
*   TreeNode(int val) { this.val = val; }
*   TreeNode(int val, TreeNode left, TreeNode right) {
*       this.val = val;
*       this.left = left;
*       this.right = right;
*   }
* }
*/
class Solution {
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        // [3,9,20,15,7]pre order ( Root, Left, Right)
        // [9,3,15,20,7] // Left, Root, Right

        // Step 1:identity which is easy to get root.
        // since preorder we have root at first so use that to find root node.
        // Step 2: add a lookup for other (inorder)

        HashMap<Integer, Integer> map = new HashMap<>();
        int index = 0;
        for (int val : inorder) {
            map.put(val, index++);
        }
        int preSt = 0;
        int preEnd = preorder.length - 1;
        int inSt = 0;
        int inEnd = inorder.length - 1;

        return helper(preorder, preSt, preEnd, map, inorder, inSt, inEnd);
    }
    private TreeNode helper(int[] preorder, int preSt, int preEnd, HashMap<Integer,
Integer> map, int[] inorder,
        int inSt, int inEnd) {

        if (preSt > preEnd || inSt > inEnd)
            return null;

        int rootVal = preorder[preSt]; // rootval;
        int ri = map.get(rootVal); // inorder found the root index;
        int cur = inorder[ri]; // get the rootvalue at the index of preorder.

        // inorder
        // [9,3,15,20,7] // Left,Root,right
        // |
        TreeNode root = new TreeNode(cur);
        root.left = helper(preorder, preSt + 1, preEnd, map, inorder, inSt, ri - 1);
        root.right = helper(preorder, preSt + ri - inSt + 1, preEnd, map, inorder, ri +
1, inEnd);

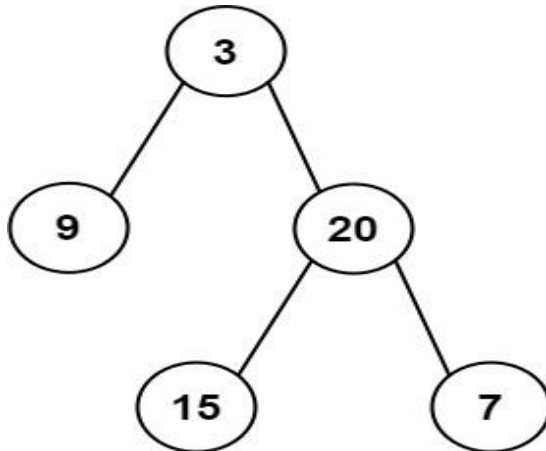
        return root;
    }
}

```

Problem:120: 106. Construct Binary Tree from Inorder and Postorder Traversal

Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]

Output: [3,9,20,null,null,15,7]

Solution

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
// Runtime: 1 ms, faster than 96.64% of Java online submissions for
Construct Binary Tree from Inorder and Postorder Traversal.
//Memory Usage: 39 MB, less than 58.88% of Java online submissions for
Construct Binary Tree from Inorder and Postorder Traversal.

    public TreeNode buildTree(int[] inorder, int[] postorder) {
        HashMap<Integer,Integer> map = new HashMap<>();

```

```
//inorder = [9,(3),15,20,7]
// L Root Right

//postorder = [9,15,7,(20),(3)]
//Left,Right,Root

//Step 1: postorder last is root.
//Step 2: assign left and right nodes in list
//      3
//     / \
//  [9]  [15,20,27]
//
int n = inorder.length;

for(int i =0;i<n;i++){
    map.put(inorder[i],i);
}

return helper(inorder,0, inorder.length-1,postorder,0,
postorder.length-1,map);
}
```

```
private TreeNode helper(int[] inorder,int instart,int inEnd,
                        int[] postorder,int pstart,int pEnd,
                        HashMap<Integer,Integer> map){

    if(instart>inEnd && pstart>pEnd)return null;
    int rval = postorder[pEnd];

    TreeNode root = new TreeNode(rval);
    int ri = map.get(rval);

    //inorder = [9,(3),15,20,7]
    // L Root Right

    //postorder = [9,15,7,(20),(3)]

    //left - [9->7] -> post order left= root of 20 is 3 in inorder
    //0 + 3-0-1 = 2 so postorder left = 9,15,7
    //Left,Right,Root
```

```

        root.left = helper(inorder,instart,ri-1,postorder,pstart,(pstart+ri-
instart-1) ,map);
        root.right = helper(inorder,ri+1,inEnd,postorder,(pstart+ri-
instart),pEnd-1, map);

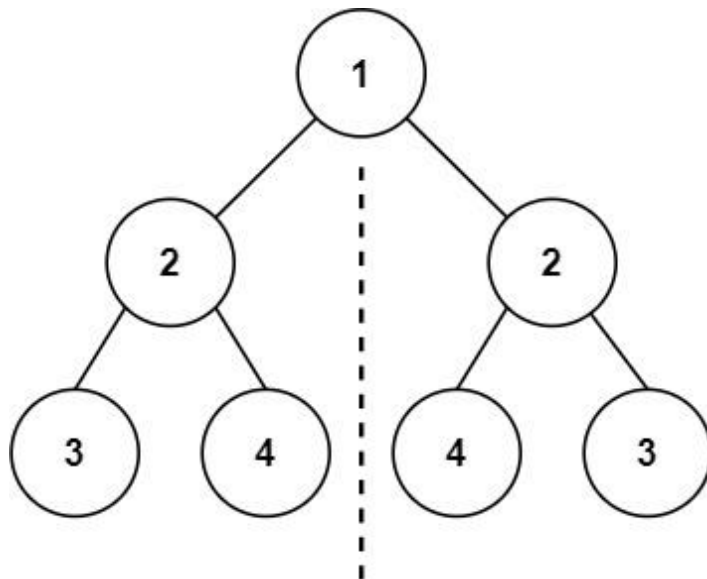
    return root;
}
}

```

Problem:121: 101. Symmetric Tree

Given the root of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

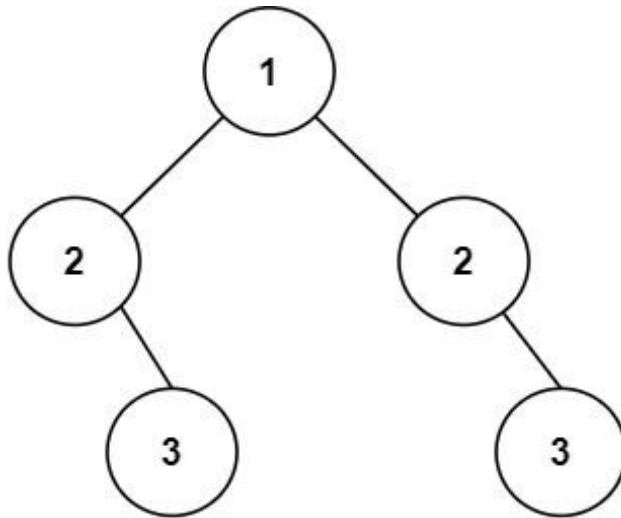
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

Solution:

/******

Following is the representation for the Binary Tree Node:

```

class BinaryTreeNode<T> {
    T data;
    BinaryTreeNode<T> left;
    BinaryTreeNode<T> right;

    public BinaryTreeNode(T data) {
        this.data = data;
    }
}
  
```

*****/

```

public class Solution {
  
```

```

    public static boolean isSymmetric(BinaryTreeNode<Integer> root) {
        if (root == null)
            return true;

        return helper(root.left, root.right);
    }
  
```

```

    private static boolean helper(BinaryTreeNode<Integer> left,
    BinaryTreeNode<Integer> right) {
        if (left == right)
            return true;
        if (left != null && right != null && left.data - right.data != 0)
  
```



```

        return false;

    return helper(left.left, right.right) && helper(left.right,
right.left);
    }
}

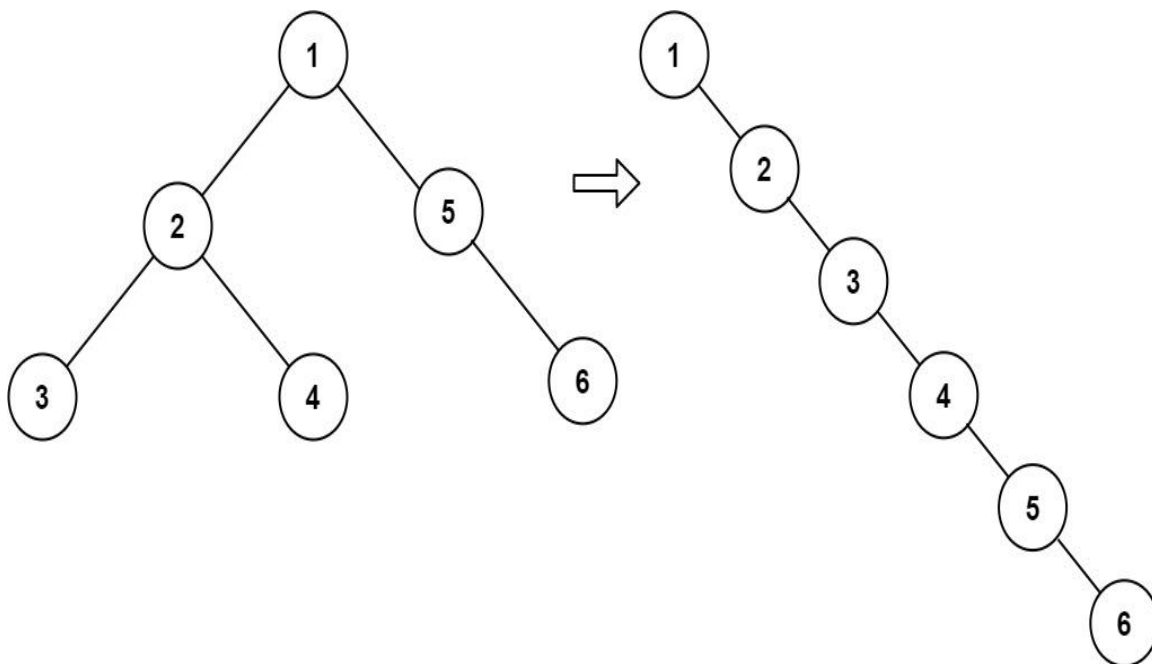
```

Problem:122: 114. Flatten Binary Tree to Linked List

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a [pre-order traversal](#) of the binary tree.

Example 1:



Input: root = [1,2,5,3,4,null,6]

Output: [1,null,2,null,3,null,4,null,5,null,6]

Example 2:

Input: root = []

Output: []

Example 3:

Input: root = [0]

Output: [0]

Solution:

```
import java.util.* ;
import java.io.*;
/*****
```

Following is the `TreeNode` class structure.

```
class TreeNode<T>
{
    public T data;
    public TreeNode<T> left;
    public TreeNode<T> right;

    TreeNode(T data)
    {
        this.data = data;
        left = null;
        right = null;
    }
}

*****/

public class Solution {
    public static TreeNode<Integer> flattenBinaryTree(TreeNode<Integer>
root) {
        TreeNode<Integer> rootNode = new TreeNode<Integer>(-1);
        TreeNode<Integer> rootHead = rootNode;
        preorder(root, rootNode);
        return rootHead.right;
    }

    private static void preorder(TreeNode<Integer> root, TreeNode<Integer>
node) {
        if (root == null)
            return;
        Stack<TreeNode> stack = new Stack<TreeNode>();
        stack.push(root);
        while (!stack.empty()) {
            TreeNode<Integer> cur = stack.pop();
            node.right = cur;
            node = node.right;
            if (cur.right != null) {
```

```

        stack.push(cur.right);
    }
    if (cur.left != null) {
        stack.push(cur.left);
    }
}
node.right = null;
}
}

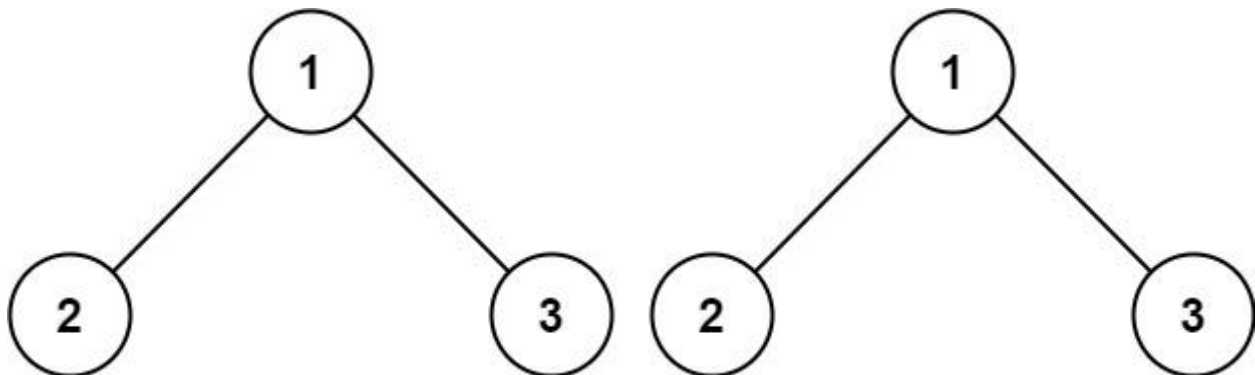
```

Problem:123: 100. Same Tree

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

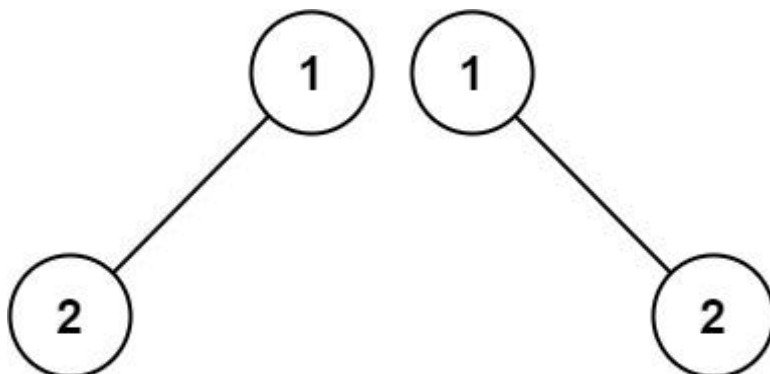
Example 1:



Input: p = [1,2,3], q = [1,2,3]

Output: true

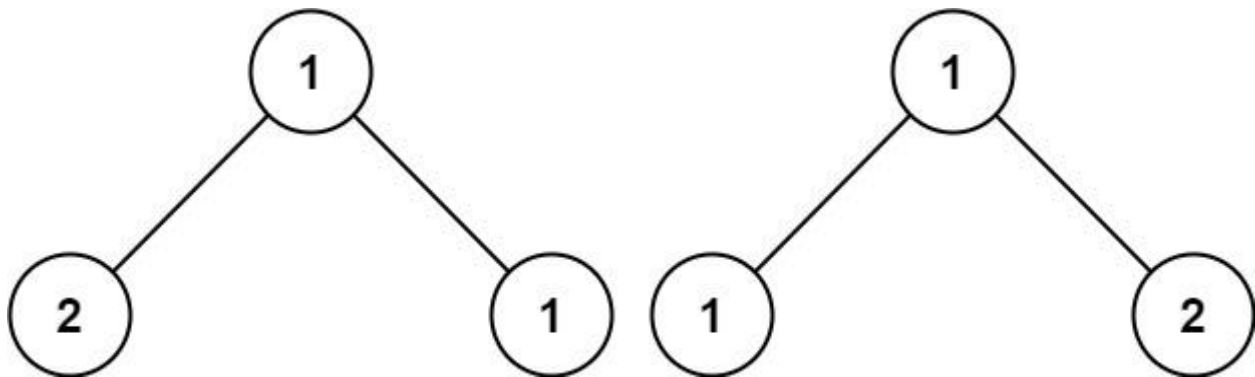
Example 2:



Input: p = [1,2], q = [1,null,2]

Output: false

Example 3:



Input: p = [1,2,1], q = [1,1,2]

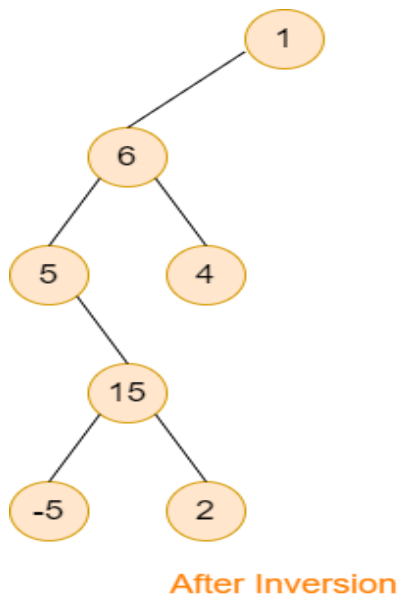
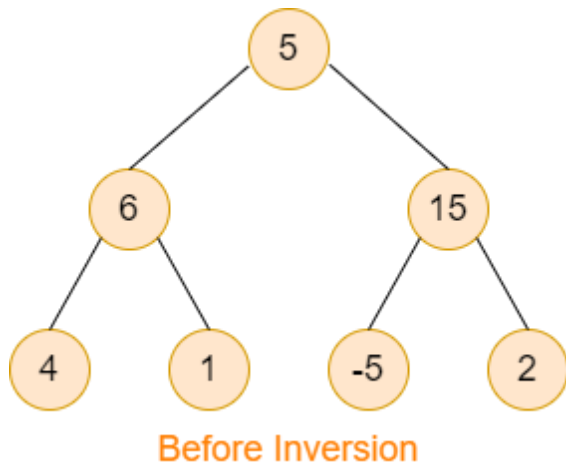
Output: false

Solution:

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if(p==null && q==null)return true;
        if(p==null || q==null)return false;
        if(p.val !=q.val)return false;

        return isSameTree(p.left,q.left) && isSameTree(p.right,q.right);
    }
}
```

Problem:124: Invert a Binary Tree



Sample Input 1:

```

1
1
5 6 15 4 1 -5 2 -1 -1 -1 -1 -1 -1 -1

```

Sample Output 1:

```

1 6 -1 5 4 -1 15 -1 -1 -5 2 -1 -1 -1 -1

```

Explanation of sample input 1:

For the first test case, the explanation is provided in the description.

Sample Input 2:

```

2
2

```

```
1 2 3 -1 -1 -1 -1
3
1 2 -1 3 -1 -1 -1
```

Solution:

```
/*
    Time complexity: O(N)
    Space complexity: O(N)

    Where 'N' is the number of nodes in the given binary tree.
*/
import java.util.Stack;

public class Solution {
    // Function which helps in filling the required path in the stack.
    private static boolean helper(TreeNode<Integer> root,
Stack<TreeNode<Integer>> st, TreeNode<Integer> leaf) {
        st.push(root);
        if (root.left == null && root.right == null) {
            if (root.data.intValue() == leaf.data.intValue()) {
                return true;
            }
            else {
                st.pop();
                return false;
            }
        }
        boolean left = false, right = false;
        if (root.left != null) {
            left = helper(root.left, st, leaf);
        }
        if (left) {
            return true;
        }
        if (root.right != null) {
            right = helper(root.right, st, leaf);
        }
        if (right) {
            return true;
        }
    }
}
```

```

    }
    st.pop();
    return false;
}

```

```

    public static TreeNode<Integer> invertBinaryTree(TreeNode<Integer> root,
TreeNode<Integer> leaf) {
        if (root == null) {
            return root;
        }
        Stack<TreeNode<Integer>> st = new Stack<TreeNode<Integer>>();
        helper(root, st, leaf);
        TreeNode<Integer> head = st.peek();
        st.pop();
        TreeNode<Integer> cur = head;

        while (!st.isEmpty()) {
            TreeNode<Integer> temp = st.peek();
            st.pop();
            if (temp.left == cur) {
                temp.left = null;
                cur.left = temp;
            } else {
                temp.right = temp.left;
                temp.left = null;
                cur.left = temp;
            }
            cur = cur.left;
        }

        return head;
    }
}

```

Problem:125: Children Sum Property

Sample Input 1 :

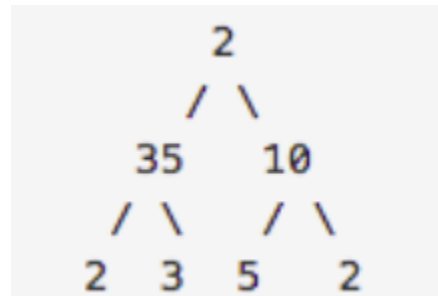
1
2 35 10 2 3 5 2 -1 -1 -1 -1 -1 -1 -1

Sample Output 1 :

Valid (One of the possible answers is : 45 35 10 32 3 8 2 -1 -1 -1 -1 -1, thus if the user modifies the given tree like this, the output printed will be valid).

Explanation For Sample Input 1 :

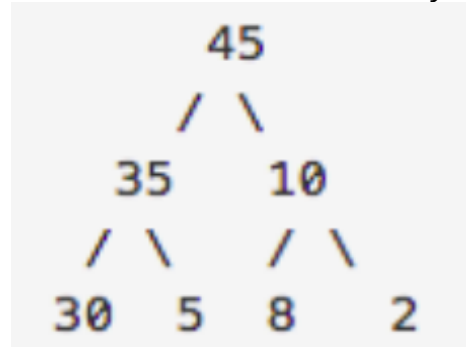
The tree can be represented as follows :



The value at the root node is 2 which is less than the sum of its children's values (35 + 10). So we simply increase the value at the root node to 45. The node with value = 35 has children with values 2 and 3 so their sum i.s $2 + 3 = 5 < 35$. As we can't decrement any value, so instead we have to increase the sum of children and thus we update 35's children (2 and 3) as 30 and 5 so that $30 + 5 = 35$ and the same is done for the children of the node with value = 10.

The final tree looks like this :

Note that there can be many other valid solutions.



Solution:

```
/******
```

Following is the Binary Tree node structure

```
class BinaryTreeNode < Integer > {  
    int data;  
    BinaryTreeNode < Integer > left;  
    BinaryTreeNode < Integer > right;  
  
    public BinaryTreeNode(int data) {  
        this.data = data;  
    }  
}
```

```
*****/
```

```
public class Solution {  
    public static void changeTree(BinaryTreeNode<Integer> root) {  
  
        postorder(root);  
    }  
  
    private static void postorder(BinaryTreeNode<Integer> root) {  
        if (root == null)  
            return;  
  
        int child = 0;  
  
        if (root.left != null) {  
            child += root.left.data;  
        }  
        if (root.right != null) {  
            child += root.right.data;  
        }  
  
        if (child >= root.data) {  
            root.data = child;  
        } else {  
            if (root.left != null) {  
                root.left.data = root.data;  
            }  
  
            if (root.right != null) {  
                root.right.data = root.data;  
            }  
        }  
    }  
}
```

```
    }  
    postorder(root.left);  
    postorder(root.right);  
    int tot = 0;  
  
    if (root.left != null) {  
        tot += root.left.data;  
    }  
    if (root.right != null) {  
        tot += root.right.data;  
    }  
    if (root.left != null || root.right != null) {  
        root.data = tot;  
    }  
    }  
}
```

DAY 20: BINARY SEARCH TREE-I:

Problem:126:116. Populating Next Right Pointers in Each Node

You are given a **perfect binary tree** where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
    int val;
    Node *left;
    Node *right;
    Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

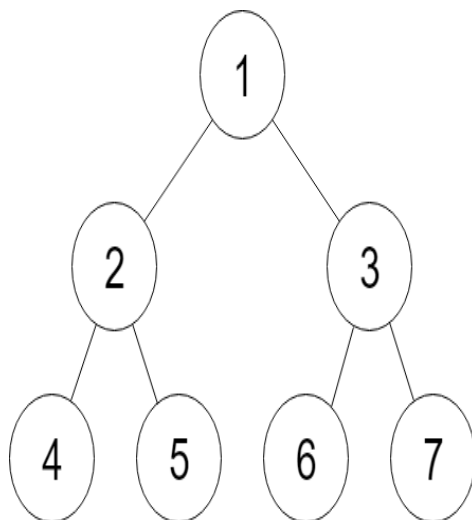
Example 1:

Figure A

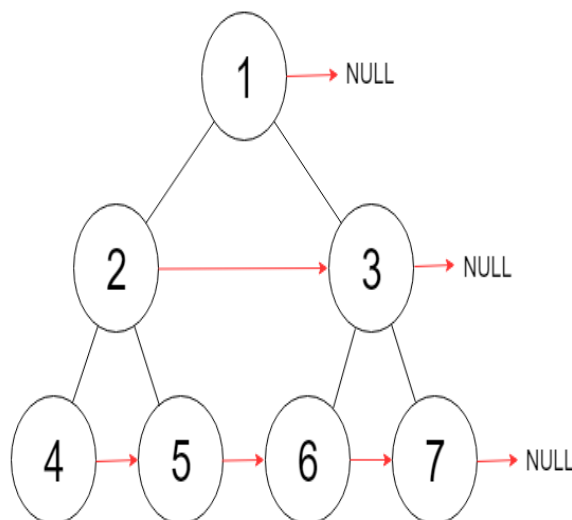


Figure B

Input: root = [1,2,3,4,5,6,7]

Output: [1,#,2,3,#,4,5,6,7,#]

Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2^{12} - 1]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

SOLUTION:

```
import java.util.* ;
import java.io.*;
/*
----- Binary Tree node class for reference -----

class BinaryTreeNode<T> {
    public T data;
    public BinaryTreeNode<T> left;
    public BinaryTreeNode<T> right;
    public BinaryTreeNode<T> next;

    BinaryTreeNode(T data) {
        this.data = data;
        left = null;
        right = null;
        next = null;
    }
};

*/

public class Solution {
    public static void connectNodes(BinaryTreeNode<Integer> root) {
        helper(root);
    }

    private static void helper(BinaryTreeNode<Integer> root){
        LinkedList<BinaryTreeNode<Integer>> queue = new LinkedList<>();
        queue.offer(root);
    }
}
```

```

        while(!queue.isEmpty()){
            int n = queue.size();
            for(int i=0;i<n;i++){
                BinaryTreeNode<Integer> cur = queue.poll();
                if(i!=n-1){
                    cur.next = queue.peek();
                }
                if(cur.left!=null){
                    queue.offer(cur.left);
                }
                if(cur.right!=null){
                    queue.offer(cur.right);
                }
            }
        }
    }
}

```

Problem:127: Search In BST

Sample Input 1:

```

2
1 1
1 -1 -1
7 8
4 2 6 1 3 5 7 -1 -1 -1 -1 -1 -1 -1 -1

```

Sample Output 1:

```

True
False

```

Explanation For Sample Input 1:

In the first test case, there is only one node and it has data 1, thus we should print 'True'.

In the second test case, there is no node having data 8. See the problem statement for the picture of this BST.

Solution:

```

import java.util.* ;
import java.io.*;

public class Solution {
    public static Boolean searchInBST(BinaryTreeNode<Integer> root, int x)
    {
        return helper (root,x);
    }

    private static boolean helper(BinaryTreeNode<Integer> root, int val){

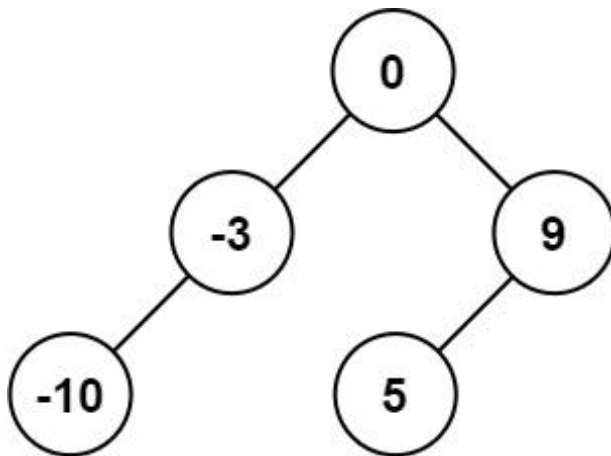
```

```
    if(root==null)return false;  
    if(root.data == val)return true;  
    if(root.data > val){  
        return helper(root.left,val);  
    }  
    return helper(root.right,val);  
}  
}
```

Problem:128: 108. Convert Sorted Array to Binary Search Tree

Given an integer array `nums` where the elements are sorted in **ascending order**, convert it to a **height-balanced** binary search tree.

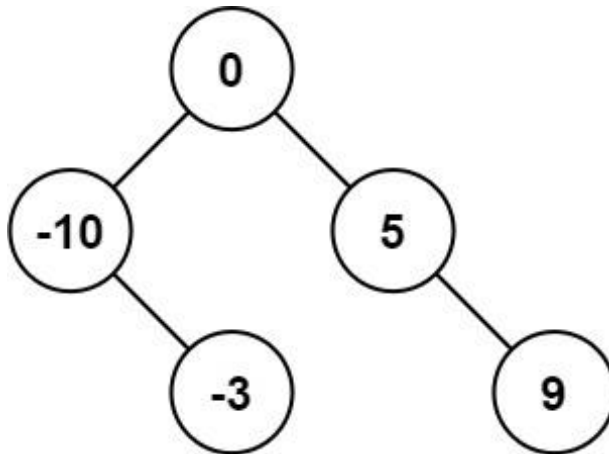
Example 1:



Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:

**Solution:**

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {

        // if(nums==null ||(nums!=null && nums.length==0))return null;

        return helper(nums, 0, nums.length - 1);
    }

    private TreeNode helper(int nums[], int low, int high) {
        if (low > high)
            return null;
        int mid = low + (high - low) / 2;
    }
}

```



```
TreeNode root = new TreeNode(nums[mid]);
root.left = helper(nums, low, mid - 1);
root.right = helper(nums, mid + 1, high);

return root;
}
```

Problem:129: 1008. Construct Binary Search Tree from Preorder Traversal

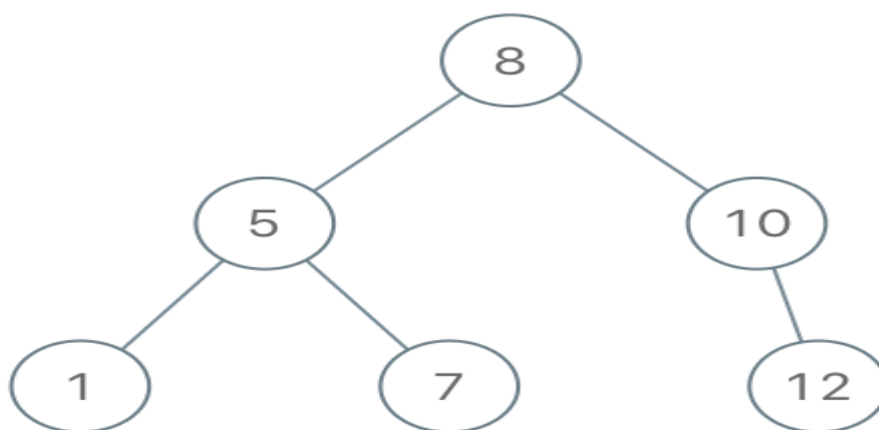
Given an array of integers preorder, which represents the **preorder traversal** of a BST (i.e., **binary search tree**), construct the tree and return its *root*.

It is **guaranteed** that there is always possible to find a binary search tree with the given requirements for the given test cases.

A **binary search tree** is a binary tree where for every node, any descendant of Node.left has a value **strictly less than** Node.val, and any descendant of Node.right has a value **strictly greater than** Node.val.

A **preorder traversal** of a binary tree displays the value of the node first, then traverses Node.left, then traverses Node.right.

Example 1:



Input: preorder = [8,5,1,7,10,12]

Output: [8,5,10,1,7,null,12]

Solution:

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
// Runtime: 0 ms, faster than 100.00% of Java online submissions for
Construct Binary Search Tree from Preorder Traversal.
// Memory Usage: 37.2 MB, less than 29.23% of Java online submissions for
Construct Binary Search Tree from Preorder Traversal.

    public TreeNode bstFromPreorder(int[] preorder) {
        TreeNode root = null;
        for (int i : preorder) {
            root = helper(root, i);
        }

        return root;
    }

    private TreeNode helper(TreeNode root, int val) {
        if (root == null) {
            return new TreeNode(val);
        } else if (root.val > val) {
            root.left = helper(root.left, val);
        } else {
            root.right = helper(root.right, val);
        }
        return root;
    }
}

```

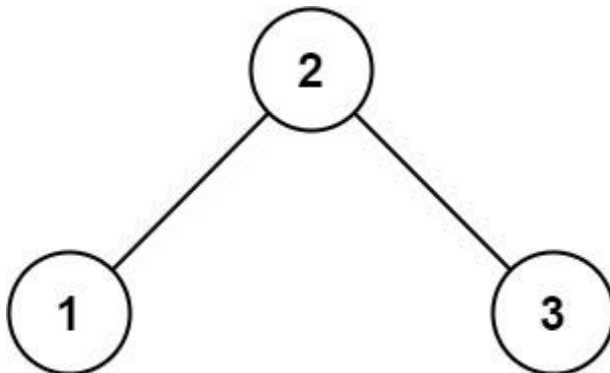
Problem:130: 98. Validate Binary Search Tree

Given the root of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

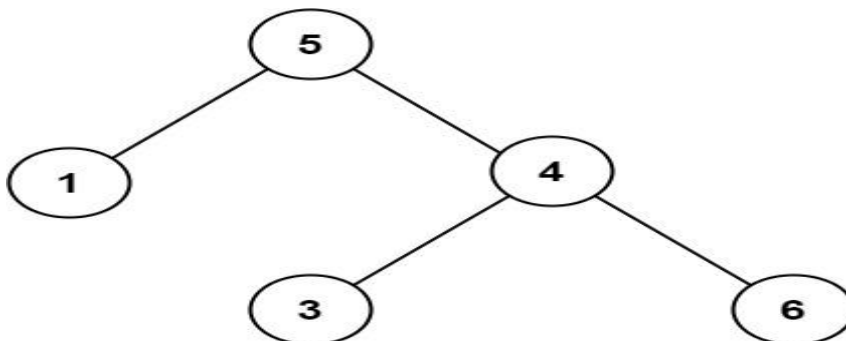
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Solution:

```
import java.util.* ;
```

```
import java.io.*;
/*****
```

Following is the Binary Tree node structure

```
class BinaryTreeNode<Integer> {
    int data;
    BinaryTreeNode<Integer> left;
    BinaryTreeNode<Integer> right;

    public BinaryTreeNode(int data) {
        this.data = data;
    }
}
```

```
*****/
```

```
public class Solution {
    public static boolean validateBST(BinaryTreeNode<Integer> root) {
        if(root==null)return true;

        return helper(root, Integer.MIN_VALUE,Integer.MAX_VALUE);
    }

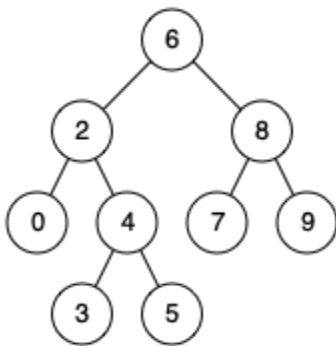
    private static boolean helper(BinaryTreeNode<Integer> root,
                                   int left, int right){
        if(root==null)return true;
        int rootVal = root.data;
        //left should <= root
        //right should >= root.
        if(left>rootVal || right<rootVal)return false;

        //every recursion -> right val for left node and left val for right
node.
        return helper(root.left,left,root.data) &&
helper(root.right,root.data,right);
    }
}
```

Problem:131: 235. Lowest Common Ancestor of a Binary Search Tree

iven a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

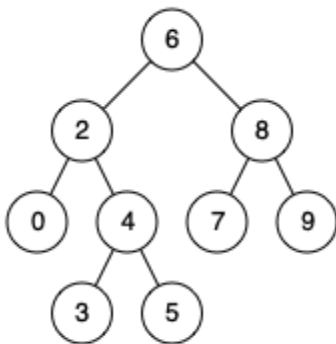
According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:

Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */

class Solution {
// Runtime: 3 ms, faster than 100.00% of Java online submissions for
// Lowest Common Ancestor of a Binary Search Tree.
// Memory Usage: 39.6 MB, less than 89.59% of Java online submissions for
// Lowest Common Ancestor of a Binary Search Tree.

    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode
q) {
        if(root==null)return null;

        return lca(root,p,q);
    }

    private TreeNode lca( TreeNode root, TreeNode p, TreeNode q){
        if(root==null)return null;

        //check whether p lies in left

        if(p.val > root.val && q.val >root.val){
            //if root val > p and q val
            //search right
            return lca(root.right,p,q);
        }else if(p.val < root.val && q.val <root.val){
            return lca(root.left,p,q);
        }

        return root;
    }
}
```

Problem:132: Predecessor And Successor In BST**Sample Input 1:**

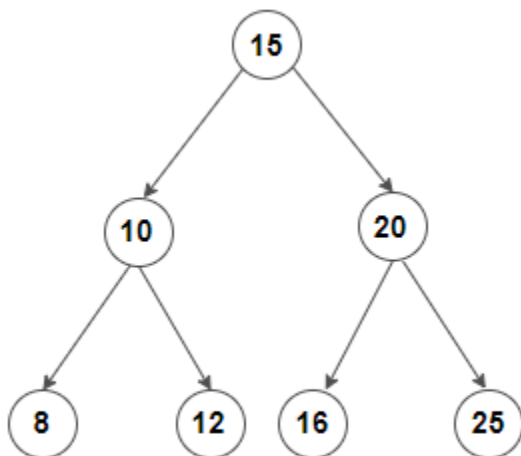
```
1
15 10 20 8 12 16 25 -1 -1 -1 -1 -1 -1 -1
10
```

Sample output 1:

```
8 12
```

Explanation of Sample output 1:

The tree can be represented as follows:



The inorder traversal of this tree will be 8 10 12 15 16 20 25.

Since the node with data 8 is on the immediate left of the node with data 10 in the inorder traversal, the node with data 8 is the predecessor.

Since the node with data 12 is on the immediate right of the node with data 10 in the inorder traversal, the node with data 12 is the successor.

Solution

```
import java.util.* ;
import java.io.*;
/*****
Following is the Binary Tree node structure

class TreeNode<T> {
    public T data;
    public TreeNode<T> left;
    public TreeNode<T> right;

    TreeNode(T data) {
        this.data = data;
        left = null;
        right = null;
    }
}

*****/

import java.util.ArrayList;

public class Solution {
    public static ArrayList<Integer> predecessorSuccessor(TreeNode<Integer>
root, int key) {
    ArrayList<Integer> result = new ArrayList<>(Arrays.asList(null,
null));
    while (root != null) {
        if (root.data == key) {
            if (root.left != null) {
                result.set(0, findMax(root.left).data);
            }
            if (root.right != null) {
                result.set(1, findMin(root.right).data);
            }
            break;
        } else if (root.data > key) {
            result.set(1, root.data);
            root = root.left;
        } else {
```



```
                result.set(0, root.data);
                root = root.right;
            }
        }
        return result;
    }

    public static TreeNode findMin(TreeNode<Integer> root) {
        while (root.left != null) {
            root = root.left;
        }
        return root;
    }

    public static TreeNode<Integer> findMax(TreeNode<Integer> root) {
        while (root.right != null) {
            root = root.right;
        }
        return root;
    }
}
```

DAY 21: BINARY SEARCH TREE-I:

Problem:133: Floor in BST**Sample Input 1:**

```

2
10 5 15 2 6 -1 -1 -1 -1 -1
7
2 1 3 -1 -1 -1 -1
2

```

Sample Output 1:

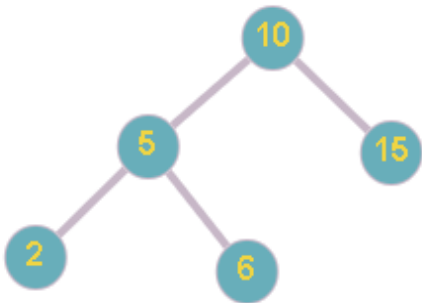
```

6
2

```

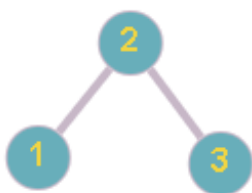
Explanation of Sample Input 1:

In the first test case, the BST looks like as below:



The greatest value node of the BST which is smaller than or equal to 7 is 6.

In the second test case, the BST looks like as below:



The greatest value node of the BST which is smaller than or equal to 2 is 2.

Solution

```

/*****

```

Following is the TreeNode class structure

```

class TreeNode<T>

```

```

{
    public:
        T data;
        TreeNode<T> left;
        TreeNode<T> right;

        TreeNode(T data)
        {
            this.data = data;
            left = null;
            right = null;
        }
};

*****/

public class Solution {
    public static int floorInBST(TreeNode<Integer> root, int X) {

        return helper(root, X);
    }

    private static int helper(TreeNode<Integer> root, int target) {
        // Base case
        if (root == null) {
            return -1;
        }

        // We found equal key
        if (root.data > target) {
            return root.data;
        }

        // If root's key is smaller,
        // ceil must be in right subtree
        if (root.data < target) {
            return helper(root.right, target);
        }

        // Else, either left subtree or root
        // has the ceil value
        int ceil = helper(root.left, target);

        return (ceil >= target) ? ceil : root.data;
    }
}

```

Problem:134: Ceil from BST

Sample Input 1:

```

2
8 5 10 2 6 -1 -1 -1 -1 -1 7 -1 -1
4
8 5 10 2 6 -1 -1 -1 -1 -1 7 -1 -1
7

```

Sample Output 1:

```

5
7

```

Explanation for Sample Output 1:

In the first test case, we traverse the tree starting from the root node which has a value of 8. Now the value of its left child is 5 and its right child is 10. Key-value 4 is less than the left child so we traverse the left subtree. Now we reach a node with value 5 and then again repeat the above process till we reach a null node. Finally, we return 5 as our answer.

In the second test case, we traverse the tree starting from the root node which has a value of 8. Now the value of its left child is 5 and its right child is 10. Key-value 7 is less than the right child and more than the left child, so we traverse the right subtree. Now we reach a node with value 10 and then again follow the same procedure and reach a node with value 7 and stop there and return 7 as our final answer.

Sample Input 2:

```

2
55 25 82 13 34 67 86 6 21 28 47 61 70 84 92 1 10 17 24 26 29 45 54 56 65 68
81 83 85 91
96 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1
34
84 19 97 0 56 96 99 -1 8 50 83 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
50

```

Sample Output 2:

```

34
50

```

Solution:

```

/*****

```

Following is the `TreeNode` class structure

```
class TreeNode<T>
{
    public:
        T data;
        TreeNode<T> left;
        TreeNode<T> right;

        TreeNode(T data)
        {
            this.data = data;
            left = null;
            right = null;
        }
};

*****/

public class Solution {
    public static int findCeil(TreeNode<Integer> node, int input) {
        return helper(node, input);
    }

    private static int helper(TreeNode<Integer> node, int input) {
        // Base case
        if (node == null) {
            return -1;
        }

        // We found equal key
        if (node.data == input) {
            return node.data;
        }

        // If root's key is smaller,
        // ceil must be in right subtree
        if (node.data < input) {
            return helper(node.right, input);
        }

        // Else, either left subtree or root
        // has the ceil value
        int ceil = helper(node.left, input);

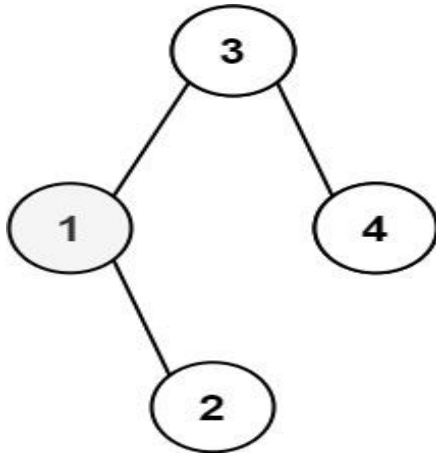
        return (ceil >= input) ? ceil : node.data;
    }
}
```

```
}
```

Problem:135: 230. Kth Smallest Element in a BST

Given the root of a binary search tree, and an integer k , return the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.

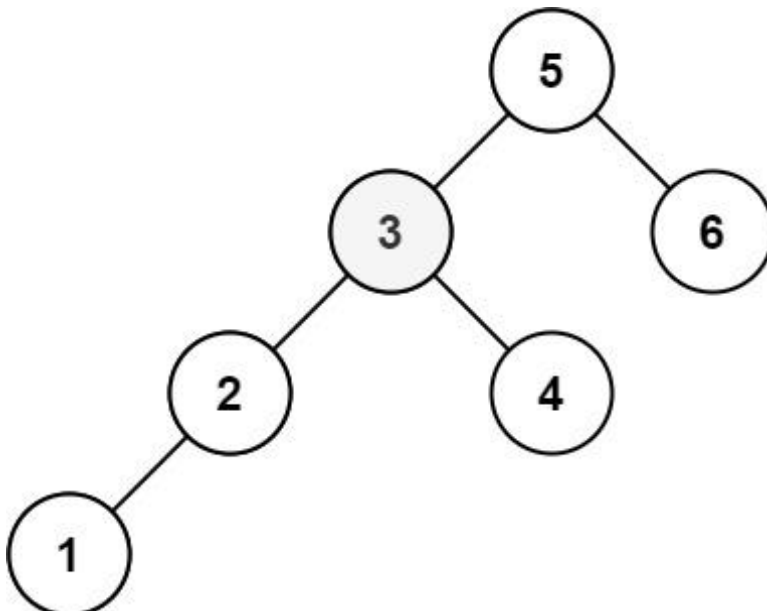
Example 1:



Input: root = [3,1,4,null,2], k = 1

Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3

Output: 3

Solution:

```
import java.util.* ;
import java.io.*;
/*****
```

Following is the Binary Search Tree node structure

```
class TreeNode<T> {
    public T data;
    public TreeNode<T> left;
    public TreeNode<T> right;

    TreeNode(T data) {
        this.data = data;
        left = null;
        right = null;
    }
}
```

```
*****/
```

```
import java.util.ArrayList;

public class Solution {
    public static int KthLargestNumber(TreeNode<Integer> root, int k) {
        TreeNode<Integer> cur = root;
        Stack<TreeNode<Integer>> st = new Stack<>();

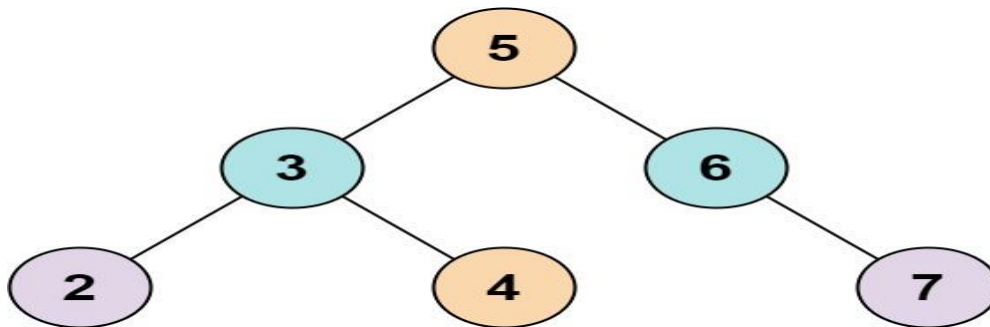
        while (cur != null) {
            st.push(cur);
            cur = cur.right;
        }
        // right is biggest from right traverse left
        while (!st.isEmpty() && k-- > 0) {
            cur = st.pop();
            if (k == 0)
                return cur.data;
            TreeNode right = cur.left;
            while (right != null) {
                st.push(right);
                right = right.right;
            }
        }

        return -1;
    }
}
```


Problem:136: 653. Two Sum IV - Input is a BST

Given the root of a binary search tree and an integer k , return true *if there exist two elements in the BST such that their sum is equal to k* , or false otherwise.

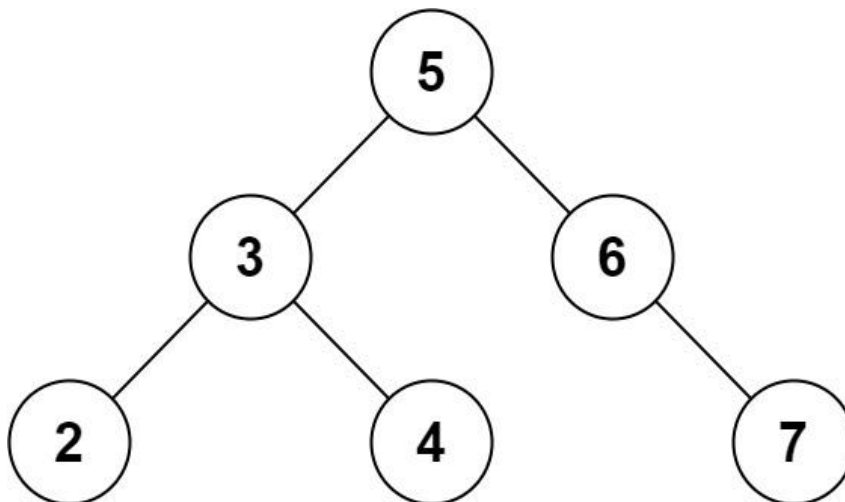
Example 1:



Input: root = [5,3,6,2,4,null,7], $k = 9$

Output: true

Example 2:



Input: root = [5,3,6,2,4,null,7], $k = 28$

Output: false

Solution:

```
class Solution {
    public boolean findTarget(TreeNode root, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();

        return helper(root, k, 0, map);
    }

    private boolean helper(TreeNode root, int k, int sum, HashMap<Integer,
Integer> map) {

        // x+y=k
        // x+(k-x)

        if (root == null)
            return false;
        if (map.get(k - root.val) != null)
            return true;
        if (map.get(sum + root.val) != null && map.get(sum + root.val) ==
k)
            return true;

        map.put(root.val, root.val);
        map.put(sum + root.val, sum + root.val);
        boolean isLeft = helper(root.left, k, sum, map);

        boolean isRight = false;
        if (!isLeft) {
            isRight = helper(root.right, k, sum, map);
        }

        return isLeft || isRight;
    }
}
```

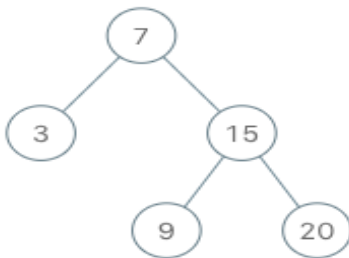
Problem:137: 173. Binary Search Tree Iterator

Implement the BSTIterator class that represents an iterator over the [in-order traversal](#) of a binary search tree (BST):

- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

Example 1:**Input**

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next",
"hasNext", "next", "hasNext"]
```

```
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
bSTIterator.next();      // return 3
bSTIterator.next();      // return 7
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 9
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 15
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 20
bSTIterator.hasNext();   // return False
```

Solution:

```
class BSTIterator {
    Stack<TreeNode> stack;
    public BSTIterator(TreeNode root) {
        stack = new Stack<>();
        TreeNode node = root;
        //update stack
        updateStack(node);
    }
    public int next() {
        TreeNode toRemove = stack.pop();
        // before return node, first update stack further
        updateStack(toRemove.right);
        return toRemove.val;
    }
    public boolean hasNext() {
        return !stack.isEmpty();
    }
    // -----
    public void updateStack(TreeNode node){
        while(node != null){
            stack.add(node);
            node = node.left;
        }
    }
}
```

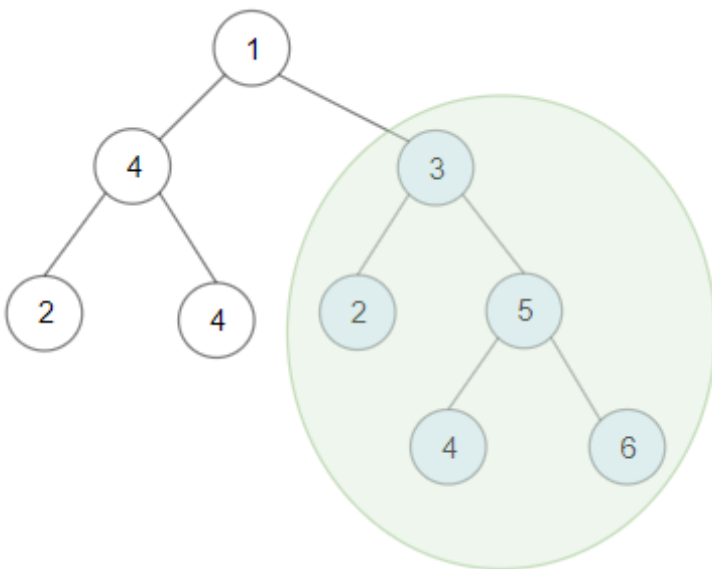
Problem:138: 1373. Maximum Sum BST in Binary Tree

Given a **binary tree** root, return the *maximum sum of all keys of **any** sub-tree which is also a Binary Search Tree (BST)*.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

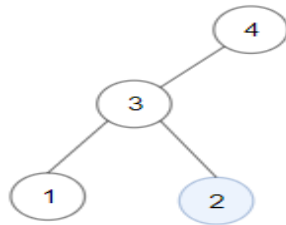
Example 1:



Input: root = [1,4,3,2,4,2,5,null,null,null,null,null,null,4,6]

Output: 20

Explanation: Maximum sum in a valid Binary search tree is obtained in root node with key equal to 3.

Example 2:

Input: root = [4,3,null,1,2]

Output: 2

Explanation: Maximum sum in a valid Binary search tree is obtained in a single root node with key equal to 2.

Example 3:

Input: root = [-4,-2,-5]

Output: 0

Explanation: All values are negatives. Return an empty BST.

Solution:

```

class Solution {
    private int maxSum = 0;

    public int maxSumBST(TreeNode root) {
        postOrderTraverse(root);
        return maxSum;
    }

    private int[] postOrderTraverse(TreeNode root) {
        if (root == null)
            return new int[] { Integer.MAX_VALUE, Integer.MIN_VALUE, 0 };
        // {min, max, sum}, initialize min=MAX_VALUE, max=MIN_VALUE
        int[] left = postOrderTraverse(root.left);
        int[] right = postOrderTraverse(root.right);
        // The BST is the tree:
        if (!(left != null // the left subtree must be BST
            && right != null // the right subtree must be BST
            // the root's key must greater than maximum keys of
            //the left subtree
            && root.val > left[1]
            // the root's key must lower than minimum keys
            //of the right subtree
            && root.val < right[0]))
            return null;
        // now it's a BST make `root` as root
        int sum = root.val + left[2] + right[2];
    }
};
  
```

```
        maxSum = Math.max(maxSum, sum);
        int min = Math.min(root.val, left[0]);
        int max = Math.max(root.val, right[1]);
        return new int[] { min, max, sum };
    }
}
```

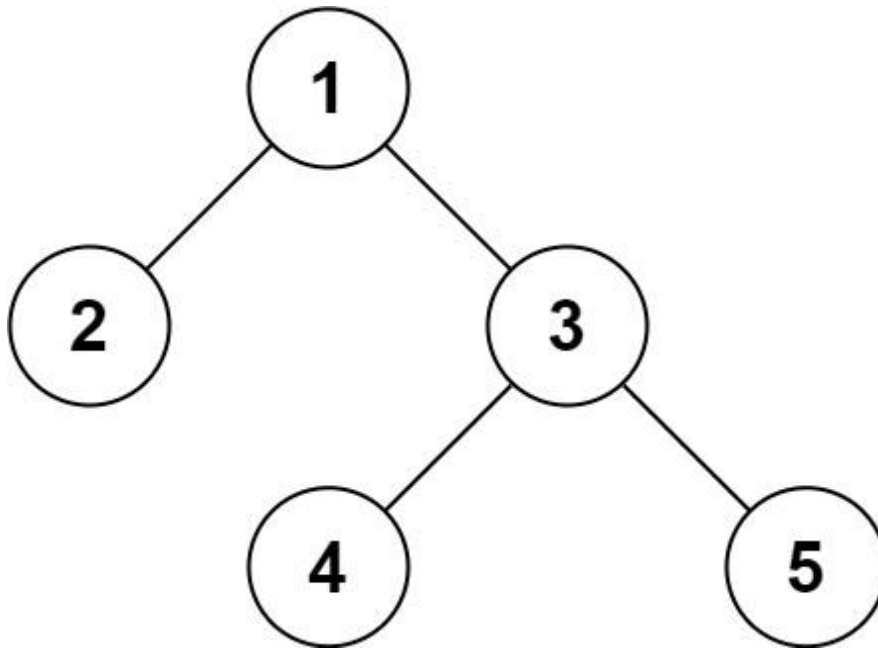
Problem:139: 297. Serialize and Deserialize Binary Tree

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](#). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Solution:

```
public class Codec {  
    // Encodes a tree to a single string.  
    public String serialize(TreeNode root) {  
        StringBuffer buffer = new StringBuffer();  
  
        return serializeHelper(root, buffer);  
    }  
  
    private String serializeHelper(TreeNode root, StringBuffer buffer) {  
        if (root == null) {  
            buffer.append("X").append(",");  
            return buffer.toString();  
        }  
  
        buffer.append(root.val).append(",");  
  
        serializeHelper(root.left, buffer);  
        serializeHelper(root.right, buffer);  
  
        return buffer.toString();  
    }  
}
```



```
// Decodes your encoded data to tree.
public TreeNode deserialize(String data) {
    String arr[] = data.split(",");

    LinkedList<String> queue = new LinkedList<>();
    for (String val : arr) {
        queue.offer(val);
    }

    return deserializeHelper(queue);
}

private TreeNode deserializeHelper(LinkedList<String> queue) {
    String val = queue.poll();
    if (val.equals("X"))
        return null;

    TreeNode root = new TreeNode(Integer.parseInt(val));
    root.left = deserializeHelper(queue);
    root.right = deserializeHelper(queue);

    return root;
}
}

// Your Codec object will be instantiated and called as such:
// Codec ser = new Codec();
// Codec deser = new Codec();
// TreeNode ans = deser.deserialize(ser.serialize(root));
```

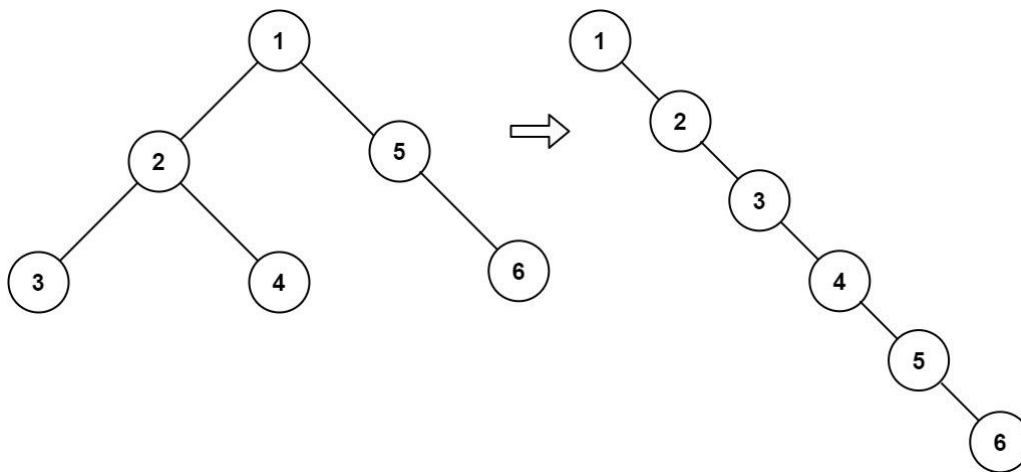
Day 22: BINARY TREES(MISCELLANEOUS)

Problem:140: 114. Flatten Binary Tree to Linked List

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a [pre-order traversal](#) of the binary tree.

Example 1:



Input: `root = [1,2,5,3,4,null,6]`

Output: `[1,null,2,null,3,null,4,null,5,null,6]`

Example 2:

Input: `root = []`

Output: `[]`

Example 3:

Input: `root = [0]`

Output: `[0]`

Solution:

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public void flatten(TreeNode root) {
        if (root == null)
            return;
        // if(root.left==null) return;

        helper(root, root.left, root.right);
    }

    private TreeNode helper(TreeNode root, TreeNode left, TreeNode right) {
        root.left = null;
        if (left != null) {
            root.right = left;
            root = helper(left, left.left, left.right);
        }

        if (right != null) {
            root.right = right;
            root = helper(right, right.left, right.right);
        }

        return root;
    }
}
```

Problem:141: 295. Find Median from Data Stream

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:**Input**

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();  
medianFinder.addNum(1);    // arr = [1]  
medianFinder.addNum(2);    // arr = [1, 2]  
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)  
medianFinder.addNum(3);    // arr[1, 2, 3]  
medianFinder.findMedian(); // return 2.0
```

Solution:

```

class MedianFinder {
    PriorityQueue<Integer> min;
    PriorityQueue<Integer> max;
    public MedianFinder() {
        min = new PriorityQueue<Integer>(1, (a,b)->(a-b));
        max = new PriorityQueue<Integer>(1, Collections.reverseOrder());
    }

    public void addNum(int num) {
        min.offer(num);

        max.offer(min.poll());
        if(min.size()<max.size()){
            min.offer(max.poll());
        }
    }

    public double findMedian() {

        //if odd
        //if odd

        //[1,2]
        //min max
        // 1

        //2 [2,1] #2
        //2 1 - if 2 elements min and max are equal.

        //[1,2,3]
        //min max
        // #1 1

        // #2 [1,2]
        //2 1

        // #3
        //[3] [1,2] //here min having 1 and max having 2
        if(min.size()==max.size()){
            return (min.peek()+max.peek())/2.0;
        }

        return min.peek();
    }
}

```

```
/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
```

Problem:142: 703. Kth Largest Element in a Stream

Design a class to find the k^{th} largest element in a stream. Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Implement KthLargest class:

- KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream of integers nums.
- int add(int val) Appends the integer val to the stream and returns the element representing the k^{th} largest element in the stream.

Example 1:

Input

```
["KthLargest", "add", "add", "add", "add", "add"]
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]
```

Output

```
[null, 4, 5, 5, 8, 8]
```

Explanation

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3);    // return 4
kthLargest.add(5);    // return 5
kthLargest.add(10);   // return 5
kthLargest.add(9);    // return 8
kthLargest.add(4);    // return 8
```

Constraints:

- $1 \leq k \leq 10^4$
- $0 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $-10^4 \leq \text{val} \leq 10^4$
- At most 10^4 calls will be made to add.
- It is guaranteed that there will be at least k elements in the array when you search for the k^{th} element.

Solution:

```

class KthLargest {
    PriorityQueue<Integer> pq;
    int k;
    public KthLargest(int k, int[] nums) {
        this.k = k;

        pq = new PriorityQueue<Integer>();

        for(int num:nums){
            add(num);
        }
        //System.out.println(pq);
    }

    //[10,9,8,5,5,4,3,2]
    //[2,3,4,5,5,8,9,10]
    //k = 3
    //[[ [4, 5, 8, 2]], [3], [5], [10], [9], [4]]

    //prun val >first in queue
    //min queue          reach size
    // 4                  5(print 4)    10
    // \                  \             \
    // 5      =>          8
    // \                  \
    //
    //
    //
    // [4, 5, 8]
    //cur removed
    // 5      4 [5, 8]
    // 10     5 [5, 8]
    // 9      5 [8, 10]

    //pq.add is faster than offer.
    public int add(int val) {
        if(pq.size()<k){
            pq.offer(val);
        }else{
            int cur = pq.peek();
            if(val>cur){
                int temp = pq.poll();
                // System.out.println(val + " "+temp + " "+pq);
                pq.offer(val);
            }
        }
    }
}

```



```

    }
    return pq.peek();
}
}

```

Problem:143: Count Distinct Element in Every K Size Window

```

2
7 4
1 2 1 3 4 2 3
5 3
1 1 2 1 3

```

Sample Output 1:

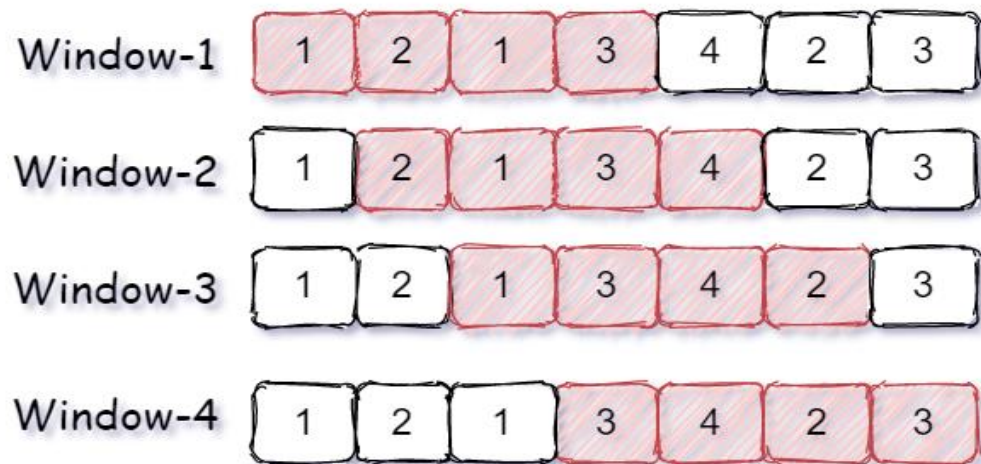
```

3 4 4 3
2 2 3

```

Explanation of sample input 1:

Test Case 1:



Window-1 has four elements { 1, 2, 1, 3 } and only three elements { 1, 2, 3 } are distinct because 1 is repeating two times.

Window-2 has four elements { 2, 1, 3, 4 } and all four elements { 2, 1, 3, 4 } are distinct.

Window-3 has four element { 1, 3, 4, 2 } and all four elements { 1, 3, 4, 2 } are distinct.

Window-4 has four element { 3, 4, 2, 3 } and only three elements { 3, 4, 2 } are distinct because 3 is repeating two times.

Hence, the count of distinct elements in all windows is { 3, 4, 4, 3}.

Solution:

```
import java.util.*;
import java.io.*;
import java.util.ArrayList;

public class Solution {

    public static ArrayList<Integer>
countDistinctElements(ArrayList<Integer> arr, int k) {
    ArrayList<Integer> ans = new ArrayList<>();
    HashMap<Integer, Integer> map = new HashMap<>();
    int n = arr.size();
    int j = 0, i = 0;
    while (j < n) {
        int cur = arr.get(j);
        map.put(cur, map.getOrDefault(cur, 0) + 1);
        if (j - i + 1 == k) {
            int next = arr.get(i);
            ans.add(map.size());
            map.put(next, map.get(next) - 1);
            if (map.get(next) == 0)
                map.remove(next);
            i++;
        }
        j++;
    }
    return ans;
}
```

Problem:143-I: 215. Kth Largest Element in an Array

Given an integer array `nums` and an integer `k`, return *the* k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`

Output: 4

Solution:

```
class Solution {  
    public int findKthLargest(int[] nums, int k) {  
        // 3,2,1,5,6,4 , k =2  
    }  
}
```

```

        // Step 1: set the pivot ,just for consistence, so set last
element as pivot
        // set 4 as pivot

        // Step 2: move all less pivot to left.
        // 3,2,1,5,6,4
        // | |

        // Step 3: swap last element less than pivot with end element.

        // step 4: if k is == last element index less than pivot return.

        // step 4.1 : if lastelementless than k < pivot swap -> (
start+1,end)
        // Step 4.2 : else swap( start ,end -1)

        // since first k elements are ignored to get k th largest element
to
        // find kth is nums.length-k

        return helper(nums, 0, nums.length - 1, nums.length - k);
    }
    private int helper(int nums[], int start, int end, int k) {
        if (start > end)
            return Integer.MAX_VALUE;

        int pivot = nums[end];

        int left = start;

        for (int i = start; i < end; i++) {
            if (nums[i] <= pivot) {
                swap(nums, left++, i);
            }
        }
        swap(nums, left, end);

        if (left == k) {
            return nums[left];
        } else if (left < k) {
            return helper(nums, start + 1, end, k);
        } else {
            return helper(nums, start, end - 1, k);
        }
    }
    private void swap(int nums[], int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];

```

```

        nums[j] = temp;
    }
}

```

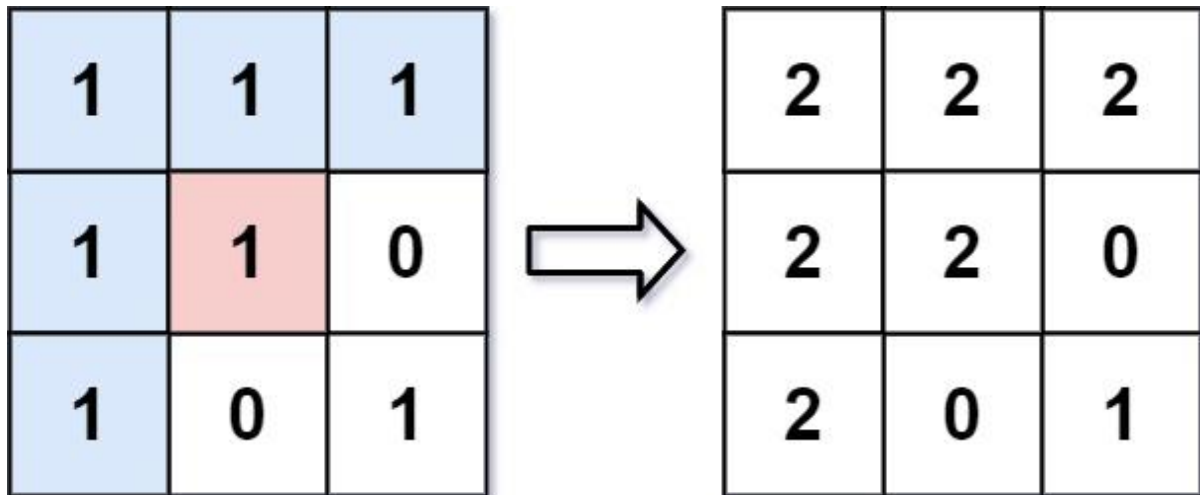
Problem:144: 733. Flood Fill

An image is represented by an $m \times n$ integer grid `image` where `image[i][j]` represents the pixel value of the image.

You are also given three integers `sr`, `sc`, and `color`. You should perform a **flood fill** on the image starting from the pixel `image[sr][sc]`.

To perform a **flood fill**, consider the starting pixel, plus any pixels connected **4-directionally** to the starting pixel of the same color as the starting pixel, plus any pixels connected **4-directionally** to those pixels (also with the same color), and so on. Replace the color of all of the aforementioned pixels with `color`.

Return the modified image after performing the flood fill.



Input: `image = [[1,1,1],[1,1,0],[1,0,1]]`, `sr = 1`, `sc = 1`, `color = 2`

Output: `[[2,2,2],[2,2,0],[2,0,1]]`

Explanation: From the center of the image with position $(sr, sc) = (1, 1)$ (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color. Note the bottom corner is not colored 2, because it is not 4-directionally connected to the starting pixel.

Solution:

```
class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {

        int r = image.length;
        int c = image[0].length;
        int res[][] = new int[r][c];

        if (image == null)
            return res;

        boolean visited[][] = new boolean[r][c];
        dfs(image, sr, sc, image[sr][sc], newColor, visited);

        // [1,1,1],
        // [1,1,0],
        // [1,0,1]]

        return image;
    }

    private void dfs(int[][] image, int x, int y, int color, int newColor,
boolean[][] visited) {
        int r = image.length;
        int c = image[0].length;

        if (x < 0 || y < 0 || x >= r || y >= c || visited[x][y] ||
image[x][y] != color)
            return;

        if (image[x][y] == color)
            image[x][y] = newColor;

        visited[x][y] = true;
        dfs(image, x + 1, y, color, newColor, visited);
        dfs(image, x - 1, y, color, newColor, visited);
        dfs(image, x, y + 1, color, newColor, visited);
        dfs(image, x, y - 1, color, newColor, visited);
    }
}
```

DAY 23: GRAPHS

Problem:144: 133. Clone Graph

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

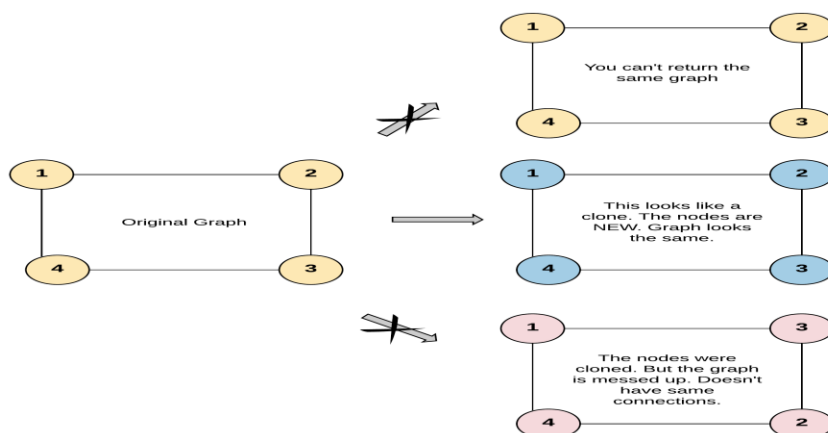
```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val == 1, the second node with val == 2, and so on. The graph is represented in the test case using an adjacency list.

An **adjacency list** is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:

Input: adjList = [[2,4],[1,3],[2,4],[1,3]]

Output: [[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph.

1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Solution:

```

/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;
    public Node() {
        val = 0;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val) {
        val = _val;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val, ArrayList<Node> _neighbors) {
        val = _val;
        neighbors = _neighbors;
    }
}
*/

class Solution {
    public Node cloneGraph(Node node) {

        HashMap<Node, Node> cache = new HashMap<>();
        return clone(node, cache);
    }

    private Node clone(Node node, HashMap<Node, Node> cache) {
        if (node == null)
            return null;
        if (cache.get(node) != null) {
            return cache.get(node);
        }
        Node cur = new Node(node.val);
        // trick is just add the reference before iterate
        cache.put(node, cur);
        for (Node next : node.neighbors) {
            cur.neighbors.add(clone(next, cache));
        }

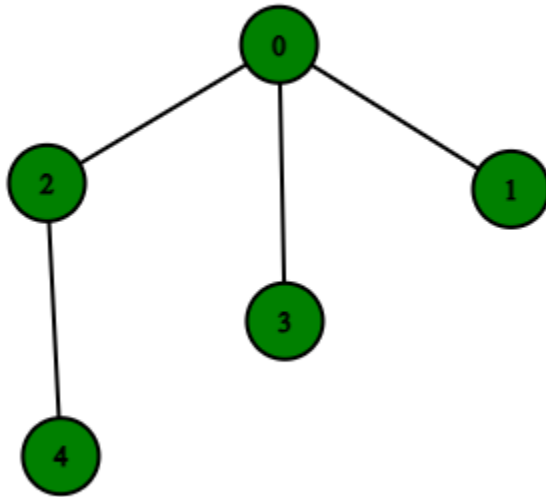
        return cur;
    }
}

```

Problem:145: DFS OF GRAPH
DFS of Graph

Example 1:

Input: $V = 5$, $adj = [[2,3,1], [0], [0,4], [0], [2]]$



Output: 0 2 4 3 1

Explanation:

0 is connected to 2, 3, 1.

1 is connected to 0.

2 is connected to 0 and 4.

3 is connected to 0.

4 is connected to 2.

so starting from 0, it will go to 2 then 4,
and then 3 and 1.

Thus dfs will be 0 2 4 3 1.

Solution:

```

// { Driver Code Starts
// Initial Template for Java
import java.util.*;
import java.lang.*;
import java.io.*;

class GFG {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine().trim());
        while (T-- > 0) {
            String[] s = br.readLine().trim().split(" ");
            int V = Integer.parseInt(s[0]);
            int E = Integer.parseInt(s[1]);
            ArrayList<ArrayList<Integer>> adj = new
ArrayList<ArrayList<Integer>>();
            for (int i = 0; i < V; i++)
                adj.add(new ArrayList<Integer>());
            for (int i = 0; i < E; i++) {
                String[] S = br.readLine().trim().split(" ");
                int u = Integer.parseInt(S[0]);
                int v = Integer.parseInt(S[1]);
                adj.get(u).add(v);
                adj.get(v).add(u);
            }
            Solution obj = new Solution();
            ArrayList<Integer> ans = obj.dfsOfGraph(V, adj);
            for (int i = 0; i < ans.size(); i++)
                System.out.print(ans.get(i) + " ");
            System.out.println();
        }
    }
}
// } Driver Code Ends

class Solution {
    // Function to return a list containing the DFS traversal of the graph.
    public ArrayList<Integer> dfsOfGraph(int V,
        ArrayList<ArrayList<Integer>> adj) {
        ArrayList<Integer> result = new ArrayList<Integer>();
        result.add(0);
        HashSet<Integer> visited = new HashSet<>();

        helper(result, visited, V, adj, 0);
    }
}

```

```

        return result;
    }

    private void helper(ArrayList<Integer> result,
                        HashSet<Integer> visited, int n,
                        ArrayList<ArrayList<Integer>> adjList, int startNode) {
        if (result.size() == n)
            return;
        // to maintain order set visited to true at first
        visited.add(startNode);

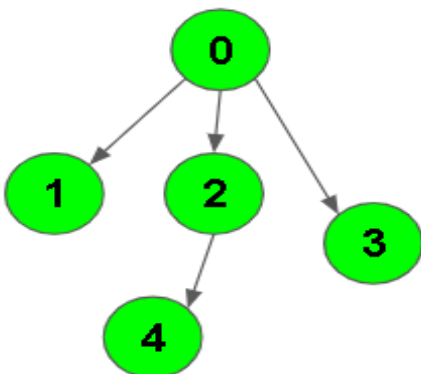
        for (int next : adjList.get(startNode)) {
            if (visited.contains(next))
                continue;
            visited.add(next);
            result.add(next);
            helper(result, visited, n, adjList, next);
        }
        // you can add visited after visiting all adjacent nodes but
        // order will change.
    }
}

```

Problem:146: BFS OF GRAPH

Example 1:

Input:



Output: 0 1 2 3 4

Explanation:

0 is connected to 1 , 2 , 3.
 2 is connected to 4.
 so starting from 0, it will go to 1 then 2
 then 3. After this 2 to 4, thus bfs will be
 0 1 2 3 4.

Solution:

```
// { Driver Code Starts
// Initial Template for Java
import java.util.*;
import java.lang.*;
import java.io.*;

class GFG {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
        InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine().trim());
        while (T-- > 0) {
            String[] s = br.readLine().trim().split(" ");
            int V = Integer.parseInt(s[0]);
            int E = Integer.parseInt(s[1]);
            ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
            for (int i = 0; i < V; i++)
                adj.add(i, new ArrayList<Integer>());
            for (int i = 0; i < E; i++) {
                String[] S = br.readLine().trim().split(" ");
                int u = Integer.parseInt(S[0]);
                int v = Integer.parseInt(S[1]);
                adj.get(u).add(v);
                // adj.get(v).add(u);
            }
            Solution obj = new Solution();
            ArrayList<Integer> ans = obj.bfsOfGraph(V, adj);
            for (int i = 0; i < ans.size(); i++)
                System.out.print(ans.get(i) + " ");
            System.out.println();
        }
    }
}

// } Driver Code Ends

class Solution {
    // Function to return Breadth First Traversal of given graph.
    public ArrayList<Integer> bfsOfGraph(int V,
```

```

        ArrayList<ArrayList<Integer>> adj) {
    ArrayList<Integer> result = new ArrayList<>();

    helper(result, adj);
    return result;
}

private void helper(ArrayList<Integer> result,
    ArrayList<ArrayList<Integer>> adj) {
    LinkedList<Integer> queue = new LinkedList<>();
    queue.offer(0);
    HashSet<Integer> visited = new HashSet<>();
    visited.add(0);
    result.add(0);
    while (!queue.isEmpty()) {
        int cur = queue.poll();
        for (int next : adj.get(cur)) {
            if (visited.contains(next))
                continue;
            result.add(next);
            visited.add(next);
            queue.offer(next);
        }
    }
}

```

Problem:147: 207. Course Schedule(UNDIRECTED GRAPH)

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq 5000$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- All the pairs $\text{prerequisites}[i]$ are **unique**.

Solution:

```
import java.util.HashSet;
import java.util.LinkedList;

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        int indegree[] = new int[numCourses + 1];
        if (prerequisites == null || (prerequisites != null &&
prerequisites.length < 2))
            return true;
        // [[0,1],[1,2]]
        LinkedList<Integer>[] adjList = new LinkedList[numCourses + 1];
        // 0->1 , 1->2
        for (int cur[] : prerequisites) {
            int course = cur[0] + 1;
            int preq = cur[1] + 1;

            if (adjList[course] == null) {
                adjList[course] = new LinkedList<>();
            }
            adjList[course].add(preq);
            // course 2->1->0 (0 has no dependency it is master or root
course)
            indegree[preq]++;
        }

        // make course start from 0.
        // arr[0]- course 0... course[n] =last course i.e (numCourses-1);
        LinkedList<Integer> queue = new LinkedList<>();
```

```

or
    // which is the first course to start if pre-Requisite count is 0
    // less than that is the basic course to start.
    HashSet<Integer> visited = new HashSet<>();
    int totalCourses = numCourses;
    for (int i = 1; i <= numCourses; i++) {
        if (indegree[i] == 0) {
            queue.offer(i);
            totalCourses--;
        }
    }
    // basic course can be completable.
    // [[0,1],[1,2]]
    while (!queue.isEmpty()) {
        int cur = queue.poll();
        // [0]-course
        // [1]-prerequisite
        if (adjList[cur] == null)
            continue;
        for (int course : adjList[cur]) {
            // just decrement the course to complete the
prerequisite.
            indegree[course]--;
            // only if it is master or root add to queue
            if (indegree[course] == 0) {
                queue.offer(course);
                totalCourses--;
            }
        }
    }

    return (totalCourses == 0) ? true : false;
}
}

```

Problem:148: 207. Course Schedule-DFS(UNDIRECTED GRAPH)

For Directed graph : use Kahn's Algorithm

```
import java.util.LinkedList;
```



```

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        int indegree[] = new int[numCourses + 1];
        if (prerequisites == null
            || (prerequisites != null && prerequisites.length <
2))
            return true;
        // [[0,1],[1,2]]
        LinkedList<Integer>[] adjList = new LinkedList[numCourses + 1];
        // 0->1 , 1->2
        for (int i = 1; i <= numCourses; i++) {
            adjList[i] = new LinkedList<>();
        }

        for (int cur[] : prerequisites) {
            int course = cur[0] + 1;
            int preq = cur[1] + 1;

            // circular loop
            if (course == preq)
                return false;

            adjList[course].add(preq);

            // course 2->1->0 (0 has no dependency it is master or root
course)
            indegree[preq]++;
        }

        // make course start from 0.
        // arr[0]- course 0... course[n] =last course i.e (numCourses-1);
        LinkedList<Integer> queue = new LinkedList<>();

        // which is the first course to start if pre-Requisite count is 0
or
        // less than that is the basic course to start.
        int totalCourses = numCourses;

        boolean res = false;
        for (int i = 1; i <= numCourses; i++) {
            if (indegree[i] == 0) {
                totalCourses--;
                totalCourses = dfs(adjList, totalCourses, indegree,
i);

                if (totalCourses == 0)
                    return true;
            }
        }
    }
}

```

```
    }

    return totalCourses == 0 ? true : false;
}

private int dfs(LinkedList<Integer>[] adjList, int totalCourses,
               int[] degree, int root) {
    if (totalCourses == 0)
        return totalCourses;

    for (int cur : adjList[root]) {
        degree[cur]--;
        if (degree[cur] == 0) {
            totalCourses = dfs(adjList, totalCourses - 1, degree,
cur);
        }
    }
    return totalCourses;
}
}
```

Problem:149: 200. Number of Islands

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

Input: `grid = [`
 `["1","1","1","1","0"],`
 `["1","1","0","1","0"],`
 `["1","1","0","0","0"],`
 `["0","0","0","0","0"]`
 `]`

Output: 1

Solution:

```

class Solution {
    public int numIslands(char[][] grid) {
        int r = grid.length;
        if (grid == null || (grid != null && r == 0))
            return 0;
        int c = grid[0].length;
        boolean visited[][] = new boolean[r][c];
        int count = 0;
        // once visited any i,j can consider
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                if (grid[i][j] == '1' && !visited[i][j]) {
                    dfs(grid, i, j, visited);
                    // we need to find how many bi-partiate graph or
                    // disjoint graph exist. so if visited still exist
                    // means disjoint graph exist and we cover at
                    // disjoint(i,j) points
                    count++;
                }
            }
        }
        // f(i,j) => { f(i+1, j), f(i-1, j), f(i, j+1), f(i, j-1) }
        // if any i on (left, right, up, down) exceeds boundary consider as water
        return count;
    }

    private void dfs(char grid[][], int i, int j, boolean visited[][]) {
        int dirs[][] = { { -1, 0 }, { 1, 0 }, { 0, 1 }, { 0, -1 } };
        int r = grid.length;
        int c = grid[0].length;

        if (grid[i][j] == '0') {
            return;
        }
        // mark starting node from where we traverse four directions as visited.
        // this is to make sure we are not in loop, from start
        // we should not go back to
        // start again.
        // we break if happens.
        visited[i][j] = true;
        int count = 0;
        for (int dir[] : dirs) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x < 0 || y < 0 || x >= r || y >= c || visited[x][y])
                continue;
            dfs(grid, x, y, visited);
        }
    }
}

```

Problem:150(A): 785. Is Graph Bipartite(BFS)?

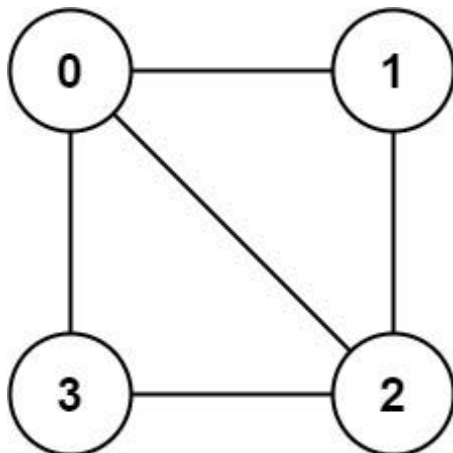
There is an **undirected** graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is **bipartite** if the nodes can be partitioned into two independent sets A and B such that **every** edge in the graph connects a node in set A and a node in set B .

Return true *if and only if* it is **bipartite**.

Example 1:



Input: `graph = [[1,2,3],[0,2],[0,1,3],[0,2]]`

Output: `false`

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Solution:

```
class Solution {
```

// Runtime: 2 ms, faster than 54.07% of Java online submissions for Is Graph Bipartite?.

// Memory Usage: 53.5 MB, less than 67.06% of Java online submissions for Is Graph Bipartite?.

```

    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int color[] = new int[n];
        Arrays.fill(color, -1);
        for (int i = 0; i < n; i++) {
            if (graph[i].length != 0 && color[i] == -1) {
                if (!bfsCheck(graph, color, i))
                    return false;
            }
        }
        return true;
    }

    private boolean bfsCheck(int[][] graph, int color[], int index) {
        LinkedList<Integer> queue = new LinkedList<>();
        color[index] = 1;
        queue.offer(index);
        while (!queue.isEmpty()) {
            int cur = queue.poll();
            for (int adj : graph[cur]) {
                if (color[adj] == -1) {
                    queue.offer(adj);
                    color[adj] = (color[cur] ^ 1);
                } else if (color[adj] == color[cur])
                    return false;
            }
        }
        return true;
    }
}

```

Problem:150-(B): 785. Is Graph Bipartite(DFS)?

```
import java.util.Arrays;
```

```

class Solution {
    // Runtime: 1 ms, faster than 83.04% of Java online submissions for Is
    // Graph Bipartite?.
    // Memory Usage: 53.2 MB, less than 76.12% of Java online submissions for Is
    // Graph Bipartite?.
    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int color[] = new int[n+1];
        Arrays.fill(color, -1);
    }
}

```

```
        for(int i=0;i<n;i++){
            if(!dfsCheck(graph,color,i)){
                return false;
            }
        }
        return true;
    }

private boolean dfsCheck(int[][] graph, int color[], int index){
    //if not colored set either (0,1)
    if(color[index]==-1){
        color[index]=1;
    }
    //check all adjacent nodes of index.
    for(int adj : graph[index]){
        //if adjacent node is not colored color
        //with alternate color of index - main node.
        if(color[adj]==-1){
            //if adjacent node not color
            color[adj]= 1-color[index];
            if(!dfsCheck(graph,color,adj)){
                return false;
            }
        }else if(color[adj]==color[index]){
            return false;
        }
    }
    //all nodes traversed default return true
    //hoping all nodes traversed no alternate
    //same color found.

    return true;
}
}
```

DAY 24:GRAPHS-II

Problem:151: 1520. Maximum Number of Non-Overlapping Substrings**Hard**

Given a string *s* of lowercase letters, you need to find the maximum number of **non-empty** substrings of *s* that meet the following conditions:

1. The substrings do not overlap, that is for any two substrings *s*[*i*..*j*] and *s*[*x*..*y*], either *j* < *x* or *i* > *y* is true.
2. A substring that contains a certain character *c* must also contain all occurrences of *c*.

Find the *maximum number of substrings that meet the above conditions*. If there are multiple solutions with the same number of substrings, *return the one with minimum total length*. It can be shown that there exists a unique solution of minimum total length.

Notice that you can return the substrings in **any** order.

Example 1:

Input: *s* = "aedefaddaccc"

Output: ["e","f","ccc"]

Explanation: The following are all the possible substrings that meet the conditions:

```
[
  "aedefaddaccc"
  "aedefadda",
  "ef",
  "e",
  "f",
  "ccc",
]
```

If we choose the first string, we cannot choose anything else and we'd get only 1. If we choose "aedefadda", we are left with "ccc" which is the only one that doesn't overlap, thus obtaining 2 substrings. Notice also, that it's not optimal to choose "ef" since it can be split into two. Therefore, the optimal way is to choose ["e","f","ccc"] which gives us 3 substrings. No other solution of the same number of substrings exist.

Solution:

```
class Solution {
    public List<String> maxNumOfSubstrings(String ss) {
        char[] s = ss.toCharArray();
        HashMap<Character, int[]> range = new HashMap<>();
        // get each character's range
        for (int i = 0; i < s.length; i++) {
            char cur = s[i];
            if (range.containsKey(cur))
                range.get(cur)[1] = i;
            else
                range.put(cur, new int[] { i, i });
        }
        // extend the range
        List<int[]> values = new ArrayList<>(range.values());
```



```

    for (int[] r : values) {
        // gather all characters in the range r[0]...r[1]
        HashSet<Character> seen = new HashSet<>();
        int size = 0, min = r[0], max = r[1];
        for (int i = r[0]; i <= r[1]; i++) {
            if (seen.add(s[i])) {
                int temp[] = range.get(s[i]);
                min = Math.min(min, temp[0]);
                max = Math.max(max, temp[1]);
            }
        }
        // extend the range until the range will
        // not cover any other new character
        while (seen.size() != size) {
            size = seen.size();
            int nMin = min;
            int nMax = max;
            for (int i = min; i < r[0]; i++) {
                if (seen.add(s[i])) {
                    int temp[] = range.get(s[i]);
                    nMin = Math.min(nMin, temp[0]);
                    nMax = Math.max(nMax, temp[1]);
                }
            }
            for (int i = r[1] + 1; i <= max; i++) {
                if (seen.add(s[i])) {
                    int temp[] = range.get(s[i]);
                    nMin = Math.min(nMin, temp[0]);
                    nMax = Math.max(nMax, temp[1]);
                }
            }
            r[0] = min; // new end point
            r[1] = max;
            min = nMin; // new start point
            max = nMax;
            // System.out.println("====");
        }
    }
    // meeting room solution
    Collections.sort(values, (a, b) -> a[1] - b[1]);
    int p = -1;
    List<String> res = new ArrayList<>();
    for (int[] r : values) {
        if (r[0] > p) { // can be accepted
            p = r[1]; // p means latest used index
            res.add(ss.substring(r[0], r[1] + 1));
        }
    }
    return res;
}
}

```

Problem:152: Implementing Dijkstra Algorithm

Example 1:

Input:

V = 2

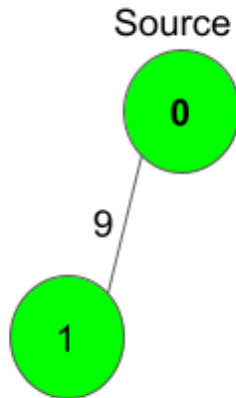
adj [] = {{{1, 9}}, {{0, 9}}}

S = 0

Output:

0 9

Explanation:



The source vertex is 0. Hence, the shortest distance of node 0 is 0 and the shortest distance from node 1 is 9

Solution:

```
// { Driver Code Starts
```

```
import java.util.*;
```

```
import java.io.*;
```

```
import java.lang.*;
```

```
class DriverClass {
```

```
    public static void main(String args[]) throws IOException {
```

```
        BufferedReader read = new BufferedReader(new
InputStreamReader(System.in));
```

```
        int t = Integer.parseInt(read.readLine());
```

```
        while (t-- > 0) {
```

```
            String str[] = read.readLine().trim().split(" ");
```

```
            int V = Integer.parseInt(str[0]);
```

```
            int E = Integer.parseInt(str[1]);
```

```
            ArrayList<ArrayList<ArrayList<Integer>>> adj =
```

```
                new ArrayList<ArrayList<ArrayList<Integer>>>();
```

```
            for (int i = 0; i < V; i++) {
```

```
                adj.add(new ArrayList<ArrayList<Integer>>());
```

```
            }
```

```
            int i = 0;
```

```
            while (i++ < E) {
```

```
                String S[] = read.readLine().trim().split(" ");
```

```

        int u = Integer.parseInt(S[0]);
        int v = Integer.parseInt(S[1]);
        int w = Integer.parseInt(S[2]);
        ArrayList<Integer> t1 = new ArrayList<Integer>();
        ArrayList<Integer> t2 = new ArrayList<Integer>();
        t1.add(v);
        t1.add(w);
        t2.add(u);
        t2.add(w);
        adj.get(u).add(t1);
        adj.get(v).add(t2);
    }

    int S = Integer.parseInt(read.readLine());

    Solution ob = new Solution();

    int[] ptr = ob.dijkstra(V, adj, S);

    for (i = 0; i < V; i++)
        System.out.print(ptr[i] + " ");
    System.out.println();
}
}
} // } Driver Code Ends

//User function Template for Java

class Solution {
    // Function to find the shortest distance of all the vertices
    // from the source vertex S.
    static int[] dijkstra(int V, ArrayList<ArrayList<ArrayList<Integer>>>
adj,
        int S) {
        LinkedList<Node>[] adjList = new LinkedList[V];

        for (int i = 0; i < V; i++) {
            adjList[i] = new LinkedList<Node>();
        }

        for (int i = 0; i < V; i++) {
            ArrayList<ArrayList<Integer>> cur = adj.get(i);
            for (ArrayList<Integer> list : cur) {
                int src = i;
                int dest = list.get(0);
                int weight = list.get(1);

                // undirected so bidirectional

```

```

        adjList[src].add(new Node(dest, weight));
        adjList[dest].add(new Node(src, weight));
    }
}

int distance[] = new int[V];
Arrays.fill(distance, -1);
// set initial node distance = 0;
// since from that node to the same node is 0.
distance[S] = 0;
bfs(adjList, distance, S);

return distance;
}

private static void bfs(LinkedList<Node>[] adjList,
    int distance[], int startNode) {

    LinkedList<Integer> queue = new LinkedList<>();
    queue.offer(startNode);

    HashSet<Integer> visited = new HashSet<>();

    visited.add(startNode);

    while (!queue.isEmpty()) {
        int cur = queue.poll();

        if (adjList[cur] == null)
            continue;
        for (Node adj : adjList[cur]) {
            int dest = adj.dest;
            int weight = adj.weight;
            if (distance[dest] == -1 || distance[dest] > weight +
distance[cur]) {
                distance[dest] = weight + distance[cur];
                queue.offer(dest);
            }
        }
    }
}

class Node {
    int dest, weight;
}

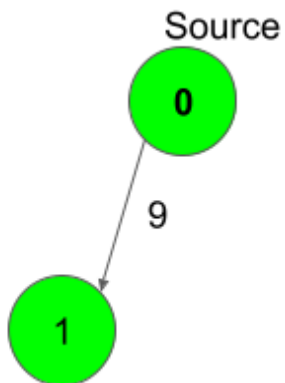
```

```
Node(int dest, int weight) {  
    this.dest = dest;  
    this.weight = weight;  
}  
}
```

Problem:153: Distance from the Source (Bellman-Ford Algorithm)

Example 1:

Input:



E = [[0,1,9]]

S = 0

Output:

0 9

Explanation:

Shortest distance of all nodes from source is printed.

Solution:

```
// { Driver Code Starts
import java.util.*;
import java.io.*;
import java.lang.*;

class DriverClass {
    public static void main(String args[]) throws IOException {

        BufferedReader read = new BufferedReader(new
InputStreamReader(System.in));
        int t = Integer.parseInt(read.readLine());
        while (t-- > 0) {
            String str[] = read.readLine().trim().split(" ");
            int V = Integer.parseInt(str[0]);
            int E = Integer.parseInt(str[1]);

            ArrayList<ArrayList<Integer>> adj = new ArrayList<>();

            int i = 0;
            while (i++ < E) {
                String S[] = read.readLine().trim().split(" ");
                int u = Integer.parseInt(S[0]);
                int v = Integer.parseInt(S[1]);
                int w = Integer.parseInt(S[2]);
                ArrayList<Integer> t1 = new ArrayList<>();
                t1.add(u);
                t1.add(v);
                t1.add(w);
                adj.add(t1);
            }

            int S = Integer.parseInt(read.readLine());

            Solution ob = new Solution();

            int[] ptr = ob.bellman_ford(V, adj, S);

            for (i = 0; i < V; i++)
                System.out.print(ptr[i] + " ");
            System.out.println();
        }
    }
}
```

```

    }
}
} // } Driver Code Ends

//User function Template for Java

/*
 * adj: vector of vectors which represents the graph S: source vertex to
start
 * traversing graph with V: number of vertices
 */
class Solution {
    static int[] bellman_ford(int V, ArrayList<ArrayList<Integer>> adj, int
S) {
        // Write your code here

        int distance[] = new int[V + 1];
        int mod = (int) 1e8;
        Arrays.fill(distance, mod);
        distance[S] = 0;
        for (int i = 0; i < V; i++) {
            for (ArrayList<Integer> cur : adj) {
                int src = cur.get(0);
                int dest = cur.get(1);
                int weight = cur.get(2);

                if ((distance[dest] > distance[src] + weight)) {
                    distance[dest] = distance[src] + weight;
                }
            }
        }

        return distance;
    }
}

class Node {
    int dest, weight;

    Node(int dest, int weight) {
        this.dest = dest;
        this.weight = weight;
    }
}

```

Problem:154: Floyd Warshall

Example 1:

Input: matrix = {{0,25},{-1,0}}

	0	1
0	0	25
1	-1	0

Output: {{0,25},{-1,0}}

	0	1
0	0	25
1	-1	0

Explanation: The shortest distance between every pair is already given(if it exists).

Solution:

```
// { Driver Code Starts
//Initial template for JAVA
```

```
import java.util.*;
import java.lang.*;
import java.io.*;
class GFG
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine().trim());
        while(T-->0)
        {
            int n = Integer.parseInt(br.readLine().trim());
            int[][] matrix = new int[n][n];
            for(int i = 0; i < n; i++){
                String[] s = br.readLine().trim().split(" ");
                for(int j = 0; j < n; j++)
                    matrix[i][j] = Integer.parseInt(s[j]);
            }
            Solution obj = new Solution();
            obj.shortest_distance(matrix);
            for(int i = 0; i < n; i++){
                for(int j = 0; j < n; j++){
                    System.out.print(matrix[i][j] + " ");
                }
                System.out.println();
            }
        }
    }
}
```



```

    }
    }
}
// } Driver Code Ends

```

```

//User function template for JAVA

```

```

class Solution
{
    public void shortest_distance(int[][] matrix)
    {
        int n = matrix.length;
        for (int k = 0; k < n; ++k)
        {
            for (int i = 0; i < n; ++i)
            {
                for (int j = 0; j < n; ++j)
                {
                    //no weight exists -1
                    //no direct path exists between (i,j)
                    //and even through (i,j)route through intermediate k
                    if(matrix[i][k]==-1 || matrix[k][j]==-1)continue;

                    //if current weight is -1 no direct path from (i,j)
                    //then sum intermediate weight
                    if(matrix[i][j] == -1){
                        matrix[i][j] = matrix[i][k] + matrix[k][j];
                        continue;
                    }
                    matrix[i][j] = Math.min(matrix[i][j], matrix[i][k] +
matrix[k][j]);
                }
            }
        }
    }
}

```

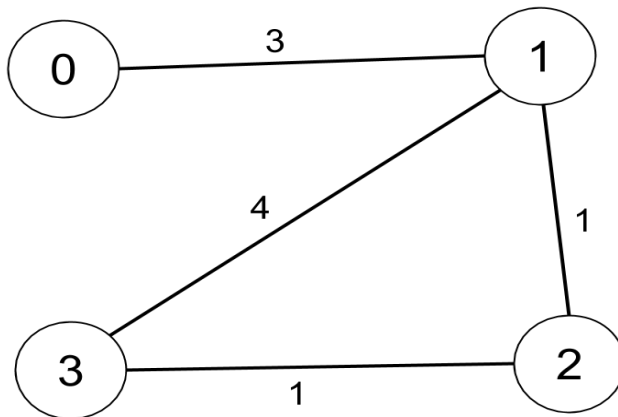
Problem:155: 1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:



Input: $n = 4$, `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`, `distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.

Solution:

```

class Solution {
// Runtime: 7 ms, faster than 97.39% of Java online submissions for Find
the City With the Smallest Number of Neighbors at a Threshold Distance.

```

// Memory Usage: 42.9 MB, less than 94.16% of Java online submissions for Find the City With the Smallest Number of Neighbors at a Threshold Distance.

```

public int findTheCity(int n, int[][] edges, int distanceThreshold) {
    int distance[][] = new int[n][n];
    // set to maxvalue or as per constraints  $10^4+1$ .
    // 5
    // [[0,1,2],[0,4,8],[1,2,10000],[1,4,2],[2,3,10000],[3,4,1]]
    // if set to MAX_VALUE it is not working.
    for (int cur[] : distance) {
        Arrays.fill(cur, Integer.MAX_VALUE);
    }

    for (int edge[] : edges) {
        int u = edge[0];
        int v = edge[1];
        int w = edge[2];
        // bi-directional
        distance[u][v] = w;
        distance[v][u] = w;
    }
    // [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4
    //
    // set self edge as 0 like from u - u
    for (int i = 0; i < n; i++) {
        distance[i][i] = 0;
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int adj1 = distance[i][k];
                int adj2 = distance[k][j];
                // if anyone is infinite we cant find shortest
                // if you set distance max values as 10001
                //below lines can be avoided.
                if (adj1 == Integer.MAX_VALUE || adj2 ==
Integer.MAX_VALUE)
                    continue;
                distance[i][j] = Math.min(distance[i][j], adj1
+ adj2);
            }
        }
    }

    int min = distanceThreshold;
    int res = 0;

```

```

    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) {
            if (distance[i][j] <= distanceThreshold) {
                count++;
            }
        }
        if (count <= min) {
            res = i;
            min = count;
        }
    }

    return res;
}
}

```

Problem:156: Minimum Spanning Tree

Prism Algorithm:

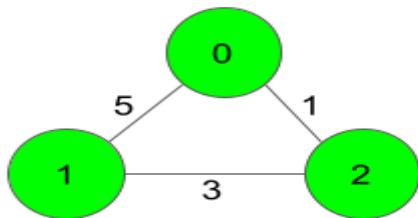
Example 1:

Input:

```

3 3
0 1 5
1 2 3
0 2 1

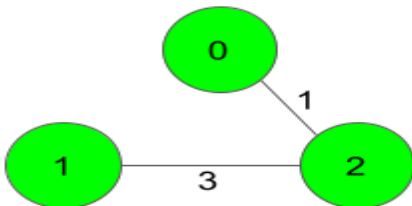
```



Output:

4

Explanation:



The Spanning Tree resulting in a weight of 4 is shown above.

Solution:

```
class Pair implements Comparable<Pair> {
    int v;
    int wt;

    Pair(int v, int wt) {
        this.v = v;
        this.wt = wt;
    }

    public int compareTo(Pair p2) {
        return this.wt - p2.wt;
    }
}

class Solution {
    static int spanningTree(int V, int E, int edges[][]){
        ArrayList<ArrayList<Pair>> list = new
ArrayList<ArrayList<Pair>>();
        for (int i = 0; i < V; i++) {
            list.add(new ArrayList<Pair>());
        }
        for (int i = 0; i < E; i++) {
            int u = edges[i][0];
            int v = edges[i][1];
            int wt = edges[i][2];

            list.get(u).add(new Pair(v, wt));
            list.get(v).add(new Pair(u, wt));
        }
        PriorityQueue<Pair> pq = new PriorityQueue<>();
        pq.add(new Pair(0, 0));
        boolean vis[] = new boolean[V];
        int s = 0;
        while (!pq.isEmpty()) {
            Pair node = pq.poll();
            int v = node.v;
            int wt = node.wt;
            if (vis[v])
                continue;

            s += wt;
            vis[v] = true;
            for (Pair it : list.get(v)) {
                if (!vis[it.v])
                    pq.add(new Pair(it.v, it.wt));
            }
        }
    }
}
```

```
        return s;  
    }  
}
```

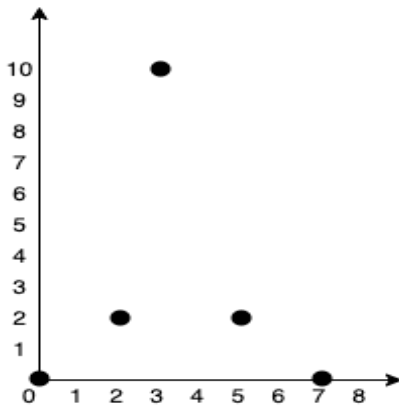
Problem:157: 1584. Min Cost to Connect All Points

You are given an array `points` representing integer coordinates of some points on a 2D-plane, where `points[i] = [xi, yi]`.

The cost of connecting two points `[xi, yi]` and `[xj, yj]` is the **manhattan distance** between them: $|x_i - x_j| + |y_i - y_j|$, where $|val|$ denotes the absolute value of val .

Return the *minimum cost to make all points connected*. All points are connected if there is **exactly one** simple path between any two points.

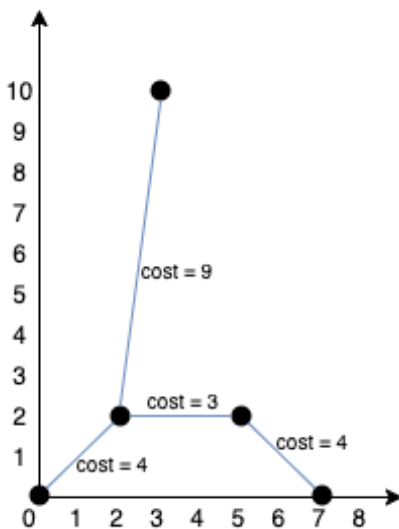
Example 1:



Input: `points = [[0,0],[2,2],[3,10],[5,2],[7,0]]`

Output: 20

Explanation:



We can connect the points as shown above to get the minimum cost of 20. Notice that there is a unique path between every pair of points.

Example 2:

Input: points = [[3,12],[-2,5],[-4,1]]

Output: 18

Solution:

```
class Solution {
    // Runtime: 802 ms, faster than 19.42% of Java online submissions for Min
    // Cost to Connect All Points.
    // Memory Usage: 59.7 MB, less than 75.86% of Java online submissions for Min
    // Cost to Connect All Points.

    public int minCostConnectPoints(int[][] points) {
        int n = points.length;

        // Sort by smaller distance or cost.
        PriorityQueue<Point> pq = new PriorityQueue<>((a, b) -> a.dist -
b.dist);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int distance = findDistance(points, i, j);
                pq.offer(new Point(i, j, distance));
            }
        }

        UnionFind uf = new UnionFind(n);
        int cost = 0;
        while (!pq.isEmpty()) {
```

```

        Point cur = pq.poll();
        int x = cur.x;
        int y = cur.y;
        int distance = cur.dist;

        // check whether if any cycle detected.
        // in minimum spanning Tree - since Tree no cycle should
exist.

        int a = uf.find(x);
        int b = uf.find(y);

        // if no cycle merge both to their parent i.e from where it
comes from.

        // steps for MST.
        // Step 1: sort by minimum distance
        // Step 2: check if no cycle exists in the path (a,b)
        // Step 3: if no cycle add to MST and track from where the
node originates from.
        // Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
        // Output: 20

        // 0 1->4
        // 0 2->13
        // 0 3->7
        // 0 4->7
        // 1 2->9
        // 1 3->3
        // 1 4->7
        // 2 3->10
        // 2 4->14
        // 3 4->4

        // (0,0) (2,2) (3,10),(5,2) ,(7,0)
        // 0 1 2 3 4

        if (a != b) {
            // add this to MST
            cost += distance;
            // Add to MST.
            uf.union(a, b);
        }
    }

    return cost;
}

```



```
// find distance from 2 points(x1,y1),(x2,y2)
private int findDistance(int points[][], int i, int j) {

    int x1 = points[i][0];
    int y1 = points[i][1];

    int x2 = points[j][0];
    int y2 = points[j][1];

    return Math.abs(x1 - x2) + Math.abs(y1 - y2);
}
}
class Point {
    int x;
    int y;
    int dist;

    // represent (x,y) and the distance.
    Point(int x, int y, int dist) {
        this.x = x;
        this.y = y;
        this.dist = dist;
    }
}
class UnionFind {
    int parent[];

    UnionFind(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    public int find(int val) {
        while (val != parent[val]) {
            val = parent[val];
        }

        return val;
    }

    public void union(int a, int b) {
        int x = find(a);
        int y = find(b);

        if (x == y)
            return;
    }
}
```

```

        parent[x] = parent[y];
    }
}

```

Problem:158: Kruskal's Minimum Spanning Tree Algorithm

Sample Input 1 :

```

2
5 6
1 2 6
2 3 5
3 4 4
1 4 1
1 3 2
3 5 3
2 1
1 2 4

```

Sample output 1 :

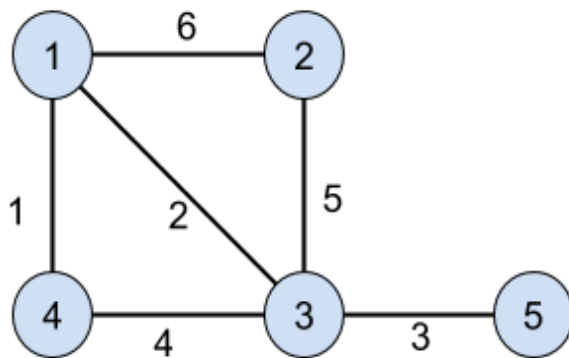
```

11
4

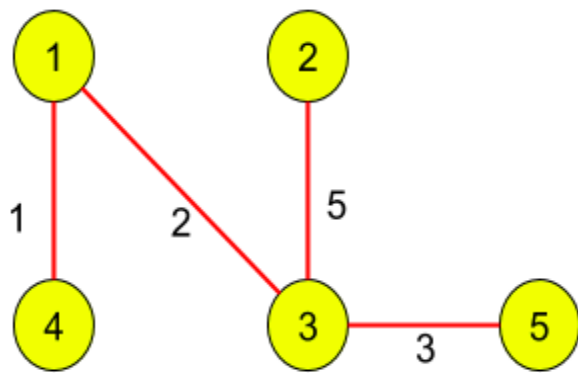
```

Explanation of Sample output 1 :

In test case 1, the graph is:



The minimum spanning tree of the graph is:



And its weight is $1 + 2 + 5 + 3 = 11$.

In test case 2, the graph has only one edge. So the minimum spanning will be the graph itself. And its weight is 4.

Solution:

```

import java.util.*;
import java.io.*;

public class Solution {
    public static int kruskalMST(int n, int m,
    ArrayList<ArrayList<Integer>> adjList) {
        Graph graph = new Graph(n);
        int len = adjList.size();
        for (int i = 0; i < len; i++) {
            graph.edgeList.add(new Edge(adjList.get(i).get(0),
adjList.get(i).get(1), adjList.get(i).get(2)));
        }
        return graph.kruskal();
    }
}

class UnionFind {
    int parent[];

    UnionFind(int n) {
        parent = new int[n + 1];
    }
}
  
```

```
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        while (parent[x] != x) {
            x = parent[x];
        }

        return x;
    }

    public void union(int x, int y) {
        parent[find(x)] = parent[find(y)];
    }
}

class Graph {
    ArrayList<Edge> edgeList = null;
    UnionFind uf;
    int n;

    Graph(int n) {
        this.n = n;
        uf = new UnionFind(n);
        edgeList = new ArrayList<Edge>();
    }

    public int kruskal() {
        Collections.sort(edgeList, (a, b) -> (a.weight - b.weight));
        int cost = 0;
        for (int i = 0; i < edgeList.size(); i++) {
            int x = edgeList.get(i).x;
            int y = edgeList.get(i).y;

            if (uf.find(x) != uf.find(y)) {
                cost += edgeList.get(i).weight;
                uf.union(x, y);
            }
        }
        return cost;
    }
}

class Edge {
    int x;
```

```
int y;  
int weight;  
  
Edge(int x, int y, int weight) {  
    this.x = x;  
    this.y = y;  
    this.weight = weight;  
}  
}
```

Problem:159: Strongly Connected Components (Tarjan's Algorithm)

Sample Input 1:

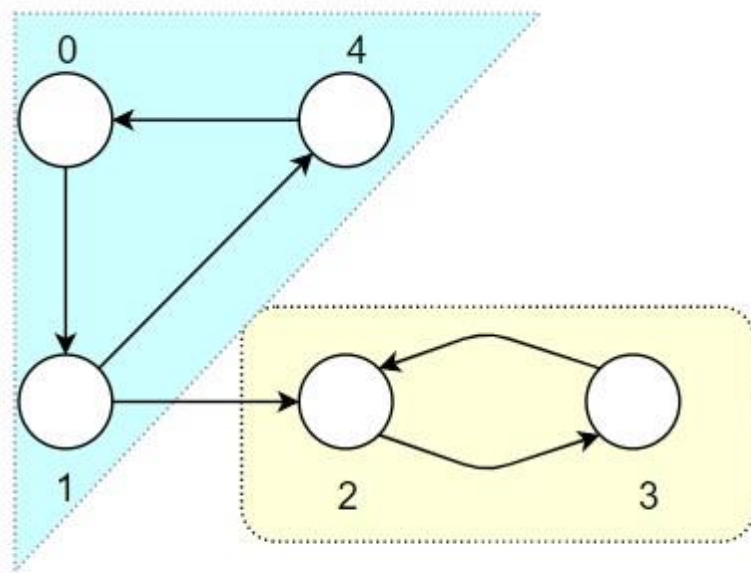
```
1  
5 6  
0 1  
1 2  
1 4  
2 3  
3 2  
4 0
```

Sample Output 1:

```
0 1 4  
2 3
```

Explanation for Sample Input 1:

For the first test case, the graph is shown below. There are two SCCs in the graph, which are enclosed in the boxes as shown in the image below.



Sample Input 2:

```

2
1 0
4 4
0 1
1 2
2 3
3 1

```

Sample Output 2:

```

0
0
1 2 3

```

Explanation for Sample Input 2:

For the first test case, the graph has 1 node(0) but there are no edges, therefore only 1 SCC exists and that is {0}.

For the second test case, one SCC is {0} and the other is {1, 2, 3}. Therefore the answer is 0, 1 2 3

Solution:

```

/*
    Time Complexity: O(V + E)
    Space Complexity: O(V)

    Where V is the number of vertices,
    E is the number of edges in the graph.
*/

```

```

import java.util.Stack;
import java.util.ArrayList;
import java.util.List;
import java.util.Arrays;
public class Solution {
    public static List<List<Integer>> stronglyConnectedComponents(int n,
int[][] edges) {
    // This stores our graph as Adjacency list
    ArrayList<Integer> graph[] = new ArrayList[n];
    for(int i = 0; i < n; i++)
    {
        graph[i] = new ArrayList();
    }

    for (int i = 0; i < edges.length; i++)
    {
        graph[edges[i][0]].add(edges[i][1]);
    }
    discoveryTime = 1;
    // Arrays to store the low-link value and discovery time of the nodes
    int disc[] = new int[n];
    Arrays.fill(disc, -1);
    int low[] = new int[n];
    Arrays.fill(low, -1);

    Stack<Integer> nodeStack = new Stack();
    boolean inStack[] = new boolean[n];

    List<List<Integer>> ans = new ArrayList();
    for (int i = 0; i < n; i++)
    {
        if (disc[i] == -1)
        {
            // Node 'i' has not been visited.
            dfs(graph, i, disc, low, inStack, nodeStack, ans);
        }
    }

    // Return the final answer
    return ans;
}
// This marks the discovery time of the nodes
private static int discoveryTime = 1;
// DFS to visit all the nodes in linear time
public static void dfs(ArrayList<Integer> graph[], int node, int disc[],
int low[], boolean inStack[], Stack<Integer> nodeStack,
List<List<Integer>> ans)

```

```

{
    disc[node] = discoveryTime;
    low[node] = discoveryTime;
    ++discoveryTime;
    nodeStack.push(node);
    inStack[node] = true;

    // Using Tarjan's algorithm here
    for (int i = 0; i < graph[node].size(); i++)
    {
        int v = graph[node].get(i);
        // Visiting all unvisited nodes
        if (disc[v] == -1)
        {
            dfs(graph, v, disc, low, inStack, nodeStack, ans);

            low[node] = Math.min(low[node], low[v]);
        }
        else if (inStack[v])
        {
            low[node] = Math.min(low[node], disc[v]);
        }
    }
    // If current node is root of a SCC
    if (low[node] == disc[node])
    {
        // component stores one of the possible SCCs
        List<Integer> component = new ArrayList();
        int u;
        int top = (int) nodeStack.peek();
        while (top != node)
        {
            u = nodeStack.peek();
            nodeStack.pop();
            inStack[u] = false;

            component.add(u);
            top = (int) nodeStack.peek();
        }
        u = nodeStack.peek();
        nodeStack.pop();
        inStack[u] = false;
        component.add(u);
        // Inserting the current SCC into the answer
        ans.add(component);
    }
}
}

```


Day 25: DYNAMIC PROGRAMMING

Problem:160: 152. Maximum Product Subarray

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Example 1:

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`

Output: 0

Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Solution:

```
class Solution {
    // Runtime: 1 ms, faster than 97.35% of Java online submissions for
    // Maximum Product Subarray.
    // Memory Usage: 42.7 MB, less than 85.79% of Java online submissions for
    // Maximum Product Subarray.
    // Next challenges:

    public int maxProduct(int[] nums) {
        int n= nums.length;
        int left =0;
        int right =0;

        //should assign first value else
        //if only one input max will be 0
        int max = nums[0];
        for(int i=0;i<n;i++){
            //previous value reset to 0 because of
            //zero in middle.
            //if <0 i.e - chance if multiple - exist
            // like -2 ,-3 will turn to +6
            //so other than 0 in middle anything else multiply
            //with previous value.
            left=((left==0)?1:left)* nums[i] ;
            right= ((right==0)?1:right)*nums[n-1-i] ;
            max = Math.max(max,Math.max(left,right));
        }

        return max;
    }
}
```

Problem:161: 300. Longest Increasing Subsequence

Given an integer array `nums`, return *the length of the longest strictly increasing subsequence*.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Solution:

```
import java.util.ArrayList;
import java.util.Collections;

class Solution {
    public int lengthOfLIS(int[] nums) {
        ArrayList<Integer> list = new ArrayList<>();
        int n = nums.length;
        for(int i = 0; i < n; i++){
            int cur = nums[i];
            int index = Collections.binarySearch(list, cur);
            if(index < 0){
                index = ~index;
            }
        }
    }
}
```

```

        if(list.size()==index){
            list.add(cur);
        }else{
            list.set(index,cur);
        }
    }

    return list.size();
}
}

```

Recursion:

```

class Solution {
    public int lengthOfLIS(int[] a) {
        int n = a.length;
        int[][] dp = new int[n][n + 1];
        for (int[] row : dp)
            Arrays.fill(row, -1);
        return help(0, -1, a, dp);
    }
    public int help(int cur, int prev, int[] arr, int[][] cache) {
        int n = arr.length;
        if (cur == n)
            return 0;
        if (cache[cur][prev + 1] != -1)
            return cache[cur][prev + 1];
        int pick = 0;
        int notPick = help(cur + 1, prev, arr, cache);
        if (prev == -1 || arr[cur] > arr[prev]) {
            pick = 1 + help(cur + 1, cur, arr, cache);
        }

        return cache[cur][prev + 1] = Math.max(pick, notPick);
    }
}

```

DP:

```

class Solution {
    public int lengthOfLIS(int[] a) {
        int n = a.length;
        int[][] dp = new int[n+1][n+1];
        for(int cur=n-1; cur>= 0; cur--){
            for(int prev=cur-1; prev >= -1; prev--){
                int notPick = dp[cur+1][prev+1];
                int pick = -1;
                if(prev == -1 || a[cur]>a[prev])
                    pick = 1 + dp[cur+1][cur+1];
                dp[cur][prev+1] = Math.max(pick,notPick);
            }
        }
    }
}

```

```
        }
    }
    return dp[0][0];
}
}
```

Space Optimized:

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int max = Integer.MIN_VALUE;

        for (int ind = 0; ind < n; ind++) {
            for (int prev = 0; prev < ind; prev++) {
                if (nums[prev] < nums[ind]) {
                    dp[ind] = Math.max(1 + dp[prev], dp[ind]);
                }
            }
            max = Math.max(max, dp[ind]);
        }

        return max;
    }
}
```

Problem:162: 1143. Longest Common Subsequence

Given two strings text1 and text2, return *the length of their Longest **common subsequence***. If there is no **common subsequence**, return 0.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

Input: text1 = "abcde", text2 = "ace"

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- 1 <= text1.length, text2.length <= 1000
- text1 and text2 consist of only lowercase English characters.

Solution:

Recursion:

```
class Solution {  
    public int longestCommonSubsequence(String text1, String text2) {  
        char arr1[] = text1.toCharArray();  
        char arr2[] = text2.toCharArray();  
        int n1 = arr1.length;  
        int n2 = arr2.length;  
        int cache[][] = new int[n1 + 1][n2 + 1];  
        for (int i = 0; i < n1; i++) {  
            Arrays.fill(cache[i], -1);  
        }  
  
        return helper(arr1, arr2, 0, 0, cache);  
    }  
}
```

```

        private int helper(char arr1[], char arr2[], int i, int j, int
cache[][]) {
            int n1 = arr1.length;
            int n2 = arr2.length;
            if (i >= n1 || j >= n2) {
                return 0;
            }
            if (cache[i][j] != -1) {
                return cache[i][j];
            }
            if (arr1[i] == arr2[j]) {
                return cache[i][j] = 1+helper(arr1, arr2, i + 1, j + 1,
cache);
            }
            int left = helper(arr1, arr2, i + 1, j, cache);
            int right = helper(arr1, arr2, i, j + 1, cache);
            return cache[i][j] = Math.max(left, right);
        }
    }
}

```

DP:

class Solution {

// Runtime: 12 ms, faster than 94.53% of Java online submissions for Longest Common Subsequence.

// Memory Usage: 46.1 MB, less than 70.48% of Java online submissions for Longest Common Subsequence.

```

    public int longestCommonSubsequence(String text1, String text2) {
        //abcef  acde
        //          f(0,0)
        //          /
        //          f(1,1) -match
        //          / \
        //          f(2,1) f(1,2) - match
        //          / \   / \
        //          f(2,2) f(3,2)       f(2,2) f(1,3)
    }
}

```

```
int n1 = text1.length();
int n2 = text2.length();

int dp[][] = new int[n1+1][n2+1];

dp[0][0]=0;

char arr1[] = text1.toCharArray();
char arr2[] = text2.toCharArray();

for(int i=1;i<=n1;i++){
    for(int j=1;j<=n2;j++){
        if(arr1[i-1] == arr2[j-1]){
            dp[i][j] = 1+dp[i-1][j-1];
        }else{
            dp[i][j]= Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
}

return dp[n1][n2];
}
```

Space Optimized:

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        // Runtime: 10 ms, faster than 98.94% of Java online submissions for
        // Longest Common Subsequence.
        // Memory Usage: 43.9 MB, less than 95.06% of Java online submissions
        // for Longest Common Subsequence.
        int n1 = text1.length();
        int n2 = text2.length();
```



```

    int prev[] = new int[n2+1];
    int cur[] = new int[n2+1];
    for(int i=0;i<=n2;i++)
        prev[i]=0;

    char arr1[] = text1.toCharArray();
    char arr2[] = text2.toCharArray();

    for(int i=1;i<=n1;i++){
        for(int j=1;j<=n2;j++){
            if(arr1[i-1] == arr2[j-1]){
                cur[j] = 1+prev[j-1];
            }else{
                cur[j]= Math.max(prev[j], cur[j-1]);
            }
        }
        //must to clone else wont work
        prev = (int[])(cur.clone());
    }

    return prev[n2];
}
}

```

Problem:163: 474. Ones and Zeroes

You are given an array of binary strings `strs` and two integers `m` and `n`.

Return the size of the largest subset of `strs` such that there are **at most** `m` 0's and `n` 1's in the subset.

A set `x` is a **subset** of a set `y` if all elements of `x` are also elements of `y`.

Example 1:

Input: `strs = ["10","0001","111001","1","0"]`, `m = 5`, `n = 3`

Output: 4

Explanation: The largest subset with at most 5 0's and 3 1's is {"10", "0001", "1", "0"}, so the answer is 4. Other valid but smaller subsets include {"0001", "1"} and {"10", "1", "0"}. {"111001"} is an invalid subset because it contains 4 1's, greater than the maximum of 3.

Example 2:

Input: `strs = ["10","0","1"]`, `m = 1`, `n = 1`

Output: 2

Explanation: The largest subset is {"0", "1"}, so the answer is 2.

Constraints:

- $1 \leq \text{strs.length} \leq 600$
- $1 \leq \text{strs}[i].\text{length} \leq 100$
- $\text{strs}[i]$ consists only of digits '0' and '1'.
- $1 \leq m, n \leq 100$

Solution:

```
import java.util.TreeMap;
//Runtime: 19 ms, faster than 99.64% of Java online submissions for Ones and
Zeroes.
//Memory Usage: 41.2 MB, less than 75.25% of Java online submissions for Ones
and Zeroes.
class Solution {
    public int findMaxForm(String[] strs, int m, int n) {
        // Input: strs = ["10","0001","111001","1","0"], m = 5, n = 3
        // m(0) n(1)
        // 0- 1 1
        // 1- 3 1
        // 2- 2 4
        // 3 -0 1
        // 4 -1 0
        int len = strs.length;
        int dp[][] = new int[m + 1][n + 1];

        for (int x = 0; x < len; x++) {
            String cur = strs[x];

            int arr[] = getCount(cur);
            int zero = arr[0];
            int one = arr[1];

            for (int i = m; i >= zero; i--) {
                for (int j = n; j >= one; j--) {
                    int curMax = dp[i - zero][j - one] + 1;
                    dp[i][j] = Math.max(dp[i][j], curMax);
                }
            }
        }
    }
}
```

```
        }

        return dp[m][n];
    }

    private int[] getCount(String val) {
        int n = val.length();
        int arr[] = new int[2];
        for (int i = 0; i < n; i++) {
            int index = val.charAt(i) - '0';
            arr[index]++;
        }
        return arr;
    }

    public static void main(String args[]) {
        String arr[] = { "10", "0001", "111001", "1", "0" };
        int res = new Solution().findMaxForm(arr, 5, 3);

        System.out.println("Solution.main()" + res);
    }
}
```

Problem:164: 72. Edit Distance

Given two strings word1 and word2, return *the minimum number of operations required to convert word1 to word2*.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse -> rorse (replace 'h' with 'r')

rorse -> rose (remove 'r')

rose -> ros (remove 'e')

Recursion:

```
class Solution {
    public int minDistance(String word1, String word2) {
        int n1= word1.length();
        int n2 = word2.length();

        int cache[][]= new int[n1][n2];
        for(int i=0;i<n1;i++){
            Arrays.fill(cache[i],-1);
        }

        return helper(word1.toCharArray(), word2.toCharArray(),0,0,cache);
    }

    private int helper(char arr1[], char arr2[],int i,int j,int cache[][]){
        int n1= arr1.length;
        int n2 = arr2.length;
        int n = Math.min(n1,n2);

        if(i == n1 && j==n2) {
            return 0;
        }

        if(i>=n1 || j>=n2) {
            if(i==n1){
                return n2-j;
            }else{
                return n1-i;
            }
        }

        if(cache[i][j]!=-1){
            return cache[i][j];
        }

        int res = 0;
        if(arr1[i]==arr2[j]){
            return cache[i][j]= helper(arr1,arr2,i+1,j+1,cache);
        }
    }
}
```

```

        //abcd acd
        int add = helper(arr1,arr2,i,j+1,cache);
        int remove = helper(arr1,arr2,i+1,j,cache);
        int replace = helper(arr1,arr2,i+1,j+1,cache);
        res = Math.min(Math.min(add,remove),replace)+1;

        return cache[i][j]= res;
    }
}

```

DP:

```

class Solution {
// Runtime: 10 ms, faster than 40.93% of Java online submissions for Edit Distance.
// Memory Usage: 45.1 MB, less than 41.36% of Java online submissions for Edit Distance.

    public int minDistance(String word1, String word2) {
        int n1 = word1.length();
        int n2 = word2.length();

        int dp[][] = new int[n1 + 1][n2 + 1];

        char arr1[] = word1.toCharArray();
        char arr2[] = word2.toCharArray();
        for (int i = 0; i <= n1; i++) {
            dp[i][0] = i;
        }
        for (int j = 0; j <= n2; j++) {
            dp[0][j] = j;
        }
        int i = 1, j = 1;
        for (i = 1; i <= n1; i++) {
            for (j = 1; j <= n2; j++) {
                if (arr1[i - 1] == arr2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    // ac abc
                    int add = dp[i][j - 1];
                    int remove = dp[i - 1][j];
                    int replace = dp[i - 1][j - 1];

                    dp[i][j] = Math.min(Math.min(add, remove),
                        replace) + 1;
                }
            }
        }
    }
}

```

```
    }  
    return dp[n1][n2];  
}  
}
```

Space optimized:

```
class Solution {  
    // Runtime: 10 ms, faster than 40.93% of Java online submissions for Edit  
    Distance.  
    // Memory Usage: 45.1 MB, less than 41.36% of Java online submissions for  
    Edit Distance.
```

```
    public int minDistance(String word1, String word2) {  
        int n1 = word1.length();  
        int n2 = word2.length();  
  
        char arr1[] = word1.toCharArray();  
        char arr2[] = word2.toCharArray();  
  
        int cur[] = new int[n2 + 1];  
        int prev[] = new int[n2 + 1];  
  
        for (int i = 0; i <= n2; i++) {  
            prev[i] = i;  
        }  
  
        int i = 1, j = 1;  
        for (i = 1; i <= n1; i++) {  
            cur[0] = i;  
            for (j = 1; j <= n2; j++) {  
                if (arr1[i - 1] == arr2[j - 1]) {  
                    cur[j] = prev[j - 1];  
                } else {  
                    // ac abc  
                    int add = cur[j - 1];  
                    int remove = prev[j];  
                    int replace = prev[j - 1];  
  
                    cur[j] = Math.min(Math.min(add, remove),  
replace) + 1;  
                }  
            }  
        }  
    }  
}
```

```
        prev = cur.clone();
    }

    return prev[n2];
}
}
```

Problem:165: Maximum sum increasing subsequence

Example 1:

Input: N = 5, arr[] = {1, 101, 2, 3, 100}

Output: 106

Explanation:The maximum sum of a increasing sequence is obtained from {1, 2, 3, 100}

Example 2:

Input: N = 3, arr[] = {1, 2, 3}

Output: 6

Explanation:The maximum sum of a increasing sequence is obtained from {1, 2, 3}

Your Task:

You don't need to read input or print anything. Complete the function **maxSumIS()** which takes **N** and array **arr** as input parameters and returns the maximum value.

Expected Time Complexity: $O(N^2)$

Expected Auxiliary Space: $O(N)$

Constraints:

$1 \leq N \leq 10^3$

$1 \leq \text{arr}[i] \leq 10^5$

Solution:

```
class Solution {
```

```
public int maxSumIS(int arr[], int n) {  
    if (n == 1) {  
        return arr[0];  
    }  
    int[] lis = new int[n];  
    // code here.  
    for (int i = 0; i < n; i++) {  
        lis[i] = arr[i];  
    }  
    int ans = 0;  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < i; j++) {  
            if (arr[i] > arr[j]) {  
                lis[i] = Math.max(lis[j] + arr[i], lis[i]);  
            }  
        }  
        ans = Math.max(ans, lis[i]);  
    }  
    return ans;  
}
```

Problem:166: Matrix Chain Multiplication

Sample Input 1:

```
2  
4  
4 5 3 2  
4  
10 15 20 25
```

Sample Output 1:

```
8000  
70
```


Sample Output Explanation 1:

In the first test case, there are three matrices of dimensions $A = [4\ 5]$, $B = [5\ 3]$ and $C = [3\ 2]$. The most efficient order of multiplication is $A * (B * C)$.

Cost of $(B * C) = 5 * 3 * 2 = 30$ and $(B * C) = [5\ 2]$ and $A * (B * C) = [4\ 5] * [5\ 2] = 4 * 5 * 2 = 40$. So the overall cost is equal to $30 + 40 = 70$.

In the second test case, there are two ways to multiply the chain - $A1*(A2*A3)$ or $(A1*A2)*A3$.

If we multiply in order- $A1*(A2*A3)$, then the number of multiplications required is 11250.

If we multiply in order- $(A1*A2)*A3$, then the number of multiplications required is 8000.

Thus a minimum number of multiplications required is 8000.

Sample Input 2:

1

4

1 4 3 2

Sample Output 2:

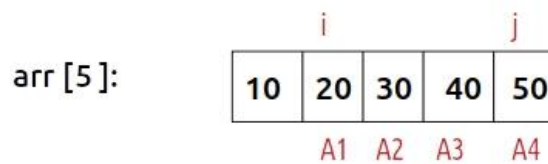
18

Explanation of Sample Output 2:

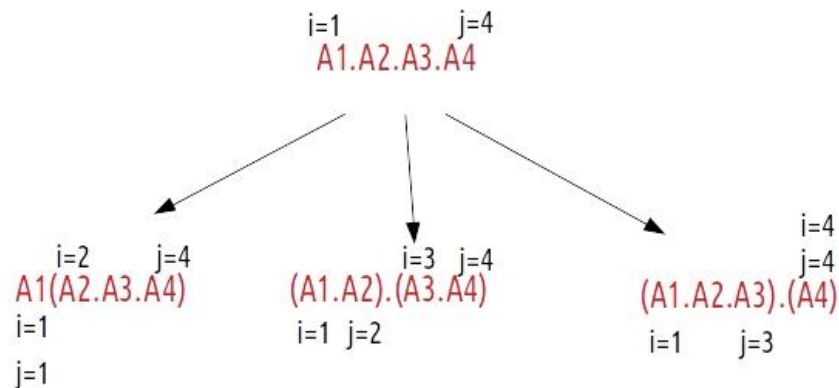
In the first test case, there are three matrices of dimensions $A = [1\ 4]$, $B = [4\ 3]$ and $C = [3\ 2]$. The most efficient order of multiplication is $(A * B) * C$.

Solution:

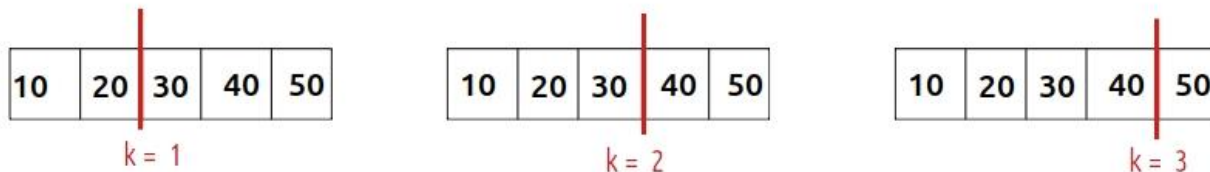
Recursion:



Minimum operations to get:



Possible partitions



```
import java.util.*;
public class Solution {
    static int mod = (int)1e9+7;
    public static int matrixMultiplication(int[] arr , int N) {
        int dp[][] = new int[N][N];
        for(int i =0;i<N;i++){
            Arrays.fill(dp[i],-1);
        }
        return helper(arr,1,N-1,dp);
    }

    private static int helper(int arr[],int i,int j,int dp[][] ){
        if(dp[i][j]!=-1){
            return dp[i][j];
        }
        if(i ==j)return 0;
        //(10 X20) * (20*30) = 10 X20X30 //where 20 is common
```

```

//      i  n
//f(i,j) = f(i,k) +f(k+1,j)

// 10,20,30,40,50 60
// 0  1  2  3  4  5
//      <----->
//      i          k <---->
//          j

//      (10,20) + f(10,10) + f(20,60)

//          (1,4)
//          / \
//      (1,1) (2,4)          // (10,20) X (60)
//          / \          // 10X20X60
//      200+ [0] (2,2) (3,4)
//          / \ / \
//          [0] (3,3) (4,4)
//          / \
//          [0] [0]

// 10,20,30,40
// 0  1  2  3
//      i  k  j
//      <---->
//          <---->

//          (1,3)
//          / \
//      (1,1) (2,3)          //(1,1)= (10,20) X (40)
//          | \ / \          // 10X20X40 = 8000 +
//      0 (1,2) (2,2) (3,3)
//          / \ | |
//      (1,1) (3,3) 0 0
//          | |
//      12000 6000

//      Solution.helper()24000 left:[2,2]0 right:[3,3]0
//      Solution.helper()8000 left:[1,1]0 right:[2,3]24000
//      Solution.helper()6000 left:[1,1]0 right:[2,2]0
//      Solution.helper()12000 left:[1,2]6000 right:[3,3]0
//      Solution.matrixMultiplication()18000

//10 20 30 40

```

```

//10 20
//      30 40
int min = Integer.MAX_VALUE;
for (int k = i; k < j; k++) {
    int root = (arr[i - 1] * arr[k] * arr[j]);
    int left = helper(arr, i, k, dp);
    int right = helper(arr, k + 1, j, dp);
    // System.out.println("Solution.helper()" + root + "
left:["+i+", "+k+"]"
    // +left + " right:["+(k+1)+", "+ j +"]"+right);
    min = Math.min(min, root + left + right);
}

return dp[i][j] = min;
}
}

```

DP:

```

import java.util.*;
import java.io.*;

public class Solution {
    // Function to find minimum cost(scalar multiplication) of matrix multiplication
    public static int matrixMultiplication(int[] arr, int N) {
        //For simplicity of the program, one extra row and one extra column are
        //allocated in 'dp[][]'. 0th row and 0th column of 'dp[][]'
        //are not used
        int dp[][] = new int[N][N];

        //State: dp[i, j] = Minimum number of scalar multiplications
        //needed to compute
        // the matrix A[i]A[i+1]...A[j] = A[i..j] where dimension of A[i]
        //is arr[i-1] x arr[i]

        // The cost of multiplying one matrix is 0
        for (int i = 1; i < N; i++) {
            // Make dp[i][i] 0
            dp[i][i] = 0;
        }
        // Run a loop from length 2 to n - 1
    }
}

```

```
for (int l = 2; l < N; l++) {
    for (int i = 1; i < N - l + 1; i++) {
        int j = i + l - 1;
        // Initialize dp[i][j] with maximum value
        dp[i][j] = Integer.MAX_VALUE;
        for (int k = i; k <= j - 1; k++) {
            // Store the temporary cost (scalar multiplications)
            //in 'temp'
            int temp = dp[i][k] + dp[k + 1][j]
                      + arr[i - 1] * arr[k] * arr[j];
            // If temporary answer 'temp' is less than actual
            //answer
            // then update actual ans i.e dp[i][j]
            dp[i][j] = Math.min(dp[i][j], temp);
        }
    }
}
// Return dp[1][N-1]
return dp[1][N - 1];
}
```

DAY 26: DYNAMIC PROGRAMMING-II

Problem:167: 64. Minimum Path Sum

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{grid}[i][j] \leq 200$

Solution:

Recursion:

```
class Solution {
// Runtime: 1 ms, faster than 99.98% of Java online submissions for
Minimum Path Sum.
```

// Memory Usage: 45.9 MB, less than 56.11% of Java online submissions for Minimum Path Sum.

```

    public int minPathSum(int[][] grid) {
        int r = grid.length - 1;
        int c = grid[0].length - 1;
        int cache[][] = new int[r + 1][c + 1];
        for (int i = 0; i <= r; i++)
            Arrays.fill(cache[i], -1);
        return helper(grid, r, c, cache);
    }
    private int helper(int grid[][], int row, int col, int cache[][]) {
        int r = grid.length - 1;
        int c = grid[0].length - 1;
        if (row == 0 && col == 0) {
            return grid[row][col];
        }
        if (cache[row][col] != -1) {
            return cache[row][col];
        }
        if (row == 0 || col == 0) {
            if (row == 0 && col != 0) {
                return cache[row][col] = grid[row][col]
                    + helper(grid, row, col - 1, cache);
            }

            if (col == 0 && row != 0) {
                return cache[row][col] = grid[row][col]
                    + helper(grid, row - 1, col, cache);
            }
        }
        int right = 0, down = 0;
        down = (grid[row][col] + helper(grid, row - 1, col, cache));
        right = (grid[row][col] + helper(grid, row, col - 1, cache));

        return cache[row][col] = Math.min(down, right);
    }
}

```

DP;

```

class Solution {
    public int minPathSum(int[][] grid) {
        int height = grid.length;
        int width = grid[0].length;
        for (int row = 0; row < height; row++) {
            for (int col = 0; col < width; col++) {

```



```

        if (row == 0 && col == 0) {
            grid[row][col] = grid[row][col];
        } else if (row == 0 && col != 0) {
            grid[row][col] = grid[row][col]
                + grid[row][col - 1];
        } else if (col == 0 && row != 0) {
            grid[row][col] = grid[row][col]
                + grid[row - 1][col];
        } else {
            grid[row][col] = grid[row][col]
                + Math.min(grid[row - 1][col], grid[row][col -
1]);
        }
    }
}
return grid[height - 1][width - 1];
}
}

```

Problem:168: 322. Coin Change

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: `coins = [1,2,5]`, `amount = 11`

Output: 3

Explanation: $11 = 5 + 5 + 1$

Solution:

Recursion(TLE):

```

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, -1);
        int res = helper (coins, amount, 0, dp);
        return res == Integer.MAX_VALUE ? -1 : res;
    }
}

```

```

    }
    public int helper (int[] coins, int n, int number, int[] dp) {
        if (n == 0)
            return 0;
        int res = Integer.MAX_VALUE;
        for (int i = 0; i < coins.length; i++) {
            int temp = 0;
            if (n - coins[i] >= 0) {
                if (dp[n - coins[i]] != -1)
                    temp = dp[n - coins[i]];
                else
                    temp = helper(coins, n - coins[i], number + 1,
dp);

                if (temp != Integer.MAX_VALUE)
                    res = Math.min(res, temp + 1);
            }
        }
        return dp[n] = res;
    }
}

```

DP:

```

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {
            for (int j = 0; j < coins.length; j++) {
                if (i - coins[j] >= 0
                    && dp[i - coins[j]] != Integer.MAX_VALUE)
                    dp[i] = Math.min(dp[i], dp[i - coins[j]] + 1);
            }
        }
        return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
    }
}

```

Problem:169: 416. Partition Equal Subset Sum

416. Partition Equal Subset Sum

Medium

Given an integer array *nums*, return *true* if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or false otherwise.

Example 1:**Input:** nums = [1,5,11,5]**Output:** true**Explanation:** The array can be partitioned as [1, 5, 5] and [11].**Example 2:****Input:** nums = [1,2,3,5]**Output:** false**Explanation:** The array cannot be partitioned into equal sum subsets.**Constraints:**

- 1 <= nums.length <= 200
- 1 <= nums[i] <= 100

Solution:

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        int n = nums.length;
        for (int i : nums)
            sum += i;
        if (sum % 2 != 0)
            return false;
        sum /= 2;
        int cache[][] = new int[n + 1][sum + 1];
        for (int i = 0; i < n; i++) {
            Arrays.fill(cache[i], -1);
        }
        return helper(nums, n - 1, sum, cache);
    }

    private boolean helper(int nums[], int index, int target, int
cache[][]) {
        int n = nums.length;
        if (target == 0) {
            return true;
        }
        if (target < 0)
            return false;
        if (cache[index][target] != -1) {
```

```

        return cache[index][target] == 1 ? true : false;
    }
    if (index == 0) {
        if (target == nums[index])
            return true;

        return false;
    }
    boolean include = false;
    include = helper(nums, index - 1, target - nums[index], cache);
    boolean exclude = helper(nums, index - 1, target, cache);
    boolean result = include || exclude;
    cache[index][target] = result ? 1 : 0;

    return result;
}
}

```

DP:

```

class Solution {
//Runtime: 35 ms, faster than 84.12% of Java online submissions for Partition
Equal Subset Sum.
//Memory Usage: 40.4 MB, less than 99.89% of Java online submissions for
Partition Equal Subset Sum.
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int cur : nums) {
            sum += cur;
        }
        // if cannot be divided by 2 half
        if (sum % 2 != 0) {
            return false;
        }
        sum = sum / 2;
        boolean dp[] = new boolean[sum + 1];

        // since 0 can be divided to 2 '0s
        dp[0] = true;
        int n = nums.length;
        // 1 ,5 ,11, 5
        // 22
        // 22/2 = 11

        // 11 -> 1
        // 11 ->5
        // 11 ->11
    }
}

```

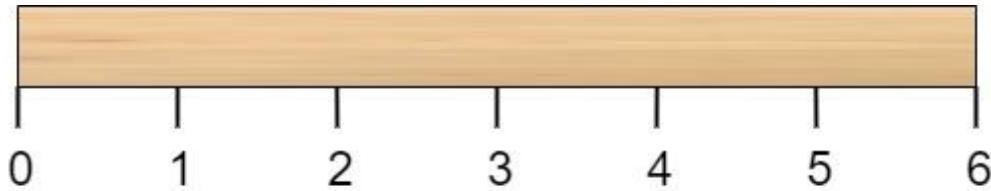
```

// 1 - n
// 11 to 1 //i=1
// 11 to 5 //i=2
// 11 to 11 //i=3i.e dp[11-nums[2]] -> dp[11-11]= dp[0]
for (int i = 1; i <= n; i++) { // 4
    for (int j = sum; j >= nums[i - 1]; j--) {
        dp[j] = dp[j] || dp[j - nums[i - 1]];
    }
}
return dp[sum];
}
}

```

Problem:170: 1547. Minimum Cost to Cut a Stick

Given a wooden stick of length n units. The stick is labelled from 0 to n . For example, a stick of length 6 is labelled as follows:



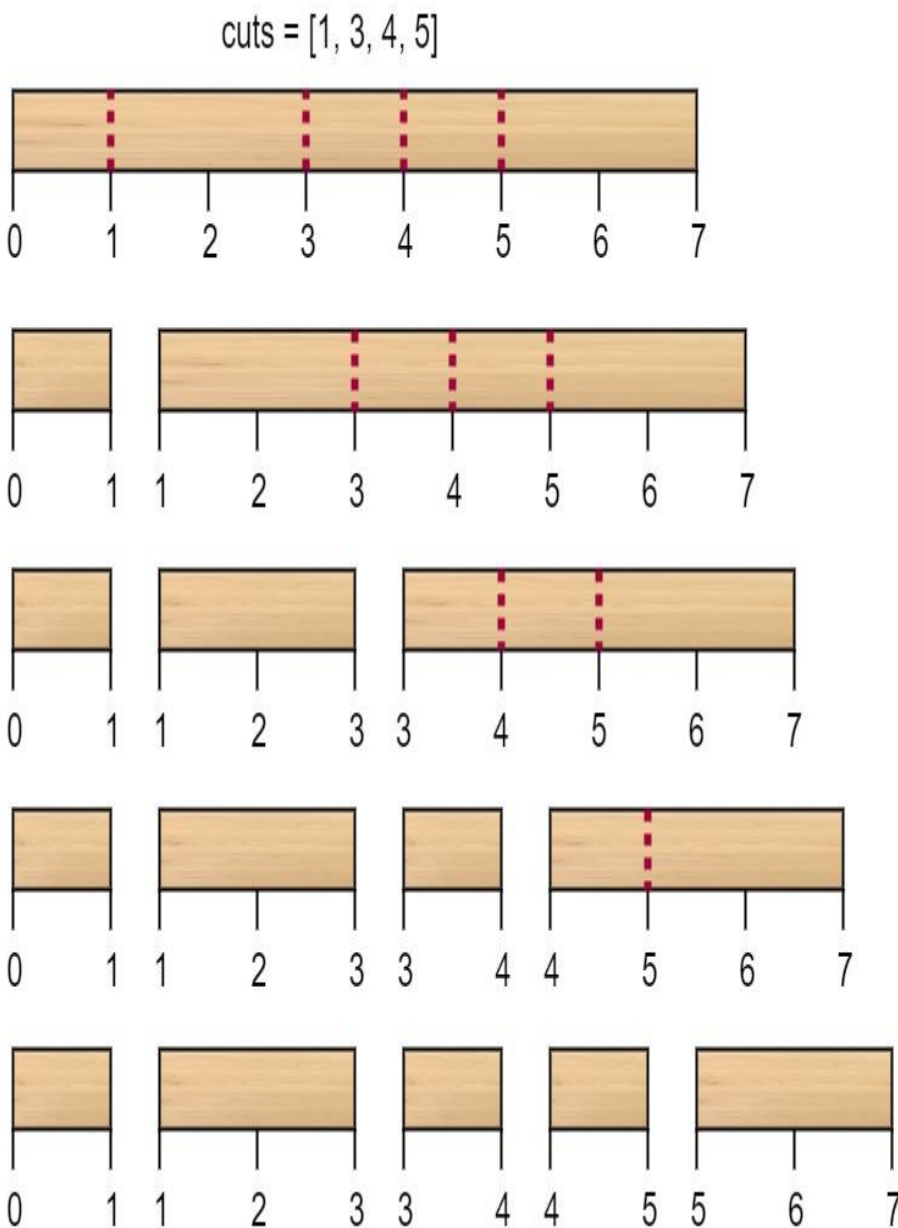
Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at.

You should perform the cuts in order, you can change the order of the cuts as you wish.

The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Example 1:



The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length 6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is to a rod of length 3. The total cost is $7 + 6 + 4 + 3 = 20$.

Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as shown in the example photo $7 + 4 + 3 + 2 = 16$).

Solution:

Recursion:

```
class Solution {
//      Runtime: 1360 ms, faster than 5.89% of Java online submissions for
Minimum Cost to Cut a Stick.
// Memory Usage: 117.8 MB, less than 8.34% of Java online submissions for
Minimum Cost to Cut a Stick.
    public int minCost(int n, int[] cuts) {
        ArrayList<Integer> list = new ArrayList<>();
        for(int val : cuts){
            list.add(val);
        }
        //added 2 extra
        list.add(0);
        Collections.sort(list);
        list.add(n);
        int len= cuts.length;
        HashMap<String,Integer> cache = new HashMap<>();
        return helper(list,1,len,cache);
    }
}
```

```
private int helper(ArrayList<Integer> list, int i, int j,
                    HashMap<String,Integer> cache){
    //0 1 2
    //in partition dp cross over means
    //all partition is solved
    if(i>j)return 0;

    // 0    1  3  5    7    5
    // -    - - - - - - - - -
```



```

// |   |   |   |   |
// i-1 i       j   (j+1)

//we sort to make sure we can do overlapping to make
// each one is dependent on others.

// let says if we dont sort instead of 1,3,5,7 we have 1,7,3,5
// 1 7 3 5
// we cant do prefix sum on this.
//after sort

// 0 1 3 5 7
// 0 1
// <-mimumum so far-->

//

//
//if 1 -> total cut 1 (1)
// 2 -> total cut 2 (1,3)

String key = i+"_"+j;
if(cache.get(key)!=null){
    return cache.get(key);
}

int min = Integer.MAX_VALUE;
for(int index = i; index<=j;index++){
    int cost = list.get(j+1)- list.get(i-1)
                +helper(list,i,index-1,cache)
                +helper(list,index+1,j,cache);

    min = Math.min(min,cost);
}

cache.put(key, min);

return min;
}
}

```

DP:

```

class Solution {
//Runtime: 23 ms, faster than 64.83% of Java online submissions for Minimum
Cost to Cut a Stick.

```

//Memory Usage: 41.4 MB, less than 76.06% of Java online submissions for Minimum Cost to Cut a Stick.

```

    public int minCost(int n, int[] cuts) {
        ArrayList<Integer> list = new ArrayList<>();
        for(int val : cuts){
            list.add(val);
        }
        //added 2 extra
        list.add(0);
        Collections.sort(list);
        list.add(n);
        int len= cuts.length;
        return helper(list );
    }
    private int helper(ArrayList<Integer> list ){
        int n = list.size()-2;
        int dp[][]= new int[n+2][n+2];
        for(int i=n;i>=1;i--){
            for(int j=1;j<=n;j++){
                if(i>j)continue;
                int min = Integer.MAX_VALUE;
                for(int index = i; index<=j;index++){
                    int cost = list.get(j+1)- list.get(i-1) +dp[i][index-1]
                        +dp[index+1][j];
                    min = Math.min(min,cost);
                }
                dp[i][j]=min;
            }
        }
        return dp[1][n];
    }
}

```

Problem:171: 887. Super Egg Drop

You are given k identical eggs and you have access to a building with n floors labeled from 1 to n .

You know that there exists a floor f where $0 \leq f \leq n$ such that any egg dropped at a floor **higher** than f will **break**, and any egg dropped **at or below** floor f will **not break**.

Each move, you may take an unbroken egg and drop it from any floor x (where $1 \leq x \leq n$). If the egg breaks, you can no longer use it. However, if the egg does not break, you may **reuse** it in future moves.

Return *the minimum number of moves that you need to determine with certainty what the value of f is.*

Example 1:

Input: $k = 1, n = 2$

Output: 2

Explanation:

Drop the egg from floor 1. If it breaks, we know that $f = 0$.

Otherwise, drop the egg from floor 2. If it breaks, we know that $f = 1$.

If it does not break, then we know $f = 2$.

Hence, we need at minimum 2 moves to determine with certainty what the value of f is.

Example 2:

Input: $k = 2, n = 6$

Output: 3

Example 3:

Input: $k = 3, n = 14$

Output: 4

Solution:

Recursion(TLE):

```
class Solution {  
    // TLE  
    public int superEggDrop(int K, int N) {  
        // k identical eggs  
        // 1 to n  
    }  
}
```

```

        // k - eggs
        // n - n floors.
        HashMap<String, Integer> cache = new HashMap<>();
        return helper(K, N, cache);
    }

    private int helper(int e, int f, HashMap<String, Integer> cache) {
        // if one floor again need to try all floor above
        // to find threshold at which it break at what floor.
        if (f == 0 || f == 1)
            return f;
        // if 1 floor 1 egg
        // if we have 1 egg safest way is
        // start from floor 1 go upwards when break
        // we find the threshold when egg break at what floor.
        if (e == 1)
            return f;

        String key = e + "_" + f;
        if (cache.get(key) != null) {
            return cache.get(key);
        }
        int min = Integer.MAX_VALUE;
        // k=n
        // ...
        // k=2
        // k=1 //f (if break then threshold is f-1 no need to go up)
        // k=0

        // if egg break no need to go up obviously it will break
        // so go down f-1;.
        // if egg not break, need to go up (f-k)-total floor -visited so far
        for (int k = 1; k <= f; k++) {

            int eggBreak = helper(e - 1, k - 1, cache);
            int notBreak = helper(e, f - k, cache);
            // worst case
            int temp = 1 + Math.max(eggBreak, notBreak);
            min = Math.min(min, temp);
        }
        cache.put(key, min);
        return min;
    }
}

```

DP:

```

class Solution {
    public int superEggDrop(int e, int f) {
        // dp: for floor f ,min egg required.
        int dp[][] = new int[f + 1][e + 1];
        int index = 0;
        // if no of egg required <f
        while (dp[index][e] < f) {
            index++;
        }
    }
}

```

```
        // try with all egg available
        for (int k = 1; k <= e; k++) {
            // broken then threshold is f-1.
            int breakEgg = dp[index - 1][k - 1];
            // since egg not broken
            int notBreakEgg = dp[index - 1][k];
            dp[index][k] = breakEgg + notBreakEgg + 1;
        }
    }
    return index;
}
```

Problem:171: 139. Word Break

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: *s* = "leetcode", *wordDict* = ["leet","code"]

Output: true

Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: *s* = "applepenapple", *wordDict* = ["apple","pen"]

Output: true

Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".

Note that you are allowed to reuse a dictionary word.

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Constraints:

- 1 <= s.length <= 300
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 20
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are **unique**.

Solution:

DP:

```
class Solution {  
    // Runtime: 8 ms, faster than 63.57% of Java online submissions for Word  
    // Break.  
  
    public boolean wordBreak(String s, List<String> wordDict) {  
  
        // Input: s = "catsandog", wordDict =  
        ["cats","dog","sand","and","cat"]  
        // Step 1: maintain a array of start and end visited  
        // if input is "a" -> [0,1] is start and end as per string  
        // we represent if present in dict as [true,true] ,  
  
        //  
        // c a t s a n d o g  
        HashSet<String> st = new HashSet<>();  
        for (String word : wordDict) {  
            st.add(word);  
        }  
        int n = s.length();  
        // since in string to retrieve last it should be n+1  
        boolean dp[] = new boolean[n + 1];  
        dp[0] = true;  
  
        for (int i = 1; i <= n; i++) {
```

```

        for (int j = 0; j < i; j++) {
            String word = s.substring(j, i);
            // dp[j] whether start of the word is in visited.
            // and end not in visited
            // in order to confirm word "cats" is visited below
            //should be true
            // dp[0]=true
            // and dp[3]=true i.e cats(0,3) is true.
            if (dp[j] && st.contains(word)) {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[n];
}
}

```

Problem:172: 131. Palindrome Partitioning

Given a string *s*, partition *s* such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of s*.

Example 1:

Input: *s* = "aab"

Output: [["a","a","b"],["aa","b"]]

Example 2:

Input: *s* = "a"

Output: [["a"]]

Constraints:

- $1 \leq s.length \leq 16$
- *s* contains only lowercase English letters.

Solution:

```

import java.util.ArrayList;
import java.util.List;

```

```

class Solution {
    // Runtime: 23 ms, faster than 32.86% of Java online submissions for
    Palindrome Partitioning.

```

```

public List<List<String>> partition(String s) {
    //      "aab"
    //      /  \
    //  [a]      [a]2 a
    //  / \      /  \
    // [a] [ab] [b]  []
    //  /
    // b[1,["a","a","b"]]
    List<List<String>> result = new ArrayList<>();
    List<String> list = new ArrayList<>();
    helper(s,list,result,0);

    return result;
}

private void helper(String s, List<String> list,
                    List<List<String>> result, int index)
{
    int n = s.length();
    if (index == n) {
        result.add(new ArrayList<String>(list));
        return;
    }
    for (int i = index; i < n; i++) {
        String cur = s.substring(index, i + 1);
        if (isPalindrome(cur)) {
            list.add(cur);
            helper(s, list, result, i + 1);
            list.remove(list.size() - 1);
        }
    }
}

private boolean isPalindrome(String cur) {
    char arr[] = cur.toCharArray();
    int low = 0;
    int high = arr.length - 1;
    while (low <= high) {
        if (arr[low] != arr[high])
            return false;

        low++;
        high--;
    }

    return true;
}

```



```

    }
}

```

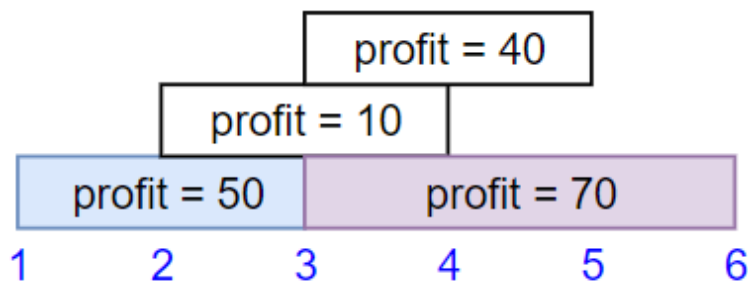
Problem:173: 1235. Maximum Profit in Job Scheduling

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:

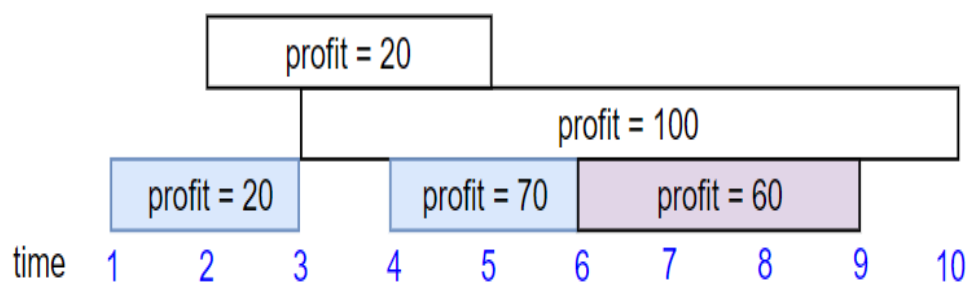


Input: $startTime = [1,2,3,3]$, $endTime = [3,4,5,6]$, $profit = [50,10,40,70]$

Output: 120

Explanation: The subset chosen is the first and fourth job.
Time range $[1-3] + [3-6]$, we get profit of $120 = 50 + 70$.

Example 2:

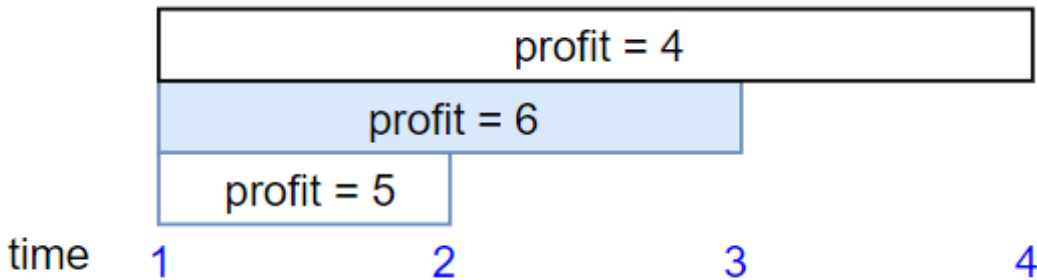


Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job. Profit obtained $150 = 20 + 70 + 60$.

Example 3:



Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]

Output: 6

Constraints:

- $1 \leq \text{startTime.length} == \text{endTime.length} == \text{profit.length} \leq 5 * 10^4$
- $1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9$
- $1 \leq \text{profit}[i] \leq 10^4$

Solution:

```
import java.util.Collections;
import java.util.LinkedList;
import java.util.TreeMap;

class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit)
    {
        LinkedList<Node> queue = new LinkedList<>();
        int n = startTime.length;
        for (int i = 0; i < n; i++) {
            int start = startTime[i];
            int end = endTime[i];
            int curprofit = profit[i];
            queue.offer(new Node(start, end, curprofit));
        }
        Collections.sort(queue, (a, b) -> (a.end - b.end));

        return helper(queue);
    }
}
```

```
}
```

```
private int helper(LinkedList<Node> queue) {
    int maxProfit = 0;
    TreeMap<Integer, Integer> map = new TreeMap<>();
    while (!queue.isEmpty()) {
        Node cur = queue.poll();
        int profit = cur.profit;

        Integer key = map.floorKey(cur.start);

        if (key != null) {
            maxProfit = Math.max(maxProfit, map.get(key) +
profit);
        } else {
            maxProfit = Math.max(maxProfit, profit);
        }
        map.put(cur.end, maxProfit);
    }

    return maxProfit;
}

class Node {
    int start;
    int end;
    int profit;

    Node(int start, int end, int profit) {
        this.start = start;
        this.end = end;
        this.profit = profit;
    }
}
```

DAY 27:TRIE:

Problem:174: 208. Implement Trie (Prefix Tree)

A [trie](#) (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string word into the trie.
- `boolean search(String word)` Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- `boolean startsWith(String prefix)` Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:**Input**

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null, true]
```

Explanation

```
Trie trie = new Trie();  
trie.insert("apple");  
trie.search("apple"); // return True  
trie.search("app");   // return False  
trie.startsWith("app"); // return True  
trie.insert("app");  
trie.search("app");    // return True
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls **in total** will be made to insert, search, and startsWith.

Solution:

```
class Trie {
    TrieRetriever trie = null;

    public Trie() {
        trie = new TrieRetriever();
    }

    public void insert(String word) {
        char arr[] = word.toCharArray();
        trie.add(word);
    }

    public boolean search(String word) {
        return trie.search(word, false);
    }

    public boolean startsWith(String prefix) {
        return trie.search(prefix, true);
    }
}

class TrieRetriever {
    TrieNode node;

    TrieRetriever() {
        this.node = new TrieNode();
    }

    public void add(String word) {
        TrieNode cur = this.node;
        for (char ch : word.toCharArray()) {
            int index = ch - 'a';
            if (cur.words[index] == null) {
                cur.words[index] = new TrieNode();
            }
            cur = cur.words[index];
        }
        cur.isWord = true;
    }
}
```

```
public boolean search(String word, boolean regex) {
    TrieNode cur = this.node;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (cur.words[index] == null) {
            return false;
        }
        cur = cur.words[index];
    }
    if (regex) {
        return true;
    }
    return cur.isWord;
}

}

class TrieNode {
    TrieNode words[] = new TrieNode[26];

    boolean isWord = false;

    TrieNode() {
    }
}

/**
 * Your Trie object will be instantiated and called as such: Trie obj = new
 * Trie(); obj.insert(word); boolean param_2 = obj.search(word); boolean
param_3
 * = obj.startsWith(prefix);
 */
```

Problem:175: Implement Trie II

Sample Input 1:

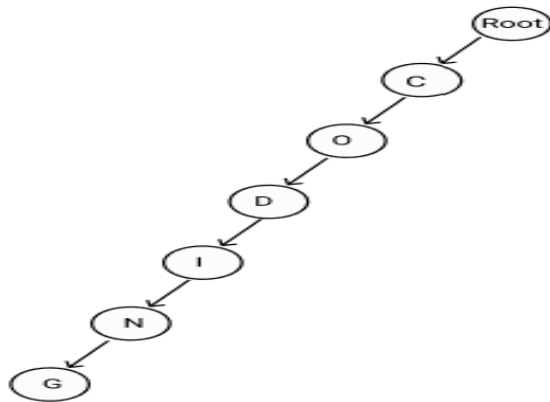
```
1
5
insert coding
insert ninja
countWordsEqualTo coding
countWordsStartingWith nin
erase coding
```

Sample Output 1:

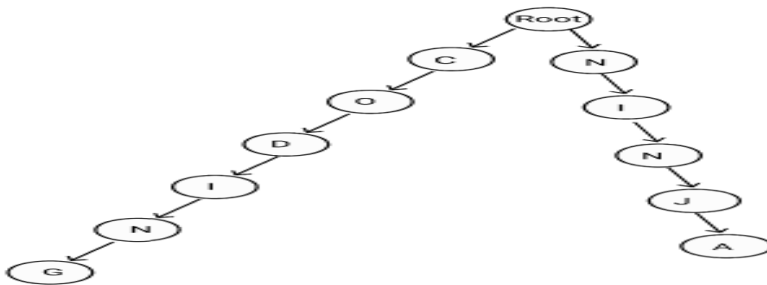
```
1
1
```

Explanation of sample input 1:

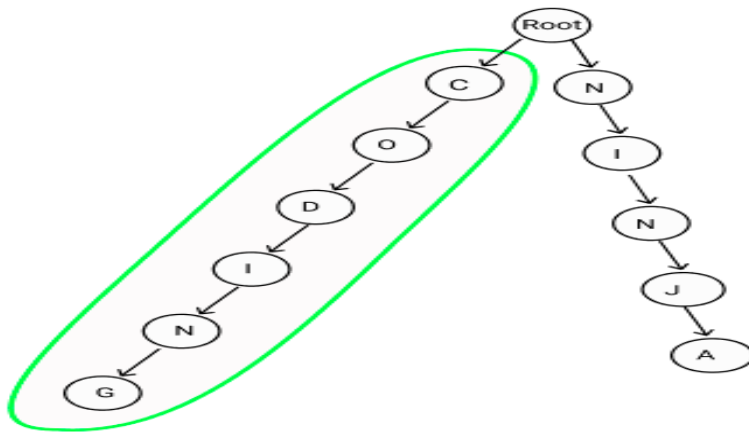
After insertion of “coding” in “TRIE”:



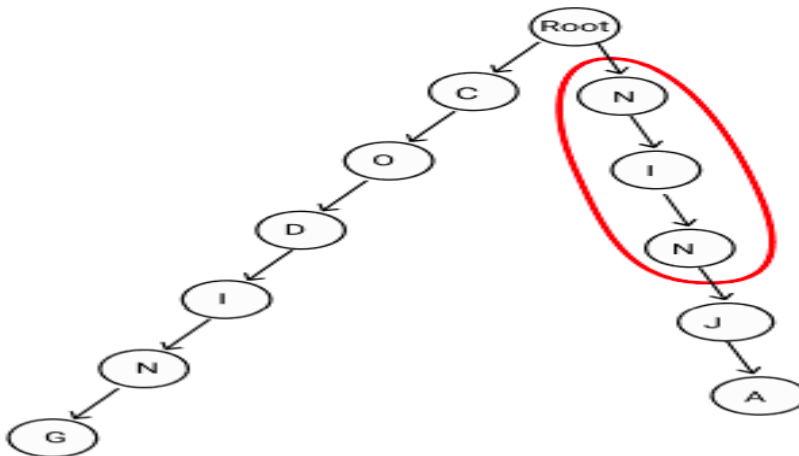
After insertion of “ninja” in “TRIE”:



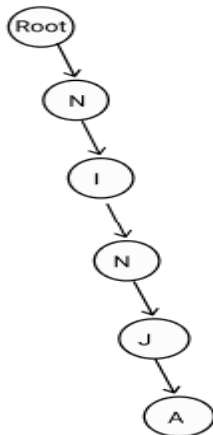
Count words equal to “coding” :



Count words those prefix is "nin":



After deletion of the word "coding", "TRIE" is:



Solution:

```
/*
```

```
Time complexity: O(N * L)
insert - O(N)
countWordsEqualTo - O(N)
countWordsStartingWith - O(N)
erase - O(N)
Space complexity: O(N * L)
```

Where 'N' and 'L' represents the numbers of words
and the longest word in words.

```
*/
```

```
class TrieNode {
    public TrieNode children[];
    public int wordCount;
    public int prefixCount;

    TrieNode() {
        children = new TrieNode[26];
        wordCount = 0;
        prefixCount = 0;
    }
};
```

```
public class Trie {
    // Declare a variable root that denotes the root of the Trie.
    public TrieNode root;
    // Initialize the Trie.
    public Trie() {
        root = new TrieNode();
    }

    // In this function we are inserting the word into our Trie.
    public void insert(String word) {
        TrieNode curr = root;

        // Iterating through the string word.
        for (int i = 0; i < word.length(); i++) {
            int index = word.charAt(i) - 'a';

            if (curr.children[index] == null) {
                curr.children[index] = new TrieNode();
            }
            curr = curr.children[index];
            curr.prefixCount++;
        }
        curr.wordCount++;
    }

    public int countWordsEqualTo(String word) {
        TrieNode curr = root;

        // Iterating through the string word.
        for (int i = 0; i < word.length(); i++) {
            int index = word.charAt(i) - 'a';

            if (curr.children[index] == null) {
                return 0;
            }
            curr = curr.children[index];
        }
        return curr.wordCount;
    }

    public int countWordsStartingWith(String word) {
        TrieNode curr = root;

        // Iterating through the string word.
        for (int i = 0; i < word.length(); i++) {
            int index = word.charAt(i) - 'a';

            if (curr.children[index] == null) {
```

```

        return 0;
    }
    curr = curr.children[index];
}
return curr.prefixCount;
}

// In this function we are removing the word from "TRIE".
public void erase(String word) {
    TrieNode curr = root;
    TrieNode toBeDeleted = null;

    // Iterating through the string word.
    for (int i = 0; i < word.length(); i++) {
        int index = word.charAt(i) - 'a';

        // Store the parent of current character.
        TrieNode parent = curr;
        curr = curr.children[index];
        curr.prefixCount--;

        if (toBeDeleted != null) {
            toBeDeleted = null;
        }

        // If the prefixCount of current node is 0 that means we
        have reached at the end
        // of the word that has to be deleted.
        if (curr.prefixCount == 0) {
            if (toBeDeleted == null) {
                parent.children[index] = null;
            }
            toBeDeleted = curr;
        }
    }

    curr.wordCount--;
    if (toBeDeleted != null) {
        toBeDeleted = null;
    }
}
}

```

Problem:176: Complete String**Sample Input 1 :**

```
2
6
n ni nin ninj ninja ninga
2
ab bc
```

Sample Output 1 :

```
ninja
None
```

Explanation Of Sample Input 1 :

For test case 1 we have,

All the prefixes of "ninja" -> "n", "ni", "nin", "ninja" and "ninja" are present in array 'A'. So, "ninja" is a valid answer whereas for "ninga", the prefix "ning" is not present in array 'A'.

So we output "ninja".

For test case 2 we have,

The prefixes of "ab" are "a" and "ab". "a" is not present in array 'A'. So, "ab" is not a valid answer.

The prefixes of "bc" are "b" and "bc". "b" is not present in array 'A'. So, "ab" is not a valid answer.

Since none of the strings is a valid answer we output "None".

Solution:

```
class Solution {
    public static String completeString(int n, String[] a) {
        Trie trie = new Trie();
        for (String word : a) {
```

```

        trie.add(word);
    }
    String res = "";
    TrieNode curNode = trie.root;

    Node prev = null;
    Node cur = null;
    for (String word : a) {
        Node tempNode = trie.search(word, curNode);
        if (tempNode != null) {
            if (word.length() >= res.length()) {
                if (prev != null && (prev.high - prev.low <
cur.high - cur.low)) {
                    res = word;
                } else {
                    res = word;
                }
            }
            cur = tempNode;
            prev = cur;
        }
    }
    if (res.length() == 0)
        return "None";
    return res;
}
}
class Trie {
    TrieNode root;

    Trie() {
        root = new TrieNode();
    }

    public void add(String word) {
        TrieNode cur = this.root;
        char wordArr[] = word.toCharArray();
        for (char c : wordArr) {
            int index = c - 'a';
            if (cur.node[index] == null) {
                cur.node[index] = new TrieNode();
            }
            cur = cur.node[index];
        }
        cur.word = word;
        cur.isEOW = true;
    }
}

```

```
public Node search(String word, TrieNode cur) {
    // TrieNode cur = this.root;
    char wordArr[] = word.toCharArray();
    int low = 0;
    int high = 0;
    for (char c : wordArr) {
        int index = c - 'a';
        low = Math.min(low, index);
        high = Math.max(high, index);
        cur = cur.node[index];
        if (!cur.isEOW)
            return null;
    }
    if (!cur.isEOW)
        return null;

    return new Node(low, high);
}

// ninja
// 13-8-13-9-0
//
// noder

class Node {
    int low;
    int high;

    Node(int low, int high) {
        this.low = low;
        this.high = high;
    }
}

class TrieNode {
    TrieNode node[] = new TrieNode[26];
    String word;
    boolean isEOW;

    TrieNode() {
    }
}
```

Problem:177: Count Distinct Substrings**Sample Input 1 :**

```
2
sds
abc
```

Sample Output 1 :

```
6
7
```

Explanation of Sample Input 1 :

In the first test case, the 6 distinct substrings are { 's', 'd', "sd", "ds", "sds", "" }.

In the second test case, the 7 distinct substrings are { 'a', 'b', 'c', "ab", "bc", "abc", "" }.

Sample Input 2 :

```
2
aa
abab
```

Sample Output 2 :

```
3
8
```

Explanation of Sample Input 2 :

In the first test case, the two distinct substrings are { 'a', "aa", "" }.

In the second test case, the seven distinct substrings are { 'a', 'b', "ab", "ba", "aba", "bab", "abab", "" }.

Solution:

```
import java.util.*;

public class Solution {
    public static int countDistinctSubstrings(String s) {
        // For storing all distinct substrings of the given string.
        HashSet<String> distinctStrings = new HashSet<String>();

        int n = s.length();
```



```
        // Iterate through the string.
        for (int i = 0; i < n; i++) {
            // Iterate till the end.
            for (int j = i; j < n; j++) {
                // Current substring
                String temp = s.substring(i, j + 1);

                // Insert the current substring into the HashSet.
                distinctStrings.add(temp);
            }
        }

        // Return the total number of distinct substrings.
        return (distinctStrings.size() + 1);
    }
}
```

Problem:178: Power Set(VERY IMPORTANT)

Input : str = "abc"

Output: a ab abc ac b bc c

Explanation : There are 7 subsequences that can be formed from abc.

Example 2:

Input: str = "aa"

Output: a a aa

Explanation : There are 3 subsequences that can be formed from aa.

Your Task:

You don't need to read input or print anything. Your task is to complete the function **AllPossibleStrings()** which takes S as the input parameter and returns a list of all possible subsequences (non-empty) that can be formed from S in lexicographically-sorted order.

Expected Time Complexity: $O(n \cdot 2^n)$ where n is the length of the String

Expected Space Complexity: $O(n \cdot 2^n)$

Solution:

```
class Solution {
    public List<String> AllPossibleStrings(String s) {
        // Code here
        int n = s.length();
        List<String> ans = new ArrayList<>();
        for (int num = 0; num < (1 << n); num++) {
            String sub = "";
            for (int i = 0; i < n; i++) {
                // check if the ith bit is set or not
                if ((num & (1 << i)) != 0) {
                    sub += s.charAt(i);
                }
            }
            if (sub.length() > 0) {
                ans.add(sub);
            }
        }
        Collections.sort(ans);
        return ans;
    }
}
```

Problem:179: 421. Maximum XOR of Two Numbers in an Array

Example 1:**Input:** nums = [3,10,5,25,2,8]**Output:** 28**Explanation:** The maximum result is 5 XOR 25 = 28.**Example 2:****Input:** nums = [14,70,53,83,49,91,36,80,92,51,66,70]**Output:** 127**Constraints:**

- $1 \leq \text{nums.length} \leq 2 * 10^5$
- $0 \leq \text{nums}[i] \leq 2^{31} - 1$

Solution:

```
public class Solution {
    public int findMaximumXOR(int[] nums) {
        int max = 0, mask = 0;
        for (int i = 31; i >= 0; i--) {
            mask = mask | (1 << i);
            Set<Integer> set = new HashSet<>();
            for (int num : nums) {
                set.add(num & mask);
            }
            int tmp = max | (1 << i);
            for (int prefix : set) {
                if (set.contains(tmp ^ prefix)) {
                    max = tmp;
                    break;
                }
            }
        }
        return max;
    }
}
```

Problem:180: 1707. Maximum XOR With an Element From Array

You are given an array nums consisting of non-negative integers. You are also given a queries array, where queries[i] = [x_i, m_i].

The answer to the ith query is the maximum bitwise XOR value of x_i and any element of nums that does not exceed m_i. In other words, the answer is

$\max(\text{nums}[j] \text{ XOR } x_i)$ for all j such that $\text{nums}[j] \leq m_i$. If all elements in nums are larger than m_i , then the answer is -1 .

Return an integer array *answer* where *answer.length* == *queries.length* and *answer[i]* is the answer to the i^{th} query.

Example 1:

Input: $\text{nums} = [0,1,2,3,4]$, $\text{queries} = [[3,1],[1,3],[5,6]]$

Output: $[3,3,7]$

Explanation:

1) 0 and 1 are the only two integers not greater than 1. $0 \text{ XOR } 3 = 3$ and $1 \text{ XOR } 3 = 2$. The larger of the two is 3.

2) $1 \text{ XOR } 2 = 3$.

3) $5 \text{ XOR } 2 = 7$.

Example 2:

Input: $\text{nums} = [5,2,4,6,6,3]$, $\text{queries} = [[12,4],[8,1],[6,3]]$

Output: $[15,-1,5]$

Constraints:

- $1 \leq \text{nums.length}, \text{queries.length} \leq 10^5$
- $\text{queries}[i].\text{length} == 2$
- $0 \leq \text{nums}[j], x_i, m_i \leq 10^9$

Solution:

```
import java.util.Arrays;
```

```
class Solution {
    public int[] maximizeXor(int[] nums, int[][] queries) {
        int queriesLength = queries.length;
        int[] ans = new int[queriesLength];
        int[][] temp = new int[queriesLength][3];
        for (int i = 0; i < queriesLength; i++) {
            temp[i][0] = queries[i][0];
            temp[i][1] = queries[i][1];
            temp[i][2] = i;
        }

        Arrays.sort(temp, (a, b) -> {
```

```
        return a[1] - b[1];
    });
    int index = 0;
    Arrays.sort(nums);
    Trie trie = new Trie();

    for (int query[] : temp) {
        while (index < nums.length && nums[index] <= query[1]) {
            trie.insert(nums[index]);
            index++;
        }
        int tempAns = -1;
        if (index != 0) {
            tempAns = trie.search(query[0]);
        }
        ans[query[2]] = tempAns;
    }

    return ans;
}
}
```

```
class Trie {
    TrieNode root;

    Trie() {
        root = new TrieNode();
    }

    public void insert(int n) {
        TrieNode node = this.root;
        for (int i = 31; i >= 0; i--) {
            int bit = (n >> i) & 1;
            if (node.nums[bit] == null) {
                node.nums[bit] = new TrieNode();
            }
            node = node.nums[bit];
        }
        node.prefixValue = n;
    }
}
```

```
public int search(int n) {
    TrieNode node = this.root;
    for (int i = 31; i >= 0; i--) {
        int bit = (n >> i) & 1;
        int requiredBit = bit == 1 ? 0 : 1;
        if (node.nums[requiredBit] != null) {
            node = node.nums[requiredBit];
        } else {
            node = node.nums[bit];
        }
    }
    return node.prefixValue ^ n;
}

}

class TrieNode {
    TrieNode nums[] = new TrieNode[2];
    int prefixValue;

    TrieNode() {
    }
}

}
```

DESIGN

Problem:181. 622. Design Circular Queue

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- MyCircularQueue(k) Initializes the object with the size of the queue to be k.
- int Front() Gets the front item from the queue. If the queue is empty, return -1.
- int Rear() Gets the last item from the queue. If the queue is empty, return -1.
- boolean enqueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
- boolean dequeue() Deletes an element from the circular queue. Return true if the operation is successful.
- boolean isEmpty() Checks whether the circular queue is empty or not.
- boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:**Input**

```
["MyCircularQueue", "enqueue", "enqueue", "enqueue", "enqueue", "Rear",  
"isFull", "dequeue", "enqueue", "Rear"]  
[[3], [1], [2], [3], [4], [], [], [], [4], []]
```

Output

```
[null, true, true, true, false, 3, true, true, true, 4]
```

Explanation

```
MyCircularQueue myCircularQueue = new MyCircularQueue(3);  
myCircularQueue.enqueue(1); // return True  
myCircularQueue.enqueue(2); // return True
```



```
myCircularQueue.enqueue(3); // return True
myCircularQueue.enqueue(4); // return False
myCircularQueue.Rear();     // return 3
myCircularQueue.isFull();   // return True
myCircularQueue.dequeue();  // return True
myCircularQueue.enqueue(4); // return True
myCircularQueue.Rear();     // return 4
```

Solution:

```
class MyCircularQueue {
    int head = 0;
    int tail = 0;
    int n = 0;
    int arr[];
    int count = 0;

    public MyCircularQueue(int k) {
        this.n = k;
        this.head = n - 1;
        this.tail = 0;
        arr = new int[n];
    }

    public boolean enqueue(int value) {
        if (isFull())
            return false;
        arr[tail] = value;
        tail = (tail + 1) % n;
        count++;
        return true;
    }

    public boolean dequeue() {
        if (isEmpty())
            return false;

        head = (head + 1) % n;
        --count;
        return true;
    }

    public int Front() {
        if (isEmpty())
            return -1;
        return arr[(head + 1) % n];
    }
}
```

```
    }

    public int Rear() {
        if (isEmpty())
            return -1;
        if (tail - 1 < 0) {
            return arr[n - 1];
        }
        return arr[(tail - 1) % n];
    }

    public boolean isEmpty() {
        if (count == 0)
            return true;
        return false;
    }

    public boolean isFull() {
        return count == n;
    }
}

/**
 * Your MyCircularQueue object will be instantiated and called as such:
 * MyCircularQueue obj = new MyCircularQueue(k); boolean param_1 =
 * obj.enqueue(value); boolean param_2 = obj.dequeue(); int param_3 =
 * obj.front(); int param_4 = obj.rear(); boolean param_5 = obj.isEmpty();
 * boolean param_6 = obj.isFull();
 */
```

Problem:182. 355. Design Twitter

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

- `Twitter()` Initializes your twitter object.

- `void postTweet(int userId, int tweetId)` Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`.
- `List<Integer> getNewsFeed(int userId)` Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themselves. Tweets must be **ordered from most recent to least recent**.
- `void follow(int followerId, int followeeId)` The user with ID `followerId` started following the user with ID `followeeId`.
- `void unfollow(int followerId, int followeeId)` The user with ID `followerId` started unfollowing the user with ID `followeeId`.

Example 1:**Input**

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed",  
"unfollow", "getNewsFeed"]
```

```
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

Output

```
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();  
twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).  
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1  
tweet id -> [5]. return [5]  
twitter.follow(1, 2);    // User 1 follows user 2.  
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).  
twitter.getNewsFeed(1); // User 1's news feed should return a list with 2  
tweet ids -> [6, 5]. Tweet id 6 should precede tweet id 5 because it is  
posted after tweet id 5.  
twitter.unfollow(1, 2); // User 1 unfollows user 2.  
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1  
tweet id -> [5], since user 1 is no longer following user 2.
```

Constraints:

- $1 \leq \text{userId}, \text{followerId}, \text{followeeId} \leq 500$
- $0 \leq \text{tweetId} \leq 10^4$
- All the tweets have **unique** IDs.
- At most $3 * 10^4$ calls will be made to `postTweet`, `getNewsFeed`, `follow`, and `unfollow`.

Solution:

```
class Twitter {
// Runtime: 22 ms, faster than 89.04% of Java online submissions for Design
// Memory Usage: 44.6 MB, less than 72.10% of Java online submissions for
Design Twitter.
    public static int timestamp = 0;
    /** Initialize your data structure here. */
    HashMap<Integer, User> map = null;
    public Twitter() {
        map = new HashMap<>();
    }
    /** Compose a new tweet. */
    public void postTweet(int userId, int tweetId) {
        if (map.get(userId) == null) {
            User user = new User(userId);
            map.put(userId, user);
        }
        map.get(userId).post(tweetId);
    }
    /**
     * Retrieve the 10 most recent tweet ids in the user's news feed. Each
item in
     * the news feed must be posted by users who the user followed or by
the user
     * herself. Tweets must be ordered from most recent to least recent.
     */
    public List<Integer> getNewsFeed(int userId) {
        LinkedList<Integer> result = new LinkedList<>();
        if (map.get(userId) == null)
            return result;
        HashSet<Integer> st = map.get(userId).followed;
        PriorityQueue<Tweet> pq = new PriorityQueue<>((a, b) -> b.time -
a.time);
        for (int user : st) {
            Tweet head = map.get(user).head;
            if (head != null)
                pq.add(head);
        }
    }
}
```

```

    }
    int n = 0;
    while (!pq.isEmpty() && n < 10) {
        Tweet tweet = pq.poll();
        result.add(tweet.tweetid);
        n++;
        if (tweet.next != null) {
            pq.add(tweet.next);
        }
    }

    return result;
}

/**
 * Follower follows a followee. If the operation is invalid, it should
be a
 * no-op.
 */
public void follow(int followerId, int followeeId) {
    if (map.get(followerId) == null) {
        User user = new User(followerId);
        map.put(followerId, user);
    }
    if (map.get(followeeId) == null) {
        User user = new User(followeeId);
        map.put(followeeId, user);
    }
    map.get(followerId).follow(followeeId);
}

/**
 * Follower unfollows a followee. If the operation is invalid, it
should be a
 * no-op.
 */
public void unfollow(int followerId, int followeeId) {
    if (map.get(followerId) == null || followerId == followeeId) {
        return;
    }
    map.get(followerId).unfollow(followeeId);
}

}

class Tweet {
    int tweetid;
    Tweet next;
    int time;
}

```

```
Tweet(int tweetid) {
    this.tweetid = tweetid;
    this.time = Twitter.timestamp++;
    this.next = null;
}

}

class User {

    Tweet head;
    int userId;
    HashSet<Integer> followed = null;

    User(int userId) {
        followed = new HashSet<>();
        this.userId = userId;
        follow(userId);
        this.head = null;
    }

    public void post(int tweetid) {
        Tweet newTweet = new Tweet(tweetid);
        newTweet.next = head;
        this.head = newTweet;
    }

    public void follow(int userId) {
        this.followed.add(userId);
    }

    public void unfollow(int userId) {
        this.followed.remove(userId);
    }
}

/**
 * Your Twitter object will be instantiated and called as such: Twitter obj =
 * new Twitter(); obj.postTweet(userId,tweetId); List<Integer> param_2 =
 * obj.getNewsFeed(userId); obj.follow(followerId,followeeId);
 * obj.unfollow(followerId,followeeId);
 */
```

Problem:183. 284. Peeking Iterator

Design an iterator that supports the peek operation on an existing iterator in addition to the hasNext and the next operations.

Implement the PeekingIterator class:

- `PeekingIterator(Iterator<int> nums)` Initializes the object with the given integer iterator iterator.
- `int next()` Returns the next element in the array and moves the pointer to the next element.
- `boolean hasNext()` Returns true if there are still elements in the array.
- `int peek()` Returns the next element in the array **without** moving the pointer.

Note: Each language may have a different implementation of the constructor and Iterator, but they all support the `int next()` and `boolean hasNext()` functions.

Example 1:**Input**

```
["PeekingIterator", "next", "peek", "next", "next", "hasNext"]  
[[[1, 2, 3]], [], [], [], [], []]
```

Output

```
[null, 1, 2, 2, 3, false]
```

Explanation

```
PeekingIterator peekingIterator = new PeekingIterator([1, 2, 3]); // [1,2,3]  
peekingIterator.next();      // return 1, the pointer moves to the next element  
[1,2,3].  
peekingIterator.peek();      // return 2, the pointer does not move [1,2,3].  
peekingIterator.next();      // return 2, the pointer moves to the next element  
[1,2,3]  
peekingIterator.next();      // return 3, the pointer moves to the next element  
[1,2,3]  
peekingIterator.hasNext();    // return False
```

Constraints:

- `1 <= nums.length <= 1000`

- `1 <= nums[i] <= 1000`
- All the calls to `next` and `peek` are valid.
- At most 1000 calls will be made to `next`, `hasNext`, and `peek`.

Follow up: How would you extend your design to be generic and work with all types, not just integer?

Solution:

```
import java.util.NoSuchElementException;
class PeekingIterator implements Iterator<Integer> {
    Integer next;
    Iterator<Integer> iter;
    boolean noSuchElement;

    public PeekingIterator(Iterator<Integer> iterator) {
        // initialize any member here.
        iter = iterator;
        advanceIter();
    }

    // Returns the next element in the iteration without advancing the
    // iterator.
    public Integer peek() {
        // you should confirm with interviewer what to return/throw
        // if there are no more values
        return next;
    }

    // hasNext() and next() should behave the same as in the Iterator
    // interface.
    // Override them if needed.
    @Override
    public Integer next() {
        if (noSuchElement)
            throw new NoSuchElementException();
        Integer res = next;
        advanceIter();
        return res;
    }

    @Override
    public boolean hasNext() {
```

```
        return !noSuchElement;
    }

    private void advanceIter() {
        if (iter.hasNext()) {
            next = iter.next();
        } else {
            noSuchElement = true;
        }
    }
}
```

Problem:184. 1396. Design Underground System

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the UndergroundSystem class:

- void checkIn(int id, string stationName, int t)
 - A customer with a card ID equal to id, checks in at the station stationName at time t.
 - A customer can only be checked into one place at a time.
- void checkOut(int id, string stationName, int t)
 - A customer with a card ID equal to id, checks out from the station stationName at time t.
- double getAverageTime(string startStation, string endStation)
 - Returns the average time it takes to travel from startStation to endStation.
 - The average time is computed from all the previous traveling times from startStation to endStation that happened **directly**, meaning a check in at startStation followed by a check out from endStation.
 - The time it takes to travel from startStation to endStation **may be different** from the time it takes to travel from endStation to startStation.
 - There will be at least one customer that has traveled from startStation to endStation before getAverageTime is called.

You may assume all calls to the checkIn and checkOut methods are consistent. If a customer checks in at time t_1 then checks out at time t_2 , then $t_1 < t_2$. All events happen in chronological order.

Example 1:

Input

```
["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]
[[],[45,"Leyton",3],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],[,"Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"Waterloo",38],["Leyton","Waterloo"]]
```

Output

```
[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]
```

Explanation

```
UndergroundSystem undergroundSystem = new UndergroundSystem();
undergroundSystem.checkIn(45, "Leyton", 3);
undergroundSystem.checkIn(32, "Paradise", 8);
undergroundSystem.checkIn(27, "Leyton", 10);
undergroundSystem.checkOut(45, "Waterloo", 15); // Customer 45 "Leyton" -> "Waterloo" in 15-3 = 12
undergroundSystem.checkOut(27, "Waterloo", 20); // Customer 27 "Leyton" -> "Waterloo" in 20-10 = 10
undergroundSystem.checkOut(32, "Cambridge", 22); // Customer 32 "Paradise" -> "Cambridge" in 22-8 = 14
undergroundSystem.getAverageTime("Paradise", "Cambridge"); // return 14.00000. One trip "Paradise" -> "Cambridge", (14) / 1 = 14
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000. Two trips "Leyton" -> "Waterloo", (10 + 12) / 2 = 11
undergroundSystem.checkIn(10, "Leyton", 24);
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 11.00000
undergroundSystem.checkOut(10, "Waterloo", 38); // Customer 10 "Leyton" -> "Waterloo" in 38-24 = 14
undergroundSystem.getAverageTime("Leyton", "Waterloo"); // return 12.00000. Three trips "Leyton" -> "Waterloo", (10 + 12 + 14) / 3 = 12
```

Example 2:

Input

```
["UndergroundSystem","checkIn","checkOut","getAverageTime","checkIn","checkOut","getAverageTime","checkIn","checkOut","getAverageTime"]
```

```
[[],[10,"Leyton",3],[10,"Paradise",8],["Leyton","Paradise"],[5,"Leyton",10],[5,"Paradise",16],["Leyton","Paradise"],[2,"Leyton",21],[2,"Paradise",30],["Leyton","Paradise"]]
```

Output

```
[null,null,null,5.00000,null,null,5.50000,null,null,6.66667]
```

Explanation

```
UndergroundSystem undergroundSystem = new UndergroundSystem();
undergroundSystem.checkIn(10, "Leyton", 3);
undergroundSystem.checkOut(10, "Paradise", 8); // Customer 10 "Leyton" ->
"Paradise" in 8-3 = 5
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.00000,
(5) / 1 = 5
undergroundSystem.checkIn(5, "Leyton", 10);
undergroundSystem.checkOut(5, "Paradise", 16); // Customer 5 "Leyton" ->
"Paradise" in 16-10 = 6
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 5.50000, (5
+ 6) / 2 = 5.5
undergroundSystem.checkIn(2, "Leyton", 21);
undergroundSystem.checkOut(2, "Paradise", 30); // Customer 2 "Leyton" ->
"Paradise" in 30-21 = 9
undergroundSystem.getAverageTime("Leyton", "Paradise"); // return 6.66667, (5
+ 6 + 9) / 3 = 6.66667
```

Constraints:

- $1 \leq id, t \leq 10^6$
- $1 \leq stationName.length, startStation.length, endStation.length \leq 10$
- All strings consist of uppercase and lowercase English letters and digits.
- There will be at most $2 * 10^4$ calls **in total** to checkIn, checkOut, and getAverageTime.
- Answers within 10^{-5} of the actual value will be accepted.

Solution:

```
class UndergroundSystem {

// INPUT1:
// =====
// 1.["EQH524YN","8AYN1B70"]-getaverg
//           115-13 + (199-145)
//           102+54 = 156/2 = 78
// 2.["Y1A2ROGU","8AYN1B70"]-getaverg
```

```

//          17-295
//          =278

// ["EQH524YN","8AYN1B70"]- getaverg
//Pair - <key,value>
//to retrieve key - getKey();
//to retrieve value - getValue();
HashMap<Integer,Pair<String,Integer>> checkInMap = new HashMap<>();//
id,{stationName,totalTime};
HashMap<String,Pair<Integer,Integer>> checkOutMap = new HashMap<>();//
route,{totalTime,count};
//stationname -id

public UndergroundSystem() {

}

public void checkIn(int id, String stationName, int t) {
    checkInMap.put(id, new Pair<String,Integer>(stationName,t));
}

public void checkOut(int id, String stationName, int t) {
    Pair<String,Integer> checkin = checkInMap.get(id);

    //10 : 3 - Leyton
    //   : 8 - Paradies
    //"Leyton"+"_"+"Paradies
    String route = checkin.getKey()+"_"+stationName;
    int totalTime = (t - checkin.getValue());
    Pair<Integer,Integer> checkout = checkOutMap.getOrDefault(route, new
Pair<Integer,Integer>(0,0));
    //
    checkOutMap.put(route, new Pair<Integer,Integer>(checkout.getKey() +
totalTime , checkout.getValue() +1));
}

public double getAverageTime(String startStation, String endStation) {
    //[10, leyton,leyton]
    //[leyton"_"+"Paradies
    String route = startStation+ "_" +endStation;

    Pair<Integer,Integer> checkout = checkOutMap.get(route);

    return (double)checkout.getKey()/checkout.getValue();
}
}

```

OTHER PROBLEMS

Problem:185.WORD LADDER-I

A **transformation sequence** from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- $s_k == \text{endWord}$

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *the number of words in the shortest transformation sequence* from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

Input: `beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`

Output: 5

Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

Example 2:

Input: `beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]`

Output: 0

Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Solution:

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String>
wordList) {
        //hit->hot->dot->dog->cog
        HashSet<String> visited = new HashSet<>();
        if(!wordList.contains(endWord))return 0;
        visited.add(beginWord);
        wordList.add(beginWord);
        // ["hot","dot","dog","lot","log","cog"]

        //hit -> hot->dot->
        //hit -> [hot]
        //hot -> [hit,dot,lot]
        //dot -> [hot,dog,lot]
        //dog -> [dot,log,cog]
        //lot -> [hot,dot,log]
        //log -> [dog,lot,cog]
        //cog -> [dog,log]
```

```

//hit -> hot ->hit
//          ->dot->dog ->
//          ->lot
Map<String, HashSet<String>> map = new HashMap<>();
int n = wordList.size();
//1 2
for(int i =0;i<n-1;i++){
    for(int j =i+1;j<n;j++){
        String word1 = wordList.get(i);
        String word2 = wordList.get(j);
        map.putIfAbsent(word1, new HashSet<>());
        map.putIfAbsent(word2, new HashSet<>());
    }
}
for(int i =0;i<n-1;i++){
    for(int j =i+1;j<n;j++){
        String word1 = wordList.get(i);
        String word2 = wordList.get(j);
        if(isOneWordApart(word1,word2)){
            map.get(word1).add(word2);
            map.get(word2).add(word1);
        }
    }
}
LinkedList<String> queue = new LinkedList<>();
LinkedList<Integer> depthQueue = new LinkedList<>();
queue.offer(beginWord);
depthQueue.offer(1);
visited.add(beginWord);
while(!queue.isEmpty()){
    String currentWord = queue.poll();
    int currentDepth = depthQueue.poll();
    //traverse adjacent words.
    if(map.get(currentWord)==null)continue;
    for(String adj: map.get(currentWord)){
        if(visited.contains(adj)) continue;
        if(adj.equals(endWord)){
            return currentDepth+1;
        }
        visited.add(adj);
        queue.offer(adj);
        depthQueue.offer(currentDepth + 1);
    }
}
return 0;
}

```

```
private boolean isOneWordApart(String word1, String word2){
    int n1 = word1.length();
    int n2 = word2.length();
    if(n1!=n2)return false;
    int count = 0;
    for(int i =0;i<n1;i++){
        if(word1.charAt(i)!= word2.charAt(i)){
            count ++;
        }
        if(count >1){
            return false;
        }
    }
    return true;
}
```

Problem:186.WORD LADDER-II

A **transformation sequence** from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord $\rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_k$ such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in wordList. Note that beginWord does not need to be in wordList.
- $s_k == \text{endWord}$

Given two words, beginWord and endWord, and a dictionary wordList, return *all the shortest transformation sequences* from beginWord to endWord, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words [beginWord, s_1 , s_2 , ..., s_k].

Example 1:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]

Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]

Explanation: There are 2 shortest transformation sequences:

"hit" $\rightarrow$ "hot" $\rightarrow$ "dot" $\rightarrow$ "dog" $\rightarrow$ "cog"

"hit" -> "hot" -> "lot" -> "log" -> "cog"

Example 2:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]

Output: []

Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Constraints:

- $1 \leq \text{beginWord.length} \leq 5$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 500$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are **unique**.
- The **sum** of all shortest transformation sequences does not exceed 10^5 .

Solution(TLE):

//Runtime: 385 ms, faster than 21.80% of Java online submissions for Word Ladder II.

//Memory Usage: 44.4 MB, less than 63.87% of Java online submissions for Word Ladder II.

```
class Solution {
    public List<List<String>> findLadders(String beginWord, String endWord,
    List<String> wordList) {
        List<List<String>> result = new ArrayList<>();
        Map<String, Node> map = new HashMap<>();
        Map<String, Integer> distanceMap = new HashMap<>();
        wordList.add(beginWord);
        // wordList.add(endWord);
        int n = wordList.size();

        // Step 1:create adjnode
        // hit-> [hot]
```

```

// hot-> [hit,dot,lot]
// dot -> [dog,lot,hot]
// dog-> [log,cog,dot]
// lot-> [log,dot,hot]
// log->[cog,lot,lot]q
// cog->[dog,log]

// step 2: calculate distance
// {lot=2, hit=0, log=3, dot=2, cog=4, hot=1, dog=3}
// hit --> [hot] --> [dot]-->[dog]---> cog
// \ /
// ----> [lot] --> [lot]-->[log] /

for (int i = 0; i < n; i++) {
    String word = wordList.get(i);
    distanceMap.put(word, -1);
    map.put(word, new Node(word));
}
for (int i = 0; i < n; i++) {
    String word1 = wordList.get(i);
    for (int j = i + 1; j < n; j++) {
        String word2 = wordList.get(j);
        if (isOneApart(word1, word2)) {
            map.get(word1).adjSet.add(word2);
            map.get(word2).adjSet.add(word1);
        }
    }
}

populateDepth(beginWord, map, distanceMap, wordList);

LinkedList<Node> queue = new LinkedList<>();
queue.offer(new Node(beginWord));
List<String> list = new ArrayList<>();
list.add(beginWord);
dfs(list, result, map, distanceMap, beginWord, endWord);

return result;
}

```

```

private void dfs(List<String> list, List<List<String>> result,
Map<String, Node> map,
Map<String, Integer> distanceMap,
String begin, String end) {

    if (begin.equals(end)) {
        result.add(new ArrayList<>(list));
        return;
    }
    for (String adj : map.get(begin).adjSet) {

        if (distanceMap.get(adj) > distanceMap.get(begin)) {
            list.add(adj);
            dfs(list, result, map, distanceMap, adj, end);
            list.remove(list.size() - 1);
        }
    }
}

private void populateDepth(String begin, Map<String, Node> map,
Map<String, Integer> distanceMap,
List<String> wordList) {
    LinkedList<Node> queue = new LinkedList<>();
    queue.offer(map.get(begin));

    HashSet<String> visited = new HashSet<>();
    visited.add(begin);
    distanceMap.put(begin, 0);

    while (!queue.isEmpty()) {
        Node current = queue.poll();
        for (String word : map.get(current.word).adjSet) {

            int distance = distanceMap.get(current.word);
            if (distanceMap.get(word) == -1) {
                distanceMap.put(word, distance + 1);
            }
            if (visited.contains(word))
                continue;
            visited.add(word);
            queue.offer(map.get(word));
        }
    }
}

```

```
private boolean isOneApart(String word1, String word2) {
    int n1 = word1.length();
    int n2 = word2.length();
    if (n1 != n2)
        return false;

    int count = 0;
    for (int i = 0; i < n1; i++) {
        if (word1.charAt(i) == word2.charAt(i)) {
            continue;
        }
        count++;
        if (count > 1)
            return false;
    }

    return true;
}
```

```
class Node {
    String word;
    HashSet<String> adjSet = new HashSet<>();

    Node(String word) {
        this.word = word;
        this.adjSet = new HashSet<>();
    }

    public String toString() {
        return this.word + " [" + adjSet + "]";
    }
}
```

Optimized Solution:

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
//Runtime: 13 ms, faster than 93.12% of Java online submissions for Word Ladder II.
//Memory Usage: 44.6 MB, less than 42.60% of Java online submissions for Word Ladder II.
class Solution {
    public List<List<String>> findLadders(String beginWord, String endWord,
        List<String> wordList) {
        List<List<String>> results = new ArrayList<>();
        if (wordList.size() == 0 || !wordList.contains(endWord))
            return results;
        int min = Integer.MAX_VALUE;
        Queue<String> queue = new LinkedList<>();
        queue.add(beginWord);
        Map<String, List<String>> adjMap = new HashMap<String, List<String>>();
        Map<String, Integer> distanceMap = new HashMap<>();
        for (String string : wordList) {
            distanceMap.put(string, Integer.MAX_VALUE);
        }
        distanceMap.put(beginWord, 0);
        while (!queue.isEmpty()) {
            String word = queue.poll();
            int step = distanceMap.get(word) + 1;
            if (step > min)
                break;
            updateAdjacentAndDistanceMap(word, adjMap, distanceMap, endWord,
step, queue);
        }
        LinkedList<String> list = new LinkedList<String>();
        list.add(0, endWord);
        backTrace(endWord, beginWord, list, adjMap, results);
        return results;
    }
}

```

```

/**
 *
 * @param word
 * @param ladder
 * @param endWord
 * @param step
 * @param queue
 * @return
 */
private void updateAdjacentAndDistanceMap(String word, Map<String,
List<String>> adjMap,
Map<String, Integer> distanceMap,
String endWord, int step, Queue<String> queue) {
    for (int i = 0; i < word.length(); i++) {
        char arr[] = word.toCharArray();
        for (char c = 'a'; c <= 'z'; c++) {
            arr[i] = c;
            String new_word = new String(arr);
            if (!distanceMap.containsKey(new_word))
                continue;

            if (step > distanceMap.get(new_word))
                continue;
            //update adjacentMap
            if (adjMap.containsKey(new_word)) {
                adjMap.get(new_word).add(word);
                continue;
            }
            //updateDistanceMap
            if (step < distanceMap.get(new_word)) {
                queue.add(new_word);
                distanceMap.put(new_word, step);
            }

            List<String> list = new LinkedList<String>();
            list.add(word);
            adjMap.put(new_word, list);
        }
    }
}

```

```

/**
 * backtrack end to begin to avoid unnecessary path.
 *
 * @param word
 * @param start
 * @param list
 */
private void backTrace(String word, String start, List<String> list,
    Map<String, List<String>> map,
    List<List<String>> results) {
    if (word.equals(start)) {
        results.add(new ArrayList<String>(list));
        return;
    }

    if (map.get(word) == null)
        return;
    for (String s : map.get(word)) {
        list.add(0, s);
        backTrace(s, start, list, map, results);
        list.remove(0);
        // since add in reverse order
        // [cog,dog,dot,hot,hit]
    }
}

public static void main(String args[]) {
    String begin = "hit";
    String end = "cog";
    List<String> wordList = new ArrayList<>();
    String wordArr[] = { "hot", "dot", "dog", "lot", "log", "cog" };
    for (String cur : wordArr) {
        wordList.add(cur);
    }
    Solution sol = new Solution();
    List<List<String>> result = sol.findLadders(begin, end, wordList);
    System.out.println("Solution.main()" + result);
}
}

```

Problem:187. 44. Wildcard Matching

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '*' where:

- '?' Matches any single character.
- '*' Matches any sequence of characters (including the empty sequence).

The matching should cover the **entire** input string (not partial).

Example 1:**Input:** s = "aa", p = "a"**Output:** false**Explanation:** "a" does not match the entire string "aa".**Example 2:****Input:** s = "aa", p = "*"**Output:** true**Explanation:** '*' matches any sequence.**Example 3:****Input:** s = "cb", p = "?a"**Output:** false**Explanation:** '?' matches 'c', but the second letter is 'a', which does not match 'b'.**Solution:**

```
class Solution {  
  
    // Runtime: 622 ms, faster than 5.02% of Java online submissions for Wildcard  
    // Matching.  
    // Memory Usage: 168 MB, less than 5.00% of Java online submissions for  
    // Wildcard Matching.  
    public boolean isMatch(String s, String p) {  
  
        char sarr[] = s.toCharArray();  
        char parr[] = p.toCharArray();  
  
        // since * means it can matches "" or any char .  
        // so *** does not make any sense so if multiple *  
        // exist remove those.  
        removeDuplicates(parr);  
        // since in this case case of overlapping subproblem  
  
        // e.g  
        // case  
        // "acb", p = "a*cb"  
        // above case we need to have 2 case include * ,not include  
        // if include *  
        // include * : a->a,c->* , then cb left so not working.  
        // exclude * : a->a,c->c,b->b it is matching. since  
        // * can match ""  
  
        HashMap<String, Boolean> cache = new HashMap<>();  
  
        return helper(sarr, parr, 0, 0, cache);  
    }  
}
```



```
private boolean helper(char sarr[], char parr[], int si, int pi,
HashMap<String, Boolean> cache) {
    int slen = sarr.length;
    int plen = parr.length;
    String key = si + "_" + pi;
    if (cache.get(key) != null) {
        return cache.get(key);
    }

    if (pi == plen) {
        return (si == slen);
    }
    if (si > slen)
        return false;

    boolean result = false;

    if (parr[pi] == '*') {
        // consider *
        boolean include = helper(sarr, parr, si + 1, pi, cache);
        // ignore *
        boolean exclude = helper(sarr, parr, si, pi + 1, cache);
        result = (include || exclude);
    }

    if ((si < slen && pi < plen) && (sarr[si] == parr[pi]
        || parr[pi] == '?')) {
        result = helper(sarr, parr, si + 1, pi + 1, cache);
    }

    cache.put(key, result);
    return result;
}
```

```
private void removeDuplicates(char[] parr) {  
  
    int i = 0;  
  
    int n = parr.length;  
    char prev = '#';  
  
    while (i < n) {  
        char cur = parr[i];  
        if (cur == '*' && prev == '*') {  
            i++;  
            continue;  
        }  
        parr[i++] = cur;  
        prev = cur;  
    }  
}
```

Problem:188. 10. Wildcard Matching

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

The matching should cover the **entire** input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"

Output: false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input: *s* = "aa", *p* = "a*"

Output: true

Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: *s* = "ab", *p* = ".*"

Output: true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq p.length \leq 20$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

Solution:

```
class Solution {
// Runtime: 2 ms, faster than 92.50% of Java online submissions for
Regular Expression Matching.
// Memory Usage: 42.6 MB, less than 56.14% of Java online submissions for
Regular Expression Matching.
    public boolean isMatch(String s, String p) {
        //. means 1 char if not matches it can be make it matched
        //* means 1 or more
        //e.g
        //aaa, pattern is .a then false
        //aa, pattern is .a then true

        int slen = s.length()-1;
        int plen = p.length()-1;

        return helper(s.toCharArray(),p.toCharArray(),slen,plen);
    }

    private boolean helper(char s[], char pattern[],
                           int si, int pi){
        int slen = s.length;
        int plen = pattern.length;

        if(si == -1 && pi == -1){
            return true;
        }

        if(pi == -1){
            return false;
        }
        boolean result = false;
```

```

    if(si >=0 && (pattern[pi]==s[si] || pattern[pi]=='.')){
        result= helper(s,pattern,si-1,pi-1);
    }
    if(pattern[pi]=='*'){
        int cur = si;
        //"mississippi"
        //"mis*is*p*."
        char rightToLeftnextPattern = pattern[pi-1]; //char
        boolean rightToLeftnextDot = (rightToLeftnextPattern=='.');

        for(int i=cur;
            (i>=0 && (rightToLeftnextDot
                ||rightToLeftnextPattern==s[i] ) )
            ;i--){
            //jump to two char since regex(*)-we checked in
            if(helper(s,pattern,i-1,pi-2)){
                return true;
            }
        }
        //"aab"
        //"c*a*b"
        result= helper(s,pattern,si,pi-2);
    }

    return result;
}
}

```

Problem:189. 17. Letter Combinations of a Phone Number

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

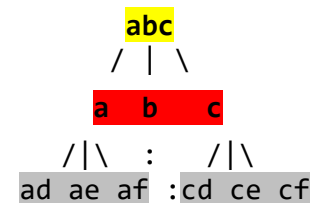
Input: digits = ""

Output: []

Example 3:

Input: digits = "2"

Output: ["a","b","c"]



Solution:

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
class Solution {
    public List<String> letterCombinations(String digits) {

        String board[] = { "*", "1", "abc", "def", "ghi", "jkl",
                           "mno", "pqrs", "tuv", "wxyz" };
        List<String> result = new ArrayList<>();
        if (digits == null || (digits != null && digits.length() == 0))
```

```

        return result;
    }
    LinkedList<String> queue = new LinkedList<>();
    queue.offer("");
    HashSet<String> visited = new HashSet<>();
    int n = digits.length();
    for (int i = 0; i < n; i++) {
        String val =
            board[Character.getNumericValue(digits.charAt(i))];
        while (queue.peek().length() == i) {
            String temp = queue.poll();
            for (char ch : val.toCharArray()) {
                String res = temp + ch;
                queue.offer(res);
            }
        }
    }

    return queue;
}
}

```

Solution-Recursion:

```

import java.util.ArrayList;
import java.util.List;
//Runtime: 5 ms, faster than 50.25% of Java online submissions for Letter
Combinations of a Phone Number.
//Memory Usage: 41.3 MB, less than 64.72% of Java online submissions for
Letter Combinations of a Phone Number.
class Solution {
    public List<String> letterCombinations(String digits) {
        String phoneArr[] = { "0", "#", "abc", "def",
                               "ghi", "jkl", "mno",
                               "pqrs", "tuv", "wxyz" };
        List<String> result = new ArrayList<>();
        if(digits.length() == 0){
            return result;
        }
        helper(digits, "", phoneArr, result, 0, 0);
        return result;
    }

    private void helper(String digits, String prefix, String phoneArr[],
        List<String> result, int index, int j){
        int len = digits.length(); //abc
        if(index == len){
            result.add(prefix);
            return ;
        }
    }
}

```

```

        String cur = phoneArr[digits.charAt(index)-'0'];
        for(int i=0;i<cur.length();i++){
            //prefix+=cur.charAt(i);
            //below won't work.
            //helper(digits,prefix,phoneArr,result,index+1,j+1);
            helper(digits,prefix+cur.charAt(i),phoneArr,result,index+1,j+1);
        }
    }
}

```

Solution-Recursion-II:

```

import java.util.ArrayList;
import java.util.List;
class Solution {
    //Runtime: 1 ms, faster than 78.46% of Java online submissions for Letter
    Combinations of a Phone Number.
    //Memory Usage: 41.1 MB, less than 76.94% of Java online submissions for Letter
    Combinations of a Phone Number.
    public List<String> letterCombinations(String digits) {

        String phoneArr[] = { "0", "#", "abc", "def" ,
                               "ghi", "jkl", "mno" ,
                               "pqrs", "tuv", "wxyz" } ;
        List<String> result = new ArrayList<>();
        if(digits.length() ==0){
            return result;
        }
        StringBuffer buffer =new StringBuffer("");
        helper(digits,buffer,phoneArr,result,0,0);

        return result;
    }

    private void helper(String digits ,StringBuffer buffer,String phoneArr[],
        List<String> result,int index,int j){
        int len = digits.length();//abc
        if(index >= len){
            result.add(buffer.toString());
            return ;
        }

        String cur = phoneArr[digits.charAt(index)-'0'];
        for(int i=0;i<cur.length();i++){
            //need to create new instance
            StringBuffer copy =new StringBuffer(buffer);
            copy.append(cur.charAt(i));
            helper(digits,copy,phoneArr,result,index+1,j+1);
        }
    }
}

```

Problem:190. 632. Smallest Range Covering Elements from K Lists**HARD**

You have k lists of sorted integers in **non-decreasing order**. Find the **smallest** range that includes at least one number from each of the k lists.

We define the range [a, b] is smaller than range [c, d] if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:**Input:** nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]**Output:** [20,24]**Explanation:**

List 1: [4, 10, 15, 24,26], 24 is in range [20,24].

List 2: [0, 9, 12, 20], 20 is in range [20,24].

List 3: [5, 18, 22, 30], 22 is in range [20,24].

Example 2:**Input:** nums = [[1,2,3],[1,2,3],[1,2,3]]**Output:** [1,1]**Constraints:**

- `nums.length == k`
- $1 \leq k \leq 3500$
- $1 \leq \text{nums}[i].\text{length} \leq 50$
- $-10^5 \leq \text{nums}[i][j] \leq 10^5$
- `nums[i]` is sorted in **non-decreasing order**.

Solution:

//632. Smallest Range Covering Elements from K Lists

//Runtime: 1355 ms, faster than 5.02% of Java online submissions for Smallest Range Covering Elements from K Lists.

//Memory Usage: 44.4 MB, less than 75.68% of Java online submissions for Smallest Range Covering Elements from K Lists.

```
class Solution {
    public int[] smallestRange(List<List<Integer>> nums) {

        int n = nums.size();
        if (n == 1) {
            int val = nums.get(0).get(0);
            return new int[] { val, val };
        }
    }
}
```



```

// create n priority queue for n list
PriorityQueue<Integer> pq[] = new PriorityQueue[n];
// store size of each arr index.
int sizearr[] = new int[n];
for (int i = 0; i < n; i++) {
    pq[i] = new PriorityQueue<>();
    sizearr[i] = nums.get(i).size();
}

// arr to store the index of k items in arr.
// i.e if 3 elements in the arr x,y,z hold the current running
index.

int indexarr[] = new int[n];

// enqueue first n items to queue.
for (int i = 0; i < n; i++) {
    pq[i].offer(nums.get(i).get(0));
    indexarr[i]++;
}

// to look for terminating condition i.e if min index found in
queue does not
// have next element.
// Input: [[4,10,15,24,26], [0,9,12,20], [5,18,22,30]]
// [4,0,5]
// [4,9,5]
// ..
// [24,20,22] // here smallest is 20 which index 1 no more
element in queue.

// here 20 is min index of

boolean isExists = true;

// to persist the range for each n queued records find the min
range for each n
// items in queue
int range[] = new int[2];

// min & max range.
int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
int diff = Integer.MAX_VALUE;

while (isExists) {
    isExists = enqueue(pq, nums, sizearr, indexarr, range, n);
    int curmin = range[0];
    int curmax = range[1];
    int curdiff = (curmax - curmin);

```

```

        if (curdiff < diff) {
            min = curmin;
            max = curmax;
            diff = curdiff;
        }
        // System.out.print("cur min :"+min+ " max:"+max +"
diff:"+diff) ;
        // System.out.println();
    }

    return new int[] { min, max };
}

private void print(PriorityQueue<Integer> pq[], int n) {
    for (int i = 0; i < n; i++) {
        System.out.print(pq[i] + " ");
    }
}

// from first n enqueue records ,find min and for that index row remove
// records and
// push next element from list for that index.
private boolean enqueue(PriorityQueue<Integer> pq[],
List<List<Integer>> nums, int sizearr[], int indexarr[],
int range[], int n) {

    int smallIndex = 0;
    int prev = pq[0].peek();
    int count = 0;

    int min = Integer.MAX_VALUE;
    int max = Integer.MIN_VALUE;

    // calculate min,max and set the range (min,max).
    for (int i = 0; i < n; i++) {
        int cur = pq[i].peek();
        if (i > 0 && cur == prev)
            count++;
        if (cur < min) {
            smallIndex = i;
        }
        min = Math.min(min, Math.min(cur, prev));
        max = Math.max(max, Math.max(cur, prev));
        prev = cur;
    }
}

```

```
        range[0] = min;
        range[1] = max;
        int curindex = indexarr[smallIndex];
        boolean isAllsame = (count == n - 1) ? true : false;
        if (isAllsame) {
            // if first index all 1 then no need to process further
            return that small index
            return false;
        } else if (curindex < sizearr[smallIndex]) {
            int val = pq[smallIndex].poll();
            pq[smallIndex].offer(nums.get(smallIndex).get(curindex));
            indexarr[smallIndex]++; // increment the index of arr i.e
            smallest index in queue.
            // print(pq,n);

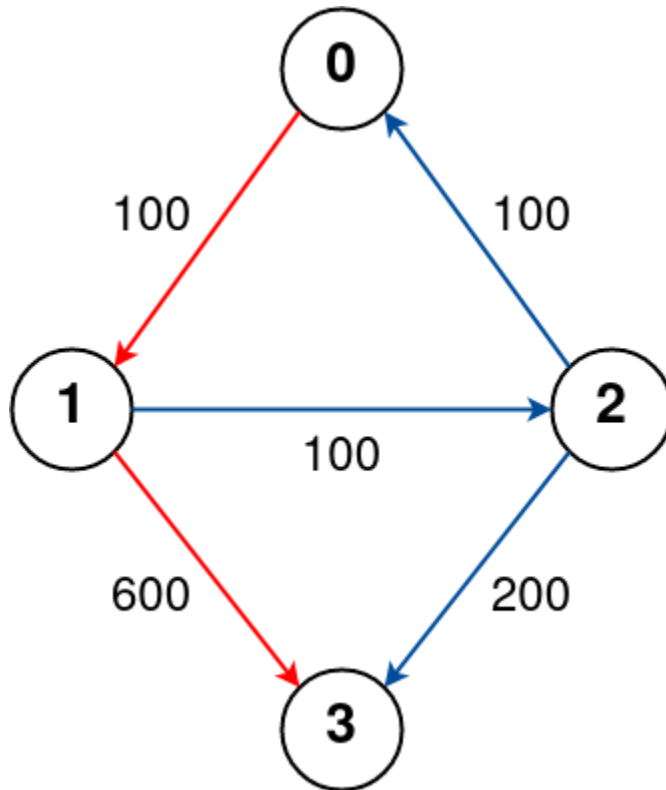
            return true;
        }
        return false;
    }
}
```

Problem:191. 787. Cheapest Flights Within K Stops

There are n cities connected by some number of flights. You are given an array flights where flights[i] = [fromi, toi, pricei] indicates that there is a flight from city fromi to city toi with cost pricei.

You are also given three integers src, dst, and k, return the cheapest price from src to dst with at most k stops. If there is no such route, return -1.

Example 1:



Input: $n = 4$, `flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

Output: 700

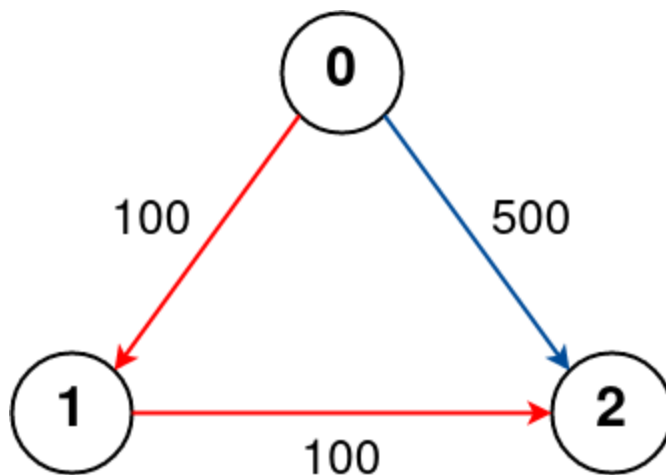
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities `[0,1,2,3]` is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 1$

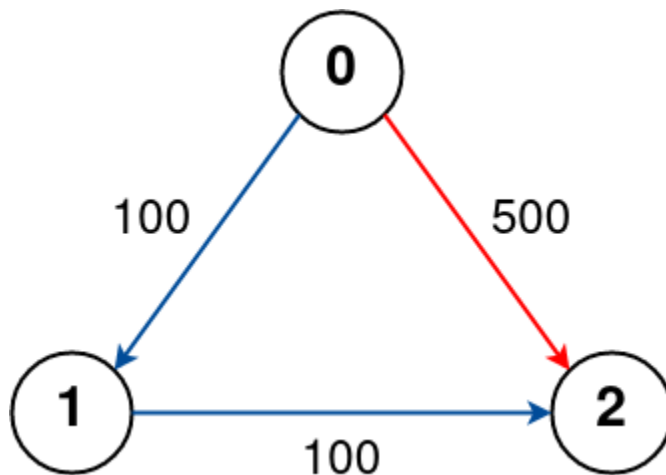
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Solution:

```

import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;
class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int K)
    {
        Map<Integer, Map<Integer, Integer>> map = new HashMap<>();

        for (int flight[] : flights) {
            int u = flight[0];
            int v = flight[1];
            int p = flight[2];

            map.putIfAbsent(u, new HashMap<Integer, Integer>());
            map.get(u).put(v, p);
        }
    }
}
  
```

```

PriorityQueue<Node> pq = new PriorityQueue<>((a, b) -> a.price -
b.price);

pq.offer(new Node(src, K + 1, 0));
boolean[][] visited = new boolean[n][K + 2];
while (!pq.isEmpty()) {
    Node cur = pq.poll();
    int des = cur.des;
    int stops = cur.stops;
    int price = cur.price;
    if (visited[des][stops])
        continue;

    if (cur.des == dst)
        return cur.price;

    visited[des][stops] = true;
    Map<Integer, Integer> adj = map.get(des);

    if (adj == null)
        continue;

    for (int key : adj.keySet()) {
        if (visited[key][stops])
            continue;
        // only k stops .
        if (stops <= 0)
            continue;
        int temp = adj.get(key);
        // decrement stops and update associated to reach current
        //stop.
        pq.offer(new Node(key, stops - 1, price + temp));
    }
}

return -1;
}
}

class Node {
    int des;
    int stops;
    int price;

    Node(int des, int stops, int price) {
        this.des = des;
        this.stops = stops;
        this.price = price;
    }
}

```

Problem:192. 417. Pacific Atlantic Water Flow

There is an $m \times n$ rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The **Pacific Ocean** touches the island's left and top edges, and the **Atlantic Ocean** touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the **height above sea level** of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is **less than or equal to** the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a **2D List of grid coordinates** `result` where `result[i] = [ri, ci]` denotes that rain water can flow from cell (r_i, c_i) to **both** the Pacific and Atlantic oceans.

Example 1:

Pacific Ocean						
Pacific Ocean	1	2	2	3	5	Atlantic Ocean
	3	2	3	4	4	
	2	4	5	3	1	
	6	7	1	4	5	
	5	1	1	2	4	
Atlantic Ocean						

Input: `heights =`

`[[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]`

Output: `[[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]`

Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:

```
[0,4]: [0,4] -> Pacific Ocean
       [0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
       [1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean
       [1,4] -> Atlantic Ocean
[2,2]: [2,2] -> [1,2] -> [0,2] -> Pacific Ocean
       [2,2] -> [2,3] -> [2,4] -> Atlantic Ocean
[3,0]: [3,0] -> Pacific Ocean
       [3,0] -> [4,0] -> Atlantic Ocean
[3,1]: [3,1] -> [3,0] -> Pacific Ocean
       [3,1] -> [4,1] -> Atlantic Ocean
[4,0]: [4,0] -> Pacific Ocean
       [4,0] -> Atlantic Ocean
```

Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.

Example 2:

Input: heights = [[1]]

Output: [[0,0]]

Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights}[r].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights}[r][c] \leq 10^5$

Solution:

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

class Solution {
    public List<List<Integer>> pacificAtlantic(int[][] heights) {

        int r = heights.length;
        int c = heights[0].length;

        int pacificArr[][] = new int[r][c];
        int atlanticArr[][] = new int[r][c];

        List<List<Integer>> result = new ArrayList<>();
        LinkedList<Node> pacificQueue = new LinkedList<>();
        LinkedList<Node> atlanticQueue = new LinkedList<>();
        // first top, left
```



```

    for (int i = 0; i < c; i++) {
        pacificArr[0][i] = 1;
        atlanticArr[r - 1][i] = 1;
        pacificQueue.offer(new Node(0, i));
        atlanticQueue.offer(new Node(r - 1, i));
    }

    for (int i = 0; i < r; i++) {
        pacificArr[i][0] = 1;
        atlanticArr[i][c - 1] = 1;
        pacificQueue.offer(new Node(i, 0));
        atlanticQueue.offer(new Node(i, c - 1));
    }

    // calculate the pacific arr, atlantic arr boundaries
    processQueue(heights, pacificQueue, pacificArr);
    processQueue(heights, atlanticQueue, atlanticArr);

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            // do both merge
            if (pacificArr[i][j] == 1 && atlanticArr[i][j] == 1) {
                List<Integer> list = new ArrayList<>();

                list.add(i);
                list.add(j);
                result.add(list);
            }
        }
    }

    return result;
}

private void processQueue(int matrix[][], LinkedList<Node> queue,
    int oceanFlow[][]) {
    int r = matrix.length;
    int c = matrix[0].length;

    int dirs[][] = { { 0, 1 }, { 0, -1 }, { 1, 0 }, { -1, 0 } };

    while (!queue.isEmpty()) {
        Node cur = queue.poll();

        for (int dir[] : dirs) {
            int x = cur.x + dir[0];
            int y = cur.y + dir[1];
            // if atlantic and pacific flow is not
            //colliding then no merge
            // so dont need to process further stream flow.
            if (x < 0 || y < 0 || x >= r || y >= c)
                continue;

```

```

        if (matrix[x][y] < matrix[cur.x][cur.y]
            || oceanFlow[x][y] == 1)
            continue;

        oceanFlow[x][y] = 1; // make it as visited.
        queue.offer(new Node(x, y));
    }
}

}

}

}

}

class Node {
    int x, y;

    Node(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

Problem:193. 1463. Cherry Pickup II

You are given a rows x cols matrix grid representing a field of cherries where grid[i][j] represents the number of cherries that you can collect from the (i, j) cell.

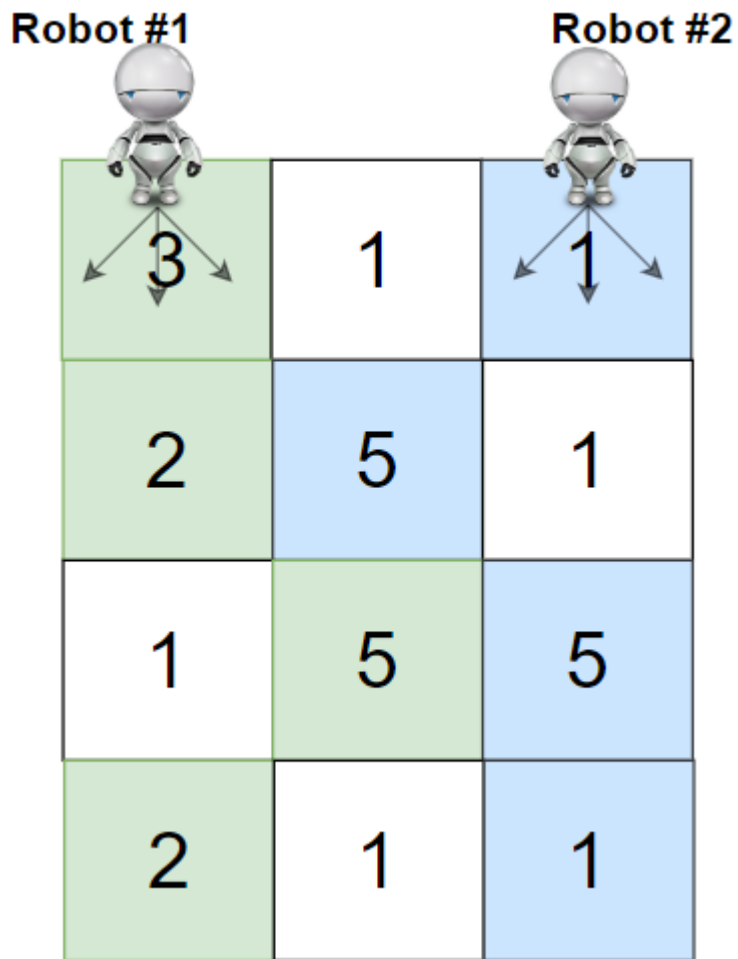
You have two robots that can collect cherries for you:

- **Robot #1** is located at the **top-left corner** (0, 0), and
- **Robot #2** is located at the **top-right corner** (0, cols - 1).

Return the maximum number of cherries collection using both robots by following the rules below:

- From a cell (i, j), robots can move to cell (i + 1, j - 1), (i + 1, j), or (i + 1, j + 1).
- When any robot passes through a cell, It picks up all cherries, and the cell becomes an empty cell.
- When both robots stay in the same cell, only one takes the cherries.
- Both robots cannot move outside of the grid at any moment.
- Both robots should reach the bottom row in grid.

Example 1:



Input: grid = [[3,1,1],[2,5,1],[1,5,5],[2,1,1]]

Output: 24

Explanation: Path of robot #1 and #2 are described in color green and blue respectively.

Cherries taken by Robot #1, $(3 + 2 + 5 + 2) = 12$.

Cherries taken by Robot #2, $(1 + 5 + 5 + 1) = 12$.

Total of cherries: $12 + 12 = 24$.

Example 2:

Robot #1



Robot #2



1	0	0	0	0	0	1
2	0	0	0	0	3	0
2	0	9	0	0	0	0
0	3	0	5	4	0	0
1	0	2	3	0	0	6

Input: grid =

```
[[1,0,0,0,0,0,1],[2,0,0,0,0,3,0],[2,0,9,0,0,0,0],[0,3,0,5,4,0,0],[1,0,2,3,0,0,6]]
```

Output: 28

Explanation: Path of robot #1 and #2 are described in color green and blue respectively.

Cherries taken by Robot #1, $(1 + 9 + 5 + 2) = 17$.

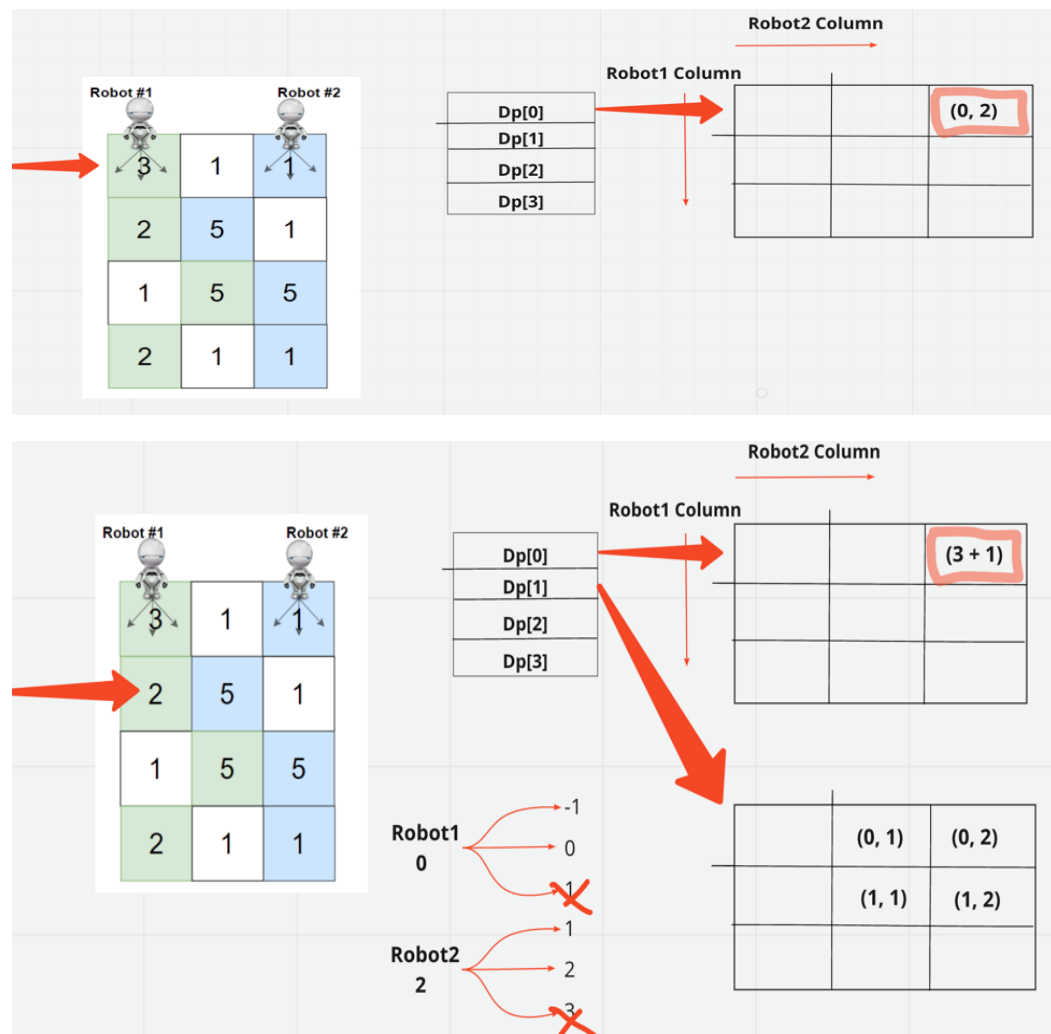
Cherries taken by Robot #2, $(1 + 3 + 4 + 3) = 11$.

Total of cherries: $17 + 11 = 28$.

Constraints:

- rows == grid.length
- cols == grid[i].length
- $2 \leq \text{rows}, \text{cols} \leq 70$

- $0 \leq \text{grid}[i][j] \leq 100$



Approach Explanation :-

1. Move both robots at a time **row by row**. Note we just need to keep track of columns where 2 of the robots were earlier.
2. At each row: If we looked at robot moves $(i + 1, j - 1)$, $(i + 1, j)$, or $(i + 1, j + 1)$, robot moves have 3 choices for column $\{-1, 0, +1\}$
3. Now, if we consider moving 2 robots to maximize cherry count - we total have **maximum 3 (1st robot) * 3 (2nd robot) = 9 choices**

4. If both robots land in the same cube - we have to just take that cherry count once. Case is handled by looking at column value of robot1 and robot2

Solution:

```
import java.util.Arrays;

class Solution {
// Runtime: 21 ms, faster than 97.36% of Java online submissions for Cherry Pickup II.
// Memory Usage: 45.5 MB, less than 17.36% of Java online submissions for Cherry Pickup II.
    public int cherryPickup(int[][] grid) {
        int r = grid.length;
        int c = grid[0].length;
        int dp[][][] = new int[r][c][c];
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                Arrays.fill(dp[i][j], -1);
            }
        }

        int max = 0;
        // f(i,c1,c2) -> store val of ith row , robot1(c1) and robot(c2)
        dp[0][0][c - 1] = grid[0][0] + grid[0][c - 1];

        //next row max will ->
        //step1 : current row max
        //          max( (r1,r2)->{col-1,samecol as current,col+1})
        //step 2.1: max(prev+currowsum ,cursum)
        //          or if both robot1 in same pos choose any one robot current sum.

        for (int i = 1; i < r; i++) {
            for (int c1 = 0; c1 < c; c1++) { // robot1
                for (int c2 = 0; c2 < c; c2++) { // robot2
                    int prev = dp[i - 1][c1][c2];
                    // if starting is zero
                    if (prev < 0)
                        continue;
                    max = getCurrentMax(i, prev, grid, dp, c1, c2, max);
                }
            }
        }

        return max;
    }

    /**
     * @param i - row
     * @param prev - prev val
     */
}
```

```

    * @param grid
    * @param dp
    * @param c1 - robot1 c1
    * @param c2 - robot2 c2
    * @param maxSoFar - current Max
    * @return maxSoFar
    */
    private int getCurrentMax(int i, int prev, int grid[][],
        int dp[][][], int c1, int c2, int maxSoFar) {
        // every row
        // only move column wise (left, mid, right)
        int dirs[] = { -1, 0, 1 };
        int c = grid[0].length;

        for (int dir1 : dirs) {
            int robot1 = c1 + dir1;
            for (int dir2 : dirs) {
                int robot2 = c2 + dir2;
                if (isInRange(robot1, c) && isInRange(robot2, c)) {
                    int robot1Sum = grid[i][robot1];
                    int robot2Sum = grid[i][robot2];
                    // total of robot1 + robot2 in that row
                    int curRowSum = robot1Sum + robot2Sum;

                    dp[i][robot1][robot2] = Math.max(
                        dp[i][robot1][robot2],
                        prev
                        + (robot1 == robot2 ? robot1Sum : curRowSum));
                    maxSoFar = Math.max(maxSoFar, dp[i][robot1][robot2]);
                }
            }
        }

        return maxSoFar;
    }

    /**
     * To check robot new position inside column matrix
     * @param val
     * @param limit
     * @return
     */
    private boolean isInRange(int val, int limit) {
        if (val >= 0 && val < limit)
            return true;

        return false;
    }
}

Time Complexity:  $O(M*N^2)$  - rows * columns
Space Complexity:  $O(M*N^2)$ 

```

Problem:194. 934. Shortest Bridge(DFS & BFS)

You are given an $n \times n$ binary matrix grid where 1 represents land and 0 represents water.

An island is a 4-directionally connected group of 1's not connected to any other 1's. There are exactly two islands in grid.

You may change 0's to 1's to connect the two islands to form one island.

Return the smallest number of 0's you must flip to connect the two islands.

Example 1:

Input: grid = [[0,1],[1,0]]

Output: 1

Example 2:

Input: grid = [[0,1,0],[0,0,0],[0,0,1]]

Output: 2

Example 3:

Input: grid = [[1,1,1,1,1],[1,0,0,0,1],[1,0,1,0,1],[1,0,0,0,1],[1,1,1,1,1]]

Output: 1

Constraints:

- $n == \text{grid.length} == \text{grid}[i].\text{length}$
- $2 \leq n \leq 100$
- $\text{grid}[i][j]$ is either 0 or 1.
- There are exactly two islands in grid.

Solution-DFS & BFS:

Runtime: 8 ms, faster than 91.76% of Java online submissions for Shortest Bridge.

Memory Usage: 44.4 MB, less than 32.70% of Java online submissions for Shortest Bridge.

```
class Solution {
    public int shortestBridge(int[][] A) {

        int r = A.length;
        int c = A[0].length;
        LinkedList<Node> queue = new LinkedList<Node>();
        boolean found = false;
        boolean[][] visited = new boolean[r][c];

        for (int i = 0; i < r; i++) {
            if (found)
                break;
            for (int j = 0; j < c; j++) {
```



```

        if (A[i][j] == 1) {
            // exit when 1 island finished.
            dfs(queue, A, i, j, visited);
            found = true;
            break;
        }
    }
}

// do bfs to find the distance to the next island
int res = bfs(queue, A, visited);

return res;
}

// queue has (0,1)
private int bfs(LinkedList<Node> queue, int[][] A,
               boolean visited[][]) {
    int dirs[][] = { { -1, 0 }, { 1, 0 }, { 0, 1 }, { 0, -1 } };

    int r = A.length;
    int c = A[0].length;

    int level = 0;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            Node temp = queue.poll();
            int x = temp.x;
            int y = temp.y;
            for (int dir[] : dirs) {
                int x1 = x + dir[0];
                int y1 = y + dir[1];

                if (x1 < 0 || y1 < 0 || x1 >= r
                    || y1 >= c || visited[x1][y1]) {
                    continue;
                }

                if (A[x1][y1] == 1)
                    return level;
                visited[x1][y1] = true;
                queue.offer(new Node(x1, y1));
            }
        }

        level++;
    }

    return -1;
}

```

```

private void dfs(LinkedList<Node> queue, int[][] A, int i,
                int j, boolean visited[][]) {

    int r = A.length;
    int c = A[0].length;

    int dirs[][] = { { -1, 0 }, { 1, 0 }, { 0, -1 }, { 0, 1 }, { 0, 0 } };

    for (int dir[] : dirs) {
        int x = i + dir[0];
        int y = j + dir[1];
        if (x < 0 || y < 0 || x >= r || y >= c
            || visited[x][y] || A[x][y] == 0) {
            continue;
        }
        visited[x][y] = true;
        queue.offer(new Node(x, y));
        dfs(queue, A, x, y, visited);
    }
}

class Node {
    int x, y;

    Node(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

Problem:195-I. 403. Frog Jump

403. Frog Jump

Hard

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones' positions (in units) in sorted **ascending order**, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either $k - 1$, k , or $k + 1$ units. The frog can only jump in the forward direction.

Example 1:

Input: stones = [0,1,3,5,6,8,12,17]

Output: true

Explanation: The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.

Example 2:

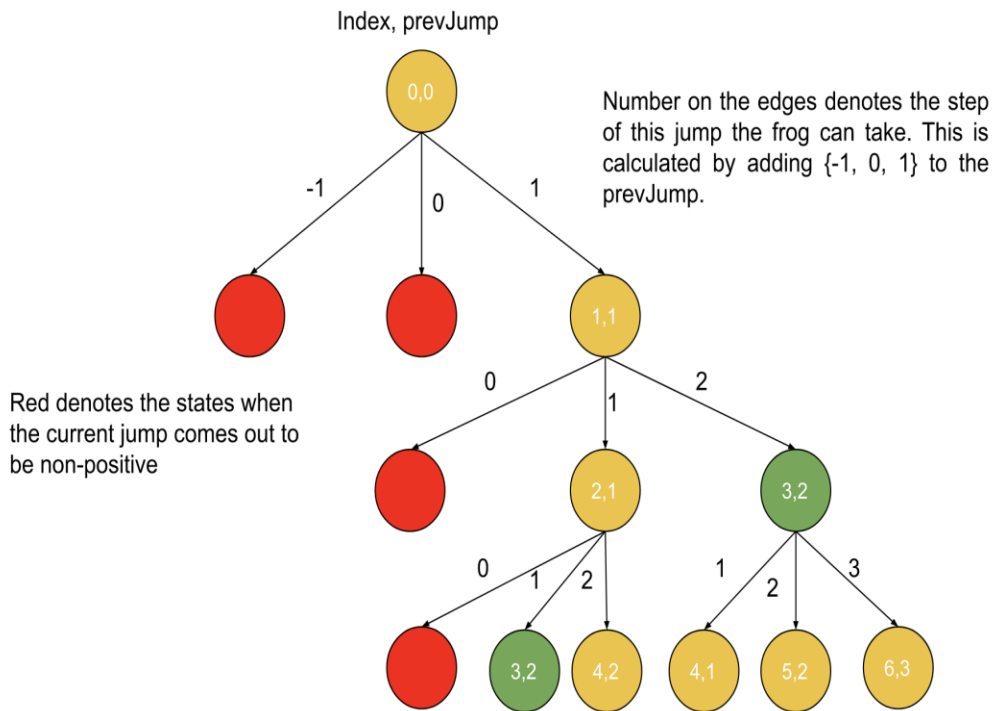
Input: stones = [0,1,2,3,4,8,9,11]

Output: false

Explanation: There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.

Constraints:

- $2 \leq \text{stones.length} \leq 2000$
- $0 \leq \text{stones}[i] \leq 2^{31} - 1$
- $\text{stones}[0] == 0$
- stones is sorted in a strictly increasing order.



Solution

```
import java.util.HashMap;
import java.util.HashSet;

class Solution {
    // Runtime: 9 ms, faster than 99.05% of Java online submissions for Frog Jump.
    // Memory Usage: 45.2 MB, less than 95.26% of Java online submissions for Frog Jump.
    public boolean canCross(int[] stones) {
        int destination = stones[stones.length-1];
        // Pass a hashmap to store calculated results
        HashMap<Integer, HashSet<Integer>> map = new HashMap<>();
        //add all available stones.
        for(int cur : stones){
            map.put(cur, new HashSet<>());
        }
        return helper( stones, map, 0, 1);
    }
}
```

```
private boolean helper(int stones[], HashMap<Integer,HashSet<Integer>> map
,
    int current, int k)
{
    int n = stones.length;
    int last = stones[n-1];
    //base conditions
    //if last stone reached
    if (current == last) return true;
    //if already visited means loop cant reach
    if ( map.get(current).contains(k)) return false;

    //move the next stone
    int next = current + k;
    if (k<=0 || !map.containsKey(next)) {
        return false;
    }

    //memoize visited stone and steps used
    map.get(current).add(k);
    //try all ways i.e first 1 step
    //          try with 2(k+1) step
    //          try with 1(k) step
    //          try with 0(k-1) false always default,
    //          but if move with K+1 2times from 1 (1,2,3)
    //          then k-1 will be 1 step again.
    return helper(stones,map,next, k + 1)
        || helper(stones,map,next, k )
        || helper(stones,map,next, k-1 );
}
}
```

Avoid unnecessary stack calls:

```

import java.util.HashMap;
import java.util.HashSet;

class Solution {
    public boolean canCross(int[] stones) {
        int destination = stones[stones.length - 1];
        // Pass a hashmap to store calculated results
        HashMap<Integer, HashSet<Integer>> map = new HashMap<>();
        // add all available stones.
        for (int cur : stones) {
            map.put(cur, new HashSet<>());
        }
        return helper(stones, map, 0, 1);
    }
    private boolean helper(int[] stones,
        HashMap<Integer, HashSet<Integer>> map, int cur, int step)
    {
        int n = stones.length;
        int lastStone = stones[n - 1];
        // base cases
        if (cur == lastStone)
            return true;
        if (step <= 0 || map.get(cur).contains(step))
            return false;
        int nextPos = cur + step;
        if (!map.containsKey(nextPos))
            return false;

        // main case
        map.get(cur).add(step);
        boolean isReachable = helper(stones, map, nextPos, step - 1);
        if (!isReachable) {
            isReachable = helper(stones, map, nextPos, step);
        }
        if (!isReachable) {
            isReachable = helper(stones, map, nextPos, step + 1);
        }
        return isReachable;
    }
}

```

Problem:195-II. 70. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Solution

```
class Solution {
    public int climbStairs(int n) {
        if(n<=3){
            return n;
        }
        HashMap<Integer,Integer> cache = new HashMap<>();

        return helper(n,cache);
    }
    private int helper(int n ,HashMap<Integer,Integer> cache)
    {
        if(n<=3)return n;
        if(cache.get(n)!=null)return cache.get(n);
        int res = helper(n-1,cache) + helper(n-2,cache);
        cache.put(n,res);
        return res;
    }
}
```

Solution-DP

```

class Solution {
    public int climbStairs(int n) {
        if (n <= 1) {
            return 1;
        }
        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
}

```

Space-Optimized (Linear-DP)

```

class Solution {
    public int climbStairs(int n) {
        if (n <= 1) {
            return 1;
        }

        int prev1 = 1;
        int prev2 = 2;

        for (int i = 3; i <= n; i++) {
            int newValue = prev1 + prev2;
            prev1 = prev2;
            prev2 = newValue;
        }

        return prev2;
    }
}

```

Problem:196. 34. Binary Search-Find First and Last Position of Element in Sorted Array(Upper & LowerBound)

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8

Output: [3,4]

Example 2:

Input: nums = [5,7,7,8,8,10], target = 6

Output: [-1,-1]

Example 3:

Input: nums = [], target = 0

Output: [-1,-1]

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- nums is a non-decreasing array.
- $-10^9 \leq \text{target} \leq 10^9$

Solution

// Runtime: 0 ms, faster than 100.00% of Java online submissions for Find First and Last Position of Element in Sorted Array.

// Memory Usage: 44.4 MB, less than 80.90% of Java online submissions for Find First and Last Position of Element in Sorted Array.

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int lowerbound = binarylowSearch(nums, target);
        if (lowerbound == -1) {
            return new int[] { -1, -1 };
        }
        int higherbound = binaryhighSearch(nums, target);

        return new int[] { lowerbound, higherbound };
    }
    private int binarylowSearch(int nums[], int target) {
        int low = 0;
        int high = nums.length - 1;
        // [5,7,7,8,8,10]
        // 0 1 2 3 4 5
        // 8
        int first = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) {
                first = mid;
                high = mid - 1;
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return first;
    }
}
```

```

private int binaryhighSearch(int nums[], int target)
{
    int low = 0;
    int high = nums.length-1;
    //if used low<high wont work
    // [5,7,7,8,8,10], target = 8
    // 0 1 2 3 4 5 6
    // |           |
    //low = 0, high =6 ,mid = 3(8) higher bound,
    //low = 4, high = 6 , mid = 5 which is 10 exit
    //so ans becomes 3 but it should be 4.

    // now with low<=high
    // [5,7,7,8,8,10], target = 8
    // 0 1 2 3 4 5
    // |           |
    //low = 0 ,high=5 mid = 2(7)
    //low =3, high = 5 mid = 4(8) update 4
    //low =5 high =5 mid =5
    //low=5high =4 exit
    int last = -1;
    //5,7,7,8
    while(low<=high)
    {
        int mid = low + ( high - low)/2;
        if(nums[mid]==target){
            last = mid;
            low = mid+1;
        }
        else if(nums[mid]<target){
            low = mid+1;
        }else{
            high = mid-1;
        }
    }

    return last;
}
}

```

Problem:197. 825. Friends Of Appropriate Ages

825. Friends Of Appropriate Ages

Medium

There are n persons on a social media website. You are given an integer array ages where ages[i] is the age of the ith person.

A Person x will not send a friend request to a person y ($x \neq y$) if any of the following conditions is true:

- $\text{age}[y] \leq 0.5 * \text{age}[x] + 7$
- $\text{age}[y] > \text{age}[x]$
- $\text{age}[y] > 100 \ \&\& \ \text{age}[x] < 100$

Otherwise, x will send a friend request to y .

Note that if x sends a request to y , y will not necessarily send a request to x . Also, a person will not send a friend request to themselves.

Return *the total number of friend requests made*.

Example 1:

Input: `ages = [16,16]`

Output: 2

Explanation: 2 people friend request each other.

Example 2:

Input: `ages = [16,17,18]`

Output: 2

Explanation: Friend requests are made $17 \rightarrow 16$, $18 \rightarrow 17$.

Example 3:

Input: `ages = [20,30,100,110,120]`

Output: 3

Explanation: Friend requests are made $110 \rightarrow 100$, $120 \rightarrow 110$, $120 \rightarrow 100$.

Constraints:

- $n == \text{ages.length}$
- $1 \leq n \leq 2 * 10^4$
- $1 \leq \text{ages}[i] \leq 120$

Solution

```
import java.util.Arrays;

// Runtime: 35 ms, faster than 23.32% of Java online submissions for Friends Of
//Appropriate Ages.
// Memory Usage: 46.5 MB, less than 69.43% of Java online submissions
//for Friends Of Appropriate Ages.
class Solution {
    public int numFriendRequests(int[] ages) {
        Arrays.sort(ages);
        int n = ages.length;
        long ans = 0;
        for (int i = n - 1; i >= 0; i--) {
            // set lower and upper bound to connect
            int x = ages[i] / 2 + 7;
            int low_bound = binarySearch(ages, x);
            int higher_bound = binarySearch(ages, ages[i]);

            int diff = (higher_bound - low_bound - 1);
            ans += Math.max(0, diff);
        }

        return (int) ans;
    }

    private int binarySearch(int ages[], int target) {
        int low = 0;
        int high = ages.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (ages[mid] <= target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return low;
    }
}
```

Solution

```

import java.util.Arrays;
// Runtime: 2 ms, faster than 88.60% of Java online submissions for Friends Of
// Appropriate Ages.
// Memory Usage: 46.3 MB, less than 83.94% of Java online submissions for Friends Of
// Appropriate Ages.
class Solution {
    public int numFriendRequests(int[] ages) {
        //max age limit is 120
        int[] agecount = new int[121];
        for (int age : ages) {
            agecount[age]++;
        }

        int cnt = 0;
        //condition for not to connect is :age[y] <= 0.5 * age[x] + 7
        // reverse is y > x/2+8 ( 1 greater than 7)
        for (int i = 1; i <= 120; i++) {
            for (int j = i / 2 + 8; j <= i; j++) {
                cnt += agecount[i] * (agecount[j] - (i == j ? 1 : 0));
            }
        }
        return cnt;
    }
}

```

Problem:198. 2262. Total Appeal of A String

The **appeal** of a string is the number of **distinct** characters found in the string.

- For example, the appeal of "abbca" is 3 because it has 3 distinct characters: 'a', 'b', and 'c'.

Given a string *s*, return the **total appeal of all of its substrings**.

A **substring** is a contiguous sequence of characters within a string.

Example 1:

Input: *s* = "abbca"

Output: 28

Explanation: The following are the substrings of "abbca":

- Substrings of length 1: "a", "b", "b", "c", "a" have an appeal of 1, 1, 1, 1, and 1 respectively. The sum is 5.

- Substrings of length 2: "ab", "bb", "bc", "ca" have an appeal of 2, 1, 2, and 2 respectively. The sum is 7.
 - Substrings of length 3: "abb", "bbc", "bca" have an appeal of 2, 2, and 3 respectively. The sum is 7.
 - Substrings of length 4: "abbc", "bbca" have an appeal of 3 and 3 respectively. The sum is 6.
 - Substrings of length 5: "abbca" has an appeal of 3. The sum is 3.
- The total sum is $5 + 7 + 7 + 6 + 3 = 28$.

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of lowercase English letters.

Solution(TLE)

```
import java.util.HashSet;

class Solution {
    public long appealSum(String s) {
        int n = s.length();
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j <= n - i; j++) {
                String cur = s.substring(j, j + i);
                sum += getDistinctCount(cur);
            }
        }
        return sum;
    }

    private int getDistinctCount(String s) {
        HashSet<Character> st = new HashSet<>();
        for (char ch : s.toCharArray()) {
            if (!st.contains(ch)) {
                st.add(ch);
            }
        }
        return st.size();
    }
}
```

Time Complexity: TLE since limit is 10^5 power 5.

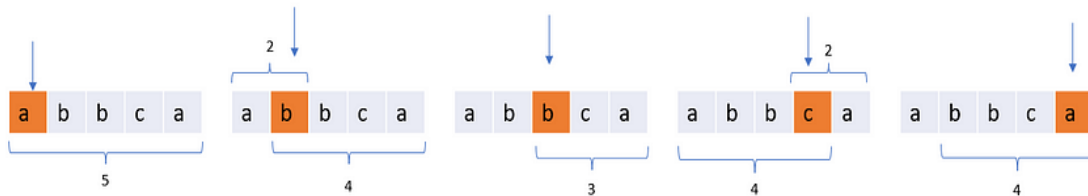
Contribution Technique:

Assume we have string xxxaxxxx..., with $s[i] = a$ and $s[j] = b$.
 $s[i]$ is the last character a before that b.

We want to count, how many substrings ending at $s[j]$ contains character a.
 They are xxxaxxxx, xxaxxxx, xaxxxx, axxxx,
 $i + 1$ substrings ending with character a at $s[i]$,
 so we do $res += i + 1$.

We repeat this for every $s[i]$ and every one of 26 characters.

In a substring, multiple same characters only get one point.
 So we need to find all substrings and count the weight of each character. If
 a character appears more than once, then its weight is only one.
 For those cases where we have duplicate chars, we can consider that the first
 occurrence gets the point. For each character $s[i]$, the substring must start
 before $s[i]$ to contain $s[i]$ and need to end after the last occurrence of
 $s[i]$, otherwise the last occurrence of the character $s[i]$ will get the score.
 In total, there are $i - \text{last}[s[i]]$ possible start positions,
 and $n - i$ possible end positions, so $s[i]$ can contribute $(i - \text{last}[s[i]]) * (n - i)$ points.

**Time Complexity:**

Time $O(n)$

Space $O(26)$

Solution

```
// 0 1 2 3 4
// a b c d b
// Appeal for all substring ending at d --F(3)
// d    -> 1
// cd   -> 2
// bcd  -> 3
// abcd -> 4

// Appeal for all substring ending at b --F(4)
// b    -> 1
// db   -> 1 + 1
// cdb  -> 2 + 1
// bcdb -> 3 + 0
// abcdb -> 4 + 0
// we can find this with the help of dp
// first find the recurrence relation
//  $F(4) = F(3) + 1 + (4 - 1) - 1$ 
// adding 1 for b but what is  $(4 - 1) - 1$  ?
//      i
// 0 1 2 3 4
// a b c d b we can see we are checking here the last occ of b
// we have added 1 , 1, 0 , 0 in the right side above
// total is 2 if b occ before , then we have added 0 so we want those
// substring in which b doesn't occur before becuz then only we will get 1
// so  $(4 - 1) - 1 = 2$  with d or cd if we add b {db, cdb} we will get 1
// so finally  $F(i) = F(i - 1) + i - \text{lastOcc}(b)$ 
```

```
class Solution {
    public long appealSum(String s) {
        int n = s.length();
        int arr[] = new int[26];
        Arrays.fill(arr, -1);
        long sum = 0;
        for (int i = 0; i < n; i++) {
            int lastIndex = s.charAt(i) - 'a';
            // last occurrence of the character.
            sum += (n - i) * (i - arr[lastIndex]);
            // update the last index.
            arr[lastIndex] = i;
        }

        return sum;
    }
}
```


Problem:199. 547.Number of Provinces-Union by Rank(Path Compression)

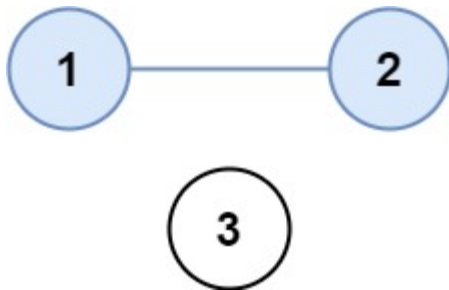
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b , and city b is connected directly with city c , then city a is connected indirectly with city c .

A **province** is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an $n \times n$ matrix `isConnected` where `isConnected[i][j] = 1` if the i^{th} city and the j^{th} city are directly connected, and `isConnected[i][j] = 0` otherwise.

Return *the total number of provinces*.

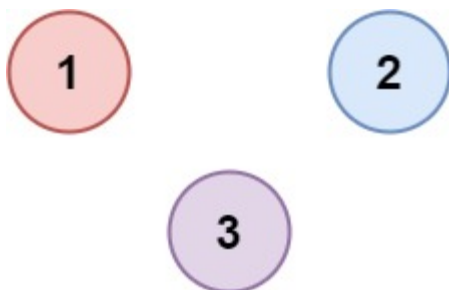
Example 1:



Input: `isConnected = [[1,1,0],[1,1,0],[0,0,1]]`

Output: 2

Example 2:



Input: `isConnected = [[1,0,0],[0,1,0],[0,0,1]]`

Output: 3

Constraints:

- $1 \leq n \leq 200$
- $n == \text{isConnected.length}$

- `n == isConnected[i].length`
- `isConnected[i][j]` is 1 or 0.
- `isConnected[i][i] == 1`
- `isConnected[i][j] == isConnected[j][i]`

Solution

```
import java.util.HashSet;
//Runtime: 4 ms, faster than 18.97% of Java online submissions
//for Number of Provinces.
//Memory Usage: 46.5 MB, less than 9.51% of Java online submissions
//for Number of Provinces.
class Solution {
// Runtime: 3 ms, faster than 27.45% of Java online submissions for Number of
// Provinces.
// Memory Usage: 46.3 MB, less than 13.15% of Java online submissions for
// Number of Provinces.
    public int findCircleNum(int[][] isConnected) {
        int r = isConnected.length;
        int c = isConnected[0].length;
        UnionFind uf = new UnionFind(r);
        for (int i = 0; i < r; i++) {
            for (int j = i + 1; j < c; j++) {
                if (isConnected[i][j] == 1 &&
                    isConnected[i][j] == isConnected[j][i]) {
                    uf.union(i, j);
                }
            }
        }
        /*HashSet<Integer> st = new HashSet<>();
        for (int i = 0; i < r; i++) {
            int parent = uf.find(i);
            st.add(parent);
        }
        return st.size();*/
        return uf.count();
    }
}

class UnionFind {
    int parent[], count;
    UnionFind(int n) {
        this.count = n;
        parent = new int[n];
    }
}
```

```
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }
    public void union(int x, int y) {
        int a = find(x);
        int b = find(y);
        if (a == b) return;
        parent[a] = b;
        //only dec if only doing union for
        //first time.
        this.count--;
    }
    public int count(){
        return this.count;
    }

    public int find(int x) {
        while (x != parent[x]) {
            x = parent[x];
        }

        return x;
    }
}

class Node {
    int x, y;
    boolean connected;

    Node(int x, int y, boolean connected) {
        this.x = x;
        this.y = y;
    }
}
```

Time Complexity: $O(N * N * \log(N))$

Space Complexity: $O(N)$

We are traversing $N * N$ size matrix and finding parent each time in vector of size N

Using a vector to store parent of size N

Weighted quickunion with path compression order of growth is very nearly, but not quite 1 (amortized)

Dynamic connectivity

Solution-UnionFind Path Compression

```

class Solution {
//Runtime: 1 ms, faster than 99.46% of Java online
//submissions for Number of Provinces.
//Memory Usage: 46.2 MB, less than 20.32% of Java online submissions
//for Number of Provinces.
    public int findCircleNum(int[][] isConnected) {
        int r = isConnected.length;
        int c = isConnected[0].length;
        UnionFind uf = new UnionFind(r);
        for (int i = 0; i < r; i++) {
            for (int j = i + 1; j < c; j++) {
                if (isConnected[i][j] == 1 && isConnected[i][j] ==
isConnected[j][i]) {
                    uf.union(i, j);
                }
            }
        }

        return uf.count();
    }
}

class UnionFind {

    int parent[], rank[];
    int count;

    UnionFind(int n) {
        count = n;
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 1; //set initial rank to 1
        }
    }

    public void union(int x, int y) {
        int a = find(x);
        int b = find(y);
        if (a == b)

```

```

        return;
        // implemented as in Algorithms book.
        // make smaller root point to larger one.
        if (rank[a] < rank[b]) {
            parent[a] = b;
            // increase rank of larger one
            rank[b] += rank[a];
        } else {
            parent[b] = a;
            rank[a] += rank[b];
        }
        count--;
    }

    public int count() {
        return count;
    }

    public int find(int x) {
        while (x != parent[x]) {
            // path compression by halving
            parent[x] = parent[parent[x]];
            x = parent[x];
        }

        return x;
    }
}

class Node {
    int x, y;
    boolean connected;

    Node(int x, int y, boolean connected) {
        this.x = x;
        this.y = y;
    }
}

```

Time Complexity:

$O(\log^*(n)) = O(m \alpha(n)) \approx O(N)$, where N is the number of vertices (and also the number of edges) in the graph, and α is the Inverse-Ackermann function. Inverse Ackerman Function is known for its extremely slow-growing property, hence it is considered constant time. m = no of Find or union operations.

Space Complexity: $O(N)$.

Since the matrix is symmetric with respect to the diagonal, we only need to iterate through either the left-bottom or the top-right triangle

We can implement path compression in `UF::find()`, which makes the height of the subset tree won't exceed 2.

We can implement union by rank/size in `UF::union()`, which speeds up path compression operation.

Those optimizations ensure the `union()` operation takes $O(1)$!

Worst case for `find()` and union is $\log n$

Solution-DFS

```
import java.util.HashSet;

class Solution {
    // Runtime: 2 ms, faster than 41.31% of Java online submissions
    // for Number of Provinces.
    // Memory Usage: 46.3 MB, less than 13.15% of Java online
    // submissions for Number of Provinces.
    public int findCircleNum(int[][] isConnected) {
        int r = isConnected.length;
        HashSet<Integer> visited = new HashSet<>();
        int count = 0;
        for(int i=0;i<r;i++){
            if(visited.contains(i)) continue;
            dfs(i,isConnected ,visited);
            count++;
        }

        return count;
    }

    private void dfs(int cur,int[][] isConnected, HashSet<Integer> visited)
    {
        int n = isConnected[cur].length;
        for(int adj=0;adj<n;adj++)
        {
            if(visited.contains(adj) || isConnected[cur][adj]==0 ) continue;
            visited.add(adj);
            dfs(adj,isConnected,visited);
        }
    }
}
```

MATH PROBLEMS

Problem-1:7. Reverse Integer

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$

Output: 321

Solution

```
class Solution {
// Runtime: 1 ms, faster than 99.30% of Java online submissions for Reverse
// Integer.
// Memory Usage: 39.8 MB, less than 45.96% of Java online submissions for
// Reverse Integer.
    public int reverse(int x) {

        int org = x;
        int rev = 0;
        boolean isNegative = false;

        if (x < 0) {
            isNegative = true;
            x = -1 * x;
        }
        while (x > 0) {
            if (rev > Integer.MAX_VALUE / 10
                || rev < Integer.MIN_VALUE / 10) {
                return 0;
            }
            int cur = x % 10;
            rev = cur + 10 * rev;
            x = x / 10;
        }

        return (isNegative) ? -1 * rev : rev;
    }
}
```


Problem-2:1492. The kth Factor of n

You are given two positive integers n and k . A factor of an integer n is defined as an integer i where $n \% i == 0$.

Consider a list of all factors of n sorted in **ascending order**, return *the k^{th} factor* in this list or return -1 if n has less than k factors.

Example 1:

Input: $n = 12, k = 3$

Output: 3

Explanation: Factors list is [1, 2, 3, 4, 6, 12], the 3rd factor is 3.

Solution

```
class Solution {
// Runtime: 0 ms, faster than 100.00% of Java online submissions for The kth
// Memory Usage: 39.3 MB, less than 59.98% of Java online submissions for The
    public int kthFactor(int n, int k) {
        for (int i = 1; i < Math.sqrt(n); i++) {
            if (n % i == 0 && --k == 0) {
                return i;
            }
        }

        for (int i = (int) Math.sqrt(n); i >= 1; --i) {
            if (n % (n / i) == 0 && --k == 0) {
                return n / i;
            }
        }

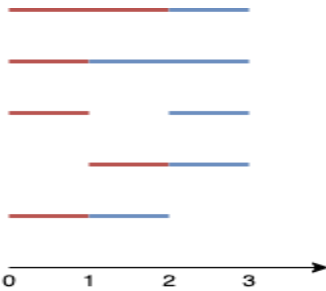
        return -1;
    }
}
```

Problem-3:1621. Number of Sets of K Non-Overlapping Line Segments

Given n points on a 1-D plane, where the i^{th} point (from 0 to $n-1$) is at $x = i$, find the number of ways we can draw **exactly k non-overlapping** line segments such that each segment covers two or more points. The endpoints of each segment must have **integral coordinates**. The k line segments **do not** have to cover all n points, and they are **allowed** to share endpoints.

Return the number of ways we can draw k non-overlapping line segments. Since this number can be huge, return it modulo $10^9 + 7$.

Example 1:



Input: $n = 4, k = 2$

Output: 5

Explanation: The two line segments are shown in red and blue.

The image above shows the 5 different ways $\{(0,2),(2,3)\}, \{(0,1),(1,3)\}, \{(0,1),(2,3)\}, \{(1,2),(2,3)\}, \{(0,1),(1,2)\}$.

Solution

```
class Solution {
// Runtime: 569 ms, faster than 17.24% of Java online submissions
// for Number of Sets of K Non-Overlapping Line Segments.
// Memory Usage: 128.6 MB, less than 17.24% of Java online submissions
// for Number of Sets of K Non-Overlapping Line Segments.

    int mod = (int)1e9+7;
    public int numberOfSets(int n, int k) {

        //1-----2-----3-----4
        //-----><----->
        //          <----->
        //----->          <----->
        //<-----><-----><----->
        //<-----><----->

        // (1,2)          (1,3)      (2,3)
        //   /  \          /          /
        //(2,3) (2,4)      (3,4)      (3,4)
        //   /  \
        //(3,4)

        Integer cache[][][] = new Integer[n+1][k+1][2];

        return helper(0,n,k,0,cache);
    }
}
```

```

// 1 2          3          4
// /
// 2
//isStart 0 - keep that index as start
//isStart 1 - keep that index as end
private int helper(int index, int n, int k, int isStart,
    Integer cache[][][]) {
    if (k == 0) {
        return 1;
    }
    if (index == n) {
        return 0;
    }

    if (cache[index][k][isStart] != null) {
        return cache[index][k][isStart];
    }

    int cur = helper(index + 1, n, k, isStart, cache);

    if (isStart == 0) {
        cur += helper(index + 1, n, k, isStart ^ 1, cache);
    } else {
        cur += helper(index, n, k - 1, isStart ^ 1, cache);
    }

    cache[index][k][isStart] = cur % mod;
    return cache[index][k][isStart];
}
}

```

Problem-4:1447. Simplified Fractions

Given an integer n , return a list of all *simplified fractions* between 0 and 1 (exclusive) such that the denominator is less-than-or-equal-to n . You can return the answer in **any order**.

Example 1:

Input: $n = 2$

Output: ["1/2"]

Explanation: "1/2" is the only unique fraction with a denominator less-than-or-equal-to 2.

Example 2:

Input: $n = 3$

Output: ["1/2", "1/3", "2/3"]

Solution

```
class Solution {
    // Runtime: 18 ms, faster than 92.45% of Java online submissions.
    // Memory Usage: 44.9 MB, less than 22.64% of Java online submissions.
    public List<String> simplifiedFractions(int n) {
        List<String> ans = new ArrayList<>();
        for (int denominator = 2; denominator <= n; ++denominator) {
            for (int numerator = 1; numerator < denominator;
                ++numerator) {
                if (gcd(numerator, denominator) == 1) {
                    ans.add(numerator + "/" + denominator);
                }
            }
        }
        return ans;
    }

    private int gcd(int x, int y) {
        return x == 0 ? y : gcd(y % x, x);
    }
}
```

Problem-5: 29. Divide Two Integers

Given two integers dividend and divisor, divide two integers **without** using multiplication, division, and mod operator.

The integer division should truncate toward zero, which means losing its fractional part. For example, 8.345 would be truncated to 8, and -2.7335 would be truncated to -2.

Return *the quotient after dividing dividend by divisor*.

Note: Assume we are dealing with an environment that could only store integers within the 32-bit signed integer range: $[-2^{31}, 2^{31} - 1]$. For this problem, if the quotient is **strictly greater than** $2^{31} - 1$, then return $2^{31} - 1$, and if the quotient is **strictly less than** -2^{31} , then return -2^{31} .

Example 1:

Input: dividend = 10, divisor = 3

Output: 3

Explanation: $10/3 = 3.33333..$ which is truncated to 3.

Example 2:**Input:** dividend = 7, divisor = -3**Output:** -2**Explanation:** 7/-3 = -2.33333.. which is truncated to -2.**Solution**

```
class Solution {
// Runtime: 1 ms, faster than 95.71% of Java online submissions for Divide
Two Integers.
// Memory Usage: 39.6 MB, less than 80.23% of Java online submissions for
Divide Two Integers.
    public int divide(int A, int B) {
        if (A == 1 << 31 && B == -1)
            return (1 << 31) - 1;
        int a = Math.abs(A), b = Math.abs(B), res = 0, x = 0;
        while (a - b >= 0) {
            for (x = 0; a - (b << x << 1) >= 0; x++)
                ;
            res += 1 << x;
            a -= b << x;
        }
        return (A > 0) == (B > 0) ? res : -res;
    }
}
```

Problem-6: 1317. Convert Integer to the Sum of Two No-Zero Integers

Easy

No-Zero integer is a positive integer that **does not contain any 0** in its decimal representation.

Given an integer n , return *a list of two integers* $[a, b]$ *where:*

- a and b are **No-Zero integers**.
- $a + b = n$

The test cases are generated so that there is at least one valid solution. If there are many valid solutions, you can return any of them.

Example 1:**Input:** $n = 2$ **Output:** $[1,1]$

Explanation: Let $a = 1$ and $b = 1$.

Both a and b are no-zero integers, and $a + b = 2 = n$.

Solution

```
class Solution {
// Runtime: 0 ms, faster than 100.00% of Java online submissions for Convert
Integer to the Sum of Two No-Zero Integers.
// Memory Usage: 40.4 MB, less than 52.23% of Java online submissions for
Convert Integer to the Sum of Two No-Zero Integers.
    public int[] getNoZeroIntegers(int n) {
        for (int i = 1; i < n; i++) {
            if (nonZero(i) && nonZero(n - i)) {
                return new int[]{i, n - i};
            }
        }
        return new int[]{-1, -1};
    }

    private boolean nonZero(int n) {
        while (n > 0) {
            if (n % 10 == 0) return false;
            n /= 10;
        }
        return true;
    }
}
```

Problem-7: 69. Sqrt(x)

Given a non-negative integer x , return *the square root of x rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: $x = 4$

Output: 2

Explanation: The square root of 4 is 2, so we return 2.

Solution

Method1:

```
class Solution {  
    // Runtime: 1 ms, faster than 99.26% of Java online submissions for Sqrt(x).  
    // Memory Usage: 39.4 MB, less than 84.90% of Java online submissions for Sqrt(x).  
    public int mySqrt(int x) {  
        long r = x;  
        while (r * r > x)  
            r = (r + x / r) / 2;  
        return (int) r;  
    }  
}
```

Method2:BinarySearch:

```
class Solution {  
    // Runtime: 1 ms, faster than 99.26% of Java online submissions for Sqrt(x).  
    // Memory Usage: 39.9 MB, less than 29.48% of Java online submissions for Sqrt(x).  
    public int mySqrt(int x) {  
        int i = 1;  
        int j = x;  
        int ans = 0;  
        while (i <= j) {  
            int mid = i + (j - i) / 2;  
            if (mid <= x / mid) {  
                i = mid + 1;  
                ans = mid;  
            } else  
                j = mid - 1;  
        }  
        return ans;  
    }  
}
```

Problem-8: 50. Pow(x, n)

Implement `pow(x, n)`, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$

Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$

Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$

Output: 0.25000

Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Solution

```
class Solution {
// Runtime: 0 ms, faster than 100.00% of Java online submissions for Pow(x, n).
// Memory Usage: 40.3 MB, less than 97.79% of Java online submissions for Pow(x, n).
    private double binaryExp(double x, long n) {
        if (n == 0) {
            return 1;
        }
        // Handle case where, n < 0.
        if (n < 0) {
            n = -1 * n;
            x = 1.0 / x;
        }
        // Perform Binary Exponentiation.
        double result = 1;
        while (n != 0) {
            // If 'n' is odd we multiply result with 'x'
            //and reduce 'n' by '1'.
            if (n % 2 == 1) {
                result = result * x;
                n -= 1;
            }
            // We square 'x' and reduce 'n'
            //by half,  $x^n \Rightarrow (x^2)^{(n/2)}$ .
            x = x * x;
            n = n / 2;
        }
        return result;
    }

    public double myPow(double x, int n) {
        return binaryExp(x, (long) n);
    }
}
```


Binary Exponentiation:

Binary exponentiation (also known as exponentiation by squaring) is a trick which allows to calculate a^n using only $O(\log n)$ multiplications (instead of

$O(n)$ multiplications required by the naive approach).

It also has important applications in many tasks unrelated to arithmetic, since it can be used with any operations that have the property of **associativity**:

The idea of binary exponentiation is, that we split the work using the binary representation of the exponent.

Let's write

n in base 2, for example

$$3^{13} = 3^{1101_2} = 3^8 \cdot 3^4 \cdot 3^1$$

$$3^1 = 3$$

$$3^2 = (3^1)^2 = 3^2 = 9$$

$$3^4 = (3^2)^2 = 9^2 = 81$$

$$3^8 = (3^4)^2 = 81^2 = 6561$$

So to get the final answer for 3^{13} , we only need to multiply three of them (skipping 3^2 because the corresponding bit in n is not set): $3^{13} = 6561 \cdot 81 \cdot 3 = 1594323$

The final complexity of this algorithm is $O(\log n)$: we have to compute $\log n$ powers of a , and then have to do at most $\log n$ multiplications to get the final answer from them.

The following recursive approach expresses the same idea:

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ (a^{\frac{n}{2}})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ (a^{\frac{n-1}{2}})^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Problem-9: 367. Valid Perfect Square

Given a positive integer num, return true if num is a perfect square or false otherwise.

A **perfect square** is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as `sqrt`.

Example 1:

Input: `num = 16`

Output: `true`

Explanation: We return true because $4 * 4 = 16$ and 4 is an integer.

Method1:

```
class Solution {  
    // Runtime: 1 ms, faster than 19.45% of Java online submissions for  
Valid  
    // Perfect Square.  
    public boolean isPerfectSquare(int num) {  
        int i = 1;  
        while (num > 0) {  
            num -= i;  
            i += 2;  
        }  
        return num == 0;  
    }  
}
```

Time Complexity: $O(\sqrt{n})$

Method 2:

```
class Solution {  
    //Runtime: 0 ms, faster than 100.00% of Java online submissions for Valid  
    //Perfect Square.  
    //Memory Usage: 38.8 MB, less than 97.95% of Java online submissions for  
    //Valid Perfect Square.  
    public boolean isPerfectSquare(int num) {  
        int low = 1, high = num;  
        while (low <= high) {  
            long mid = (low + high) >>> 1;  
            if (mid * mid == num) {  
                return true;  
            } else if (mid * mid < num) {  
                low = (int) mid + 1;  
            } else {  
                high = (int) mid - 1;  
            }  
        }  
        return false;  
    }  
}
```

Time Complexity: $O(\log(n))$

Method 3 : Newton Method:

```
class Solution {  
    //Runtime: 0 ms, faster than 100.00% of Java online submissions for Valid  
    //Perfect Square.  
    //Memory Usage: 38.6 MB, less than 99.16% of Java online submissions for  
    //Valid Perfect Square.  
    public boolean isPerfectSquare(int num) {  
        long x = num;  
        while (x * x > num) {  
            x = (x + num / x) >> 1;  
        }  
        return x * x == num;  
    }  
}
```

Problem-10: 204. Count Primes

Given an integer n , return the number of prime numbers that are strictly less than n .

Example 1:

Input: $n = 10$

Output: 4

Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.

Solution:

```
public class Solution {  
    // Runtime: 110 ms, faster than 73.19% of Java online submissions for Count  
    // Primes.  
    // Memory Usage: 47.5 MB, less than 94.91% of Java online submissions for  
    // Count Primes.  
    public int countPrimes(int n) {  
        boolean[] notPrime = new boolean[n];  
        int count = 0;  
        for (int i = 2; i < n; i++) {  
            if (notPrime[i] == false) {  
                count++;  
                for (int j = 2; i * j < n; j++) {  
                    notPrime[i * j] = true;  
                }  
            }  
        }  
        return count;  
    }  
}
```

Problem-11: 342. Power of Four

Given an integer n , return *true* if it is a power of four. Otherwise, return *false*.

An integer n is a power of four, if there exists an integer x such that $n == 4^x$.

Example 1:

Input: n = 16

Output: true

EXPLANATION:

1. num>0 then only it can be power of 4 . $4^0=1$, $4^1=4$..
2. only 1 bit is on then number is power of 2 .
therefore in every power of 4 number will have only 1 set bit.
3. $(4^n - 1) \% 3 == 0$
Mathematical proof:
3.1 $4^n - 1 = (2^n + 1) * (2^n - 1)$
3.2 among any 3 consecutive numbers, there must be one that is a multiple of 3
among (2^{n-1}) , (2^n) , (2^{n+1}) , one of them must be a multiple of 3, and (2^n) cannot be the one, therefore either (2^{n-1}) or (2^{n+1}) must be a multiple of 3, and 4^n-1 must be a multiple of 3 as well

Solution:

Method 1:

```
class Solution {  
    // Runtime: 1 ms, faster than 91.62% of Java online submissions for  
    // Power of Four.  
    // Memory Usage: 39.4 MB, less than 89.75% of Java online submissions for  
    // Power of Four.  
    public boolean isPowerOfFour(int n) {  
        if (n <= 0)  
            return false;  
        if ((n & (n - 1)) != 0)  
            return false;  
        return (n - 1) % 3 == 0;  
    }  
}
```

Method 2:

```
class Solution {  
    public boolean isPowerOfFour(int num) {  
        if(num<=0 )return false;  
  
        //16/4 = 4d  
        return ( Math.Log(num)/Math.Log(4)%1)==0?true:false;  
    }  
}
```

Method 3:

```
public class Solution {  
    //Runtime: 0 ms, faster than 100.00% of Java online submissions  
    //for Power of Four.  
    public boolean isPowerOfFour(int num) {  
        return (num > 0) && ((num & (num - 1)) == 0)  
            && ((num & 0x55555555) == num);  
    }  
}
```

The basic idea is from power of 2, We can use " $n \& (n-1) == 0$ " to determine if n is power of 2. For power of 4, the additional restriction is that in binary form, the only "1" should always located at the odd position. For example, $4^0 = 1$, $4^1 = 100$, $4^2 = 10000$. So we can use " $num \& 0x55555555 == num$ " to check if "1" is located at the odd position.

Problem-11b: 231. Power of Two**Method 1:**

```
class Solution {  
    // Runtime: 1 ms, faster than 95.95% of Java online  
    //submissions for Power of Two.  
    // Memory Usage: 39.6 MB, less than 71.73% of Java online submissions for  
    //Power of Two.  
    public boolean isPowerOfTwo(int n) {  
        if (n <= 0)  
            return false;  
  
        if((n&(n-1))!=0) return false;  
  
        return true;  
    }  
}
```

Method 2:

```
class Solution {  
    // Runtime: 1 ms, faster than 95.95% of Java online submissions for Power of  
    //Two.  
    // Memory Usage: 39.6 MB, less than 71.73% of Java online submissions for  
    //Power of Two.  
    public boolean isPowerOfTwo(int n) {  
        if (n <= 0)  
            return false;  
  
        return (Math.Log10(n) / Math.Log10(2)) % 1 == 0;  
    }  
}
```

Problem-11c: 326. Power of Three**Method 1:**

```

class Solution {
// Runtime: 7 ms, faster than 100.00% of Java online submissions for Power of
// Three.
// Memory Usage: 43.3 MB, less than 17.76% of Java online submissions
    public boolean isPowerOfThree(int n) {
        while (n >= 3) {
            if (n % 3 != 0)
                return false;
            n /= 3;
        }
        return n == 1;
    }
}

```

Complexity:

Time: $O(\log_3(n))$ and Space: $O(1)$

Method 2: Mathematics

$$n = 3^i$$

$$i = \log_3(n)$$

$$i = \frac{\log_b(n)}{\log_b(3)}$$

Here if n is actually a exact power of 3 then i must be integer . so $i\%1$ will be equal to 0.

In java we find log using `Math.logb` and output will be in double.

```

class Solution {
// Runtime: 8 ms, faster than 91.25% of Java online submissions for
// Power of Three.
    public boolean isPowerOfThree(int n) {
        return (Math.Log10(n) / Math.Log10(3)) % 1 == 0;
    }
}

```

Method 3:

In most programming competitions, we are required to answer the result in 10^9+7 modulo. The reason behind this is, if problem constraints are large integers, only efficient algorithms can solve them in an allowed limited time.

What is modulo operation:

The remainder obtained after the division operation on two operands is known as modulo operation. The operator for doing modulus operation is '%'. For ex: $a \% b = c$ which means, when a is divided by b it gives the remainder c , $7\%2 = 1$, $17\%3 = 2$.

Why do we need modulo:

- The reason of taking Mod is to prevent integer overflows

How modulo is used:

A few distributive properties of modulo are as follows:

1. $(a + b) \% c = ((a \% c) + (b \% c)) \% c$
2. $(a * b) \% c = ((a \% c) * (b \% c)) \% c$
3. $(a - b) \% c = ((a \% c) - (b \% c)) \% c$
4. $(a / b) \% c = ((a \% c) / (b \% c)) \% c$

So, modulo is distributive over +, * and – but not over / (division).

NOTE: The result of $(a \% b)$ will always be less than b .

In the case of computer programs, due to the size of variable limitations, we **perform modulo M at each intermediate stage** so that range overflow never occurs.

Example:

```
a = 145785635595363569532135132
b = 3151635135413512165131321321
c = 999874455222222200651351351
m = 1000000007
Print (a*b*c)%m.
```

Method 1:

First, multiply all the number and then take modulo:

```
(a*b*c)%m = (459405448184212290893339835148809
515332440033400818566717735644307024625348601572) %
1000000007
```

$a*b*c$ does not fit even in the unsigned long long int due to which system drop some of its most

significant digits. Therefore, it gives the wrong answer.
 $(a*b*c)\%m = 798848767$

Method 2:

Take modulo at each intermediate steps:

```
i = 1
i = (i*a) % m    // i = 508086243
i = (i*b) % m    // i = 144702857
i = (i*c) % m    // i = 798848767
i = 798848767
```

Method 2 always gives the correct answer.

```
public class Solution {
    public static void main(String args[]) {
        int maxInt = (int)1e9;//1000000000
        int maxInt1 = (int)1e10;//2147483647//truncate remaining
        //int i=1000000000 //compile time error
        //1000000000
        System.out.println("Solution.main()" + maxInt);
        System.out.println("Solution.main()" + maxInt1);
    }
}
```

Output:

```
//Solution.main()1000000000
//Solution.main()2147483647
```

If somehow we know the largest power of 3 which is in the range of 3 our work is done just check largest power $\%n == 0$.

Let largest power of 3 be x for a while and $n=3$ for simplicity.

Note : $x\%3 == 0$ and $x\%-3 == 0$ so make a check n should be greater than 0.

This idea is applicable to all Prime number

```
class Solution {
    // Runtime: 7 ms, faster than 100.00% of Java online
    // submissions for Power of Three.
    // Memory Usage: 43 MB, less than 49.33% of Java online submissions.
    public boolean isPowerOfThree(int n) {
        // 3^19 = 1162261467,
        // 3^20 is exceeding Integer_Range So 3^19 is the highest
        // power in Integer Range
        return (n > 0 && 1162261467 % n == 0);
    }
}
```

Same Solution easy to remember:

```
class Solution {  
    // Runtime: 8 ms, faster than 90.78% of Java online submissions  
    //for Power of Three.  
    // Memory Usage: 42.6 MB, less than 92.25% of Java online.  
    public boolean isPowerOfThree(int n) {  
        int maxPow3 = (int) Math.pow(3, 19);  
  
        return n > 0 && maxPow3 % n == 0;  
    }  
}
```

Problem-12: 1415. The k-th Lexicographical String of All Happy Strings of Length n

A happy string is a string that:

- consists only of letters of the set ['a', 'b', 'c'].
- $s[i] \neq s[i + 1]$ for all values of i from 1 to $s.length - 1$ (string is 1-indexed).

For example, strings "abc", "ac", "b" and "abcbabcbcb" are all happy strings and strings "aa", "baa" and "ababbc" are not happy strings.

Given two integers n and k , consider a list of all happy strings of length n sorted in lexicographical order.

Return the k th string of this list or return an empty string if there are less than k happy strings of length n .

Example 1:

Input: $n = 1, k = 3$

Output: "c"

Explanation: The list ["a", "b", "c"] contains all happy strings of length 1. The third string is "c".

Example 2:

Input: $n = 1, k = 4$

Output: ""

Explanation: There are only 3 happy strings of length 1.

Solution:

```
class Solution {  
    // Runtime: 1 ms, faster than 100.00% of Java online submissions for The k-th  
    // Lexicographical String of All Happy Strings of Length n.  
    public String getHappyString(int n, int k) {  
        int prem = 1 << (n - 1);  
        if (k > 3 * prem)  
            return "";  
        int ch = 'a' + (k - 1) / prem;  
        StringBuilder sb = new StringBuilder(Character.toString(ch));  
        while (prem > 1) {  
            k = (k - 1) % prem + 1;  
            prem >>= 1;  
            int avalue = 'a' + (ch == 'a' ? 1 : 0);  
            int bvalue = 'b' + (ch != 'c' ? 1 : 0);  
            ch = (k - 1) / prem == 0 ? avalue : bvalue;  
            sb.append((char) ch);  
        }  
        return sb.toString();  
    }  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ – not considering output

For the string of size n , we can build $3 * \text{pow}(2, n - 1)$ strings. So, if $k \leq \text{pow}(2, n - 1)$, then the first letter is a, $k \leq 2 * \text{pow}(2, n - 1)$ – then b, otherwise c. We can also return empty string right away if $k > 3 * \text{pow}(2, n - 1)$.

BIT MANIPULATION

BASIC OPERATIONS IN BIT MASK

We have considered the below facts in this article -

- 0 based indexing of bits from right to left.
- Setting i-th bit means, turning i-th bit to 1
- Clearing i-th bit means, turning i-th bit to 0

1) Clear all bits from LSB to ith bit

```
mask = ~(1 << i+1 ) - 1;  
x &= mask;
```

Logic: To clear all bits from LSB to i-th bit, we have to AND x with mask having LSB to i-th bit 0. To obtain such mask, first left shift 1 i times. Now if we minus 1 from that, all the bits from 0 to i-1 become 1 and remaining bits become 0. Now we can simply take the complement of mask to get all first i bits to 0 and remaining to 1.
Example-

```
x = 29 (00011101) and we want to clear LSB to 3rd bit, total 4 bits  
mask -> 1 << 4 -> 16(00010000)  
mask -> 16 - 1 -> 15(00001111)  
mask -> ~mask -> 11110000  
x & mask -> 16 (00010000)
```

2) Clearing all bits from MSB to i-th bit

```
mask = (1 << i) - 1;  
x &= mask;
```

Logic: To clear all bits from MSB to i-th bit, we have to AND x with mask having MSB to i-th bit 0. To obtain such mask, first left shift 1 i times. Now if we minus 1 from that, all the bits from 0 to i-1 become 1 and the remaining bits become 0.

Example-

```
x = 215 (11010111) and we want to clear MSB to 4th bit, total 4 bits  
mask -> 1 << 4 -> 16(00010000)  
mask -> 16 - 1 -> 15(00001111)  
x & mask -> 7(0000111)
```

3) Divide by 2

x >>= 1;

Logic: When we do arithmetic right shift, every bit is shifted to right and blank position is substituted with sign bit of number, 0 in case of positive and 1 in case of negative number. Since every bit is a power of 2, with each shift we are reducing the value of each bit by factor of 2 which is equivalent to division of x by 2.

Example-

x = 18(00010010)

x >> 1 = 9 (00001001)

4) Multiplying by 2

x <<= 1;

Logic: When we do arithmetic left shift, every bit is shifted to left and blank position is substituted with 0. Since every bit is a power of 2, with each shift we are increasing the value of each bit by a factor of 2 which is equivalent to multiplication of x by 2.

Example-

x = 18(00010010)

x << 1 = 36 (00100100)

5) Upper case English alphabet to lower case

ch |= ' ';

Logic: The bit representation of upper case and lower case English alphabets are -

A -> 01000001	a -> 01100001
B -> 01000010	b -> 01100010
C -> 01000011	c -> 01100011
.	.
.	.
Z -> 01011010	z -> 01111010

As we can see if we set 5th bit of upper case characters, it will be converted into lower case character. We have to prepare a mask having 5th bit 1 and other 0 (00100000). This mask is bit representation of space character (' '). The character 'ch' then ORed with mask.

Example-

ch = 'A' (01000001)

mask = ' ' (00100000)

ch | mask = 'a' (01100001)

6) Lower case English alphabet to upper case

ch &= '_' ;

Logic: The bit representation of upper case and lower case English alphabets are -

A -> 01000001	a -> 01100001
B -> 01000010	b -> 01100010
C -> 01000011	c -> 01100011
.	.
.	.
Z -> 01011010	z -> 01111010

As we can see if we clear 5th bit of lower case characters, it will be converted into upper case character. We have to prepare a mask having 5th bit 0 and other 1 (11011111). This mask is bit representation of underscore character ('_'). The character 'ch' then AND with mask.

Example-

```
ch = 'a' (01100001)
mask = '_' (11011111)
ch & mask = 'A' (01000001)
```

7) Count set bits in integer (Brian Kernighan's algorithm)

```
int countSetBits(int x)
{
    int count = 0;
    while (x!=0)
    {
        x &= (x-1);
        count++;
    }
    return count;
}
```

Input: 15

output: 4

8) Find log base 2 of 32 bit integer

```
public int log2(int N)
{
    // calculate log2 N indirectly
    // using log() method
    int result = (int)(Math.log(N) / Math.log(2));
    return result;
}
```

Input: 1024

output: 10

Logic: We right shift x repeatedly until it becomes 0, meanwhile we keep count on the shift operation. This count value is the $\log_2(x)$.

9) Checking if given 32 bit integer is power of 2

```
boolean isPowerof2(int n) {
    return (n & n - 1) == 0;
}
```

Input: 9

output: false

Logic: All the power of 2 have only single bit set e.g. 16 (00010000). If we minus 1 from this, all the bits from LSB to set bit get toggled, i.e., $16-1 = 15$ (00001111). Now if we AND x with (x-1) and the result is 0 then we can say that x is power of 2 otherwise not. We have to take extra care when $x = 0$.

Example

$x = 16$ (00010000)

$x - 1 = 15$ (00001111)

$x \& (x-1) = 0$

so 16 is power of 2

10) Find the last set bit

```
static int Right_most_setbit(int num)
{
    int pos = 1;
    //16 8 4 2 1
    // 1 0 0 1 0
    // counting the position of first set bit
    for (int i = 0; i < 32; i++) {
        if ((num & (1 << i)) == 0)
            pos++;

        else
            break;
    }
    return pos;
}
```

Input: 18

output: 2

The logarithmic value of AND of x and $-x$ to the base 2 gives the index of the last set bit(for 0-based indexing).

Interesting Facts About Bitwise Operators

1. The left-shift and right-shift operators should not be used for negative numbers.

If the second operand(which decides the number of shifts) is a negative number, it results in undefined behavior in C. For example results of both $1 \ll -1$ and $1 \gg -1$ is undefined. Also, if the number is shifted more than the size of the integer, the behavior is undefined. For example, $1 \ll 33$ is undefined if integers are stored using 32 bits. Another thing is NO shift operation is performed if the additive expression (operand that decides no of shifts) is 0. See [this](#) for more details.

Note: In C++, this behavior is well-defined.

2. Interestingly!! The bitwise OR of two numbers is just the sum of those two numbers if there is no carry involved, otherwise, you just add their bitwise AND.

Let's say, we have $a=5(101)$ and $b=2(010)$, since there is no carry involved, their sum is just $a|b$. Now, if we change 'a' to 6 which is 110 in binary, their sum would change to $a|b + a\&b$ since there is a carry involved.

3. The bitwise XOR operator is the most useful operator from a technical interview perspective.

It is used in many problems. A simple example could be "Given a set of numbers where all elements occur an even number of times except one number, find the odd occurring number" This problem can be efficiently solved by just doing XOR to all numbers.

ADVANCED BITMASK

1.How to set a bit in the number 'num':

If we want to set a bit at n th position in the number 'num', it can be done using the 'OR' operator($|$).

- First, we left shift '1' to n position via $(1 \ll n)$
- Then, use the 'OR' operator to set the bit at that position. 'OR' operator is used because it will set the bit even if the bit is unset previously in the binary representation of the number 'num'.

Note: If the bit would be already set then it would remain unchanged.

```
class Solution {
    public static void main (String[] args) {
        int num = 4, pos =1;
        num = set(num, pos);
        System.out.println(num);
    }
    public static int set(int num, int pos){
        // First step is shift '1', second
        // step is bitwise OR
        num |= (1 << pos);
        return num;
    }
}
```

Output:

6

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

We have passed the parameter by 'call by reference' to make permanent changes in the number.

2. How to unset/clear a bit at n'th position in the number 'num' :

Suppose we want to unset a bit at nth position in number 'num' then we have to do this with the help of 'AND' (&) operator.

- First, we left shift '1' to n position via $(1 \ll n)$ then we use bitwise NOT operator '~' to unset this shifted '1'.
- Now after clearing this left shifted '1' i.e making it to '0' we will 'AND'(&) with the number 'num' that will unset bit at nth position.

```
class Solution {
    public static void main(String[] args)
    {
        int num = 7, pos = 1;
        num = unset(num, pos);
        System.out.println(num);
    }
    public static int unset(int num, int pos)
    {
        // Second step is to bitwise and this number with
        // given number
        num = num & ~(1 << pos);
        return num;
    }
}
```

Output:

5

Time Complexity: $O(1)$ Auxiliary Space: $O(1)$ **3. Toggling a bit at nth position :**

Toggling means to turn bit 'on'(1) if it was 'off'(0) and to turn 'off'(0) if it was 'on'(1) previously. We will be using the 'XOR' operator here which is this '^'. The reason behind the 'XOR' operator is because of its properties.

- Properties of 'XOR' operator.
 - $1^1 = 0$
 - $0^0 = 0$
 - $1^0 = 1$
 - $0^1 = 1$
- If two bits are different then the 'XOR' operator returns a set bit(1) else it returns an unset bit(0).

```
class Solution {  
    public static void main (String[] args) {  
        int num = 4, pos =1;  
        num = toggle(num, pos);  
        System.out.println(num);  
    }  
    public static int toggle(int num, int pos){  
        // First step is to shift 1,Second step is to XOR with given number  
        num ^= (1 << pos);  
        return num;  
    }  
}
```

Output:

6

Time Complexity: $O(1)$ Auxiliary Space: $O(1)$

4. Checking if bit at nth position is set or unset:

It is quite easily doable using the 'AND' operator.

- Left shift '1' to given position and then 'AND'('&')

```
class Solution {
    public static void main(String[] args)
    {
        int num = 5;
        int pos = 0;
        int bit = at_position(num, pos);
        System.out.println(bit);
    }
    public static int at_position(int num, int pos)
    {
        int bit = num & (1 << pos);
        return bit;
    }
}
```

Output: 1

Time Complexity: O(1)

Auxiliary Space: O(1)

Observe that we have first left shifted '1' and then used 'AND' operator to get bit at that position. So if there is '1' at position 'pos' in 'num', then after 'AND' our variable 'bit' will store '1' else if there is '0' at position 'pos' in the number 'num' then after 'AND' our variable bit will store '0'.

Some more quick hacks:

1)Inverting every bit of a number/1's complement: If we want to invert every bit of a number i.e change bit '0' to '1' and bit '1' to '0'.We can do this with the help of '~' operator. For example : if number is num=00101100 (binary representation) so '~num' will be '11010011'.

This is also the '1s complement of number'.

```
class Solution {
    public static void main (String[] args) {
        int num = 4;

        // Inverting every bit of number num
        num = (~num);
        System.out.println(num);
    }
}
```

Output: -5

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

2)Two's complement of the number: 2's complement of a number is 1's complement + 1.

So formally we can have 2's complement by finding 1s complement and adding 1 to the result i.e $(\sim \text{num} + 1)$ or what else we can do is using '-' operator.

```
class Solution {
    public static void main(String[] args)
    {
        int num = 4;
        int twos_complement = -num;
        System.out.println("This is two's complement "
            + twos_complement);
        System.out.println("This is also two's complement "
            + (~num + 1));
    }
}
```

Output:

This is two's complement -4

This is also two's complement -4

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Stripping off the lowest set bit :

In many situations we want to strip off the lowest set bit for example in Binary Indexed tree data structure, counting number of set bit in a number. We do something like this:

$X = X \& (X-1)$

But how does it even work? Let us see this by taking an example, let $X = 1100$.

- $(X-1)$ inverts all the bits till it encounters the lowest set '1' and it also inverts that lowest set '1'.
- $X-1$ becomes 1011. After 'ANDing' X with $X-1$ we get the lowest set bit stripped.

```
class Solution {
    public static void main(String[] args)
    {
        int num = 7;
        num = strip_last_set_bit(num);
        System.out.println(num);
    }
    public static int strip_last_set_bit(int num)
    {
        return num & (num - 1);
    }
}
```

Output:

6

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Getting the lowest set bit of a number:

This is done by using the expression ' $X \&(-X)$ '. Let us see this by taking an example: Let $X = 00101100$. So $\sim X$ (1's complement) will be '11010011' and 2's complement will be $(\sim X + 1 \text{ or } -X)$ i.e. '11010100'. So if we 'AND' original number 'X' with its two's complement which is '-X', we get the lowest set bit.

```
00101100
& 11010100
-----
00000100
```

```
class Solution {
    public static void main(String[] args)
    {
        int num = 10;
        int ans = lowest_set_bit(num);
        System.out.println(ans);
    }
    public static int lowest_set_bit(int num)
    {
        int ret = num & (-num);
        return ret;
    }
}
```

Output:

2

Time Complexity: O(1)

Auxiliary Space: O(1)

Division by 2 and Multiplication by 2 are very frequently that too in loops in Competitive Programming so using Bitwise operators can help in speeding up the code.

Divide by 2 using the right shift operator:

00001100 >> 1 (00001100 is 12)

00000110 (00000110 is 6)

```
class Solution {
    public static void main(String[] args)
    {
        int num = 12;
        int ans = num >> 1;
        System.out.println(ans);
    }
}
```

Output:

6

Time Complexity: O(1)

Auxiliary Space: O(1)

Multiply by 2 using the left shift operator:

00001100 << 1 (00001100 is 12)

00011000 (00000110 is 24)

```
class Solution {
    public static void main (String[] args) {
        int num = 12;
        int ans = num<<1;
        System.out.println(ans);
    }
}
```

Output:

24

Time Complexity: O(1)

Auxiliary Space: O(1)

Problem-1. 705. Design HashSet

Easy

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Example 1:

Input

```
["MyHashSet", "add", "add", "contains", "contains", "add", "contains",  
"remove", "contains"]  
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
```

Output

```
[null, null, null, true, false, null, true, null, false]
```

Explanation

```
MyHashSet myHashSet = new MyHashSet();  
myHashSet.add(1);      // set = [1]  
myHashSet.add(2);      // set = [1, 2]  
myHashSet.contains(1); // return True  
myHashSet.contains(3); // return False, (not found)  
myHashSet.add(2);      // set = [1, 2]  
myHashSet.contains(2); // return True  
myHashSet.remove(2);   // set = [1]  
myHashSet.contains(2); // return False, (already removed)
```

Constraints:

- $0 \leq \text{key} \leq 10^6$
- At most 10^4 calls will be made to add, remove, and contains.

Solution

In Java a int variable has 32 bits, we can use each one of them as a boolean:

- 1 is True
- 0 is False

If $0 \leq \text{key} \leq 10^6$ we need at least 1000000 bits so $1000000 / 32 + 1 = 31251$ int variables.

```
table = new int[31251];
```

To i_{th} -key is assigned ($i \% 32$)th-bit of table[i/32]

```
class MyHashSet {
// Runtime: 10 ms, faster than 99.97% of Java online submissions for Design
// Memory Usage: 46.5 MB, less than 91.78% of Java online submissions for Design
HashSet.
    int[] table;

    public MyHashSet() {
        // 1000000 / 32 + 1 = 31251
        this.table = new int[31251];
    }

    public void add(int key) {
        this.table[key / 32] |= 1 << key % 32;
    }

    public void remove(int key) {
        this.table[key / 32] &= ~(1 << key % 32);
    }

    public boolean contains(int key) {
        return (this.table[key / 32] & 1 << (key % 32)) != 0;
    }
}
```

NOTE:

$\&= \sim(1 \ll \text{key \% } 32)$; as you can see $\sim$ negation denotes remove/clear even key not exists

Problem-2.1097B - Petr and a Combination Lock(CODE FORCES)

B. Petr and a Combination Lock

time limit per test

1 second

memory limit per test

256 megabytes

input

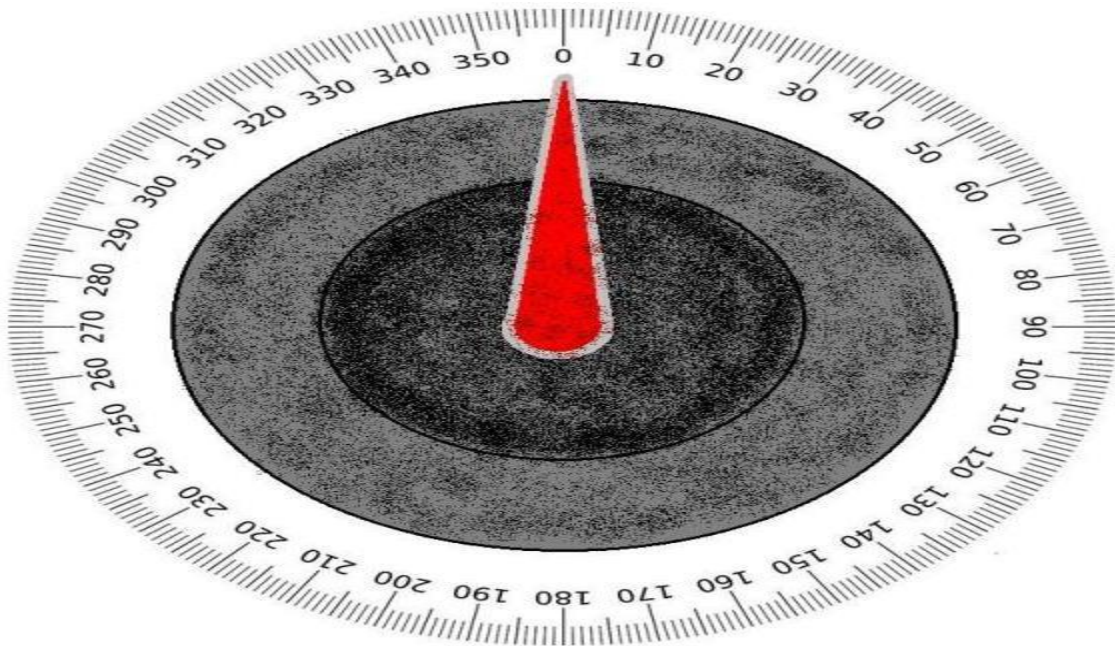
standard input

output

standard output

Petr has just bought a new car. He's just arrived at the most known Petersburg's petrol station to refuel it when he suddenly discovered that the petrol tank is secured with a combination lock! The lock has a scale of 360

degrees and a pointer which initially points at zero:



Petr called his car dealer, who instructed him to rotate the lock's wheel exactly n

times. The i -th rotation should be a_i degrees, either clockwise or counterclockwise, and after all n

rotations the pointer should again point at zero.

This confused Petr a little bit as he isn't sure which rotations should be done clockwise and which should be done counterclockwise. As there are many possible ways of rotating the lock, help him and find out whether there exists at least one, such that after all n

rotations the pointer will point at zero again.

Input

The first line contains one integer n

($1 \leq n \leq 15$

) – the number of rotations.

Each of the following n

lines contains one integer a_i ($1 \leq a_i \leq 180$) – the angle of the i -th rotation in degrees.

Output

If it is possible to do all the rotations so that the pointer will point at zero after all of them are performed, print a single word "YES". Otherwise, print "NO". Petr will probably buy a new car in this case.

You can print each letter in any case (upper or lower).

Examples**Input**

```
3
10
20
30
```

Output

```
YES
```

Solution

```
public class Solution {
    public boolean check(int n, int arr[]) {
        int end = 1 << n;
        for (int mask = 0; mask < end; mask++) {
            int total = 0;
            //split in to 2 half
            //one who's i'th bit set and one not set.
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) == 0)
                    total += arr[i];
                else
                    total = total - arr[i];
            }
            //total 360 is clock total max possible value
            if (total % 360 == 0) {
                return true;
            }
        }
        return false;
    }
}
```

```
public final static void main(final String args[]) {
    int arr[] = { 10, 20, 30 };
    boolean res = new Solution().check(3, arr);
    System.out.println("Solution.main()" + res); //true

    boolean res1 = new Solution().check(3, new int[] { 10, 10, 10 });
    System.out.println("Solution.main()" + res1); //false

    boolean res3 = new Solution().check(3, new int[] { 120, 120, 120 });
    System.out.println("Solution.main()" + res3); //true
}
}
```

Problem-3: 2151. Maximum Good People Based on Statements(BITMASK)

Hard

There are two types of persons:

- The **good person**: The person who always tells the truth.
- The **bad person**: The person who might tell the truth and might lie.

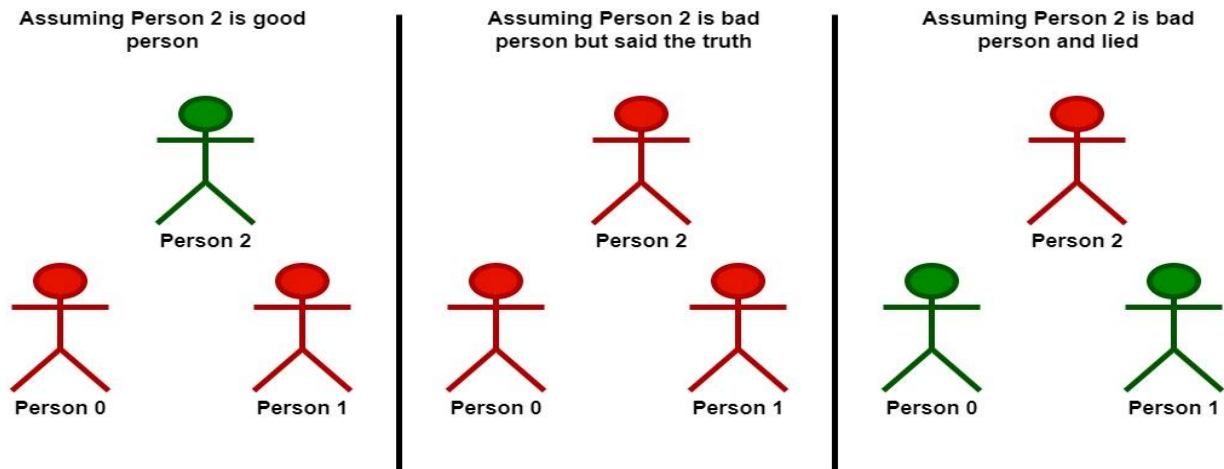
You are given a **0-indexed** 2D integer array `statements` of size `n x n` that represents the statements made by `n` people about each other. More specifically, `statements[i][j]` could be one of the following:

- `0` which represents a statement made by person `i` that person `j` is a **bad** person.
- `1` which represents a statement made by person `i` that person `j` is a **good** person.
- `2` represents that **no statement** is made by person `i` about person `j`.

Additionally, no person ever makes a statement about themselves. Formally, we have that `statements[i][i] = 2` for all $0 \leq i < n$.

Return the *maximum* number of people who can be **good** based on the statements made by the `n` people.

Example 1:



Input: statements = [[2,1,2],[1,2,2],[2,0,2]]

Output: 2

Explanation: Each person makes a single statement.

- Person 0 states that person 1 is good.
- Person 1 states that person 0 is good.
- Person 2 states that person 1 is bad.

Let's take person 2 as the key.

- Assuming that person 2 is a good person:
 - Based on the statement made by person 2, person 1 is a bad person.
 - Now we know for sure that person 1 is bad and person 2 is good.
 - Based on the statement made by person 1, and since person 1 is bad,

they could be:

- telling the truth. There will be a contradiction in this case and this assumption is invalid.

- lying. In this case, person 0 is also a bad person and lied in their statement.

- **Following that person 2 is a good person, there will be only one good person in the group.**

- Assuming that person 2 is a bad person:

- Based on the statement made by person 2, and since person 2 is bad, they could be:

- telling the truth. Following this scenario, person 0 and 1 are both bad as explained before.

- **Following that person 2 is bad but told the truth, there will be no good persons in the group.**

- lying. In this case person 1 is a good person.

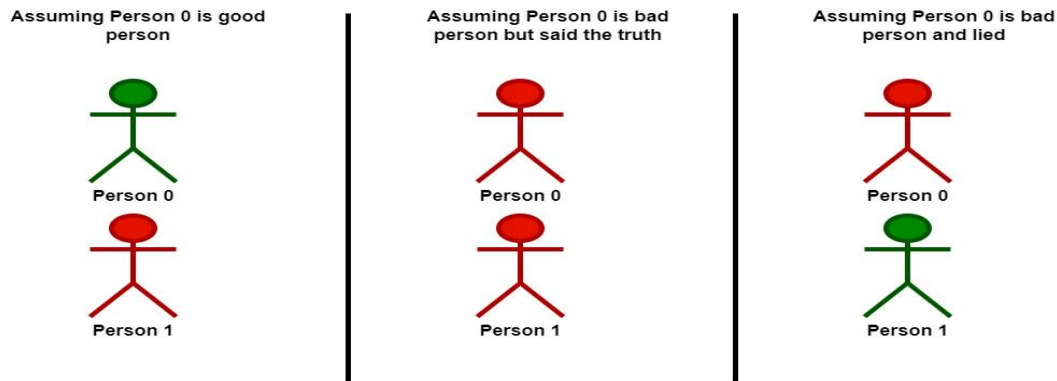
- Since person 1 is a good person, person 0 is also a good person.

- **Following that person 2 is bad and lied, there will be two good persons in the group.**

We can see that at most 2 persons are good in the best case, so we return 2.

Note that there is more than one way to arrive at this conclusion.

Example 2:



Input: statements = [[2,0],[0,2]]

Output: 1

Explanation: Each person makes a single statement.

- Person 0 states that person 1 is bad.
- Person 1 states that person 0 is bad.

Let's take person 0 as the key.

- Assuming that person 0 is a good person:
 - Based on the statement made by person 0, person 1 is a bad person and was lying.
 - **Following that person 0 is a good person, there will be only one good person in the group.**
- Assuming that person 0 is a bad person:
 - Based on the statement made by person 0, and since person 0 is bad, they could be:
 - telling the truth. Following this scenario, person 0 and 1 are both bad.
 - **Following that person 0 is bad but told the truth, there will be no good persons in the group.**
 - lying. In this case person 1 is a good person.
 - **Following that person 0 is bad and lied, there will be only one good person in the group.**

We can see that at most, one person is good in the best case, so we return 1. Note that there is more than one way to arrive at this conclusion.

Constraints:

- `n == statements.length == statements[i].length`
- `2 <= n <= 15`
- `statements[i][j]` is either 0, 1, or 2.
- `statements[i][i] == 2`

Solution

```

class Solution {
    public int maximumGood(int[][] statements) {
        int result = 0;
        int n = (1 << statements.length) - 1;

        for (int mask = 1; mask <= n; mask++) {
            if (isValid(statements, mask)) {
                result = Math.max(result, Integer.bitCount(mask));
            }
        }

        return result;
    }

    public boolean isValid(int[][] statements, int mask) {
        for (int i = 0; i < statements.length; i++) {
            // if i is not a good person in the current mask,
            // we don't need to validate the
            // statements
            if (((1 << i) & mask) == 0) {
                continue;
            }

            for (int j = 0; j < statements[0].length; j++) {
                if (statements[i][j] == 2) {
                    continue;
                }

                // i said j bad but j is not bad (0)
                // || i said j good, but j is not good
                if ((statements[i][j] == 0 && ((1 << j) & mask) != 0)
                    || (statements[i][j] == 1
                        && ((1 << j) & mask) == 0)) {
                    return false;
                }
            }
        }

        return true;
    }
}

```

Problem-4: 1799. Maximize Score After N Operations

You are given `nums`, an array of positive integers of size $2 * n$. You must perform n operations on this array.

In the i^{th} operation (**1-indexed**), you will:

- Choose two elements, x and y .
- Receive a score of $i * \text{gcd}(x, y)$.
- Remove x and y from `nums`.

Return *the maximum score you can receive after performing n operations*.

The function `gcd(x, y)` is the greatest common divisor of x and y .

Example 1:

Input: `nums = [1,2]`

Output: 1

Explanation: The optimal choice of operations is:
 $(1 * \text{gcd}(1, 2)) = 1$

Example 2:

Input: `nums = [3,4,6,8]`

Output: 11

Explanation: The optimal choice of operations is:
 $(1 * \text{gcd}(3, 6)) + (2 * \text{gcd}(4, 8)) = 3 + 8 = 11$

Example 3:

Input: `nums = [1,2,3,4,5,6]`

Output: 14

Explanation: The optimal choice of operations is:
 $(1 * \text{gcd}(1, 5)) + (2 * \text{gcd}(2, 4)) + (3 * \text{gcd}(3, 6)) = 1 + 4 + 9 = 14$

Constraints:

- $1 \leq n \leq 7$
- `nums.length == 2 * n`
- $1 \leq \text{nums}[i] \leq 10^6$

Solution

```

import java.util.Arrays;

class Solution {
// Runtime: 37 ms, faster than 83.67% of Java online submissions for Maximize Score
// After N Operations.
// Memory Usage: 41.2 MB, less than 68.88% of Java online submissions for Maximize
// Score After N Operations.
    public int maxScore(int[] nums) {
        int n = nums.length;
        int k = n/2;
        int gcdArr[][] = new int [n][n];

        //      [1,2,3,4 ,5,6]
        //      /           \
        // (1,2) [3,4,5,6] \ (3,4) [1,2,5,6](remaining)
        //      /           \
        //      /           \
        // (3,4)           (1,2) [5,6]
        //      /           \
        // (5,6)           (5,6) //overlapping can cache
        //can use boolean arr or bitmask since the
        //max len of arr is 14.
        //so max it will be 2 power 14 and since
        //integer can store up to 2 power 31 (32 bits include 0).

        //precompute gcd arr
        for(int i =0;i<n;i++){
            for(int j =0;j<n;j++){
                gcdArr[i][j]= gcd(nums[i],nums[j]);
            }
        }

        //[1,  2,  3,  4,  5,  6]
        //[0/1, 0/1  0/1 0/1  0/1  0/1 -> 2 pow 6(set vs not set)

        int mask = (1<<n) ;
        int dp[] = new int[mask];
        Arrays.fill(dp,-1);

        return helper(nums, gcdArr,0,0,dp);
    }

    private int helper(int nums[], int[][] gcdArr,
        int index, int mask, int dp[]) {
        int n = nums.length;
        int k = n / 2;
        if (index > k) {
            return 0;
        }

```

```

    if (dp[mask] != -1) {
        return dp[mask];
    }

    // 3 2 1 0
    //
    int maxScore = 0;
    for (int i = 0; i < n; i++) {
        // mask ith bit and check whether already visited
        if (((mask >> i) & 1) == 1)
            continue;
        for (int j = i + 1; j < n; j++) {
            // mask jth bit and check whether already visited
            if (((mask >> j) & 1) == 1)
                continue;

            // set the ith and jth bit as visited.
            int newMask = mask | (1 << i) | (1 << j);
            // both are same + or |
            // int newMask = mask + (1<<i) + (1<<j);
            int curScore = (index + 1) * gcdArr[i][j];
            int remainingScore = helper(nums, gcdArr, index + 1,
                                         newMask, dp);
            maxScore = Math.max(maxScore, curScore + remainingScore);
        }
    }

    return dp[mask] = maxScore;
}

private int gcd(int x, int y) {
    if (y == 0) {
        return x;
    }

    return gcd(y, x % y);
}
}

```

Complexity:

Time: $O(n * n! / \sum i! (n - i)!)$, where i in $[2, 4, \dots, n - 2]$. On each step, we can choose 2 options from the remaining ones. So, for the first step we can have $C(14, 2)$ options, on the second - $C(14, 4)$ options and so on. Here are the actual numbers for each step 1 to 7 (total 8191 operations):

Problem-5: 2002. Maximum Product of the Length of Two Palindromic Subsequence

Given a string s , find two **disjoint palindromic subsequences** of s such that the **product** of their lengths is **maximized**. The two subsequences are **disjoint** if they do not both pick a character at the same index.

Return the *maximum possible product of the lengths of the two palindromic subsequences*.

A **subsequence** is a string that can be derived from another string by deleting some or no characters without changing the order of the remaining characters. A string is **palindromic** if it reads the same forward and backward.

Example 1:

subsequence1:

s:

l	e	e	t	c	o	d	e	c	o	m
---	---	---	---	---	---	---	---	---	---	---

subsequence2:

Input: $s = \text{"leetcodecom"}$

Output: 9

Explanation: An optimal solution is to choose "ete" for the 1st subsequence and "cdc" for the 2nd subsequence.

The product of their lengths is: $3 * 3 = 9$.

Example 2:Input: $s = \text{"bb"}$

Output: 1

Explanation: An optimal solution is to choose "b" (the first character) for the 1st subsequence and "b" (the second character) for the 2nd subsequence.

The product of their lengths is: $1 * 1 = 1$.

Example 3:Input: $s = \text{"accbcaxxcxx"}$

Output: 25

Explanation: An optimal solution is to choose "acca" for the 1st subsequence and "xxcxx" for the 2nd subsequence.

The product of their lengths is: $5 * 5 = 25$.

Constraints:

- $2 \leq s.length \leq 12$
- s consists of lowercase English letters only.

Solution

```
import java.util.HashMap;
// Runtime: 438 ms, faster than 66.95% of Java online submissions
//for Maximum Product of the Length of Two Palindromic Subsequences.
// Memory Usage: 44.3 MB, less than 37.77% of Java online submissions
//for Maximum Product of the Length of Two Palindromic Subsequences.
class Solution {
    public int maxProduct(String s) {
        HashMap<Integer,Integer> map = new HashMap<>();
        int n = s.length();
        //11 (bit mask) - 2^11

        //l e e t c o d e c o m
        //0 1 2 3 4 5 6 7 8 9 10

        //0 0 0 0 0 0 0 0 0 0 0
        //1 0 0 0 0 0 0 0 0 0 0 -> l
        //0 1 0 1 0 0 0 1 0 0 0 -> ete
        //0 0 0 0 1 0 1 0 1 0 0 -> cdc
        //::::::::::::::::::::
        //1 1 1 1 1 1 1 1 1 1 1 -> whole word
        for(int mask=1;mask < (1<<n) ;mask++){
            StringBuffer buffer = new StringBuffer();
            for(int i =0;i<n;i++){
                //add matching mask bit to current
                //string e.g(eta,cta) to buffer
                //1<<0 = 0000000001(1)
                if( (mask & (1<<i))> 0){
                    buffer.append(s.charAt(n-1-i));
                }
            }

            if(buffer.toString().equals(buffer.reverse().toString())){
                //add mask(key,len of mask)
                map.put(mask, buffer.length());
            }
        }

        int max =0;
        //iterate 2^n 2 times -> O(n)=4^n
        for(int key1 : map.keySet()){
```

```

        for(int key2 : map.keySet()){
            if((key1 & key2 )==0){
                //check whether it is disjoint
                //and not colliding with
                //i.e (eta & cdc) index not colliding.
                int sum = map.get(key1)*map.get(key2);
                max = Math.max(max,sum);
            }
        }
    }

    return max;
}

public static void main(String args[]) {
    int res = new Solution().maxProduct("leetcodecom");
    System.out.println("Solution.main()"+res);
}
}

```

Problem-6: 2212. Maximum Points in an Archery Competition

Alice and Bob are opponents in an archery competition. The competition has set the following rules:

1. Alice first shoots numArrows arrows and then Bob shoots numArrows arrows.
2. The points are then calculated as follows:
 1. The target has integer scoring sections ranging from 0 to 11 **inclusive**.
 2. For **each** section of the target with score k (in between 0 to 11), say Alice and Bob have shot a_k and b_k arrows on that section respectively. If $a_k \geq b_k$, then Alice takes k points. If $a_k < b_k$, then Bob takes k points.
 3. However, if $a_k == b_k == 0$, then **nobody** takes k points.
- For example, if Alice and Bob both shot 2 arrows on the section with score 11, then Alice takes 11 points. On the other hand, if Alice shot 0 arrows on the section with score 11 and Bob shot 2 arrows on that same section, then Bob takes 11 points.

You are given the integer `numArrows` and an integer array `aliceArrows` of size 12, which represents the number of arrows Alice shot on each scoring section from 0 to 11. Now, Bob wants to **maximize** the total number of points he can obtain.

Return *the array* `bobArrows` which represents the number of arrows Bob shot on *each* scoring section from 0 to 11. The sum of the values in `bobArrows` should equal `numArrows`.

If there are multiple ways for Bob to earn the maximum total points, return **any** one of them.

Example 1:

Scoring Section	0	1	2	3	4	5	6	7	8	9	10	11
Alice's Arrows	1	1	-	1	-	-	2	1	-	1	2	-
Bob's Arrows	-	-	-	-	1	1	-	-	1	2	3	1
Points Obtained by Bob	-	-	-	-	4	5	-	-	8	9	10	11

Input: `numArrows = 9, aliceArrows = [1,1,0,1,0,0,2,1,0,1,2,0]`

Output: `[0,0,0,0,1,1,0,0,1,2,3,1]`

Explanation: The table above shows how the competition is scored.

Bob earns a total point of $4 + 5 + 8 + 9 + 10 + 11 = 47$.

It can be shown that Bob cannot obtain a score higher than 47 points.

Example 2:

Scoring Section	0	1	2	3	4	5	6	7	8	9	10	11
Alice's Arrows	-	-	1	-	-	-	-	-	-	-	-	2
Bob's Arrows	-	-	-	-	-	-	-	-	1	1	1	-
Points Obtained by Bob	-	-	-	-	-	-	-	-	8	9	10	-

Input: `numArrows = 3, aliceArrows = [0,0,1,0,0,0,0,0,0,0,0,2]`

Output: `[0,0,0,0,0,0,0,0,1,1,1,0]`

Explanation: The table above shows how the competition is scored.

Bob earns a total point of $8 + 9 + 10 = 27$.

It can be shown that Bob cannot obtain a score higher than 27 points.

Constraints:

- $1 \leq \text{numArrows} \leq 10^5$
- $\text{aliceArrows.length} == \text{bobArrows.length} == 12$
- $0 \leq \text{aliceArrows}[i], \text{bobArrows}[i] \leq \text{numArrows}$
- $\text{sum}(\text{aliceArrows}[i]) == \text{numArrows}$

Solution

```
class Solution {
    public int[] maximumBobPoints(int numArrows, int[] aliceArrows) {
        int maxArr[][] = new int[2][12];
        int bob[] = new int[12];
        int maxsum[] = new int[2];

        //greedy start from index 11.
        helper(numArrows, aliceArrows, bob, maxArr, 0, maxsum, 11, 0);

        //max of pick or not-pick
        return (maxsum[0] > maxsum[1]) ? maxArr[0] : maxArr[1];
    }

    //state - 0 pick
    //        - 1 not pick
    private void helper(int numArrows, int[] aliceArrows, int bob[],
        int[][] maxArr, int sum, int[] maxSum, int index,
        int state) {
        if (numArrows < 0)
            return;
        if (index < 0 || numArrows <= 0) {
            if (numArrows > 0) {
                //if still more Arrows left add to first index.
                bob[0] += numArrows;
            }
            if (sum > maxSum[state]) {
                maxSum[state] = sum;
                maxArr[state] = bob.clone();
            }
        }

        return;
    }

    //add arrows to index 1 more than alice arrows.
    if (numArrows >= aliceArrows[index] + 1) {
        bob[index] = aliceArrows[index] + 1;
        helper(numArrows - aliceArrows[index] - 1, aliceArrows,
            bob, maxArr, sum + index, maxSum, index - 1, 0);
        bob[index] = 0;
    }
}
```

```

        //don't pick
        helper(numArrows, aliceArrows, bob, maxArr, sum, maxSum, index - 1, 1);
        bob[index] = 0;
    }
}

```

Solution-BitMASKING

```

class Solution {
    public int[] maximumBobPoints(int numArrows, int[] aliceArrows) {
        final int allMask = (1 << 12) - 1;
        int maxPoint = 0;
        int maxMask = 0;

        for (int mask = 0; mask < allMask; ++mask) {
            Node node = getShorableAndPoint(mask, numArrows, aliceArrows);
            final boolean shorable = node.isArrowsLeft;
            final int point = node.point;
            if (shorable && point > maxPoint) {
                maxPoint = point;
                maxMask = mask;
            }
        }

        return getBobsArrows(maxMask, numArrows, aliceArrows);
    }

    /**
     * Return node - with whether any left arrows and max
     *                  point scored
     * - add arrows +1 more than aliceArrows
     * @param mask
     * @param leftArrows
     * @param aliceArrows
     * @return
     */
    private Node getShorableAndPoint(int mask, int leftArrows,
                                     int[] aliceArrows) {
        int point = 0;
        for (int i = 0; i < 12; ++i)
            if ((mask >> i & 1) == 1) {
                leftArrows -= aliceArrows[i] + 1;
                point += i;
            }

        return new Node(leftArrows >= 0, point);
    }

    /**
     * To get the bob Arrows array based on alice -
     * get 1 more than alice
     */
}

```



```
    * @param mask
    * @param leftArrows
    * @param aliceArrows
    * @return
    */
    int[] getBobsArrows(int mask, int leftArrows,
        int[] aliceArrows) {
        int[] bobsArrows = new int[12];
        for (int i = 0; i < 12; ++i)
            if ((mask >> i & 1) == 1) {
                bobsArrows[i] = aliceArrows[i] + 1;
                leftArrows -= aliceArrows[i] + 1;
            }
        bobsArrows[0] = leftArrows;
        return bobsArrows;
    }
}

class Node {
    boolean isArrowsLeft;
    int point;

    Node(boolean isArrowsLeft, int point) {
        this.isArrowsLeft = isArrowsLeft;
        this.point = point;
    }
}
```

RECURSION & Backtracking PROBLEMS

BackTracking:

Generally **backtracking does not require memoization** since the testcase size are less try finding all the possibilities with the given condition and if it satisfies all the condition stop the recursion

Returning cumulative values in recursion:

```
public class RecursiveExample {
    public static int cumulativeSum(int[] arr, int index, int sumSoFar) {
        if (index >= arr.length) {
            return sumSoFar;
        }
        int currentElement = arr[index];
        int newSum = sumSoFar + currentElement;
        return cumulativeSum(arr, index + 1, newSum);
    }

    public static void main(String[] args) {
        int[] arr = { 1, 2, 3, 4 };
        int totalSum = cumulativeSum(arr, 0, 0);
        System.out.println("Cumulative Sum: " + totalSum);
    }
}
```

cumulative values in recursion:

Identifying whether to return a **cumulative value or a value for a single case in the base case of a recursive function** depends on the problem's nature and what the base case represents in the context of the problem. Here are some considerations that can help you make this decision:

1. Nature of the Problem:

- If the problem inherently involves calculating some form of cumulative result or aggregation as you traverse the recursion, then returning a cumulative value in the base case might be more suitable.
- If the problem involves evaluating a condition or producing a single result for a specific element or state, returning a value for a single case in the base case could be more appropriate.

2. Problem Description:

- Carefully read and understand the problem description. Pay attention to the base case scenario and what the desired output is for that case.

- Consider whether the cumulative sum or a value for a single element aligns with the problem's goal.

3. Logic and Recursive Calls:

- Examine how the recursive logic unfolds. If the base case contributes to the cumulative result you're trying to calculate, returning a cumulative value could make sense.
- If the recursive calls lead to a single value or condition that represents a solution, returning that value for a single case in the base case might be more suitable.

4. Real-World Interpretation:

- Think about how the base case translates to a real-world scenario. Does the cumulative value make sense in that context? Does the single value align with the problem's requirements?

5. Examples and Patterns:

- Look at examples and patterns within the problem. Analyze how the values evolve as you go through the recursion. This might give you insight into whether cumulative or single values are more appropriate.

6. Test Cases:

- Consider different test cases and their outcomes. Think about what the base case should return to satisfy the problem's requirements for each test case.

7. Discuss with Others:

- Discussing the problem with peers or mentors can provide different perspectives and insights that might help you decide on the most appropriate approach.

Remember that every problem is unique, and the decision of whether to return cumulative or single values in the base case depends on the specific problem's characteristics. Over time and with practice, you'll become more adept at identifying the appropriate approach based on your experience and problem-solving skills

Choose whether to pass values in function or as global variable:

Deciding whether to calculate a sum by adding to a variable outside the recursion or passing the sum inside the recursive function involves considering factors such as clarity, modularity, and the problem's

requirements. Let's explore both approaches and the considerations for choosing between them:

Approach 1: Accumulate Sum in a Variable Outside Recursion

In this approach, you maintain a variable outside the recursive function to accumulate the sum. The recursive function updates this variable with the sum as it processes elements.

Considerations for Choosing an Approach:

1. **Clarity and Readability:** The approach that leads to clearer and more readable code is generally preferable. Choose the approach that makes the code's intent and logic more understandable.
2. **Modularity:** If the sum calculation is closely tied to the recursive process, passing the sum inside the recursive function can encapsulate the logic and make the function more self-contained.
3. **Global State:** Accumulating a sum in a variable outside the recursive function might introduce global state, which can make code harder to reason about and maintain.
4. **Function Purity:** Passing the sum inside the function makes the function more self-contained and doesn't rely on external variables, which can align with the principles of functional programming.
5. **Parameter Management:** Passing the sum as a parameter might lead to a slightly larger number of parameters, but it's generally manageable and often improves code organization.
6. **Consistency:** Consider the overall coding style of your project or team. Choose the approach that maintains consistency with your team's practices.

Both approaches are valid, and the choice depends on the specific problem, code design principles, and readability. Ultimately, strive for code that is easy to understand, maintain, and modify.

Approach 1: Passing Parameters Inside the Function

In this approach, you pass the necessary parameters, such as the count, as arguments to the recursive function. The function updates the parameters as it traverses through the recursion.

Advantages:

- **Modularity:** Each recursive call operates independently with its own set of parameters, making the function more self-contained and modular.

- **Clearer Code:** The explicit passing of parameters helps clarify what information is being used and manipulated within the function.
- **No Side Effects:** This approach avoids side effects and potential interference with other parts of the code, as parameters are explicitly passed and not modified globally.

When to Choose:

- Choose this approach when you want to keep each recursive call self-contained, especially when multiple parameters are involved.
- It's a good choice when you prioritize clear code organization and avoiding global state.

Example:

```
public static int countElements(int[] arr, int index,
int target, int countSoFar) {
    if (index >= arr.length) {
        return countSoFar;
    }
    if (arr[index] == target) {
        countSoFar++;
    }
    return countElements(arr, index + 1, target, countSoFar);
}
```

Approach 1: Optimization:How Tail Recursion (with Accumulator)

To make use of tail call optimization, a function must use *tail recursion*. In tail recursion, the recursive function call is the last action of a recursive function. In code, this looks like a return statement returning the results of a recursive call.

```
function factorial(number, accum=1) {
    if (number === 1) {
        // BASE CASE
        return accum;
    } else {
        // RECURSIVE CASE
        return factorial(number - 1, accum * number);
    }
}
```

A stack overflow happens when too many function calls are made without returning, causing the number of frame objects to exceed the capacity

of the call stack. This capacity is 1,000 function calls for Python and about 10,000 for JavaScript programs. While these amounts are more than enough for typical programs, recursive algorithms could exceed this limit and cause a stack overflow that crashes your program.

As the frame object stores the local variables in the function call as well as the return address of the instruction to return to when the function finishes. However, if the last action in the recursive case of a function is to return the results of a recursive function call, there's no need to retain the local variables. The function does nothing involving the local variables after the recursive call, so the current frame object can be deleted immediately. **The next frame object's return address information can be the same as the old deleted frame object's return address**

Since the current frame object is deleted instead of retained on the call stack, the call stack never grows in size and can never cause a stack overflow!

Accumulators in Tail Recursion

The disadvantage of tail recursion is that it requires rearranging your recursive function so that the last action is returning the recursive call's return value.

In the case of our tail call factorial() function, a new parameter named **accum** follows the calculated product as recursive function calls are made. This is known as an **accumulator** parameter, and it keeps track of a **partial result of a calculation** that would **otherwise** have been stored in a **local variable**. **Not all tail recursive functions use accumulators, but they act as a workaround for tail recursion's inability to use local variables after the final recursive call.**

```
function factorial(number) {  
    if (number == 1) {  
        // BASE CASE  
        return 1;  
    } else {  
        // RECURSIVE CASE  
        return number * factorial(number - 1);  
    }  
}
```

Changed to below to facilitate tail recursion:

```
function factorial(number, accum = 1) {  
  if (number === 1) {  
    // BASE CASE  
    return accum;  
  } else {  
    // RECURSIVE CASE  
    return factorial(number - 1, accum * number);  
  }  
}
```

Issue or Disadvantage of Tail Recursion:

Since tail call optimization removes the need for a call stack, we are effectively using recursion to simulate a loop's iterative code

Without a call stack, no tail recursive function could possibly do any backtracking work. In my view, every algorithm that can be implemented with tail recursion would be easier and more readable to implement with a loop instead. There's nothing automatically more elegant about using recursion for recursion's sake.

Personally, I take the stance that the **tail recursion technique should never be used**. any recursive algorithm can be implemented with a loop and a stack. Tail call optimization prevents stack overflows by effectively removing the use of the call stack. Therefore, **all tail recursive algorithms can be implemented with a loop alone**. Since the code for loops is much simpler than a recursive function, a loop should be used wherever tail call optimization could be employed.

Additionally, potential problems exist even if tail call optimization is implemented. Since tail recursion is possible only when the last action of a function is returning the recursive call's return value, **it's impossible to do tail recursion for algorithms that require two or more recursive calls**. For example, take the Fibonacci sequence algorithm calls fibonacci(n - 1) and fibonacci(n - 2). **While tail call optimization can be performed for the latter recursive call, the first recursive call will cause a stack overflow for large-enough arguments.**

Limitations of Tail Recursion

Tail recursive functions require rearranging their code to make them suitable for the tail call optimization feature of the compiler or interpreter. However, not all compilers and interpreters offer tail call optimization as a feature.

Tail Recursive Exponents

Below programs have two recursive cases for when the `n` parameter is even or odd. This is fine: as long as all the recursive cases return the return value of the recursive function call as their last action, the function can use tail call optimization.

```
function exponentByRecursion(a, n) {
  if (n == 1) {
    return a;
  }
  else if (n % 2 === 0) {
    // RECURSIVE CASE (when n is even)
    result = exponentByRecursion(a, n / 2);
    return result * result;

  } else if (n % 2 == 1) {

    result = exponentByRecursion(a, Math.floor(n / 2));
    return result * result * a;
  }
}
```

This recursive case does not have the recursive call as its last action. We could get rid of the `result` local variable, and instead call the recursive function twice. This would reduce the recursive case to the following:

```
return exponentByRecursion(a, n / 2) * exponentByRecursion(a, n / 2).
```

now we have two recursive calls to `exponentByRecursion()`. Not only does this needlessly double the amount of computation the algorithm performs, but the last action performed by the function is to multiply the two return values.

TailRecursion Case Studies

Tail Recursive Odd-Even

To determine whether an integer is odd or even, you can use the modulus operator. The expression `number % 2 == 0` will be `True` if number is even, and `False` if number is odd. However, if you'd prefer to overengineer a more "elegant" recursive algorithm

```
function isOdd(number) {
  if (number === 0) {
    // BASE CASE
    return false;
  } else {
```

```
    // RECURSIVE CASE
    return !isOdd(number - 1);
  }
}
```

Above code - Calling `isOdd(100000)` results in 100,001 function calls without returning—which far exceeds the capacity of any call stack. We can rearrange the code in the function so that the last action of the recursive case is returning the results of the recursive function call, making the function tail recursive

Tail Recursion Optimization used in below code:

```
function isOddTailCall(number, inversionAccum) {
  if (inversionAccum === undefined) {
    inversionAccum = false;
  }

  if (number === 0) {
    // BASE CASE
    return inversionAccum;
  } else {
    // RECURSIVE CASE
    return isOddTailCall(number - 1, !inversionAccum);
  }
}
```

JavaScript code is run by an interpreter that supports tail call optimization, calling `isOddTailCall(100000)` won't result in a stack overflow

Approach 2: Adding within the Recursive Method

In this approach, the recursive function returns a value for each call, and you add these values as the recursive calls unwind, typically in the calling code.

Advantages:

- **Simplicity:** This approach keeps the recursive function cleaner and simpler, without the need to pass additional parameters.
- **Elegant Code:** The recursive function focuses solely on its task, making it more elegant and easy to read.
- **Reduced Parameter Complexity:** It's suitable when you have a single parameter to accumulate, simplifying the function signature.

When to Choose:

- Choose this approach when the recursive logic itself remains simple and the accumulation doesn't require complex parameter interactions.
- It's appropriate when you want to keep the recursive function's signature clean and straightforward.

Example:

```
public static int countElements(int[] arr, int index, int target) {  
    if (index >= arr.length) {  
        return 0;  
    }  
    int count = 0;  
    if (arr[index] == target) {  
        count = 1;  
    }  
    return count + countElements(arr, index + 1, target);  
}
```

Choosing Between the Approaches: Choose the approach that aligns with the nature of your problem, code organization preferences, and the complexity of parameter interactions. Both approaches are valid, and the choice depends on the specific context of your problem and your coding style.

Identify Backtracking vs Recursion Problems

Distinguishing between a backtracking problem and a recursion with memoization problem can sometimes be challenging, but there are a few key characteristics and patterns you can look for to help identify which type of problem you're dealing with:

Backtracking:

1. **Exploring Choices:** Backtracking often involves exploring multiple choices at each step to find a solution.
2. **Exploration and Undoing:** It typically involves exploring one path, and if it doesn't lead to a valid solution, backtracking to a previous state and trying a different path.
3. **Exponential Complexity:** Backtracking problems can have exponential time complexity due to the branching of possibilities.
4. **No Shortcuts:** Backtracking often doesn't have "shortcuts" or subproblems that overlap, meaning that recomputing is necessary.
5. **Examples:** Problems like N-Queens, Sudoku, and solving mazes often involve backtracking.

Recursion with Memoization:

1. **Overlapping Subproblems:** Problems that can benefit from memoization have overlapping subproblems, meaning the same subproblem is solved multiple times.
2. **Optimal Substructure:** They often have optimal substructure, meaning that solving a larger problem involves solving smaller subproblems.
3. **Reduced Redundancy:** Memoization reduces redundant computation by storing previously computed results.
4. **Dynamic Programming:** Many memoization problems can also be solved using dynamic programming bottom-up approach.
5. **Examples:** Fibonacci sequence, Coin Change problem, Longest Common Subsequence, etc.

Identifying Strategies:

1. **Read the Problem Carefully:** Pay attention to whether the problem description suggests exploring choices and backtracking or optimizing with subproblems and memoization.
2. **Look for Recomputation:** If you notice that the same problem is being solved multiple times with the same inputs, memoization is likely beneficial.
3. **Analyze Time Complexity:** If the problem exhibits an exponential increase in time complexity, it's more likely to be a backtracking problem.
4. **Focus on Subproblems:** If the problem naturally breaks down into smaller subproblems, it's a clue that memoization could be useful.
5. **Dynamic Programming:** If you can build up solutions for larger problems from smaller ones, dynamic programming (with memoization) might be applicable.

Solve Recurrence Relation

Arriving at a recurrence relation for a complex problem involves breaking down the problem into smaller subproblems and identifying how the solution to the larger problem depends on the solutions to those subproblems. Here's a step-by-step process using a complex problem as an example:

Example Problem: Counting Beautiful Subsets

Let's consider the "Counting Beautiful Subsets" problem you mentioned earlier. Given an array `nums` and a positive integer `k`, you need to find the number of non-empty beautiful subsets of the array. A subset is beautiful if it doesn't contain two integers with an absolute difference equal to `k`.

Step 1: Identify Base Cases and Subproblems

Start by identifying base cases and how the problem can be divided into smaller subproblems:

- **Base Case:** When the array is empty, the count is 0 (an empty subset is not beautiful).
- **Subproblems:** You can solve subproblems by considering whether to include or exclude the current element from the subset.

Step 2: Describe the Recurrence Relation

Next, describe how the solution to the larger problem depends on the solutions to the subproblems. In this case, you need to account for both the choices of including and excluding the current element:

- If you include the current element, the problem reduces to counting beautiful subsets for the remaining elements, with the constraint that the difference between any two included elements is not k .
- If you exclude the current element, the problem reduces to counting beautiful subsets for the remaining elements without any additional constraint.

Step 3: Write the Recurrence Relation

Translate the relationship described in Step 2 into a mathematical recurrence relation:

Let `countBeautifulSubsets(nums, index, k)` be the function that returns the count of beautiful subsets starting from index `index` in the array `nums`.

- **Base Case:** `countBeautifulSubsets(nums, index, k)` returns 0 if `index` is beyond the bounds of the array.
- **Recursive Case:**
 - If you include `nums[index]` in the subset:
`countBeautifulSubsets(nums, index + 1, k)` (with constraint that difference is not k)
 - If you exclude `nums[index]` from the subset:
`countBeautifulSubsets(nums, index + 1, k)` (without constraint)

Now, you have a recurrence relation that describes how the problem is solved based on smaller subproblems.

Step 4: Solve the Recurrence

Solving the recurrence might involve solving for specific cases, using techniques like memoization or dynamic programming, or analyzing the complexity further.

In this problem, due to the presence of multiple recursive calls and constraints, the solution may be quite complex. Solving the recurrence relation might require a combination of mathematical analysis and optimization techniques.

Remember, this is a simplified explanation, and real-world problems may involve additional complexities. Practice and experience will help you improve your ability to arrive at accurate recurrence relations for complex problems.

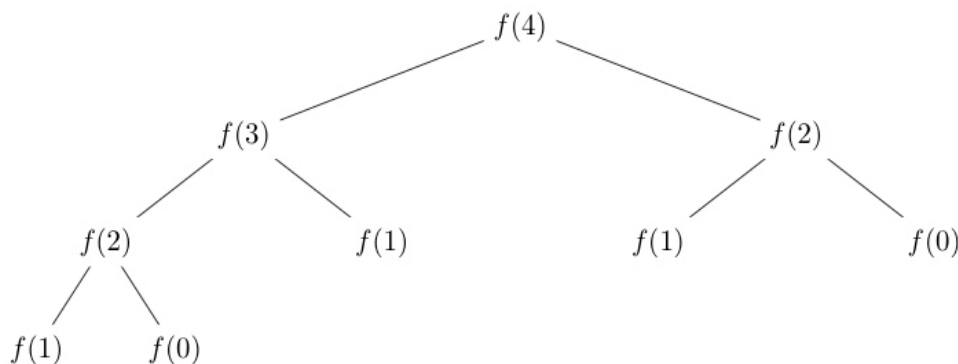
Calculate Time Complexity of Recursion:

Remember that some problems might exhibit characteristics of both backtracking and memoization. In such cases, carefully analyzing the nature of the problem and its requirements will help you determine the appropriate approach.

Example: Fibonacci Sequence

The Fibonacci sequence is defined as follows: $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n > 1$.

Here's a naive recursive implementation:



```

public int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
  
```

Estimating Time Complexity:

In this example, at each recursive call, we make two more recursive calls: one for $n - 1$ and one for $n - 2$. This leads to an exponential growth in the number of recursive calls.

1. Number of Recursive Calls: At each level of the recursion, the number of calls doubles (approximately). This is because each recursive call branches into two more calls.

2. Work per Call: The work done in each call is relatively small. It involves simple addition and return operations.

3. Recurrence Relation: Let $T(n)$ be the number of recursive calls for the Fibonacci function with input n .

- $T(0) = 1$ (base case)
- $T(1) = 1$ (base case)
- $T(n) = T(n - 1) + T(n - 2) + 1$ (for $n > 1$, including the current call)

4. Solve the Recurrence: Solving this recurrence relation isn't straightforward and might require mathematical tools beyond simple substitution. However, it's clear that the growth is exponential due to the repeated branching. The time complexity is approximately exponential: $O(2^n)$.

Conclusion: The naive recursive Fibonacci algorithm has an exponential time complexity due to the exponential branching of recursive calls. This is a classic example of an algorithm where the inefficiency of the recursive approach becomes evident, and optimization techniques like memoization or dynamic programming are often used to improve the performance.

Backtracking and Branch & Bound

Backtracking and branch and bound are both techniques used in computer science and mathematics to solve optimization and search problems, but they differ in their approaches and applications. Let's explore the key differences between these two techniques:

1. Objective:

- **Backtracking:** Backtracking is primarily used for solving problems where you need to find one or more solutions within a search space. It's often employed for combinatorial problems like the N-Queens problem or Sudoku, where you want to find a single valid solution or enumerate all possible solutions.
- **Branch and Bound:** Branch and bound is used for optimization problems, where the goal is to find the best solution (maximize or minimize an objective function) within a search space. It's commonly used in problems like the traveling salesman problem or the knapsack problem.

2. Search Strategy:

- **Backtracking:** Backtracking explores the search space recursively by trying out various options and undoing choices if they lead to dead-ends. It systematically explores the entire search space and often involves depth-first search. It may not guarantee finding the optimal solution for optimization problems.
- **Branch and Bound:** Branch and bound divides the search space into subproblems and uses an upper and lower bound strategy to prune branches that are guaranteed to not contain optimal solutions. It actively seeks to find the best solution by pruning unpromising branches early in the search.

3. Termination:

- **Backtracking:** Backtracking terminates when it finds a valid solution or exhausts the entire search space, depending on the specific problem. It may not always find the optimal solution.
- **Branch and Bound:** Branch and bound terminates when it has found the optimal solution or determined that it's impossible to improve upon the current best solution. It guarantees an optimal solution for optimization problems.

4. Memory Usage:

- **Backtracking:** Backtracking usually uses less memory because it explores one branch at a time and often only stores a small amount of information about the current state.
- **Branch and Bound:** Branch and bound can use more memory because it maintains a priority queue or a stack of subproblems to explore. The memory usage can grow with the size of the search space.

5. Applications:

- **Backtracking:** Backtracking is suitable for problems like permutations, combinations, and constraint satisfaction problems. It's used when you need to enumerate or find all valid solutions within a search space.
- **Branch and Bound:** Branch and bound is used for optimization problems in various fields, such as operations research, logistics, and network design, where finding the best solution is essential.

In summary, backtracking and branch and bound are problem-solving techniques with distinct objectives and strategies. Backtracking is employed for finding one or more valid solutions within a search space, while branch and bound are

used to optimize an objective function by efficiently exploring the search space and guaranteeing the optimal solution. The choice between these techniques depends on the nature of the problem you are trying to solve.

Code:

```
import java.util.Arrays;

public class Solution {
    public static double knapsackBranchBound(int[] values,
        int[] weights, int capacity) {
        int n = values.length;
        Integer[] indices = new Integer[n];
        for (int i = 0; i < n; i++) {
            indices[i] = i;
        }
        Arrays.sort(indices, (a, b) -> {
            double val1 = (double) values[b] / weights[b];
            double val2 = (double) values[a] / weights[a];

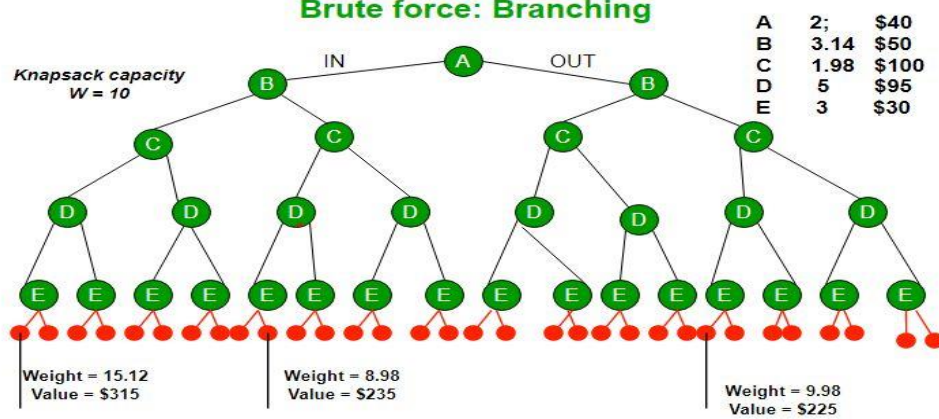
            return Double.compare(val1, val2);
        });
        int currentWeight = 0;
        double bound = 0.0;
        double maxValue = 0.0;
        for (int i = 0; i < n; i++) {
            if (currentWeight + weights[indices[i]] <= capacity) {
                currentWeight += weights[indices[i]];
                maxValue += values[indices[i]];
            } else {
                double remainingCapacity = capacity - currentWeight;
                double cval =
                    (double) values[indices[i]] / weights[indices[i]];
                bound = maxValue + (remainingCapacity * cval);
                break;
            }
        }

        return bound;
    }

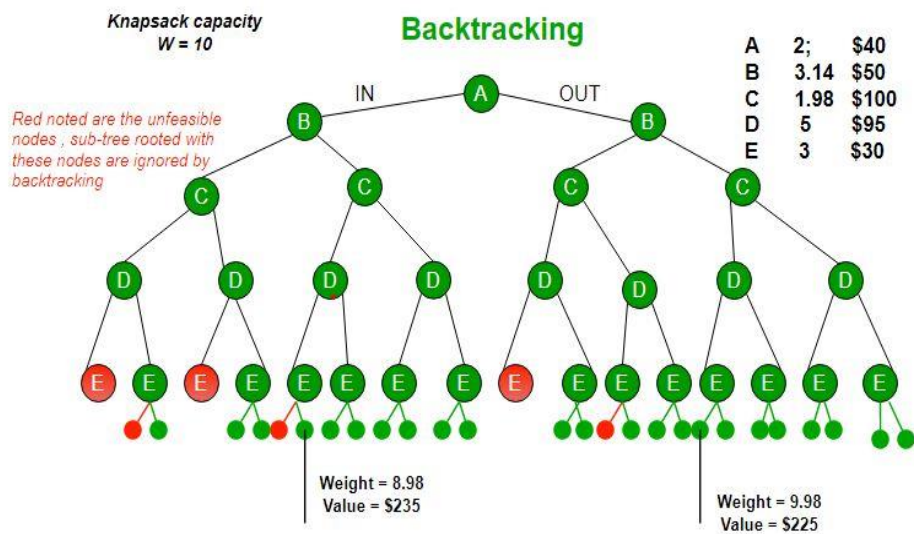
    public static void main(String[] args) {
        int[] values = { 60, 100, 120 };
        int[] weights = { 10, 20, 30 };
        int capacity = 50;

        double maxKnapsackValue = knapsackBranchBound(values,
            weights, capacity);
        System.out.println("Maximum knapsack: " + maxKnapsackValue); //240.0
    }
}
```

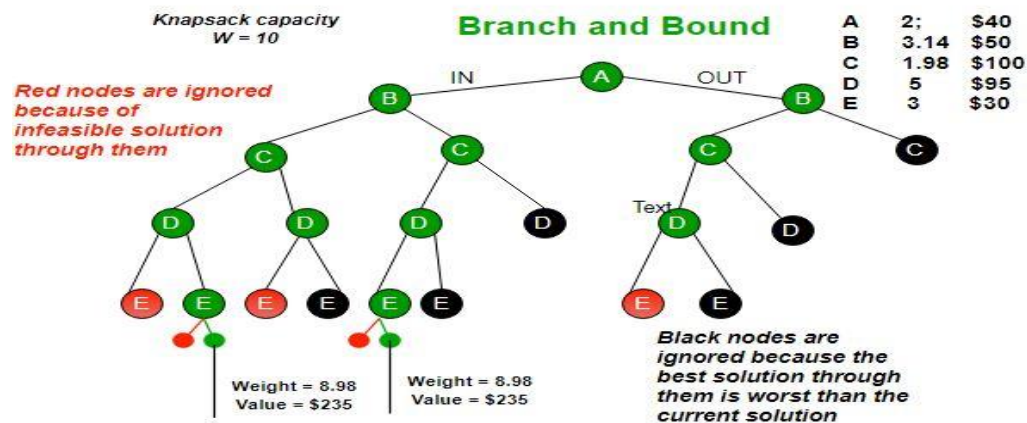
Brute force: Branching



Backtracking



Branch and Bound



Branch and Bound The backtracking based solution works better than brute force by ignoring infeasible solutions. We can do better (than backtracking) if we know a bound on best possible solution subtree rooted with every node. If the best in subtree is worse than current best, we can simply ignore this node and its subtrees. So we compute bound (best solution) for every node and compare the bound with current best solution before exploring the node. Example bounds used in below diagram are, A down can give \$315, B down can \$275, C down can \$225, D down can \$125 and E down can \$30. In the [next article](#), we have discussed the process to get these bounds.

Branch and bound is very useful technique for searching a solution but in worst case, we need to fully calculate the entire tree. At best, we only need to fully calculate one path through the tree and prune the rest of it.

0/1 Knapsack problem using branch and bound doesn't include the recursive approach typically associated with backtracking algorithms. Instead, it uses an iterative approach with a branch-and-bound strategy. This is a common way to solve the 0/1 Knapsack problem efficiently.

Problem-1.39. Combination Sum(BACKTRACKING)

Given an array of **distinct** integers candidates and a target integer target, return a list of all **unique combinations** of candidates where the chosen numbers sum to target. You may return the combinations in **any order**.

The **same** number may be chosen from candidates an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Solution

//Runtime: 1 ms, faster than 100.00% of Java online submissions for Combination Sum.

```
//Memory Usage: 43.8 MB, less than 72.42% of Java online submissions for Combination
Sum.
import java.util.ArrayList;
import java.util.List;
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        // Arrays.sort(candidates);
        //no sorting
        List<List<Integer>> result = new ArrayList<>();

        //          [2,3,6,7]
        //          /      \
        //          [2](0,5)  [7](0,7)
        //          /
        //          [2,2](0,3)
        //          /
        //          [2,2,3](0,0)
        //          /
        List<Integer> list = new ArrayList<>();
        helper(candidates,result,list,0,target);
        return result;
    }

    private void helper(int[] candidates, List<List<Integer>> result,
List<Integer> list, int index, int target) {
        int n = candidates.length;
        if (target == 0) {
            result.add(new ArrayList<Integer>(list));
            return;
        }
        if (index == n || target<0) {
            return;
        }
        if (target - candidates[index] >= 0) {
            list.add(candidates[index]);
            helper(candidates, result, list,
                index, target - candidates[index]);
            list.remove(list.size()-1); // backtrack
        }
        // ignore move to next
        helper(candidates, result, list, index + 1, target);
    }
}

```

NOTES

1. No Sorting required, since going to find combination which make target
2. Branch recursion with pattern- include & reduce the target or increase index.

3. Make base case `target==0`(if include) and `index==n`(if exclude)
4. Since we need either include or exclude at each step no need of loop.
5. Need backtracking for cases like `[[2,2,2,2],[2,3,3]]` after Making combination `[[2,2,2,2]]` backtrack and add `[2,3,3]`

```
//
//           0   1   2
//           [2, 3, 5] : target =8
//           /      \
//       [2](0,2)    [3] (1,5)
//           /      \
//       [2,2](0,4)  [2,3] (2,0)
//           /      \
//       [2,2,2](0,6) [2,2,3](1,1)
//           /      \
//       [2,2,2,2](0,8) [2,2,3,3](1,-2)
//           /      \
//       (target)    target <0
```

Problem-2.40. Combination Sum II(BACKTRACKING)

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

Input: candidates = [10,1,2,7,6,1,5], target = 8

Output:

```
[
[1,1,6],
[1,2,5],
[1,7],
[2,6]
]
```

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

```
[
[1,2,2],
]
```

```
[5]
]
```

Solution

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

class Solution {
// Runtime: 2 ms, faster than 99.73% of Java online submissions for Combination Sum II.
// Memory Usage: 42 MB, less than 94.83% of Java online submissions for Combination Sum II.
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {

        //          [10,1,2,7,6,1,5]
        //          0  1 2 3 4 5 6
        //          [1, 1,2,5,6,7,10], target = 8
        //          /                \
        //          [1](0,7)           [2,5](4,0)
        //          /                \
        //          [1,1](1,6)         [1,7](3,0)
        //          /                \
        //          [1,1,2] (2,5)      \
        //          /                \
        //          [1,1,2,5](5,0)      [1,1,5](4,0) (backtrack)
        //          /
        //          // exceed target break

        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        List<Integer> list = new ArrayList<>();
        helper(candidates,result,list,0,target);

        return result;
    }

    private void helper(int candidates[], List<List<Integer>> result,
        List<Integer> list, int index, int target) {
        int n = candidates.length;
        if (target == 0) {
            result.add(new ArrayList<Integer>(list));
            return;
        }
        if (index == n || target < 0) {
            return;
        }
    }
}
```

```

    for (int i = index; i < n; i++) {
        if (i != index && candidates[i] == candidates[i - 1])
            continue;
        // since sorted if current exceeds target ,
        // further values must be > current index so no need to check
        // other indexes to avoid branch out stack.
        if (target - candidates[i] < 0)
            break;
        list.add(candidates[i]);
        helper(candidates, result, list, i + 1, target - candidates[i]);
        list.remove(list.size() - 1); //backtracking
    }
}

```

TimeComplexity: $O(2^n \cdot k)$

NOTE:

- Index refer to stack index and i refer to local index.

Problem-3: 1723. Find Minimum Time to Finish All Jobs (Backtracking)

You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the i^{th} job.

There are k workers that you can assign jobs to. Each job should be assigned to **exactly** one worker. The **working time** of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the **maximum working time** of any worker is **minimized**.

Return the minimum possible maximum working time of any assignment.

Example 1:

Input: jobs = [3,2,3], k = 3

Output: 3

Explanation: By assigning each person one job, the maximum time is 3.

Example 2:

Input: jobs = [1,2,4,7,8], k = 2

Output: 11

Explanation: Assign the jobs the following way:

Worker 1: 1, 2, 8 (working time = $1 + 2 + 8 = 11$)

Worker 2: 4, 7 (working time = $4 + 7 = 11$)

The maximum working time is 11.

Constraints:

- $1 \leq k \leq \text{jobs.length} \leq 12$
- $1 \leq \text{jobs}[i] \leq 10^7$

Recurrence:

- 1) Take first worker and assign tasks to him
- 2) For e.g for [1,2,4,7,8] and k=2 worker1 will be initially 22 then backtrack
- 3) For each worker1 accumulate task reduce and find minimum

Solution(BACKTRACKING & update Global Shared Object)

```
import java.util.Arrays;
```

```
class Solution {  
    // Runtime: 26 ms, faster than 61.03% of Java online submissions for  
    // Find Minimum Time to Finish All Jobs.  
    // Memory Usage: 43.7 MB, less than 15.07% of Java online  
    // submissions for Find Minimum Time to Finish All Jobs.  
    //      0 1 2 3      4  
    //      [1,2, 4, 7, 8], k = 2  
    //      /          \  
    //      1(0,w1)    4(2,w2) - worker2  
    //      /          \  
    //      3(1,w1)    11(3,w2) [4,7]remaining - worker2  
    //      /          \  
    //      11 (2,w1)  
    //  
    public int minimumTimeRequired(int[] jobs, int k) {  
        int length = jobs.length;  
        // Arrays.sort(jobs);  
        int res[] = new int[1];  
        res[0] = Integer.MAX_VALUE;  
        backtrack(jobs, length - 1, new int[k], res);  
        return res[0];  
    }  
}
```



```
public void backtrack(int[] jobs, int current, int[] workers, int res[]) {
    if (current < 0) {
        int curmax = Arrays.stream(workers).max().getAsInt();
        res[0] = Math.min(res[0], curmax);
        return;
    }
    int curmax = Arrays.stream(workers).max().getAsInt();
    if (curmax >= res[0])
        return;

    for (int i = 0; i < workers.length; i++) {
        // pruning
        if (i > 0 && workers[i] == workers[i - 1]) {
            continue;
        }
        // make choice
        workers[i] += jobs[current];
        // backtrack
        backtrack(jobs, current - 1, workers, res);
        // undo the choice
        workers[i] -= jobs[current];
    }
}
```

Problem-4: 698. Partition to K Equal Sum Subsets(BACKTRACKING,BUCKET,BITMASK)

Given an integer array nums and an integer k, return true if it is possible to divide this array into k non-empty subsets whose sums are all equal.

Example 1:

Input: nums = [4,3,2,3,5,2,1], k = 4

Output: true

Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.

Example 2:

Input: nums = [1,2,3,4], k = 3

Output: false

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 16$
- $1 \leq \text{nums}[i] \leq 10^4$
- The frequency of each element is in the range $[1, 4]$.

Approach:

```
//          0          1  2  3  4  5  6
//      [ 4,          3 ,2, 3,  5,  2,  1] sum=20,k=4 ,avg=5
//      /      |      \      |      |      |
//      /      |      \      |      |      |
//      3      2:::1(index=6)  |      |      |
//      /      |      \      |      |      |
//      x(7>5)  x(6>5)  5(met avg)  5(met)  5(met avg)
```

Solution(BACKTRACKING)

```
class Solution {
// Runtime: 1697 ms, faster than 11.88% of Java online submissions
// for Partition to K Equal Sum Subsets.
// Memory Usage: 39.3 MB, less than 99.71% of Java online
// submissions for Partition to K Equal Sum Subsets.
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int n = nums.length;
        int total = 0;
        for (int i = 0; i < n; i++) {
            total += nums[i];
        }

        int avg = total / k;
        //if avg % total not divisible then can't be split to k blocks
        if (total % avg != 0) {
            return false;
        }
        boolean visited[] = new boolean[n];

        return helper(nums, visited, 0, 0, avg, k);
    }
}
```

```

// [4,3,2,3,5,2,1]
//
private boolean helper(int nums[], boolean visited[],
    int sum, int index, int avg, int k) {
    int n = nums.length;
    //last partition
    if (k == 0) {
        return true;
    }
    if (sum > avg)
        return false;

    if (sum == avg) {
        return helper(nums, visited, 0, 0, avg, k - 1);
    }

    for (int i = index; i < n; i++) {
        if (visited[i]) {
            continue;
        }
        visited[i] = true;
        if (helper(nums, visited, sum + nums[i],
            i + 1, avg, k)) {
            return true;
        }
        visited[i] = false;
    }

    return false;
}
}

```

Approach-2:Solution(ADD TO BUCKET till reach k)

```

class Solution {
//Runtime: 1495 ms, faster than 24.36% of Java online submissions
//for Partition to K Equal Sum Subsets.
//Memory Usage: 40 MB, less than 70.30% of Java online submissions
//for Partition to K Equal Sum Subsets.
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int n = nums.length;
        int total = 0;
        for (int i = 0; i < n; i++) {
            total += nums[i];
        }
    }
}

```

```

    int avg = total / k;
    //if avg % total not divisible then can't be split to k buckets
    if (total % avg != 0) {
        return false;
    }
    boolean visited[] = new boolean[n];

    return helper(nums, visited, 0, 0, avg, k);
}

// [4,3,2,3,5,2,1]
//
private boolean helper(int nums[], boolean visited[],
    int bucket, int index, int avg, int k) {
    if (k == 0) {
        return true;
    }
    if (bucket == avg) {
        return helper(nums, visited, 0, 0, avg, k-1);
    }

    for (int i = index; i < nums.length; i++) {
        if (visited[i]) {
            continue;
        }
        if (nums[i] + bucket > avg) {
            continue;
        }
        visited[i] = true;
        //add to bucket till avg
        bucket += nums[i];
        if (helper(nums, visited, bucket, i+1, avg, k)) {
            return true;
        }
        visited[i] = false;
        //remove from current bucket if not reaches avg
        bucket -= nums[i];
    }
    return false;
}
}

```

Approach-3:Solution(BIT Mask and cache States +k times BUCKET meets target)

```

import java.util.HashMap;

class Solution {
// Runtime: 38 ms, faster than 73.64% of Java online submissions
// for Partition to K Equal Sum Subsets.
// Memory Usage: 43.5 MB, less than 25.13% of Java online submissions
// for Partition to K Equal Sum Subsets.
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int n = nums.length;
        int total = 0;
        for (int i = 0; i < n; i++) {
            total += nums[i];
        }

        int avg = total / k;
        // if avg % total not divisible then can't be split to k blocks
        if (total % avg != 0) {
            return false;
        }
        HashMap<Integer, Boolean> cache = new HashMap<>();
        return helper(nums, 0, cache, 0, 0, avg, k);
    }

// [4,3,2,3,5,2,1]
//
    private boolean helper(int nums[], int visited,
        HashMap<Integer, Boolean> cache,
        int bucket, int index, int avg, int k) {
        if (k == 0) {
            return true;
        }

        if (bucket == avg) {
            boolean res = helper(nums, visited, cache, 0, 0, avg, k-1);
            cache.put(visited, res);
            return res;
        }
        //cache all states
        if (cache.containsKey(visited)) {
            return cache.get(visited);
        }
    }
}

```

```

    for (int i = index; i < nums.length; i++) {
        if ((visited >> i & 1) == 1) {
            continue;
        }
        if (nums[i] + bucket > avg) {
            continue;
        }
        bucket += nums[i];
        visited |= 1 << i;
        if (helper(nums, visited, cache, bucket, i + 1, avg, k)) {
            return true;
        }
        bucket -= nums[i];
        visited ^= 1 << i;
    }
    return false;
}
}

```

Problem-5: 93. Restore IP Addresses(Reduce by Index vs Unprocessed String)

A **valid IP address** consists of exactly four integers separated by single dots. Each integer is between 0 and 255 (**inclusive**) and cannot have leading zeros.

- For example, "0.1.2.201" and "192.168.1.1" are **valid** IP addresses, but "0.011.255.245", "192.168.1.312" and "192.168@1.1" are **invalid** IP addresses.

Given a string *s* containing only digits, return *all possible valid IP addresses that can be formed by inserting dots into s*. You are **not** allowed to reorder or remove any digits in *s*. You may return the valid IP addresses in **any** order.

Example 1:

2

Input: *s* = "25525511135"

Output: ["255.255.11.135", "255.255.111.35"]

- 1 <= *s*.length <= 20
- s* consists of digits only.

SolutionApproach 1: Reduce subproblem by index:

```
// 2 5 5 2 5 5 1 1 1 3 5
// 0 1 2 3 4 5 6 7 8 9 10

// 2 5 5|2 5 5|1 1 1|3 5
// 2 5 5|2 5 5|1 1 1|3 5

//          2(0,"2") - (index, string)
//          /
//          5(1,"25")
//          /
//          5 (2,"255")
//          / \
//          x  2(3,"255.2")
//          \
//          5(4,"255.25")
```

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;

class Solution {
// Runtime: 11 ms, faster than 9.98% of Java
// online submissions for Restore IP Addresses.
// Memory Usage: 44.5 MB, less than 6.25% of Java online
// submissions for Restore IP Addresses.
    public List<String> restoreIpAddresses(String s) {
        HashSet<String> result = new HashSet<>();
        StringBuffer buffer = new StringBuffer("");
        helper(s, 0, 1, buffer, result, 0);
        return new ArrayList<String>(result);
    }
}
```

```

private void helper(String s, int index, int j,
    StringBuffer buffer, HashSet<String> result,
    int inc) {
    int n = s.length();
    if (index >= n && inc == 4) {
        int lastIndex = buffer.lastIndexOf(".");
        int len = buffer.length();
        buffer = buffer.replace(lastIndex, len, "");
        result.add(buffer.toString());
        return;
    }

    // 25 525 511 135(invalid since 511 not valid ip)
    // 255 255 11 135(valid)
    // 255 255 111 35(valid)
    for (int i = 1; i <= 3; i++) { // local stack
        if (index + i > n) {
            return;
        }
        StringBuffer copy = new StringBuffer(buffer);
        String ip = s.substring(index, index + i);
        if (!isValidIp(ip))
            return;
        copy.append(ip).append(".");
        //reduce subproblem by index keeping input s same
        helper(s, index + i, 1, copy, result, inc + 1);
    }
}

private boolean isValidIp(String val) {
    if (val.length() > 3)
        return false;
    if (val.startsWith("0") && val.length() > 1)
        return false;
    int cur = Integer.parseInt(val);
    if (cur < 0 || cur > 255) {
        return false;
    }
    return true;
}
}

```


Approach 2: minimize input problem by process only unprocessed input String:

```
// 2 5 5 2 5 5 1 1 1 3 5
// 0 1 2 3 4 5 6 7 8 9 10

// 2 5 5 | 2 5 5 | 1 1 1 | 3 5 ( sol 1)
// 2 5 5 | 2 5 5 | 1 1 | 1 3 5 ( sol 2)
//
//First(index, str)(0,3) | process | Remaining (3 - n)
//-----|-----
//      2( 0 , "2")      |      2(0, "255.2")
//      /                |      /
//      5(1, "25")        |      5(1, "255.25")
//      /                |      /
//      5(2, "255")       |      5
//      / \              |      / \
// 2  exit rem(3, "255.") | 1  1 (2, "255.255.1")...
// |                    | |
// x(invalid ip-exit)    | x(invalid ip -255.2551)
```

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
```

```
//Runtime: 3 ms, faster than 62.15% of Java online
//submissions for Restore IP Addresses.
//Memory Usage: 42.1 MB, less than 35.29% of Java online submissions for
Restore IP Addresses.
```

```
class Solution {

    public List<String> restoreIpAddresses(String s) {
        List<String> res = new ArrayList<>();
        StringBuffer buffer = new StringBuffer();
        helper(s, buffer, res, 0);
        return res;
    }

    public void helper(String s, StringBuffer buffer,
        List<String> result, int n) {
        if (n == 4) {
            if (s.length() == 0) {
                int lastIndex = buffer.lastIndexOf(".");
                int len = buffer.length();
                buffer = buffer.replace(lastIndex, len, "");
                result.add(buffer.toString());
            }
            return;
        }
    }
}
```

```

    for (int k = 1; k <= 3; k++) {
        if (s.length() < k)
            continue;
        StringBuffer copy = new StringBuffer(buffer);
        String cur = s.substring(0, k);
        String rem = s.substring(k);
        if (!isValidIp(cur)) {
            continue;
        }
        copy.append(cur).append(".");
        //reduce input ie. Remaining string after (0,k)
        helper(rem, copy, result, n + 1);
    }
}

private boolean isValidIp(String val) {
    if (val.length() > 3)
        return false;
    if (val.startsWith("0") && val.length() > 1)
        return false;
    int cur = Integer.parseInt(val);
    if (cur < 0 || cur > 255) {
        return false;
    }
    return true;
}
}

```

Problem-6: 1593. Split a String Into the Max Number of Unique Substrings(BACKTRACKING)

Given a string *s*, return the maximum number of unique substrings that the given string can be split into.

You can split string *s* into any list of **non-empty substrings**, where the concatenation of the substrings forms the original string. However, you must split the substrings such that all of them are **unique**.

A **substring** is a contiguous sequence of characters within a string.

Example 1:

Input: *s* = "ababccc"

Output: 5

Explanation: One way to split maximally is ['a', 'b', 'ab', 'c', 'cc']. Splitting like ['a', 'b', 'a', 'b', 'c', 'cc'] is not valid as you have 'a' and 'b' multiple times.

Constraints:

- $1 \leq s.length \leq 16$

- s contains only lower case English letters.

Approach:

```
// 0 1 2 3 4 5 6
// a b a b c c c

// a      [a]
//  b     [a,b]
//   a b  [a,b,ab]// 'a' already exist, so extend
//    c   [a,b,ab,c]
//   c    [a,b,ab,c,cc]
```

Solution 1:

```
import java.util.HashSet;
//Runtime: 28 ms, faster than 86.03% of Java online submissions
//for Split a String Into the Max Number of Unique Substrings.
//Memory Usage: 43.8 MB, less than 92.14% of Java online submissions
//for Split a String Into the Max Number of Unique Substrings.
class Solution {
    public int maxUniqueSplit(String s) {
        HashSet<String> st = new HashSet<>();
        int res[] = new int[1];
        helper(s, 0, st, res);

        return res[0];
    }

    private void helper(String s, int index,
                        HashSet<String> st, int res[]) {
        int n = s.length();
        if (index >= n) {
            res[0] = Math.max(res[0], st.size());
            return;
        }
        for (int i = index; i < n; i++) {
            String cur = s.substring(index, i + 1);
            if (cur.length() == 0 || st.contains(cur)) {
                continue;
            }
            st.add(cur);
            helper(s, i + 1, st, res);
            st.remove(cur);
        }
    }
}
```

NOTE:

if backtracking not used it does recurse stack unwind and generate different combination of string to avoid this remove last character.

while this problem doesn't involve undoing choices to find a solution, it does involve exploring different choices at each step in a recursive manner, and that's why the term "backtracking" is used. It's more about efficiently searching through the solution space rather than solving a constraint satisfaction problem as in N-Queens or Sudoku.

1. **Backtracking in MaxUniqueSplit:** In this problem, we use backtracking not to explore all possible combinations but to efficiently explore the solution space while avoiding redundant work. The primary goal is to find the maximum number of unique substrings without generating all possible combinations upfront, which can be computationally expensive. Backtracking is applied to make choices (including or excluding characters) and prune branches that cannot lead to a better solution, thus reducing the recursion depth.
2. **Avoiding Stack Unwind:** One of the main reasons for using backtracking in this problem is to avoid unnecessary stack unwind. When we find a substring that has already been seen, instead of continuing with the current branch of recursion, we backtrack (move to the next character) and try other possibilities. This avoids the need to unwind the stack to explore other branches, making the solution more efficient.

In summary, while the term "backtracking" is used in the MaxUniqueSplit problem, it's applied differently from classic backtracking problems where all combinations are explored. Here, it's used as a technique to efficiently navigate the solution space and reduce the complexity of the algorithm by avoiding stack unwind when redundant work can be eliminated. This may appear unconventional but is a valid application of backtracking in a problem-solving context.

input : `ababccc`

```
//a b a b c c c
//0 1 2 3 4 5 6
[a, b, ab, c, cc, //initial stack (0,6)
ccc, //stack unwind of (5,6)-cc already exists so break and (4,6)
abc, //stack unwind of (2,4)
abcc, //stack unwind of (2,5)
```

```
abccc, //stack unwind of (2,6)
ba,   //stack unwind of (1,2)
bc,   //stack unwind of (3,4)
bcc   //stack unwind of (3,5)
...//continues
bcc//
bab   //stack unwind of (1,3)
```

Another Solution 2:

```
import java.util.HashSet;

class Solution {
    // Runtime: 33 ms, faster than 52.84% of Java online submissions for
    // Split a String Into the Max Number of Unique Substrings.
    // Memory Usage: 44.1 MB, less than 61.57% of Java online
    // submissions for Split a String Into the Max Number of Unique Substrings
    public int maxUniqueSplit(String s) {
        HashSet<String> st = new HashSet<>();
        int res[] = new int[1];
        helper(s, 0, st, res);

        return res[0];
    }

    private void helper(String s, int index,
        HashSet<String> st, int res[]) {
        int n = s.length();
        if (index >= n) {
            res[0] = Math.max(res[0], st.size());
            return;
        }

        for (int i = index; i < n; i++) {
            String cur = s.substring(index, i + 1);
            String rem = s.substring(i + 1);
            if (cur.length() == 0 || st.contains(cur)) {
                continue;
            }
            st.add(cur);
            helper(rem, i + 1, st, res);
            st.remove(cur);
        }
    }
}
```

Problem-7: 2056. Number of Valid Move Combinations On Chessboard

There is an 8 x 8 chessboard containing n pieces (rooks, queens, or bishops). You are given a string array `pieces` of length n , where `pieces[i]` describes the type (rook, queen, or bishop) of the i^{th} piece. In addition, you are given a 2D integer array `positions` also of length n , where `positions[i] = [ri, ci]` indicates that the i^{th} piece is currently at the **1-based** coordinate (r_i, c_i) on the chessboard.

When making a **move** for a piece, you choose a **destination** square that the piece will travel toward and stop on.

- A rook can only travel **horizontally or vertically** from (r, c) to the direction of $(r+1, c)$, $(r-1, c)$, $(r, c+1)$, or $(r, c-1)$.
- A queen can only travel **horizontally, vertically, or diagonally** from (r, c) to the direction of $(r+1, c)$, $(r-1, c)$, $(r, c+1)$, $(r, c-1)$, $(r+1, c+1)$, $(r+1, c-1)$, $(r-1, c+1)$, $(r-1, c-1)$.
- A bishop can only travel **diagonally** from (r, c) to the direction of $(r+1, c+1)$, $(r+1, c-1)$, $(r-1, c+1)$, $(r-1, c-1)$.

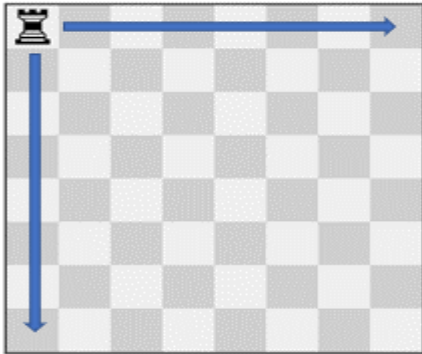
You must make a **move** for every piece on the board simultaneously. A **move combination** consists of all the **moves** performed on all the given pieces. Every second, each piece will instantaneously travel **one square** towards their destination if they are not already at it. All pieces start traveling at the 0^{th} second. A move combination is **invalid** if, at a given time, **two or more** pieces occupy the same square.

Return *the number of valid move combinations*.

Notes:

- **No two pieces** will start in the **same** square.
- You may choose the square a piece is already on as its **destination**.
- If two pieces are **directly adjacent** to each other, it is valid for them to **move past each other** and swap positions in one second.

Example 1:

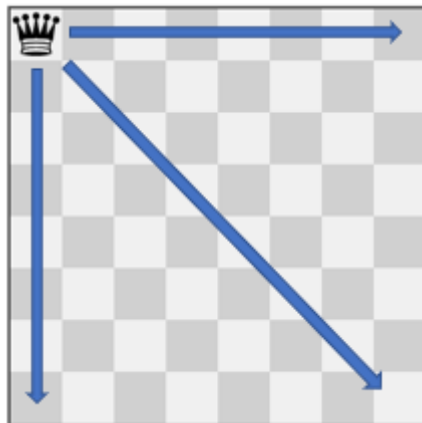


Input: pieces = ["rook"], positions = [[1,1]]

Output: 15

Explanation: The image above shows the possible squares the piece can move to.

Example 2:

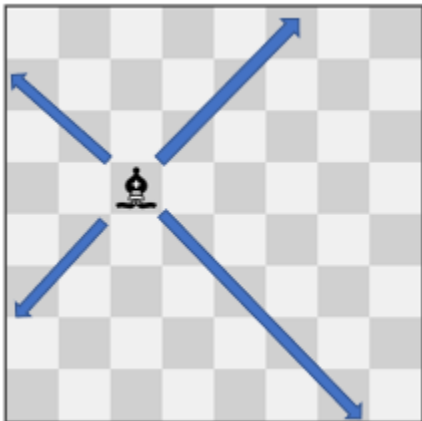


Input: pieces = ["queen"], positions = [[1,1]]

Output: 22

Explanation: The image above shows the possible squares the piece can move to.

Example 3:



Input: pieces = ["bishop"], positions = [[4,3]]

Output: 12

Explanation: The image above shows the possible squares the piece can move to.

Constraints:

- n == pieces.length
- n == positions.length
- 1 <= n <= 4
- pieces only contains the strings "rook", "queen", and "bishop".
- There will be at most one queen on the chessboard.
- 1 <= x_i, y_i <= 8
- Each positions[i] is distinct.

Solution

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

//Runtime: 303 ms, faster than 23.08% of Java online submissions for Number
//of Valid Move Combinations On Chessboard.
//Memory Usage: 53.8 MB, less than 11.54% of Java online submissions for Number
//of Valid Move Combinations On Chessboard.
class Solution {
    //count combination
    public int countCombinations(String[] pieces, int[][] positions) {
        final int n = pieces.length;
        Set<Long> ans = new HashSet<>();
        List<List<int[]>> combMoves = new ArrayList<>();
        getCombMoves(pieces, 0, new ArrayList<>(), combMoves);

        for (List<int[]> combMove : combMoves) {
            dfs(positions, n, combMove, (1 << n) - 1, ans);
        }

        return ans.size();
    }

    /**
     * get directions based on piece
     * @param piece
     */
}
```



```

    * @return
    */
    private int[][] getDirections(String piece) {
        // (r+1, c), (r-1, c), (r, c+1), or (r, c-1).
        int rook[][] = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 } };
        // (r+1, c), (r-1, c), (r, c+1), (r, c-1), (r+1, c+1),
        // (r+1, c-1), (r-1, c+1),
        // (r-1, c-1).
        int queen[][] = { { 1, 0 }, { -1, 0 }, { 0, 1 }, { 0, -1 },
                          { 1, 1 }, { 1, -1 }, { -1, 1 }, { -1, -1 } };

        // (r+1, c+1), (r+1, c-1), (r-1, c+1), (r-1, c-1).
        int bishop[][] = { { 1, 1 }, { 1, -1 }, { -1, 1 }, { -1, -1 } };
        if (piece.equals("rook")) {
            return rook;
        } else if (piece.equals("queen")) {
            return queen;
        } else {
            return bishop;
        }
    }
}

/**
 * get all combo moves in 8x8 matrix
 * @param pieces
 * @param ithPiece
 * @param path
 * @param combMoves
 */
private void getCombMoves(String[] pieces, int ithPiece, List<int[]> path,
                          List<List<int[]>> combMoves) {
    if (ithPiece == pieces.length) {
        combMoves.add(new ArrayList<>(path));
        return;
    }

    for (int[] move : getDirections(pieces[ithPiece])) {
        path.add(move);
        getCombMoves(pieces, ithPiece + 1, path, combMoves);
        path.remove(path.size() - 1);
    }
}

```

```

/**
 * do dfs on the board combo move for all states
 * @param board
 * @param n
 * @param combMove
 * @param activeMask
 * @param ans
 */
private void dfs(int[][] board, int n, List<int[]> combMove,
                int activeMask, Set<Long> ans) {
    if (activeMask == 0) {
        return;
    }

    ans.add(getHash(board));
    for (int nextActiveMask = 1; nextActiveMask < (1 << n);
        ++nextActiveMask)
    {
        if ((activeMask & nextActiveMask) != nextActiveMask) {
            continue;
        }
        // Make sure to copy the board
        int[][] nextBoard = new int[n][];
        for (int i = 0; i < n; ++i) {
            nextBoard[i] = board[i].clone();
        }

        // Move pieces that are active in this turn
        for (int i = 0; i < n; ++i) {
            if (((nextActiveMask >> i) & 1) == 1) {
                nextBoard[i][0] += combMove.get(i)[0];
                nextBoard[i][1] += combMove.get(i)[1];
            }
        }
        // Check no two or more pieces occupy the same square
        if (getUniqueSize(nextBoard, activeMask) < n) {
            continue;
        }

        // Check if all in boundary
        if (checkInBounds(nextBoard)) {
            dfs(nextBoard, n, combMove, nextActiveMask, ans);
        }
    }
}

```

```
/**
 * to check out of bounds condition
 * @param nextBoard
 * @return
 */
private boolean checkInBounds(int[][] nextBoard) {
    for (int cur[] : nextBoard) {
        int x = cur[0];
        int y = cur[1];
        if (x < 1 || y < 1 || x > 8 || y > 8) {
            return false;
        }
    }

    return true;
}

/**
 * to generate unique hash
 * @param board
 * @return
 */
private long getHash(int[][] board) {
    long hash = 0;
    for (int[] pos : board) {
        // multiply by 8 to get unique hash
        // e.g row 1 col 7 == row 7 col 1 to generate unique distribution
        hash = (hash * 64) + ((pos[0] - 1) * 8) + (pos[1] - 1);
    }
    return hash;
}

/**
 * multiple by 8 to generate unique hash
 * @param board
 * @param activeMask
 * @return
 */
private int getUniqueSize(int[][] board, int activeMask) {
    Set<Integer> unique = new HashSet<>();
    for (int[] pos : board) {
        unique.add(pos[0] * 8 + pos[1]);
    }

    return unique.size();
}
}
```

Problem-8: 967. Numbers With Same Consecutive Differences(Backtrack primitive Object)

Given two integers n and k , return an array of all the integers of length n where the difference between every two consecutive digits is k . You may return the answer in any order.

Note that the integers should not have leading zeros. Integers as 02 and 043 are not allowed.

Example 1:

Input: $n = 3, k = 7$

Output: [181,292,707,818,929]

Explanation: Note that 070 is not a valid number, because it has leading zeroes.

Example 2:

Input: $n = 2, k = 1$

Output: [10,12,21,23,32,34,43,45,54,56,65,67,76,78,87,89,98]

Constraints:

- $2 \leq n \leq 9$
- $0 \leq k \leq 9$

Approach:

```
//          1 ----- (index=1) (n,k) = (3,7)
//          / \
//         /   \
//        8     -6 ----- (index =2)
//       / \   \
//      15  1    x ----- (index=3) -k
//     /  \   \
//    x    [181] ----- base case met index == n
```

E.g: start with 1 $(+k, -k) = (8, -6)$ on each subsequent recursive call

Solution(BACKTRACK PRIMITIVE integer by diving num by 10)

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;

class Solution {
    public int[] numsSameConsecDiff(int n, int k) {

        List<Integer> result = new ArrayList<>();
        int dirs[] = { -k, +k };
        HashSet<Integer> st = new HashSet<>();
        for (int i = 1; i <= 9; i++) {
            helper(i, i, result, dirs, n, k, 1, i, st);
        }
        int count = 0;
        int res[] = new int[result.size()];
        for (int cur : result) {
            res[count++] = cur;
        }

        return res;
    }

    /**
     *
     * @param cursum - current cumulative sum
     * @param prev - prev value
     * @param result - final list
     * @param dir - dir (+k,-k)
     * @param n = total number
     * @param k = difference between cur -prev
     * @param index = current running index
     * @param sum = total sum(prev + cur)
     * @param visited - already result found avoid duplicate
     */
    private void helper(int cursum, int prev, List<Integer> result,
        int dir[], int n, int k, int index, int sum,
        HashSet<Integer> visited) {
        if (index == n) {
            if (cursum > 0 && !visited.contains(cursum)) {
                visited.add(cursum);
                result.add(cursum);
            }

            return;
        }
    }
}

```

```

        for (int x : dir) {
            //add (+k) , remove(-K)
            int cur = sum + x;
            if (cur >= 10 || cur < 0) {
                continue;
            }

            if (isValid(prev, cur, index, k)) {
                cursum = (cursum * 10) + cur;
                helper(cursum, cur, result, dir, n, k,
                    index + 1, cur, visited);
                cursum = (cursum / 10);
                //(backtrack primitive also -by delete last i.e num/10)
            }
        }
    }

    private boolean isValid(int prev, int val, int index, int k) {
        if (index == 0) {
            return true;
        }
        //prev - cur is k then valid
        int cur = Math.abs(prev - val);

        return (cur == k);
    }

    public static void main(String args[]) {
        Solution sol = new Solution();
        int n = 3;
        int k = 7;

        int arr[] = sol.numsSameConsecDiff(n, k);
        System.out.println("Solution.main()" + Arrays.toString(arr));
    }
}

```

Problem-9: 691. Stickers to Spell Word(Recursion+MAP as Memoization key)

We are given n different types of stickers. Each sticker has a lowercase English word on it.

You would like to spell out the given string `target` by cutting individual letters from your collection of stickers and rearranging them. You can use each sticker more than once if you want, and you have infinite quantities of each sticker.

Return *the minimum number of stickers that you need to spell out target*. If the task is impossible, return -1.

Note: In all test cases, all words were chosen randomly from the 1000 most common US English words, and `target` was chosen as a concatenation of two random words.

Example 1:

Input: `stickers = ["with","example","science"], target = "thehat"`

Output: 3

Explanation:

We can use 2 "with" stickers, and 1 "example" sticker.

After cutting and rearrange the letters of those stickers, we can form the target "thehat".

Also, this is the minimum number of stickers necessary to form the target string.

Example 2:

Input: `stickers = ["notice","possible"], target = "basicbasic"`

Output: -1

Explanation:

We cannot form the target "basicbasic" from cutting letters from the given stickers.

Constraints:

- $n == \text{stickers.length}$
- $1 \leq n \leq 50$
- $1 \leq \text{stickers}[i].\text{length} \leq 10$
- $1 \leq \text{target.length} \leq 15$
- `stickers[i]` and `target` consist of lowercase English letters.

Solution

```

import java.util.HashMap;
import java.util.Map;
import java.util.Set;
class Solution {
    public int minStickers(String[] stickers, String target) {
        Map<Character, Integer> targetFrequencyMap = getFrequency(target);
        Map<String, Integer> cache = new HashMap<>();
        int ans = helper(stickers, targetFrequencyMap, cache);
        if (ans == Integer.MAX_VALUE)
            return -1;
        return ans;
    }

    /**
     * recursion function to calculate the count required to achieve
     target.
     * @param stickers
     * @param targetFrequencyMap
     * @param cache
     * @return
     */
    private int helper(String stickers[],
                       Map<Character, Integer> targetFrequencyMap,
                       Map<String, Integer> cache) {
        String curHash = getHash(targetFrequencyMap);
        if (curHash == null)
            return 0;
        if (cache.get(curHash) != null) {
            return cache.get(curHash);
        }
        int result = Integer.MAX_VALUE;
        int n = stickers.length;
        for (int i = 0; i < n; i++) {
            String cur = stickers[i];
            Map<Character, Integer> curFrequencyMap = getFrequency(cur);
            if (isAnyExists(curFrequencyMap, targetFrequencyMap)) {
                Map<Character, Integer> new_FrequencyMap =
                    subtractFrequency(curFrequencyMap, targetFrequencyMap);
                int temp = helper(stickers, new_FrequencyMap, cache);
                if (temp != Integer.MAX_VALUE) {
                    result = Math.min(result, temp+1);
                }
            }
        }
        cache.put(curHash, result);
        return result;
    }
}

```



```
/**
 * to check any character of current frequencyMap exists in
targetFrequency Map
 * if so subtract frequency or target character found and search
remaining
 * character in same or different stickers.
 *
 * @param currentMap
 * @param targetMap
 * @return
 */
private boolean isAnyExists(Map<Character, Integer> currentMap,
    Map<Character, Integer> targetMap) {
    for (char ch : targetMap.keySet()) {
        if (currentMap.get(ch) != null)
            return true;
    }

    return false;
}

/**
 * Generate a hash of the frequency Map
 *
 * @param map
 * @return
 */
private String getHash(Map<Character, Integer> frequencyMap) {
    StringBuffer buffer = new StringBuffer();
    for (char ch : frequencyMap.keySet()) {

        int val = frequencyMap.get(ch);
        buffer.append(ch + ":" + val);
        buffer.append("|");
    }

    int index = buffer.toString().lastIndexOf("|");
    String result = "";
    if (index != -1) {
        result = buffer.substring(0, index);

        return result;
    }

    return null;
}
```

```
/**
 * To get the frequency for the source
 *
 * @param target
 * @return
 */
private Map<Character, Integer> getFrequency(String source) {
    Map<Character, Integer> frequencyMap = new HashMap<>();
    for (char ch : source.toCharArray()) {
        frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
    }

    return frequencyMap;
}

/**
 * subtract target with current till now frequency and return remaining
 * frequency to achieve target frequency
 *
 * @param sourceMap
 * @param targetMap
 * @return
 */
private Map<Character, Integer> subtractFrequency(
    Map<Character, Integer> sourceMap,
    Map<Character, Integer> targetMap) {
    Map<Character, Integer> frequencyMap = new HashMap<>();
    Set<Character> st = targetMap.keySet();
    for (char ch : st) {
        Integer sfrequency = sourceMap.get(ch);
        Integer tfrequency = targetMap.get(ch);
        if (sfrequency == null) {
            frequencyMap.put(ch, tfrequency);
            continue;
        }
        int diff = (tfrequency - sfrequency);

        if (diff > 0) {
            frequencyMap.put(ch, diff);
        }
    }

    return frequencyMap;
}
```

```

    public static void main(String args[]) {
        String res[] = { "with", "example", "science" };
        int ans = new Solution().minStickers(res, "thehat");
        System.out.println("Solution.main()::::::::::" + ans);
    }
}

```

Problem-10:1476:Probability of a Two Boxes Having The Same Number of Distinct Balls

RECURSION & MEMOIZATION:

Given $2n$ balls of k distinct colors. You will be given an integer array `balls` of size k where `balls[i]` is the number of balls of color i .

All the balls will be **shuffled uniformly at random**, then we will distribute the first n balls to the first box and the remaining n balls to the other box (Please read the explanation of the second example carefully).

Please note that the two boxes are considered different. For example, if we have two balls of colors a and b , and two boxes `[]` and `()`, then the distribution `[a] (b)` is considered different than the distribution `[b] (a)` (Please read the explanation of the first example carefully).

Return *the probability* that the two boxes have the same number of distinct balls. Answers within 10^{-5} of the actual value will be accepted as correct.

Example 1:

Input: `balls = [1,1]`

Output: 1.00000

Explanation: Only 2 ways to divide the balls equally:

- A ball of color 1 to box 1 and a ball of color 2 to box 2
- A ball of color 2 to box 1 and a ball of color 1 to box 2

In both ways, the number of distinct colors in each box is equal. The probability is $2/2 = 1$

Example 2:

Input: `balls = [2,1,1]`

Output: 0.66667

Explanation: We have the set of balls `[1, 1, 2, 3]`

This set of balls will be shuffled randomly and we may have one of the 12 distinct shuffles with equal probability (i.e. $1/12$):

`[1,1 / 2,3]`, `[1,1 / 3,2]`, `[1,2 / 1,3]`, `[1,2 / 3,1]`, `[1,3 / 1,2]`, `[1,3 / 2,1]`,
`[2,1 / 1,3]`, `[2,1 / 3,1]`, `[2,3 / 1,1]`, `[3,1 / 1,2]`, `[3,1 / 2,1]`, `[3,2 / 1,1]`

After that, we add the first two balls to the first box and the second two balls to the second box.

We can see that 8 of these 12 possible random distributions have the same number of distinct colors of balls in each box.

Probability is $8/12 = 0.66667$

Example 3:

Input: balls = [1,2,1,2]

Output: 0.60000

Explanation: The set of balls is [1, 2, 2, 3, 4, 4]. It is hard to display all the 180 possible random shuffles of this set but it is easy to check that 108 of them will have the same number of distinct colors in each box.

Probability = $108 / 180 = 0.6$

Constraints:

- $1 \leq \text{balls.length} \leq 8$
- $1 \leq \text{balls}[i] \leq 6$
- $\text{sum}(\text{balls})$ is even.

When it comes to generic software engineering interview questions, probability problems are not as hard as they seem. Their solutions are often the same as your average backtracking (and DP if it's a harder problem) problem.

Let's define the following terms:

A *permutation* is some unique combination of the 2N-length array. If our input is [2,1,1] like in the second test case, an example permutation is [2 / 1,1,3].

A *valid permutation* is a permutation that can be considered in our answer, but may or may not meet the specific requirements of what we're counting. The above example for a permutation is not a *valid* permutation, because the number of balls in the first half is not equal to the number of balls in the second half. We want to divide the array in half, after all - the above permutation doesn't do that. An example of a valid permutation is [1, 1 / 2, 3].

A *successful* permutation is a valid permutation that meets the requirements of what we're counting. The above example of a valid permutation isn't *successful* because the number of unique balls in the first half (1) isn't the same as the number of unique balls in the second half (2). An example of a *successful* permutation is [1, 2 / 1, 3].

A simple strategy, which actually works here, is to just count the number of "successful" permutations and divide it by the number of "valid" permutations. I do this in my solution below.

It makes me sad to see candidates mess up on probability questions like this, because if you know how to backtrack, you can solve problems like this. Good luck in your future interviews!

Solution

```

import java.util.HashMap;
import java.util.Map;
class Solution {
    public double getProbability(int[] balls) {
        int[] firstHalf = new int[balls.length];
        int[] secondHalf = new int[balls.length];
        double[] good = new double[] { 0.0 };
        double[] all = new double[] { 0.0 };
        Map<Double, Double> memorizedFactorial = new HashMap<>();

        dfs(0, firstHalf, secondHalf, balls, good, all, memorizedFactorial);
        return good[0] / all[0];
    }

    private void dfs(int i, int[] firstHalf, int[] secondHalf,
        int[] balls, double[] good, double[] all,
        Map<Double, Double> memorizedFactorial) {
        if (i == balls.length) {
            int firstHalfSum = 0;
            int secondHalfSum = 0;
            for (int value : firstHalf) {
                firstHalfSum += value;
            }
            for (int value : secondHalf) {
                secondHalfSum += value;
            }

            if (firstHalfSum != secondHalfSum) {
                return;
            }
            double p1 = permutation(firstHalf, memorizedFactorial);
            double p2 = permutation(secondHalf, memorizedFactorial);
            int cnt1 = 0;
            int cnt2 = 0;
            for (int value : firstHalf) {
                if (value > 0) {
                    cnt1++;
                }
            }
            for (int value : secondHalf) {
                if (value > 0) {
                    cnt2++;
                }
            }
            all[0] += p1 * p2;
            good[0] += (cnt1 == cnt2) ? p1 * p2 : 0.0;

            return;
        }
    }
}

```

```

        for (int j = 0; j <= balls[i]; j++) {
            firstHalf[i] = j;
            secondHalf[i] = balls[i] - j;
            dfs(i + 1, firstHalf, secondHalf, balls,
                good, all, memorizedFactorial);
            firstHalf[i] = 0;
            secondHalf[i] = 0;
        }
    }

    private double permutation(int[] arr,
        Map<Double, Double> memorizedFactorial) {
        double prod = 1.0;
        int sum = 0;
        for (int value : arr) {
            prod *= factorial((double) value, memorizedFactorial);
            sum += value;
        }
        return factorial((double) sum, memorizedFactorial) / prod;
    }

    private double factorial(double value,
        Map<Double, Double> memorizedFactorial) {
        if (memorizedFactorial.containsKey(value)) {
            return memorizedFactorial.get(value);
        }
        double res = (value != 0) ? value *
            factorial(value - 1, memorizedFactorial) : 1.0;
        memorizedFactorial.put(value, res);
        return res;
    }
}

```

Problem-11-I. 403. Frog Jump

403. Frog Jump

Hard

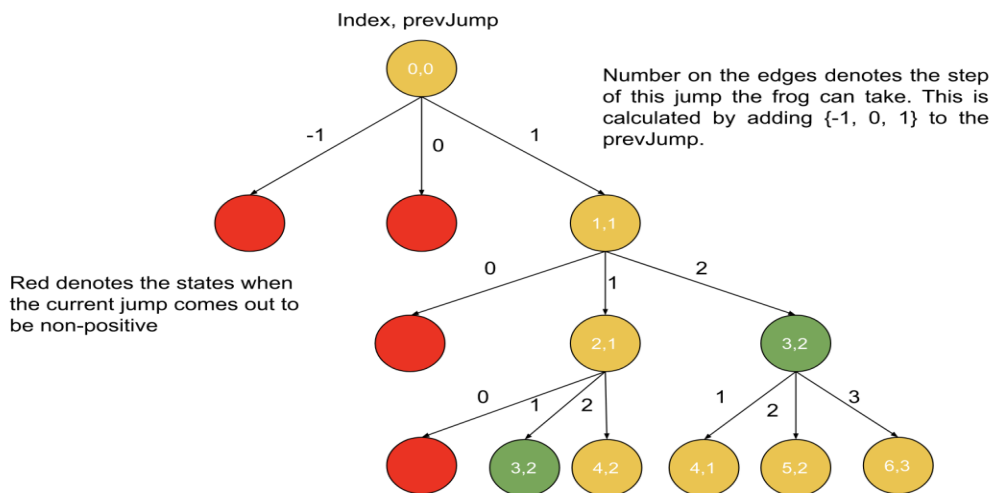
A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones' positions (in units) in sorted **ascending order**, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either $k - 1$, k , or $k + 1$ units. The frog can only jump in the forward direction.

Example 1:**Input:** stones = [0,1,3,5,6,8,12,17]**Output:** true**Explanation:** The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.**Example 2:****Input:** stones = [0,1,2,3,4,8,9,11]**Output:** false**Explanation:** There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.**Constraints:**

- $2 \leq \text{stones.length} \leq 2000$
- $0 \leq \text{stones}[i] \leq 2^{31} - 1$
- $\text{stones}[0] == 0$
- stones is sorted in a strictly increasing order.

**Solution(Recursion & update global shared object in Recursion)**

```
import java.util.HashMap;
import java.util.HashSet;
```

```

class Solution {
// Runtime: 9 ms, faster than 99.05% of Java online submissions for Frog
Jump.
// Memory Usage: 45.2 MB, less than 95.26% of Java online submissions for
Frog Jump.
    public boolean canCross(int[] stones) {
        int destination = stones[stones.length-1];
        // Pass a hashmap to store calculated results
        HashMap<Integer,HashSet<Integer>> map = new HashMap<>();
        //add all available stones.
        for(int cur : stones){
            map.put(cur,new HashSet<>());
        }
        return helper( stones,map,0,1);
    }

    private boolean helper(int stones[], HashMap<Integer,HashSet<Integer>> map
, int current, int k)
    {
        int n = stones.length;
        int last = stones[n-1];
        //base conditions
        //if last stone reached
        if (current == last) return true;
        //if already visited means loop cant reach
        if ( map.get(current).contains(k)) return false;

        //move the next stone
        int next = current + k;
        if (k<=0 || !map.containsKey(next)) {
            return false;
        }

        //memoize visited stone and steps used
        map.get(current).add(k);
        //try all ways i.e first 1 step
        //          try with 2(k+1) step
        //          try with 1(k) step
        //          try with 0(k-1) false always default,
        //          but if move with K+1 2times from 1 (1,2,3)
        //          then k-1 will be 1 step again.
        return helper(stones,map,next, k + 1)
            || helper(stones,map,next, k )
            || helper(stones,map,next, k-1 );
    }
}

```


Avoid unnecessary stack calls:

```

import java.util.HashMap;
import java.util.HashSet;

class Solution {
    public boolean canCross(int[] stones) {
        int destination = stones[stones.length - 1];
        // Pass a hashmap to store calculated results
        HashMap<Integer, HashSet<Integer>> map = new HashMap<>();
        // add all available stones.
        for (int cur : stones) {
            map.put(cur, new HashSet<>());
        }
        return helper(stones, map, 0, 1);
    }
    private boolean helper(int[] stones,
        HashMap<Integer, HashSet<Integer>> map, int cur, int step)
    {
        int n = stones.length;
        int lastStone = stones[n - 1];
        // base cases
        if (cur == lastStone)
            return true;
        if (step <= 0 || map.get(cur).contains(step))
            return false;
        int nextPos = cur + step;
        if (!map.containsKey(nextPos))
            return false;

        // main case -memoization
        map.get(cur).add(step);
        boolean isReachable = helper(stones, map, nextPos, step - 1);
        if (!isReachable) {
            isReachable = helper(stones, map, nextPos, step);
        }
        if (!isReachable) {
            isReachable = helper(stones, map, nextPos, step + 1);
        }
        return isReachable;
    }
}

```

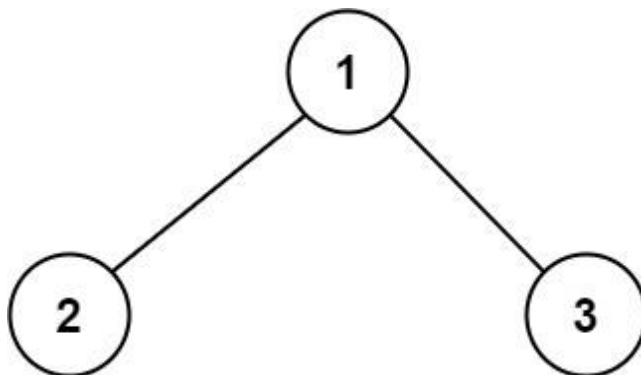
Problem-12: 124. Binary Tree Maximum Path Sum(Pass Global Shared Object in RECURSION)

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the root of a binary tree, return *the maximum path sum of any non-empty path*.

Example 1:

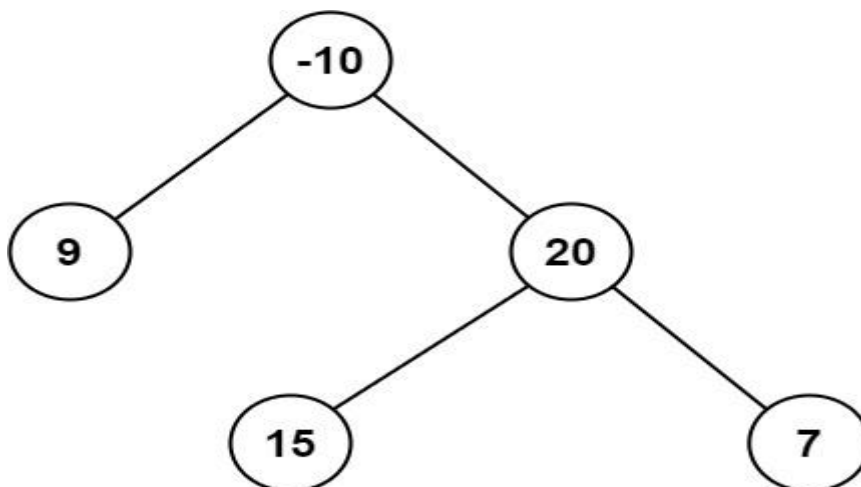


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Solution

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int maxPathSum(TreeNode root) {
        if(root==null)return 0;

        int res[] = new int[1];
        res[0]=Integer.MIN_VALUE;
        helper(root,0,res);
        return res[0];
    }

    private int helper(TreeNode root,int val, int res[]){
        if(root==null)return 0;

        int left =0,right =0;
        left+=helper(root.left,val,res);
        right+=helper(root.right,val,res);

        res[0]= Math.max(res[0],left+right+root.val);

        int result =Math.max(left,right)+root.val;
        return result>=0?result:0;
    }
}
```

Problem-13: 1774. Closest Dessert Cost(RECURSION + pass Global Shared Object in RECURSION)

You would like to make dessert and are preparing to buy the ingredients. You have n ice cream base flavors and m types of toppings to choose from. You must follow these rules when making your dessert:

- There must be **exactly one** ice cream base.
- You can add **one or more** types of topping or have no toppings at all.
- There are **at most two** of **each type** of topping.

You are given three inputs:

- `baseCosts`, an integer array of length n , where each `baseCosts[i]` represents the price of the i^{th} ice cream base flavor.
- `toppingCosts`, an integer array of length m , where each `toppingCosts[i]` is the price of **one** of the i^{th} topping.
- `target`, an integer representing your target price for dessert.

You want to make a dessert with a total cost as close to `target` as possible.

Return *the closest possible cost of the dessert to target*. If there are multiple, return *the Lower one*.

Example 1:

Input: `baseCosts = [1,7]`, `toppingCosts = [3,4]`, `target = 10`

Output: 10

Explanation: Consider the following combination (all 0-indexed):

- Choose base 1: cost 7
- Take 1 of topping 0: cost $1 \times 3 = 3$
- Take 0 of topping 1: cost $0 \times 4 = 0$

Total: $7 + 3 + 0 = 10$.

Example 2:

Input: `baseCosts = [2,3]`, `toppingCosts = [4,5,100]`, `target = 18`

Output: 17

Explanation: Consider the following combination (all 0-indexed):

- Choose base 1: cost 3
- Take 1 of topping 0: cost $1 \times 4 = 4$
- Take 2 of topping 1: cost $2 \times 5 = 10$
- Take 0 of topping 2: cost $0 \times 100 = 0$

Total: $3 + 4 + 10 + 0 = 17$. You cannot make a dessert with a total cost of 18.

Example 3:

Input: `baseCosts = [3,10]`, `toppingCosts = [2,5]`, `target = 9`

Output: 8

Explanation: It is possible to make desserts with cost 8 and 10. Return 8 as it is the lower cost.

Constraints:

- $n == \text{baseCosts.length}$
- $m == \text{toppingCosts.length}$
- $1 \leq n, m \leq 10$
- $1 \leq \text{baseCosts}[i], \text{toppingCosts}[i] \leq 10^4$
- $1 \leq \text{target} \leq 10^4$

```
//      [3,      10] [2,5]
//      / \      \
//      3   \      10
//      / \   \   / \ \
//      3+2 3+4 3+0 10+2 10+4 10+0
//      / |   \
//      3+5 3+10 3+4+0
```

Solution

```
class Solution {
    public int closestCost(int[] baseCosts, int[] toppingCosts, int target) {
        int res[] = new int[1];
        res[0]=baseCosts[0];
        for(int base: baseCosts)
            helper(base, toppingCosts, 0, target,res);
        return res[0];
    }
    private void helper(int current, int[] toppingCosts, int index,
        int target,int res[]) {
        if( Math.abs(target - current) < Math.abs(target - res[0])
            || Math.abs(target - current) == Math.abs(target - res[0])
            && current < res[0])
            res[0] = current;
        if(index==toppingCosts.length || current >= target) {
            return;
        }
        helper(current, toppingCosts, index + 1, target,res);
        helper(current + toppingCosts[index], toppingCosts,
            index + 1, target,res);
        helper(current + toppingCosts[index]*2, toppingCosts,
            index + 1, target,res);
    }
}
```

TimeComplexity: $O(n * 3^m)$
`n = baseCosts.length`
`m = toppingCosts.length`

Another Solution

```
class Solution {
    public int closestCost(int[] baseCosts, int[] toppingCosts, int target) {

        int res[] = new int[1];
        res[0] = baseCosts[0];
        for (int base : baseCosts)
            helper(toppingCosts, target, base, 0, res);
        return res[0];
    }

    private void helper(int[] top, int target, int cost, int idx, int res[]) {
        if (Math.abs(target - res[0]) > Math.abs(target - cost)) {
            res[0] = cost;
        } else if (Math.abs(target - res[0]) == Math.abs(target - cost)) {
            res[0] = Math.min(res[0], cost);
        }

        if (idx == top.length) return;

        //either 0, 1 or 2 toppings for each topping
        for (int i = 0; i <= 2; i++) {
            helper(top, target, cost + top[idx] * i, idx + 1, res);
        }
    }
}
```

Problem-14: 2151. Maximum Good People Based on Statements(BITMASK)

Hard

There are two types of persons:

- The **good person**: The person who always tells the truth.
- The **bad person**: The person who might tell the truth and might lie.

You are given a **0-indexed** 2D integer array `statements` of size `n x n` that represents the statements made by `n` people about each other. More specifically, `statements[i][j]` could be one of the following:

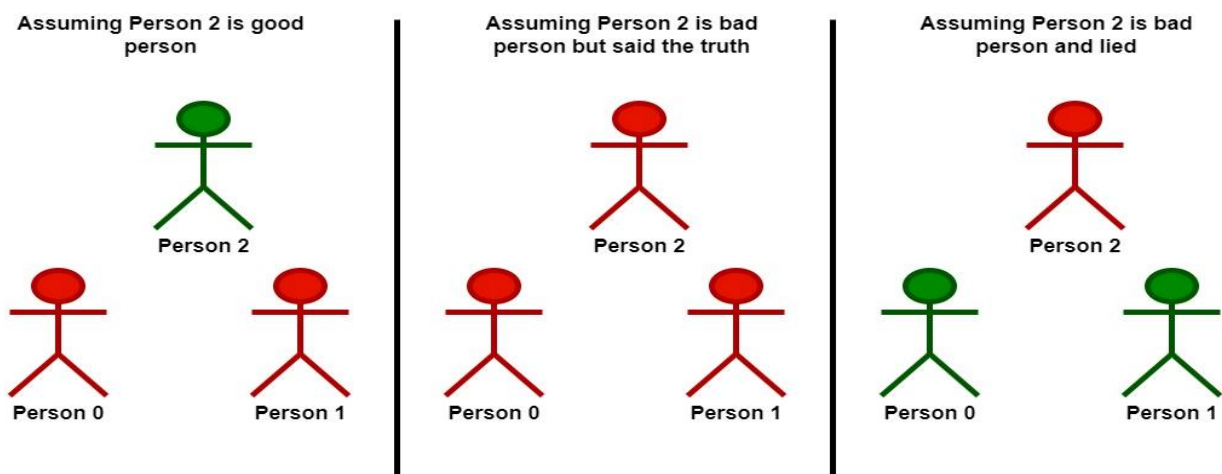
- `0` which represents a statement made by person `i` that person `j` is a **bad** person.

- 1 which represents a statement made by person i that person j is a **good** person.
- 2 represents that **no statement** is made by person i about person j .

Additionally, no person ever makes a statement about themselves. Formally, we have that $\text{statements}[i][i] = 2$ for all $0 \leq i < n$.

Return the *maximum* number of people who can be **good** based on the statements made by the n people.

Example 1:



Input: `statements = [[2,1,2],[1,2,2],[2,0,2]]`

Output: 2

Explanation: Each person makes a single statement.

- Person 0 states that person 1 is good.
- Person 1 states that person 0 is good.
- Person 2 states that person 1 is bad.

Let's take person 2 as the key.

- Assuming that person 2 is a good person:
 - Based on the statement made by person 2, person 1 is a bad person.
 - Now we know for sure that person 1 is bad and person 2 is good.
 - Based on the statement made by person 1, and since person 1 is bad, they could be:

- telling the truth. There will be a contradiction in this case and this assumption is invalid.

- lying. In this case, person 0 is also a bad person and lied in their statement.

- **Following that person 2 is a good person, there will be only one good person in the group.**

- Assuming that person 2 is a bad person:

- Based on the statement made by person 2, and since person 2 is bad, they could be:

- telling the truth. Following this scenario, person 0 and 1 are both bad as explained before.

- Following that person 2 is bad but told the truth, there will be no good persons in the group.

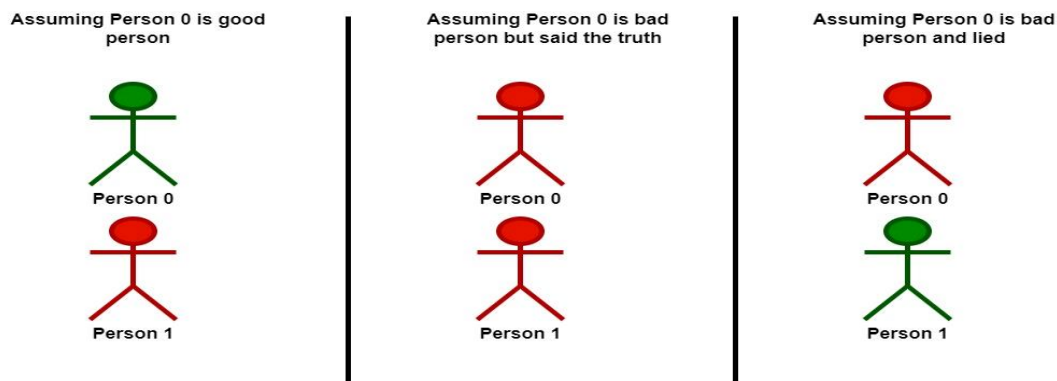
- lying. In this case person 1 is a good person.

- Since person 1 is a good person, person 0 is also a good person.

- Following that person 2 is bad and lied, there will be two good persons in the group.

We can see that at most 2 persons are good in the best case, so we return 2. Note that there is more than one way to arrive at this conclusion.

Example 2:



Input: statements = [[2,0],[0,2]]

Output: 1

Explanation: Each person makes a single statement.

- Person 0 states that person 1 is bad.

- Person 1 states that person 0 is bad.

Let's take person 0 as the key.

- Assuming that person 0 is a good person:

- Based on the statement made by person 0, person 1 is a bad person and was lying.

- Following that person 0 is a good person, there will be only one good person in the group.

- Assuming that person 0 is a bad person:

- Based on the statement made by person 0, and since person 0 is bad, they could be:

- telling the truth. Following this scenario, person 0 and 1 are both bad.

- Following that person 0 is bad but told the truth, there will be no good persons in the group.

- lying. In this case person 1 is a good person.

- Following that person 0 is bad and lied, there will be only one good person in the group.

We can see that at most, one person is good in the best case, so we return 1.

Note that there is more than one way to arrive at this conclusion.

Constraints:

- `n == statements.length == statements[i].length`
- `2 <= n <= 15`
- `statements[i][j]` is either 0, 1, or 2.
- `statements[i][i] == 2`

Solution

if there are 2 people
then possible configurations are

00 - 0 bad 1 bad
01 - 0 bad 1 good
10 - 0 good 1 bad
11 - 0 good 1 good

```
class Solution {
    //Runtime: 0 ms, faster than 100.00% of Java online submissions for
    //Maximum Good People Based on Statements.
    //Memory Usage: 40.2 MB, less than 100.00% of Java online submissions
    //for Maximum
    //Good People Based on Statements.
    public int maximumGood(int[][] statements) {
        // Declare variable to store the max number of good people
        int[] maxGoodPeople = new int[1];
        // Recursive function to find the maximum number of good people
        maximumGood(statements, new boolean[statements.length],
            0, 0, maxGoodPeople);
        return maxGoodPeople[0];
    }

    private void maximumGood(int[][] statements, boolean[] goodPeople,
        int currentPerson, int currentGoodPeople,
        int[] maxGoodPeople) {
        // Base case: if all people have been checked
        if (currentPerson == goodPeople.length) {
            // Update maxGoodPeople if necessary
            maxGoodPeople[0] = Math.max(maxGoodPeople[0], currentGoodPeople);

            return;
        }

        // Set current person as good
        goodPeople[currentPerson] = true;
        // If remaining people cannot beat the current max, return
        if (goodPeople.length - currentPerson +
            currentGoodPeople < maxGoodPeople[0]) {
            return;
        }
        // If current configuration is valid, move forward
```

```

    if (isValid(statements, goodPeople, currentPerson)) {
        maximumGood(statements, goodPeople, currentPerson + 1,
                    currentGoodPeople + 1, maxGoodPeople);
    }

    // Set current person as bad
    goodPeople[currentPerson] = false; //backtracking
    // If remaining people cannot beat the current max, return
    if (goodPeople.length - currentPerson
        - 1 + currentGoodPeople < maxGoodPeople[0]) {

        return;
    }
    // If current configuration is valid, move forward
    if (isValid(statements, goodPeople, currentPerson)) {
        maximumGood(statements, goodPeople, currentPerson + 1,
                    currentGoodPeople, maxGoodPeople);
    }

    return;
}

// Function to check if the current configuration is valid
private boolean isValid(int[][] statements,
    boolean[] goodPeople, int currentPerson) {
    // Check if all previous people's statements are consistent with
    // current
    // configuration
    for (int i = 0; i < currentPerson; ++i) {
        if (goodPeople[i]) {
            if (goodPeople[currentPerson]
                && statements[i][currentPerson] == 0) {
                return false;
            }
            if (!goodPeople[currentPerson]
                && statements[i][currentPerson] == 1) {
                return false;
            }
        }
    }
    // Check if current person's statements are consistent with
    // current
    // configuration

```

```

        if (goodPeople[currentPerson]) {
            for (int i = 0; i < currentPerson; ++i) {
                if (goodPeople[i]
                    && statements[currentPerson][i] == 0) {
                    return false;
                }
                if (!goodPeople[i]
                    && statements[currentPerson][i] == 1) {
                    return false;
                }
            }
        }
        return true;
    }
}

```

Another Solution - BITMASK Solution

Use constraint - $2 \leq n \leq 15$ and do store states of all people in bitmask

We need to check all configurations of good and bad people and find out which one has maximum good people

a person will be good, if and only if his statements don't contradict
i.e. if in our configuration person i is assumed to be good, then if he says person j is good then j should be good in our configuration and vice versa

Example:

if there are 2 people
then possible configurations are

00 - 0 bad 1 bad
01 - 0 bad 1 good
10 - 0 good 1 bad
11 - 0 good 1 good

and for each of these configurations we will check if any statement contradicts or not

if a person is bad in our configuration, then his opinion doesn't matter, so no need to check

if a person i is good, then if $\text{statements}[i][j] = 1$ then j should be 1 in our configuration and

if $\text{statements}[i][j] = 0$ then j should be 0 in our configuration

so if any of above two statements contradict then our configuration / assumption is invalid and we will have to check for other configurations (make other assumptions)

and we will take the configuration that has maximum number of good people

BITMASK BASICS

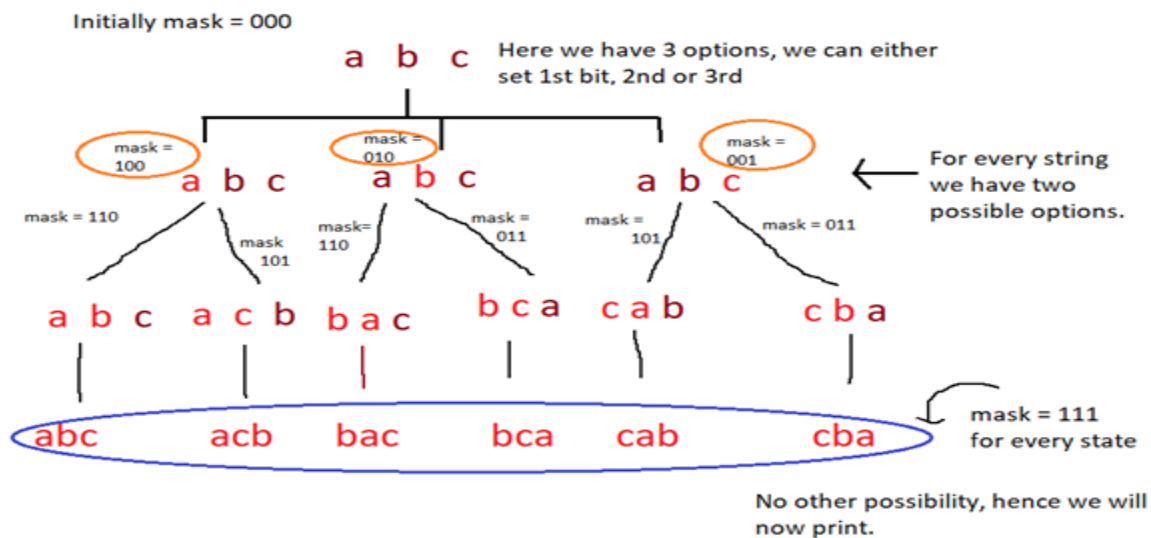
A binary digit is used as a flag in **bitmasking** to denote the status or existence of a feature or trait. To accomplish this, certain bits within a binary number are **set or reset to reflect a particular state** or value.

Some of the most commonly used bitwise operations in bitmasking are:

- **OR (|)** – sets a bit to 1 if either of the corresponding bits in the operands is 1.
- **AND (&)** – sets a bit to 1 if both the corresponding bits in the operands are 1.
- **XOR (^)** – sets a bit to 1 if the corresponding bits in the operands are different.
- **NOT (~)** – flips the bits in the operand, i.e., sets 0 bits to 1 and 1 bits to 0.

General way to implement Bitmasking:

- Create the list or array you wish to manipulate with bitmasking.
- Establish the list or array's size to determine how many bits you'll need for the binary representation.
- Check the corresponding bits for each index, then carry out the required operation based on their values. This could entail performing an operation, including or excluding entries from a subset, or both.
- Repeat the above operation in a loop until every element of the list has been checked.
- Utilize the outcomes as necessary for your specific application.



```
String s; ← contains our original string
int len; ← length of our original string, s
void permute(String s1, int mask)
{
    if (mask == ((1 << len) - 1))
    {
        print(s1);
        return;
    }

    for (int i = 0; i < len; i++)
    {
        if ((mask & (1 << i)) == 0)
        {
            s1 = s1 + s[i];
            permute(s1, mask | (1 << i));
            s1 = s1.substring(0, s1.length() - 1);
        }
    }

    return;
}
```

This will have diff. mask at each iteration as discussed in the explanation above.

Checking if all the states are reached then, print the string.
len actually has number of char.
Doing $1 \ll \text{len}$ will give us 1000 (8) and max possible value for mask is 111. Hence we subtract 1 here. And if $\text{mask} == 1 \ll \text{len} - 1$ then we print String as no further choices are left. (Assuming string is of length 3)

Checking if the ith bit is set or not in mask. (Checking if a particular char. has been visited or not.)
If ith bit is not set ($= 0$) then we need to visit it, or else if it is set then we need not do anything and check for other bits.

Adding ith char. to our string
Now, our task will be to set the ith bit so that same char. may not appear again in a particular string, generated by this recursive call. We've already learnt we can set the ith bit by performing | operation with the number and $(1 \ll i)$

This is the most imp. part of the code. Here we remove the last character that we have added to our string so that other characters also get the chance and we can generate all possible permutations.

Applications of Bitmasking:

Bitmasking is an effective method with numerous uses in computer science and technologies, such as:

- **Optimization:** Algorithm optimization can be achieved using bitmasking, which substitutes bit-level operations for expensive ones. As an

illustration, right shifting by one is a substantially faster operation than dividing by two.

- **Memory efficiency:** There are several circumstances in which storing data as a bitmask can be more memory-efficient than utilizing conventional data structures. For instance, you can use a single bitmask to indicate the presence or absence of a collection of items or list rather than an array of Boolean values.
- **Data compression:** By encoding data as a series of bits, bitmasking can be utilized in data compression methods to minimize the size of data.
- **Graphics:** In computer graphics, bitmasking is used to alter pixels and run various operations on images.
- **Networking:** Network protocols employ bitmasking to specify flags and options that are used to modify the protocol's behavior.

Advantages of Bitmasking:

- **Greater speed:** Bit-level operations typically occur more quickly than more conventional operations like addition or multiplication. Bitmasking enables a speedy and effective execution of sophisticated operations on massive data sets.
- **Memory efficiency:** Bitmasking can be used for compact and memory-efficient storage of big collections of data.
- **Code simplicity:** By swapping out complex conditional statements with straightforward bit-level operations, bitmasking can make code simpler and more elegant.
- **Versatility:** Bitmasking is a flexible method that can be applied to a variety of tasks, including network protocols, cryptography, and data compression.
- **Data masking and filtering:** By selectively turning on or off specific bits, bitmasks can be used to mask or filter data.
- **Bit-level programming:** For low-level programming activities like creating device drivers or operating systems, where it is important to directly manipulate bits and bytes, bitmasks are a crucial tool.

Disadvantages of Bitmasking:

- **Limited range:** Bitmasks have a limited range since they can only represent a finite range of values. Bitmasking becomes impossible if there are more potential values than there are bits in the bitmask.
- **Code complexity:** Bitmasking can make a program harder to comprehend, especially if it uses complicated bitwise operations.

- **Debugging:** Bitwise operations have the potential to bring elusive bugs into programs. A misplaced bitwise operator, for instance, can radically alter the meaning of an expression.
- **Restricted readability:** While using bitmasks can make code shorter, those unfamiliar with the method may find it difficult to understand. As a result, maintaining and updating the code over time may be challenging.

```
class Solution {
    public int maximumGood(int[][] statements) {
        int result = 0;
        int n = (1 << statements.length) - 1;

        for (int mask = 1; mask <= n; mask++) {
            if (isValid(statements, mask)) {
                result = Math.max(result, Integer.bitCount(mask));
            }
        }

        return result;
    }

    public boolean isValid(int[][] statements, int mask) {
        for (int i = 0; i < statements.length; i++) {
            // if i is not a good person in the current mask,
            // we don't need to validate the
            // statements
            if (((1 << i) & mask) == 0) {
                continue;
            }

            for (int j = 0; j < statements[0].length; j++) {
                if (statements[i][j] == 2) {
                    continue;
                }

                // i said j bad but j is not bad (0)
                // || i said j good, but j is not good
                if ((statements[i][j] == 0 && ((1 << j) & mask) != 0)
                    || (statements[i][j] == 1
                        && ((1 << j) & mask) == 0)) {
                    return false;
                }
            }
        }

        return true;
    }
}
```

we will only care about only good persons and ignore bad persons
bcoz good persons will always tells truth.

We assume person i is good so he always speaks
truth so there are 2 possible contradictions :

(i) Person i said that person j is good but we assume person j is bad so,
here it contradict.

(ii) Person i said that person j is bad but we assume person j is good so,
here it contradict.

if everything is okay (no contradiction)
then just count the no. of good peoples in this combination
Similarly do for all combinations and return max of them.

Problem-15: 1307. Verbal Arithmetic Puzzle (Backtracking)

Given an equation, represented by words on the left side and the result on the right side.

You need to check if the equation is solvable under the following rules:

- Each character is decoded as one digit (0 - 9).
- No two characters can map to the same digit.
- Each words[i] and result are decoded as one number **without** leading zeros.
- Sum of numbers on the left side (words) will equal to the number on the right side (result).

Return true *if the equation is solvable, otherwise return false.*

Example 1:

Input: words = ["SEND","MORE"], result = "MONEY"

Output: true

Explanation: Map 'S' -> 9, 'E' -> 5, 'N' -> 6, 'D' -> 7, 'M' -> 1, 'O' -> 0, 'R' -> 8, 'Y' -> 2

Such that: "SEND" + "MORE" = "MONEY" , 9567 + 1085 = 10652

//Step 1: Generate all unique charset

//Step 2: Generate score of all char count arr (add for source , remove for target)

// if more than single word -> no leading zero.

//Step 3 :Generate uniquecharset,charcount and non-leadingzero arr for all words

// do for source,target.

//Step 4:create a list of all unique character using in source,target.

//Step 5:do backtrack by assigning (0-9) to all char list and try all combinations

// keeping base case when diff becomes zero return.

Solution

```

import java.util.HashSet;
import java.util.Set;
class Solution {
    public boolean isSolvable(String[] words, String result) {
        Set<Character> st = new HashSet<>();
        // 65-91(upper case) , 26 (lower case letter)
        int cscount[] = new int[26];
        boolean noPrefixZero[] = new boolean[26];
        for (String word : words) {
            updateCharacter(word, cscount, noPrefixZero, st, 1);
        }
        updateCharacter(result, cscount, noPrefixZero, st, 2);
        char csList[] = new char[st.size()];
        int i = 0;
        for (char ch : st) {
            csList[i++] = ch;
        }
        boolean used[] = new boolean[10];
        return backtrack(csList, cscount, noPrefixZero, 0, used, 0);
    }

    /**
     * To update the character score
     */
    private void updateCharacter(String word, int cscount[],
                                boolean noPrefixZero[], Set<Character> st, int param) {
        int n = word.length();
        for (int i = 0; i < n; i++) {
            char ch = word.charAt(i);
            if (i == 0 && word.length() > 1) {
                noPrefixZero[ch - 'A'] = true;
            }
            st.add(ch);
            // score from right to left for scoring.
            if (param == 1) {
                cscount[ch - 'A'] += Math.pow(10, n - i - 1);
            } else {
                cscount[ch - 'A'] -= Math.pow(10, n - i - 1);
            }
        }
        // SEND,MORE = MONEY
        // [0, 0, 0, 10, 1000, 0, 0, 0, 0, 0, 0, 0, 0, 100, 0, 0,
        // 0, 0, 10000, 0, 0, 0, 0, 0, 0, 0]
    }
}

```

```
/**
 * To do backtrack by trying all possibilities of character
 *
 * @param csList
 * @param cscount
 * @param noPrefixZero
 * @param diff
 * @param used
 * @param index
 * @return
 */
private boolean backtrack(char csList[], int[] cscount,
                          boolean noPrefixZero[], int diff, boolean used[],
                          int index) {
    int n = csList.length;
    if (index == n) {
        return (diff == 0);
    }
    for (int d = 0; d <= 9; d++) {
        char ch = csList[index];
        if (used[d] || (d == 0 && noPrefixZero[ch - 'A'])) {
            continue;
        }

        used[d] = true;
        if (backtrack(csList, cscount, noPrefixZero,
                      diff + cscount[ch - 'A'] * d, used, index + 1))
        {
            return true;
        }
        used[d] = false;
    }

    return false;
}
}
```